INTERVIEW PREP · FINAL REVISION FILE

Aestheticommerce

Multi-vendor handmade art & crafts marketplace, MERN stack.

Part 0: zaroori warnings, “I” vs “we”, notes kitne reliable hain, 10-minute revision sheet.

Step 2–4: overview, pitch, tech stack, architecture, request flows, har module ka deep dive.

Step 5–6: design decisions, sabse mushkil problems, kamiyan, scaling, 60 interview questions with model answers.

Step 7: har technical concept ka simple explanation + project mein kahan use hua.

Appendix A–B: local pe app chalana, folder tree aur har file ka kaam.

PROJECT_NOTES.md ka saaf, organized version (Hinglish) · 14 October 2026

Contents

Is file ko kaise use karein

Part 0: Pehle ye padho

0.1 Teen zaroori baatein

0.2 Attribution: "I" kab bolna, "we" kab

0.3 Ye notes kaise bane, aur inpe kitna bharosa karein

0.4 10-minute revision sheet

Step 2: High-level overview

2.1 Project kya hai?

2.2 Kis problem ko solve karta hai? (inference)

2.3 Ek order shuru se aakhir tak (interview mein ye flow sunana)

2.4 30-second elevator pitch (English, interview ke liye)

2.5 Full tech stack, aur "isse kyun choose kiya?"

2.6 Honest reality check: interview mein ye "done" claim mat karna

Step 3: Architecture aur request flow

3.1 Overall architecture

3.2 Ek request shuru se aakhir tak

3.3 Folder structure conventions

3.4 Design patterns jo project mein use hue (asli code ke saath)

Step 4: Har part ka deep dive

4.1 Database models: har field, relationships, indexes, validations

4.2 Saare API endpoints

4.3 Authentication aur authorization

4.4 Frontend state management

4.5 Routing

4.6 Error handling aur validation

4.7 Real-time feature: Socket.io (rider ko delivery request)

4.8 File uploads (Multer → AWS S3 → CloudFront)

4.9 Payments: Razorpay aur COD (line by line)

4.10 Email

4.11 Caching

4.12 Cron job: badge automation

4.13 Baaki features: PDF, notifications, geo, SEO

4.14 Security measures: kya hai, kya nahi hai

4.15 Performance optimizations

4.16 Testing setup

4.17 Deployment, hosting aur CI/CD

4.18 Step 3 + 4 ke naye findings: quick list (Section 2.6 ki list ke alawa)

Step 5: Design decisions, sabse mushkil problems, kamiyan aur scaling

5.1 Key design decisions aur unke trade-offs

5.2 Sabse mushkil 5 technical problems ("the hardest problem I solved")

5.3 Kamiyan, bugs, aur "what would you do differently?"

5.4 Project ko scale aur extend kaise karein

Step 6: Interview questions aur model answers

6.1 Basic questions (15): stack aur project kya hai

6.2 Intermediate questions (15): features kaise kaam karte hain

6.3 Advanced questions (15): security, scaling, edge cases, trade-offs

6.4 Tricky follow-ups (10): "kya tumne sach mein banaya?"

6.5 Behavioral / STAR questions (5)

6.6 Quick revision: numbers aur facts jo yaad rakhne hain

6.7 Interview mein ye cheezein "done" mat bolna

Step 7: Key concepts jo is project mein use hue

7.0 Sabse pehle ye concepts pakke karo

7.1 JavaScript aur Node.js basics

7.2 Web aur HTTP basics

7.3 Express backend ke concepts

7.4 Authentication aur security

7.5 MongoDB aur Mongoose

7.6 Real-time: WebSocket aur Socket.io

7.7 File upload aur media

7.8 Payments

7.9 Background jobs, caching aur email

7.10 React ke concepts

7.11 Redux Toolkit (state management)

7.12 Frontend routing (React Router 7)

7.13 HTTP client aur browser APIs

7.14 UI aur styling libraries

7.15 SEO aur analytics

7.16 Tooling, build aur deployment

7.17 Architecture aur design patterns

7.18 Concepts jo project mein NAHI hain (interviewer pooche to sach bolo)

Appendix A: Local pe kaise chalayein (revision ke liye)

A.1 Start aur stop

A.2 URLs aur demo logins

A.3 Setup mein kya kiya gaya

A.4 Local pe kya kaam nahi karega (expected)

A.5 Revision ke liye suggested walkthrough (yahi interview wala flow hai)

Appendix B: Folder tree aur har file ka kaam (Step 1)

1.1 Big picture: ek folder, 5 apps

1.2 Backend/ (Node.js + Express): poore system ka dimaag

1.3 FrontEnd/: customer website (React 19 + Vite 6)

1.4 Seller-Panel/: seller dashboard (React 19 + Vite 7 + Redux Toolkit + MUI 7)

1.5 Admin-Panel/: admin dashboard (React 19 + Vite 7, Redux nahi)

1.6 delivery-dashboard/: rider app (React 19 + Vite 7 + Redux Toolkit + Socket.io client)

> Ye `PROJECT_NOTES.md` ke saare notes ka saaf, ek jagah organized version hai (2026-10-14). Content wahi hai jo `PROJECT_NOTES.md` mein hai; bas order aur headings saaf kiye gaye hain, aur sabse upar Part 0 joda gaya hai (warnings, attribution, notes kaise bane, aur 10-minute revision sheet). Section numbers (2.3, 4.9.5, D6, H1, A4, 7.5.15…) `PROJECT_NOTES.md` wale hi rakhe gaye hain, taaki "Section 4.9 dekho" jaisa koi bhi reference dono files mein same jagah le jaaye.

Is file ko kaise use karein

Part	Kya milega	Kab padhna
Part 0	Zaroori warnings, "I" vs "we", notes kitne reliable hain, 10-minute revision sheet	Sabse pehle, aur interview se 10 minute pehle
Step 2	Project kya hai, kis problem ko solve karta hai, order flow, pitch, tech stack aur "kyun", honest reality check	Pehla din
Step 3	Architecture diagram, ek request shuru se aakhir tak, folder conventions, design patterns	Pehla din
Step 4	Har module ka deep dive: models, saare APIs, auth, state, routing, errors, Socket.io, uploads, payments, email, caching, cron, security, performance, testing, deployment	Doosra din (sabse lamba part)
Step 5	Design decisions D1–D16, sabse mushkil problems H1–H5 (English narration ke saath), kamiyan, scaling	Teesra din
Step 6	60 interview questions + model answers, numbers ki table, "done mat bolna" list	Teesra din, aur interview se ek din pehle
Step 7	Har technical concept: simple explanation + project mein kahan use hua	Jab bhi koi shabd samajh na aaye
Appendix A	Local pe app chalana, demo logins, walkthrough	Demo ya hands-on revision ke liye
Appendix B	Folder tree aur har file ka kaam (Step 1)	Reference, jab koi file dhoondhni ho

Agar basics kamzor hain (JWT, middleware, index, transaction jaise shabd naye lagte hain): Step 2 ke baad seedha Step 7 padho (pehle 7.0 ki table), phir Step 3 aur 4.

Revision plan:

- **3 din:** upar wali table ka order.
- **1 din:** Part 0 → Step 2 → 3.1 aur 3.2 → 5.1 aur 5.2 → 6.1 se 6.4 → 7.0.
- **1 ghanta:** 0.4 sheet → 2.4 ka pitch bol ke practice → H1–H5 ki narrations (5.2) → 6.6 aur 6.7.

Symbols jo poori file mein hain:

- **(verified)**: local pe app chala ke check kiya gaya (2026-10-10).
- **(inference)**: andaaza; code mein iska seedha saboot nahi hai.
- **(naya)**: Step 5 mein naya mila, sirf code padh ke.
- ⚠: trap ya honest caveat. Ise zaroor padhna.

Part 0: Pehle ye padho

0.1 Teen zaroori baatein

1. **Leaked password:** `Backend/generateSiteMap.js` line 6 mein MongoDB Atlas ka connection string **username aur password ke saath hardcoded** hai.
   - Agar ye repo GitHub pe hai to DB user ka password turant badlo (rotate karo).
   - URI ko file se hatao aur uski jagah `process.env.DB_CONNECT` use karo.
   - Interviewer tumhara GitHub khol sakta hai.
2. **Team project:** code comments mein alag-alag teammates ke naam hain, aur DB ka naam `InternshipDb` hai, to ye team ya internship project lagta hai.
   - Is folder mein git history nahi hai, isliye code se pata nahi chal sakta ki kaunsa part tumne likha.
   - Interview se pehle tay kar lo: "*Maine X aur Y module banaye.*"
3. **Kuch features adhoore hain:** kai cheezein sirf stub (placeholder) hain ya incomplete hain. Unki list Section 2.6 mein hai. Inko interview mein "done" claim mat karna.

0.2 Attribution: "I" kab bolna, "we" kab

Step 6 ke answers first-person mein hain ("I built…"), jaisa tumne maanga. Lekin code comments mein kai teammates ke naam hain, aur git history nahi hai. Isliye:

- Jo module tumne khud banaya, uske liye "I" bolo.
- Jo teammate ne banaya, uske liye "we" ya "my teammate built X, I worked on Y" bolo.
- Interviewer 2–3 follow-up mein pakad leta hai ki kisne kya banaya. Galat claim se achha honest "we" hai.

0.3 Ye notes kaise bane, aur inpe kitna bharosa karein

- **2026-10-10, Step 1–2:** poora code padh ke likhe gaye. Backend ki har file line-by-line padhi gayi, aur chaaron frontend apps ki har JS/JSX/config file bhi. CSS sirf skim ki gayi.
- **2026-10-10, Step 3–4:** backend ki core files (models, middleware, payment, order, delivery, auth, services) dobara line-by-line padhi gayi, aur important claims code khol ke cross-check kiye gaye. Phir project local pe chalaya gaya, aur poora flow API se run kiya gaya (Appendix A). Jo baatein wahan check hui, unpe **(verified)** likha hai.
- **2026-10-14, Step 5–6:** Steps 1–4 ke notes pe based. Saath mein ye files dobara padhi gayi: `controllersDeliveryPanel/delivery.controller.js`, `services/deliveryBroadcast.service.js`, `services/deliveryTask.service.js`, `services/home.services.js`, `routes/product.routes.js` (listing pipeline), `utils/productForClient.js`, aur `routes/payment.routes.js` (helpers). Is step mein kuch bhi run nahi kiya gaya.
- **2026-10-14, Step 7:** concepts ke naye snippets code khol ke check kiye gaye; kuch run nahi kiya gaya.
- **Correction:** Step 5 likhte waqt notes mein likha gaya tha ki `local-dev/` folder project mein nahi hai. 2026-10-14 ko dobara check kiya: folder maujood hai, aur paanchon apps ki `.env` files aur `node_modules` bhi hain (sirf `ls` se check kiya, app start karke nahi).
- Code snippets kahin-kahin chhote kiye gaye hain. `// ...` ka matlab hai beech ka code hataya gaya hai.
- Code mein kahin nahi likha ki koi tool ya design *kyun* chuna gaya. "Kyun" wale points standard engineering reasons hain jo code ke design se match karte hain; unhe apne words mein confirm kar lena.

0.4 10-minute revision sheet

30-second pitch (English, Section 2.4):

> "Aestheticommerce is a multi-vendor marketplace for handmade art and crafts. It's four React apps — a customer store, a seller dashboard, an admin panel and a delivery-rider app — running on one Node/Express and MongoDB backend. Customers filter products and pay through Razorpay or cash on delivery, and the server always recalculates prices so the client can't tamper with them. Sellers sign up with email OTP and KYC, upload images and video to S3, and admins approve every product before it goes live. For delivery, a MongoDB geospatial query and Socket.io push each order to riders within 5 km, and a database transaction makes sure two riders can't take the same task."

Pitch ke turant baad: "*My part was ___.*" (Part 0.2 dekho.)

Ek order ka safar, ek line mein (detail: 2.3): checkout → server DB se price nikaalta hai → Razorpay order + Pending order (30 min TTL) → HMAC verify ya webhook → Paid → har seller ka delivery task → `$geoNear` 5 km + Socket.io push → transaction se accept → rider status badalta hai aur order sync hota hai.

5 sabse mushkil problems (detail: 5.2):

#	Problem	Solution ek line mein	Honest caveat
H1	Do riders ek task accept na karein	`findOneAndUpdate({ _id, status: "Pending" })` + rider lock, dono ek transaction mein	`assignDefaultRider` ye pattern follow nahi karta
H2	Payment kho na jaaye, nakli payment na chale	HMAC verify + webhook (raw body) + idempotent `markRazorpayOrderPaid` + TTL cleanup	"Already Paid" check atomic nahi; rebuild branch amount match nahi karta
H3	Chhupi fees ke saath price pe filter/sort/pagination	Pipeline mein `$addFields` se `offeredPrice`, `$facet` se count + page, `$project` se fees hatao	Price ka formula do jagah hai
H4	Ek order, kai sellers ka status sync	Har seller ka alag task + state machine + order sync, sab ek transaction mein	Final states pe state machine ka check skip hota hai
H5	Personalised home feed, fast bhi	`Promise.all` se 3 signals + tag scoring + popular data ki TTL cache	COD orders recommendation signal nahi bante

Numbers jo zubaan pe hon (poori table: 6.6): 4 frontends + 1 backend, 17 models, ~140 endpoints (lagbhag 56 bina auth), rider radius 5 km aur max 10 riders, fallback 15 second (popup 10 second), Pending order TTL 30 min, GST 5%, JWT 30 din, bcrypt 10 rounds, listing 15 per page (max 50), badge cron roz 00:00 UTC.

"Done" mat bolna (poori list: 6.7): asli rider location / geocoding / live tracking, stock management, refunds aur payouts, delivery OTP, seller approval, seller/admin notifications, tests / CI / Docker, customer aur seller ke liye real-time, coupon banane ka UI.

Koi kami pooche to formula: "*That part isn't finished. Here's what exists today, and here's exactly how I'd complete it.*"

"What would you do differently?" ke 4 shabd (detail: 5.3): Security, Consistency, Tests, Scale. Har ek ke saath project ka ek asli example do.

Step 2: High-level overview

2.1 Project kya hai?

Aestheticommerce ek **multi-vendor marketplace** hai **handmade art & crafts** ke liye. Isme 6 main categories hain: Visual Arts, Paper Crafts, Traditional & Cultural Crafts, Sculptural & 3D, Textile Arts, Decorative Arts.

Products ko **occasions** ke hisaab se bhi dhoondha ja sakta hai: Diwali, Eid, Christmas, birthday, housewarming.

Project **India ke liye bana hai**. Iske saboot code mein hain:

- prices INR mein
- GST 5%
- Razorpay (UPI, cards)
- seller KYC ke liye Aadhar aur PAN
- IST time ke hisaab se delivery window
- 6-digit pincode

Is platform ke 4 tarah ke users hain, aur har ek ki apni app hai:

User	App	Ye user kya kar sakta hai
Customer	`FrontEnd`	Products browse aur filter karna, cart, wishlist, coupon, Razorpay ya COD se checkout, orders track karna, review dena, invoice download karna
Seller	`Seller-Panel`	Email OTP se signup, KYC onboarding, product add karna (images + video), pricing set karna, orders dekhna, delivery rider request karna, stats dekhna
Admin	`Admin-Panel`	Har product approve ya reject karna (quality control), badges dena, sellers block ya delete karna, riders approve karna, banners manage karna, analytics dekhna
Delivery rider	`delivery-dashboard`	Register karna (admin approval ke baad login), 5 km ke andar ke tasks dekhna, real-time request accept karna, status update karna (jisse order ka status bhi apne aap update hota hai)

2.2 Kis problem ko solve karta hai? (inference)

Code mein koi problem statement likha hua nahi hai. Ye categories, policies aur features dekh kar nikala gaya hai, to apne shabdon mein confirm kar lena.

- **Artisans ki problem:** chhote artisans aur ghar se kaam karne wale creators ke paas apni website, payment gateway aur delivery ka setup nahi hota.
- **Customer ki problem:** bade generic marketplaces mein handmade aur occasion-wise gifts dhoondhna mushkil hota hai.
- **Ye platform kya deta hai:**
  - curated storefront
  - online payment
  - hyperlocal delivery partner network (rider ko 5 km ke andar se match karna)
  - admin approval se quality control

2.3 Ek order shuru se aakhir tak (interview mein ye flow sunana)

1. Customer checkout karta hai. `POST /api/payment/create-order` call hota hai.

2. Server items ka price **DB se dobara nikalta hai** (`buildPaymentOrderItems`), aur GST + coupon khud calculate karta hai (`resolveOrderPricing`).
3. Server Razorpay order banata hai, aur DB mein `Pending` order save karta hai. Is order pe 30 min ka TTL hai.
4. Browser mein Razorpay ka popup khulta hai. Payment ke baad `POST /api/payment/verify` call hota hai, aur server **HMAC-SHA256 signature** check karta hai.
5. Agar user payment ke baad tab band kar de, tab bhi kaam nahi rukta: Razorpay ka `webhook` (`payment.captured`) order ko `Paid` mark kar deta hai.
6. Paid hone ke baad `createDeliveryTasksForOrder()` har seller ke liye ek `DeliveryTask` banata hai. Phir `$geoNear` se 5 km ke riders dhoondhe jaate hain, aur unhe Socket.io pe `newDeliveryRequest` bheja jaata hai.
7. Rider accept karta hai. `acceptTask` MongoDB **transaction** ke andar chalta hai, isliye do riders ek hi task nahi le sakte.
8. Rider status badalta hai: PickedUp → OutForDelivery → Delivered. Har step pe `syncOrderStatusFromTask()` customer ke order item ka status bhi update kar deta hai.
9. Sellers ko DB mein notification milta hai. Raat ko cron job sales ke hisaab se product badges update karta hai.

2.4 30-second elevator pitch (English, interview ke liye)

> "Aestheticommerce is a multi-vendor marketplace for handmade art and crafts. It's four React apps — a customer store, a seller dashboard, an admin panel and a delivery-rider app — running on one Node/Express and MongoDB backend. Customers filter products and pay through Razorpay or cash on delivery, and the server always recalculates prices so the client can't tamper with them. Sellers sign up with email OTP and KYC, upload images and video to S3, and admins approve every product before it goes live. For delivery, a MongoDB geospatial query and Socket.io push each order to riders within 5 km, and a database transaction makes sure two riders can't take the same task."

Hinglish mein gist: handmade products ka marketplace hai, jisme 4 apps ek backend share karte hain. Payment safe hai kyunki server price khud calculate karta hai. Sellers ki KYC hoti hai aur admin har product approve karta hai. Delivery ke liye geo query, real-time socket, aur transaction lock.

Pitch ke baad turant ek line apne kaam ki jodna: "*My part was ___ (jaise: payment + order flow / delivery module / seller panel).*"

2.5 Full tech stack, aur "isse kyun choose kiya?"

> Code mein kahin likha nahi hai ki koi tool kyun choose hua. Neeche wale reasons **standard, defend kiye ja sakne wale reasons** hain. Jahan code mein koi saboot hai, wo bhi likha hai.

Backend

Tech (version)	Project mein kahan use hua	Kyun choose kiya / alternatives se comparison
Node.js + Express 5.2	Poora API (`app.js`)	Frontend aur backend dono mein ek hi language (JavaScript). Ecosystem bahut bada hai. Express 5 async errors apne aap error handler tak bhej deta hai. **NestJS** zyada structured hai par boilerplate bhi zyada. **Django/Spring** ke liye alag language seekhni padti.

Tech (version)	Project mein kahan use hua	Kyun choose kiya / alternatives se comparison
MongoDB + Mongoose 8.21	Saare models	Schema flexible hai: badges object, embedded reviews, order items ka array. **GeoJSON + 2dsphere index built-in hai**, isliye rider proximity query asaan hai. Aggregation pipeline se filters aur analytics ban jaate hain. Atlas pe replica set milta hai, isliye transactions chal jaate hain. **PostgreSQL** mein geo ke liye PostGIS lagta; orders aur payments ke liye SQL ke joins aur constraints zyada strong hote (ye honest trade-off hai).
jsonwebtoken 9 (JWT)	Chaaron roles ka auth, 30 din expiry	Stateless hai: 4 alag clients ek hi backend use karte hain, aur server pe koi session store nahi rakhna padta. **Session + Redis** mein extra infrastructure lagta.
bcryptjs 3	Password hashing (10 salt rounds)	Pure JavaScript library hai, native build ka jhanjhat nahi. (`bcrypt` bhi `package.json` mein hai par use nahi hota.)
Zod 4 + **express-validator 7**	Zod: delivery routes, seller product routes, admin badges. express-validator: user, address, blog routes	Zod schema reuse ho sakta hai, `safeParse` saaf data deta hai, aur unknown keys strip ho jaati hain (mass-assignment se bachav). Do validation libraries ka hona inconsistency hai: purane routes ek mein, naye doosre mein.
Multer 2 (`memoryStorage`)	Har file upload	Multipart data parse karta hai. File buffer seedha S3 pe chala jaata hai, disk pe temp file nahi banti.
AWS S3 (`@aws-sdk/client-s3` **v3) + CloudFront**	`utils/s3Upload.js` : product media, KYC docs, avatars, banners, blogs	Sasta aur durable storage, aur CloudFront CDN se fast delivery. Project pehle **Cloudinary** pe tha (`scripts/migrateCloudinaryToS3.js` iska saboot hai). Cloudinary mein image transforms built-in hote hain, par cost aur limits ka issue hota hai.
Razorpay 2.9	`payment.routes.js`	India ka standard gateway (UPI, cards, netbanking). Order API + HMAC signature + webhook sab deta hai. **Stripe** India mein UPI ke liye utna strong nahi.
Socket.io 4.8	`server.js` , `deliveryBroadcast.service.js`	Riders ko server khud request push kar sakta hai. Auto-reconnect aur fallback built-in hain. **Polling** se load zyada hota, **SSE** sirf ek taraf (server → client) chalta hai.
node-cron 4	`jobs/badgeCron.js`	Roz raat ka badge job usi process mein chal jaata hai. Agar server ke multiple instances hon to job har instance pe chalega; tab **BullMQ ya Agenda** jaise queue ki zaroorat padegi.
Resend 6	`utilsSellerPanel/sendEmail.js` : OTP, order, contact emails	API se email bhejta hai, deliverability achhi hai, setup simple hai. Gmail SMTP pe sending limits hoti hain.
nodemailer 8	Sirf newsletter ka dead route	Gmail SMTP. Purana sendEmail code comment out hai.
PDFKit 0.17	Invoice aur shipping label	Server pe halka PDF banake seedha stream karta hai. **Puppeteer** (headless Chrome) bahut heavy hai.
swagger-jsdoc + swagger-ui-express	`/api-docs`	Code ke comments se API documentation ban jaata hai.
sitemap 9	`routes/sitemap.js` , `generateSiteMap.js`	SEO ke liye sitemap
cors, dotenv	`app.js`	Doosre origins se requests allow karna, aur env variables load karna
`package.json` mein hain par use nahi hote	`bcrypt` , `@mui/material` , `@emotion/*` , `router` , `crypto`	`crypto` npm package ki zaroorat nahi, Node ka built-in `crypto` use hota hai. `@mui` frontend library hai jo galti se backend mein install hui. Ulta, `mongodb` driver `sitemap.js` mein use hota hai par `package.json` mein declared nahi hai.

Frontend (chaaron apps)

Tech	Kahan	Kyun choose kiya / alternatives se comparison
React 19	Chaaron apps	Component-based UI, sabse bada ecosystem
Vite (FrontEnd mein 6, baaki mein 7)	Build tool	Dev server bahut fast hai, aur HMR (code save karte hi page update) milta hai. **CRA** ab deprecated hai. **Next.js** SSR deta jo customer store ke SEO ke liye behtar hota; ye project pure client-side SPA hai, ye iski limitation hai.
React Router 7	Chaaron apps	Client-side routing
Redux Toolkit 2 + react-redux 9	FrontEnd, Seller, Delivery (Admin mein sirf `useState`)	Global state (auth, cart, filters, tasks) ek jagah rehta hai. `createAsyncThunk` se API calls ka pattern ek jaisa ho jaata hai. **Context API** mein re-render issues aate hain. **Zustand** halka option hai. **RTK Query / React Query** se caching milti, jo yahan use nahi hui.
Axios 1.x	Saari HTTP calls (beech beech mein `fetch` bhi)	Interceptors milte hain: FrontEnd mein 401 aane pe logout, delivery app mein token apne aap attach hota hai. JSON apne aap handle hota hai.
UI libraries	FrontEnd: custom CSS + shadcn-style Radix kit + styled-components (2 files). Seller: **MUI 7** (tables, dialogs). Admin: custom CSS, MUI sirf ek page pe	MUI data-heavy tables ke liye ready components deta hai (sort, pagination). Radix primitives accessible hote hain.
Charts	Seller: **nivo** (bar, line). Admin: **recharts**	Dono React chart libraries hain. Do alag libraries hona dikhata hai ki alag log alag panels pe kaam kar rahe the **(inference)**.
@mdxeditor/editor	Seller product description likhta hai; customer use read-only dekhta hai	Markdown rich-text editor. Read-only display ke liye bahut heavy hai.
react-select (CreatableSelect)	Seller ke category aur tags fields	Dropdown se choose karo, ya naya type karke bana do
socket.io-client 4.8	Sirf delivery-dashboard	Server ke push events sunne ke liye
framer-motion	Product gallery, footer	Animations
lucide-react, react-icons	Icons	
Toasts	FrontEnd + Delivery: react-toastify. Admin: react-hot-toast	Seller-Panel mein `toast()` call hota hai par `<Toaster/>` mount nahi hai, isliye toast dikhta hi nahi.
react-helmet-async	FrontEnd SEO	Har page ke meta tags + JSON-LD (Product, BlogPosting)
date-fns	Seller notifications	"2 hours ago" jaisa relative time
Google Analytics (gtag)	Har `index.html`	Traffic tracking
Install hain par use nahi hote	FrontEnd: MUI, emotion, prop-types, redux, tailwind, sonner. Seller: bcrypt, jsonwebtoken, d3-*, react-spring. Admin: react-toastify. Delivery: MUI, react-icons	Cleanup ka kaam baaki hai

Third-party services aur hosting

- **MongoDB Atlas**: connection string `mongodb+srv://…mongodb.net` hai, isliye Atlas.
- **AWS S3 + CloudFront**, **Razorpay**, **Resend**, **Gmail SMTP**
- **OpenStreetMap Nominatim**: address form mein coordinates se address nikalne ke liye (reverse geocoding)
- **Google Maps search links**: rider app mein

- **Hosting (inference):**
  - Backend shayad **Render** pe hai, kyunki `server.js` mein comment hai: "Render will provide PORT".
  - FrontEnd shayad **Vercel** pe hai, kyunki `.gitignore` mein `.vercel` hai.

Dev tools

- ESLint 9 (flat config) har app mein
- npm + bun, dono ke lockfiles
- Swagger

Ye nahi hain: tests (Jest, Vitest, Testing Library), TypeScript, Docker, CI/CD.

2.6 Honest reality check: interview mein ye "done" claim mat karna

Neeche wali cheezein code mein adhoori hain ya sirf stub hain. Interviewer pooche to sach bolo, aur saath mein batao ki tum ise kaise fix karoge.

1. **Rider location asli nahi hai:**
   - Socket connect hote hi rider ki location hardcoded New Delhi `[77.209, 28.613]` pe set ho jaati hai.
   - Delivery task ke pickup aur drop coordinates bhi yahi placeholder hain. Address se coordinates nikalne ka code (geocoding) nahi hai.
   - Live tracking nahi hai. Asli GPS sirf "Share My Location" button dabane pe ek baar jaata hai.
2. **Delivery OTP nahi hai:** `DeliveryTask.otp` field model mein hai, lekin koi code use generate ya verify nahi karta.
3. **Stock kam nahi hota, aur check bhi nahi hota:** order hone pe `inStock` kabhi decrement nahi hota, aur order banate waqt stock check bhi nahi hota.
   - Local run mein ye pakka hua: "Macrame Wall Art Festive 6-3" ka `inStock: 0` hai, phir bhi uska 2 units ka COD order ban gaya.
4. **Seller approval adhoora hai:**
   - Admin **products** approve karta hai.
   - Lekin `seller.status` ya `verificationStatus` ko "approved/verified" karne ka koi endpoint ya UI nahi hai.
5. **Kai endpoints bina auth ke hain:**
   - `GET /api/orders` (saare orders) aur `/api/orders/:id`
   - `/api/document/invoice|label/:id`
   - `/api/dashboard/*`
   - `/api/contact`
   - admin ke rider routes
   - blog create
6. **Password reset flow mein gaps hain:**
   - Customer `reset-password` sirf email + naya password maangta hai, OTP proof nahi.
   - `verifyOtpController` jo OTP field padhta hai, wo `select:false` hai, isliye code ke hisaab se OTP check hamesha fail lagta hai. (Code run nahi kiya, sirf padha hai.)
   - FrontEnd ka `ChangePassword.jsx` render hote hi crash hota hai, kyunki `password` aur `showPassword` variables declare hi nahi hain.

7. **Seller aur admin notifications nahi dikhte:** `/api/notifications` customer wala `protect` use karta hai, aur seller/admin tokens ka format alag hai.
8. **Login response mein password hash leak hota hai:** customer login response ke `user` object mein password hash bhi chala jaata hai, kyunki query mein `select('+password')` hai aur wahi object return hota hai.
9. **Tests zero hain.** JWT `localStorage` mein rehta hai (XSS ka risk). Code mein bahut saare `console.log` hain, kuch mein token bhi print hota hai.

> Detail: security bugs Section 4.14.3 mein, priority ke hisaab se list 5.3 mein, aur "done mat bolna" wali list 6.7 mein.

Step 3: Architecture aur request flow

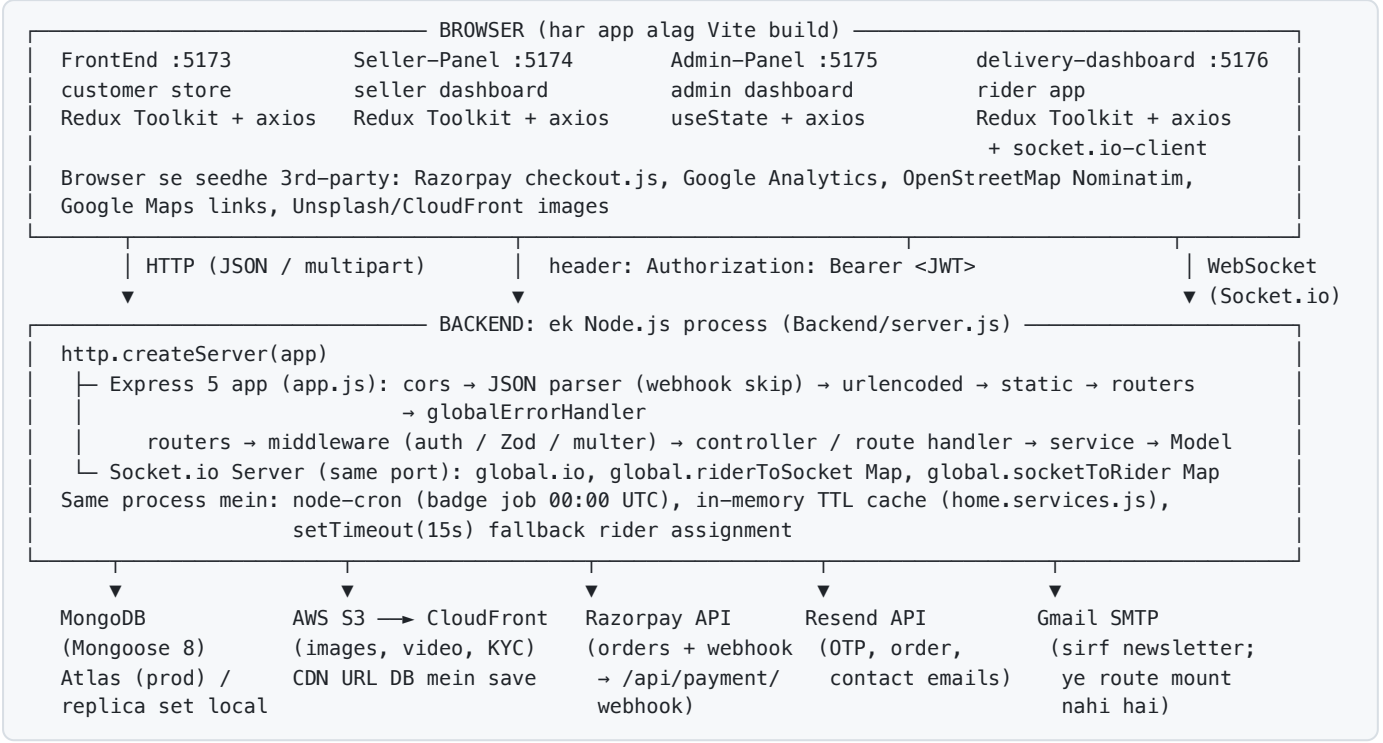
> Step 3 aur Step 4 2026-10-10 ko likhe gaye. Backend ki core files (models, middleware, payment, order, delivery, auth, services) dobara line-by-line padhi gayi. Chaaron frontends aur poore API catalog ke liye code alag se padha gaya, aur important claims ko seedha code kholke cross-check kiya gaya. Jo baat **local app chala ke verify** hui hai, uspe **(verified)** likha hai. Jo sirf code padh ke nikali hai, wahan "code padh ke" ya kuch nahi likha. **(inference)** ka matlab hai andaaza, code mein saboot nahi.

3.1 Overall architecture

Ek line mein

4 alag React SPAs (Single Page Applications) → **ek Node/Express backend** (REST API + Socket.io, dono ek hi process aur ek hi port pe) → **ek MongoDB database**, aur saath mein third-party services (S3, Razorpay, Resend).

Isse **"monolith backend + multiple frontends"** kehte hain. Microservices nahi hain: saara backend code ek hi deployable app mein hai, bas folders panel-wise bante hain.



Har layer ki zimmedari

Layer	Kya karta hai	Kya nahi karta
React apps	UI dikhana, form input lena, Redux mein state rakhna, API call karna, token ko localStorage mein rakhna	Price ka final calculation (sirf display ke liye karta hai), security decisions
Express routes + middleware	URL ko handler se jodna, token verify karna, input validate karna, files parse karna	
Controllers / route handlers	Request padhna, business logic chalana (kai jagah logic seedha route file mein hai), response bhejna	
Services	Reusable business logic: delivery task banana, rider broadcast, recommendations, badges, stats	

Layer	Kya karta hai	Kya nahi karta
Models (Mongoose)	Data ka shape, validation, hooks, indexes	
MongoDB	Data store, aggregations, geo queries, transactions, TTL deletion	
Socket.io	Server se rider ko turant "new delivery request" push karna	Customer/seller ko live update (ye use nahi hota)

Kuch architecture decisions aur unke trade-offs (interview ke liye)

1. **Stateless JWT auth:** server pe session store nahi hai, isliye backend ko scale karna asaan hai. Lekin token revoke nahi ho sakta (Section 4.3).
2. **Socket.io usi process mein:** setup simple hai. Lekin `global.riderToSocket` ek in-memory `Map` hai, isliye **2 backend instances chalayein to rider ka socket doosre instance pe ho sakta hai** aur message nahi pahunchega. Scale karne ke liye Socket.io Redis adapter chahiye.
3. **Cron aur `setTimeout` bhi usi process mein:** server restart hua to pending 15 second ka timer kho jaata hai, aur multiple instances pe cron har instance pe chalega.
4. **Ek database, kai collections:** saare panels same DB share karte hain. Isliye "order ka status rider ne badla, to customer ko turant dikhe" bina kisi message queue ke ho jaata hai.
5. **4 alag frontends:** har role ka bundle chhota rehta hai aur alag team alag app pe kaam kar sakti hai. Lekin common code (auth, axios setup, BackButton, toasts) chaaron mein copy hai.

3.2 Ek request shuru se aakhir tak

3.2.1 Generic flow (har API call aise chalti hai)

```
[1] User click
      |
[2] React component ka event handler  ──►  dispatch(thunk(args))            (Redux Toolkit createAsyncThunk)
      |                                          |
      |                                    [3] thunk → *Service.js → axios.METHOD(`${VITE_API_URL}/path`, body,
      |                                         { headers: { Authorization: `Bearer ${token}` } })
      |                                         |  network (JSON)
      ▼                                         ▼
[4] server.js ka http server → Express app (app.js) middleware chain, upar se neeche, order mein:
      a. cors()                              → Access-Control-* headers
      b. custom JSON parser                  → req.body (sirf /api/payment/webhook skip)
      c. express.urlencoded()                → form data
      d. express.static('uploads'), ('public') → file match ho to yahin response, warna next()
      e. app.use('/prefix', router)          → URL prefix se router chuna jaata hai
      |
[5] Router ke andar route-level middleware (order mein):
      auth (protect / protectSeller / protectAdmin / protectDelivery) → req.user
      → multer (agar file upload) → req.file(s)
      → validation (express-validator rules ya zodValidate) → saaf req.body
      |
[6] Controller / handler → service functions → Mongoose Model → MongoDB query
      |
[7] res.status(code).json(data)          (error aaye to next(err) → globalErrorHandler → JSON error)
      |
[8] axios promise resolve/reject → thunk fulfilled/rejected → slice ka reducer state badalta hai
      |                              (401 aaye to FrontEnd ka axios interceptor → logoutUser)
[9] useSelector se jude components re-render → UI update (+ toast)
```

3.2.2 Real example: customer "Add to Cart" dabata hai

```
ProductCard.jsx:93  handleAddToCart()
   if (!user || !token) → toast.error('Please login…'), return            ← client-side guard
   dispatch(addToCartWithBackendSync({ id, name, price: product.offeredPrice, image })).unwrap()
        │
cartSlice.js:66  createAsyncThunk('cart/addToCartWithBackendSync')
   cartService.add(getState, { productId: item.id, quantity: item.quantity ?? 1, price })
        │
cartService.js:14  axios.post(`${API_URL}/users/cart`, {productId, quantity, price},
                           { headers: { Authorization: `Bearer ${getState().user.token}` } })
        │  ─── HTTP POST /users/cart ──►
app.js:90  app.use("/users", cartRoutes)            (cart router pehle mount hai, comment: "Cart routes first")
cart.routes.js  router.post('/cart', protect, cartController.addToCart)
        │
authMiddleware.js  protect: header se token → jwt.verify(token, JWT_SECRET) → User.findById(decoded._id)
                   → req.user = user  (fail → 401 {message:"Not authorized, token failed"})
        │
cart.controllers.js  addToCart:
   Product.findById(productId)                        → nahi mila to 404
   Cart.findOne({ user: req.user._id }) || new Cart  → product pehle se hai to quantity += , warna push
   cart.save() → Cart.findById().populate('items.product') → toClientProduct (fees hata ke offeredPrice)
        │  ◄── 200 { message: 'Added to cart', cart: [ {product:{…offeredPrice}, quantity} ] }
cartSlice.js:244  fulfilled → state.cartItems = mapBackendCartToItems(payload)
                  totalQuantity / totalAmount = reduce(...)
        │
Navbar.jsx:56  useSelector(state.cart.totalQuantity) → badge number badal jaata hai; toast.success('Added to cart!')
```

Dhyan do: client `price` bhejta hai, par server use ignore karta hai. Server cart mein sirf `product` ID aur `quantity` rakhta hai, aur price hamesha DB se nikalta hai. Ye sahi design hai.

3.2.3 Real example: COD order + rider ko real-time request

```
Customer (Checkout.jsx)                Backend                                        Rider app (socket on)
──────────────────────                 ─────                                          ─────────────────────
"Place Order" (COD)
dispatch(createOrder({...orderData,
  paymentMethode:"COD"})).unwrap()
  ── POST /api/orders ──────────────► protect → order.routes.js:69
                                       1. items: har product DB se, price = offeredPrice
                                       2. resolveOrderPricing: subtotal + 5% GST − coupon
                                       3. new Order(...).save()  (pre-save: orderId "OD…", itemIds)
                                       4. CouponUsage.create + Coupon.usedCount $inc
                                       5. createDeliveryTasksForOrder(order)
                                            har seller → DeliveryTask(status "Pending")
                                            broadcastToNearbyRiders(task):
                                              DeliveryPartner.aggregate($geoNear 5 km,
                                                isActive && isAvailable) → max 10 riders
                                              riderToSocket.get(riderId) → socketId
                                              io.to(socketId).emit("newDeliveryRequest", ──► useRiderSocket: Redux mein
                                                {taskId, pickup, drop})                       task add → popup (10s)
                                              setTimeout(15s → assignDefaultRider)
                                       6. sendOrderEmails (await nahi)
                                       7. Cart khaali karo (sirf agar body mein products nahi)
                                       8. Notification.insertMany (sellers ko)
  ◄── 201 order JSON ──────────────────
createOrderSection():
  clearCartFromBackend()  (client products bhejta hai, isliye server cart clear nahi karta)
  navigate('/order-placed/:id', {state})
                                                                                  Rider "Accept":
                                       ◄── POST /api/delivery/accept-task {taskId} ──
                                       protectDelivery → zodValidate → acceptTask:
                                         transaction start
                                         rider: {isAvailable:true} → false  (lock)
                                         task:  {status:"Pending"} → "Accepted"
                                         commit (koi bhi fail → abort, dono rollback)
                                       ── 200 ──────────────────────────────────────►
                                       Agar 15s mein kisi ne accept nahi kiya:
                                         assignDefaultRider → task.status = "Assigned"
```

```
                                    rider = delivery@test.com
```

3.2.4 Real example: Razorpay online payment

```
Checkout.jsx                          Backend (payment.routes.js)                  Razorpay
────────────                          ──────────────────────────                   ────────
dispatch(createRazorpayOrder(orderData))
  ── POST /api/payment/create-order ─► buildPaymentOrderItems (DB price)
                                       resolveOrderPricing (GST + coupon)
                                       razorpay.orders.create({amount paise}) ──────────► order_XYZ
                                       Order.create({Pending, razorpayOrderId,
                                                     expireAt: +30 min})
  ◄── {orderId, key_id, amount} ──────
new window.Razorpay(options).open()  ──────────────────────────────────────────────────► popup, user pays
handler(response) ◄──────────────────────────── {razorpay_order_id, payment_id, signature}
dispatch(verifyRazorpayPayment({...response, orderData}))
  ── POST /api/payment/verify ──────► HMAC_SHA256(order_id|payment_id, SECRET) === signature ?
                                       Pending order → Paid, expireAt hatao, coupon usage
                                       createDeliveryTasksForOrder + seller notifications
  ◄── 201 order ──────────────────────
navigate('/order-placed/…')
                                       Backup: ◄── POST /api/payment/webhook (payment.captured) ── Razorpay
                                       raw body HMAC verify → order Paid (agar /verify nahi chala)
Popup band kiya (ondismiss)
  ── DELETE /api/payment/cancel-pending/:rzpOrderId ─► Pending order delete
Kuch nahi kiya → MongoDB TTL 30 min baad Pending order khud delete kar deta hai
```

3.3 Folder structure conventions

Backend (`Backend/`)

```
server.js            → entry: DB connect, cron, HTTP + Socket.io server
app.js               → Express app: global middleware + saare routers mount
db/db.js             → mongoose.connect
config/razorpay.js   → SDK instance
models/              → 17 Mongoose schemas (sabhi panels share karte hain)
routes*/             → URL → handler mapping            (routes/, routes-seller-panel/, routes-admin-panel/, routes-delivery-panel/)
Middleware*/         → auth, validation, error handler  (Middleware/, MiddlewareSellerPanel/, MiddlewareAdminPanel/,
     MiddlewareDeliveryPanel/)
controllers*/        → request/response handling        (controllers/, controllersSellerPanel/, controllersAdminPanel/,
     controllersDeliveryPanel/)
services*/           → reusable business logic          (services/, servicesSellerPanel/)
validators/          → Zod schemas (panel-wise subfolders)
utils*/              → helpers: AppError, asyncHandler, pricing, S3, PDF, email templates
jobs/                → cron jobs
scripts/, seedFile/  → one-time scripts (migration, seed data)
docs/ + swagger.js   → API documentation
```

Convention: har layer ke folder ke naam ke aage panel ka naam (`SellerPanel`, `AdminPanel`, `DeliveryPanel`) lagta hai. Customer ka code bina suffix ke hai. Iska fayda hai ki har team apne folder mein kaam kar sakti hai. Nuksaan ye hai ki duplicates ban gaye (do `AppError`, do `protectSeller`, do `statsService`).

Consistency kitni hai? Poora project ek jaisa nahi hai:

- **Achha layered:** delivery panel (route → Zod → controller → model), seller auth (route → controller → service).
- **Sab kuch route file mein:** `product.routes.js` (749 lines), `order.routes.js` (697), `payment.routes.js`, `addProduct.routes.js`, `productInfo.routes.js`, `dashboard.routes.js`, `invoice.routes.js`. Inme controller layer hai hi nahi.

Frontends

```
FrontEnd/ (aur Seller-Panel/ same pattern)
├── index.html, vite.config.js, package.json, env.example
├── pages/          → src ke BAHAR: har route ka patla wrapper (Navbar + component + Footer)
└── src/
    ├── main.jsx    → providers (Redux, Helmet)
    ├── App.jsx     → BrowserRouter + saare <Route>
    ├── Store/store.js
    ├── Redux/      → xyzService.js (sirf axios calls) + xyzSlice.js (thunks + reducers)
    ├── components/ → feature-wise folders, har component ke saath uski .css
    ├── hooks/, utils/, api/, ui/ (FrontEnd), Images/
```

- Admin-Panel: `pages/` (har page apna data khud fetch karta hai) + `src/component/` + `src/layouts/`. Redux nahi hai.
- delivery-dashboard: `src/Redux/`, `src/services/` (axios instance + socket singleton), `src/hooks/`, `src/components/`, `src/pages/`.

3.4 Design patterns jo project mein use hue (asli code ke saath)

Pattern	Kahan	Ek line mein samjho
Layered / MVC-jaisa (Route → Controller → Service → Model)	Backend	"V" (view) nahi hai, kyunki backend sirf JSON deta hai. View ka kaam React karta hai.
Middleware chain (Chain of Responsibility)	`router.post('/accept-task', protectDelivery, zodValidate(schema), acceptTask)`	Har middleware ya to `next()` call karke aage bhejta hai, ya khud response bhej ke request rok deta hai
Factory / Higher-order function	`zodValidate(schema, 'body')`, `asyncHandler(fn)`	Function jo naya middleware banake return karta hai
Custom error class + central error handler	`AppError` + `globalErrorHandler`	Saare errors ek jagah format hote hain
DTO / Mapper (data ko client ke liye shape karna)	`toClientProduct()`, `orderWithClientProducts()`, `taskToDashboardShape()`	DB document ko seedha nahi bhejte: internal fees hatate hain, extra fields jodte hain
Adapter	`uploadToS3` Cloudinary jaisa `{secure_url}` return karta hai	Naya provider, purana interface
Singleton (lazy)	S3 `client` (`s3Upload.js`), socket client (`delivery-dashboard/src/services/socket.js`)	Ek hi instance banta hai aur reuse hota hai
State machine	`updateTaskStatus` ka `allowedTransitions` (`Accepted → PickedUp → OutForDelivery → Delivered`)	Sirf allowed status changes hi ho sakte hain
Optimistic concurrency via conditional update	`acceptTask`: `findOneAndUpdate({_id, status:"Pending"}, ...)` + `{isAvailable:true}`	"Update tabhi karo jab state abhi bhi wahi ho." Isse race condition se bachte hain.
Transaction (Unit of Work)	`acceptTask`, `updateTaskStatus`, `cancelTask`, `markAsPickedUp`	Kai document updates ya to sab hon, ya ek bhi na ho
Cache-aside (TTL)	`withTtlCache()` in `home.services.js`	Pehle cache dekho, miss ho to DB se lao aur cache mein rakho

Pattern	Kahan	Ek line mein samjho
Publish / push	`io.to(socketId).emit("newDeliveryRequest")`	Server khud client ko message bhejta hai, client ko poochna nahi padta
Polymorphic reference	`Notification.recipient` + `refPath: 'recipientModel'`	Ek field teen alag collections ko point kar sakta hai
Soft delete	`Product.isDeleted` + `deletedAt`	Data delete nahi hota, sirf hide hota hai
Snapshot / denormalization	Order item mein `price`, `productName`, `storeName`	Purane order ka data baad mein nahi badalta
Scheduled job	`jobs/badgeCron.js`	
Frontend: Service layer	`cartService.js`, `orderService.js`	Components ko pata nahi ki API ka URL kya hai
Frontend: Thunk (async action)	`createAsyncThunk` (har slice mein)	API call ki pending / fulfilled / rejected states apne aap bante hain
Frontend: Interceptor	`FrontEnd/src/axiosSetup.js` (401 → logout), `delivery-dashboard/src/services/deliveryService.js` (token lagao)	Har request/response pe common logic
Frontend: Custom hook	`useCartPersistence`, `useRiderSocket`	Reusable side-effect logic
Frontend: URL as state	`ShopM.jsx` : filters URL query params mein	Link share karo to wahi filters khulte hain

Jo pattern use nahi hue: Repository pattern (models seedha controllers mein use hote hain), Dependency Injection, Context API (sirf libraries ke providers hain), RTK Query, aur `ProtectedRoute` jaisa route guard component.

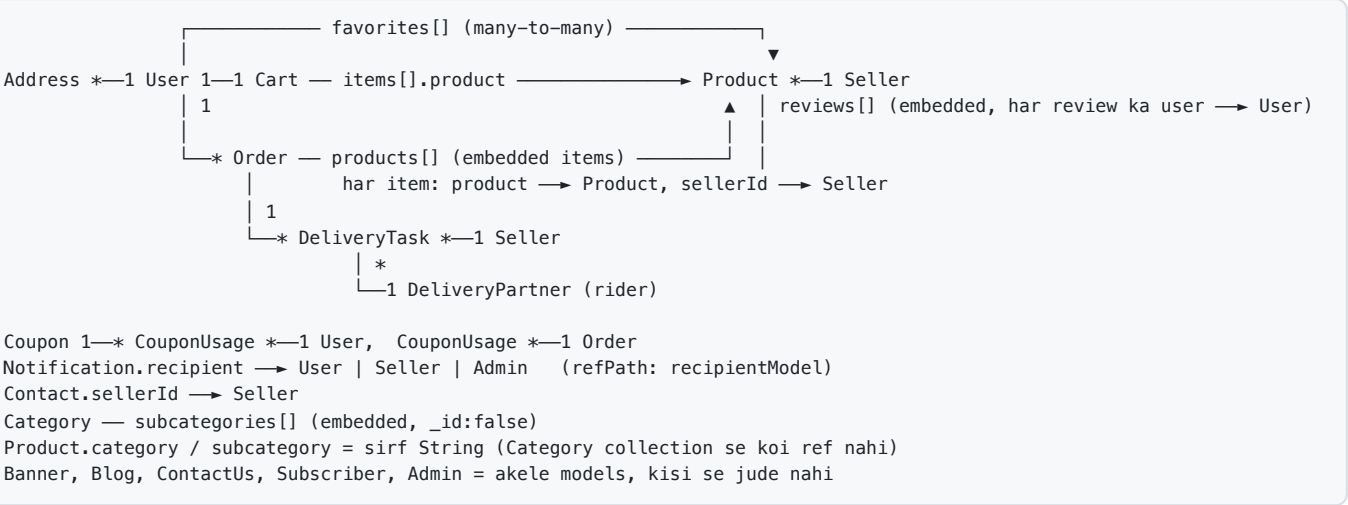
Step 4: Har part ka deep dive

> Is step mein har module ko andar se dekha gaya hai: database, APIs, auth, state, routing, errors, real-time, uploads, payments, email, caching, cron, security, performance, testing aur deployment. Code snippets project ki asli files se liye gaye hain. Kahin-kahin unhe chhota kiya gaya hai, aur `// ...` ka matlab hai ki beech ka code hataya gaya hai.

4.1 Database models: har field, relationships, indexes, validations

> Saare models `Backend/models/` mein hain, aur sab Mongoose schemas hain. Ek baat yaad rakho: Mongoose mein `unique: true` **validator nahi hai**. Ye sirf MongoDB mein unique index banata hai. Duplicate value aaye to Mongoose validation error nahi aata, balki MongoDB ka `E11000 duplicate key` error aata hai (code `11000`). `globalErrorHandler` isi code ko pakadta hai (Section 4.6).

4.1.1 Relationships ka diagram (ER diagram)



Embedded vs referenced (interview mein poochha jaata hai):

- **Embedded** (parent document ke andar hi array): `Order.products[]`, `Product.reviews[]`, `Cart.items[]`, `Category.subcategories[]`, `DeliveryTask.attempts[]`.
  - Fayda: ek hi query mein poora data aa jaata hai, aur ek document ka update MongoDB mein atomic hota hai.
  - Nuksaan: document bada hota jaata hai (MongoDB document ki limit 16 MB hai). Ek popular product ke hazaaron reviews ho jaayein to problem hogi.
- **Referenced** (sirf `ObjectId` store hota hai, `populate()` se data aata hai): `Order.user`, `Product.sellerId`, `DeliveryTask.order`, etc.
- **Denormalization (jaan-boojh kar data copy karna):** order item mein `productName`, `sellerName`, `storeName` aur `price` copy karke save hote hain.
  - Isse order ka record "us waqt" jaisa tha waisa hi rehta hai. Baad mein product ka price badle, tab bhi purane order ka price nahi badalta.
  - Ye ek sahi design decision hai, interview mein bata sakte ho.

4.1.2 `user.model.js` (model name `User`): customer

Field	Type	Validation / default	Matlab
`fullname.firstname`	String	`required`, `minlength: 3`	First name
`fullname.lastname`	String	`minlength: 3`	Schema mein optional hai, lekin `registerUser` aur `createUser` ise zaroori maante hain
`email`	String	`required`, `unique`, `minlength: 5`	Login ID. Schema mein email format check nahi hai, wo route ke `express-validator` mein hota hai. Lowercase bhi nahi kiya jaata, isliye `A@x.com` aur `a@x.com` do alag users ban sakte hain.
`password`	String	`required`, `minlength: 6`, `select: false`	bcrypt hash. `select:false` ka matlab: normal `find()` mein password nahi aata, use `.select('+password')` se maangna padta hai.
`phone`	String	`minlength: 10`	
`address.country/city/postalCode`	String	default `""`	Profile page wala simple address. Asli delivery addresses `Address` collection mein hain.
`avatar`	String	default `''`	S3/CloudFront URL
`forgot_password_otp`	String	default `null`, `select: false`	Password reset OTP. (Isi `select:false` ki wajah se OTP verify ka bug hai, Section 4.3 dekho.)
`forgot_password_expiry`	Date	default `null`	OTP kab expire hoga
`savedAddresses[]`	`{fullName, phone, street, city, state, pincode}`	—	**Legacy.** Naya code `Address` model use karta hai.
`favorites[]`	ObjectId → `Product`	—	Wishlist
`createdAt`, `updatedAt`	Date	`{timestamps:true}` se apne aap	

- **Methods:**
  - `generateAuthToken()` (`user.model.js:96`): `jwt.sign({ _id: this._id }, JWT_SECRET, { expiresIn: "30d" })`.
  - `comparePassword(password)`: `bcrypt.compare`.
- **Static:** `hashPassword(password)`: `bcrypt.hash(password, 10)`. Static isliye hai kyunki register ke waqt user document abhi bana hi nahi hota.
- **Indexes:** sirf `email` (unique).
- **Missing:** `lastLogin` field nahi hai. Admin ka "weekly active users" chart isi field pe query karta hai, isliye wo hamesha khaali aata hai (Section 4.14).

4.1.3 `seller.model.js` (`Seller`)

Group	Fields	Validation / default
Basic	`sellerName`, `storeName`, `bio` (trim)	`storeName` unique **nahi** hai schema mein; uniqueness sirf `updateSellerProfile` mein code se check hoti hai
	`email`	`required`, `unique`, `lowercase`, `trim`

Group	Fields	Validation / default
	`password`	`required`, `select:false` (hash `sellerPanelService.js` mein hota hai, model mein koi hook nahi)
	`phone`	`required`, `unique`, `trim`
	`role`	default `"seller"`
	`status`	enum `pending / approved / suspended / rejected`, default `pending`. **Isko "approved" karne wala koi endpoint nahi hai.**
	`isBlocked`	default `false`. `protectSeller` isse check karta hai (403).
Business	`businessType`	enum `individual / partnership / private_limited / other`
	`gstNumber`, `registrationNumber`	String, koi format check nahi
	`businessAddress{street, city, state, country, postalCode}`	Delivery task ka pickup address yahin se banta hai
Bank	`bankDetails{accountNumber, ifscCode, accountHolderName, upiId}`	trim. Koi encryption nahi, plain text.
KYC	`aadharNumber`, `aadharFile` (S3 URL), `panNumber`, `panFile`, `addressProof`, `governmentIdProof`	Aadhar/PAN number plain text mein save hote hain
	`verificationStatus`	enum `pending / verified / rejected`. Isko bhi badalne wala koi endpoint nahi.
Tracking	`totalProducts`, `totalOrders`, `ratings`	default 0. **Kahin update nahi hote** (stats aggregation se nikalte hain).
OTP	`isEmailVerified`, `emailOtp` (`select:false`), `otpExpiresAt`	Registration aur forgot-password dono yahi fields use karte hain
Links	`orders[]` → Order, `products[]` → Product	**Kabhi fill nahi hote** (dead fields)
	`onboardingCompleted`	KYC wizard poora hua ya nahi
	`dateJoined`	default `Date.now`. `timestamps` option nahi hai.

Indexes: `email` aur `phone` (dono unique).

4.1.4 `admin.model.js` (`Admin`)

- Fields: `email` (`required`, `unique`), `password` (`required`). `select:false` nahi hai.
- `pre('save')` **hook** (`admin.model.js:17`): password ko bcrypt (10 rounds) se hash karta hai.
  - **Bug:** `if (!this.isModified("password")) { next(); }` ke baad `return` nahi hai, isliye function aage chalta rehta hai aur **already-hashed password ko dobara hash** kar deta hai.
  - Abhi ye bug dikhta nahi, kyunki admin document ek baar banne ke baad kabhi dobara `save()` nahi hota. Lekin agar koi "update admin email" feature aaya, to admin ka login toot jaayega.
  - `deliveryPartner.model.js:89` mein yahi code sahi likha hai: `return next();`.
- Method: `matchPassword(entered)` → `bcrypt.compare`.

4.1.5 `product.model.js` (`Product`)

Field	Type / validation	Matlab
`name`, `description`	String, `required`	Description Markdown hoti hai (seller MDXEditor se likhta hai)
`sellingPrice`	Number, `required`	Seller ko jo milna chahiye
`platformFee`	default 0	Seller-Panel ka form isse 15% calculate karke bhejta hai (server khud calculate nahi karta)
`logisticsFee`	default **60**	
`transactionFee`	default **10**	
`discountPercentage`	default 0	
`mrp`	Number, `required`	`addProduct.routes.js:59–61` mein calculate hota hai (neeche formula)
`images`	`[String]`, custom validator `arrayLimit`: `val.length <= 5`	Error message: `"{PATH} exceeds the limit of 5"`
`productVideo`	String, default `""`	
`category`	String, `required`	Category collection ka naam (string ki tarah, ref nahi)
`subcategory`	String	
`tags`	`[String]`, **setter** `normalizeTagsArray`	Har tag lowercase + trim hota hai, duplicates hat jaate hain
`badges.{bestSeller, trending, featured, newArrival, mostPopular, topRated, onSale, limitedStock}`	Boolean, default false	7 badges cron job set karta hai, `featured` sirf admin
`rating`	Number, default **1**	Reviews ka average. Seller form se bhi set ho sakta hai (Section 4.14).
`inStock`	Number, `required`, default 0	Naam "inStock" hai par ye **quantity** hai, boolean nahi
`reviews[]`	`{user → User, stars (1–5, required), text, photos[], helpfulCount (default 1), helpfulVoters[] → User, createdAt}`	Embedded reviews
`sellerId`	ObjectId → `Seller`, `required`	
`approvalStatus`	enum `pending / approved / rejected`, default `pending`	Customer ko sirf `approved` dikhte hain
`adminNotes`	String	Reject karne ki wajah
`isDeleted`, `deletedAt`	soft delete	Product DB mein rehta hai, purane orders ke liye
`createdAt`	default `Date.now`	`timestamps` nahi, isliye `updatedAt` bhi nahi

- **Tags normalize karne ke do raaste:**
  1. `set: normalizeTagsArray`: jab `new Product()` ya `doc.tags = [...]` ho.

2. `pre(["findOneAndUpdate","updateOne","updateMany"])` hook (`product.model.js:165`): update queries mein setter nahi chalta, isliye ye hook `update.tags` aur `update.$set.tags` ko bhi normalize karta hai.
    - Interview point: "*Mongoose setters `save()` pe chalte hain, query updates pe nahi. Isliye update ke liye alag hook lagaya.*"

- **Indexes** (`product.model.js:175–182`):
    - `{approvalStatus:1, isDeleted:1, inStock:1, rating:-1}` : home page ka "approved + not deleted + in stock, rating ke hisaab se sort" query isi pe chalti hai.
    - `{sellerId:1, rating:-1, createdAt:-1}` : seller-wise product lists.
    - `{tags:1}` : **multikey index** (array field pe index), recommendation ke `tags: {$in: [...]}` ke liye.
- **MRP ka formula (ek inconsistency):**

```
// addProduct.routes.js:59–61
const calculatedPrice = sellingPriceNum + logisticsFee + transactionFee + platformFeeNum;
const discountAmount = (calculatedPrice * discountNum) / 100;
const mrp = calculatedPrice + discountAmount;      // MRP = base × (1 + d%)
```

  Customer ka price `toClientProduct` mein `base × (1 − d%)` hai. Matlab 20% discount pe MRP = 1.2×base aur price = 0.8×base, to customer ko jo discount dikhega wo ~33% hoga, 20% nahi. Sahi formula `mrp = base / (1 − d%)` hota.

4.1.6 `order.model.js` (`Order`)

Field	Type / validation	Matlab
`orderId`	String, `unique` , `index:true`	Human-readable ID: `"OD"` + 13 digit timestamp + 5 random digits. (`unique` already index banata hai, `index:true` faltu hai.)
`user`	ObjectId → User, `required`	
`products[].itemId`	String, `index:true`	Har item ki ID: base number + item index (BigInt se)
`products[].product`	→ Product, `required`	
`products[].productName`	String	Snapshot
`products[].sellerId`	→ Seller, `required`	
`products[].sellerName` , `storeName`	String, `required`	Snapshot (missing ho to "Unknown Seller" / "Marketplace")
`products[].quantity`	Number, `required` , `min: 1`	
`products[].price`	Number, `required`	Order ke waqt ka `offeredPrice`
`products[].status`	enum `Pending / Dispatched / Shipped / Out for Delivery / Delivered / Cancelled`	**Item-level status**: ek order mein alag sellers ke items alag stage pe ho sakte hain
`products[].trackingInfo{courier, trackingNumber, updatedAt}`		courier/trackingNumber kabhi fill nahi hote
`products[].cancelReason` , `cancelledBy`	`cancelledBy` enum `user / delivery_partner / null`	
`paymentMethode` (spelling aisi hi hai)	enum `Online / COD` , `required`	

Field	Type / validation	Matlab
`razorpayOrderId`, `razorpayPaymentId`	String	**Index nahi hai**, jabki `/verify` aur webhook isi pe search karte hain
`expireAt`	Date	**TTL timer**
`totalAmount`	Number, `required`	
`pricingBreakdown{subtotal, gstPercent (5), gstAmount, shippingCharge, deliveryCharge, discountCode, discountType (flat/percent), discountValue, discountAmount}`		Server ka calculate kiya hua breakdown
`paymentStatus`	enum `Pending / Paid`	`Failed` ya `Refunded` jaisa koi state nahi
`deliveryStatus`	same enum as item status	Order-level status, items se derive hota hai (`syncOrderStatusFromTask`)
`shippingAddress{fullName, phone, street, city, state, pincode}`		Snapshot. Client jo bhejta hai wahi save hota hai, server validate nahi karta.
`cancelledAt`	Date	
`rating{stars (1–5), text, photos[], submittedAt}`		Order-level rating
`createdAt`	default `Date.now`	

- **TTL index:** `orderSchema.index({ expireAt: 1 }, { expireAfterSeconds: 0 })` (`order.model.js:139`).
  - MongoDB ka background TTL monitor har ~60 second mein chalta hai, aur jin documents ka `expireAt` time nikal chuka ho unhe **delete** kar deta hai.
  - Online order banate waqt `expireAt = now + 30 min` set hota hai. Payment ho jaaye to `order.expireAt = undefined` se field hat jaata hai, aur order bach jaata hai.
  - COD orders mein `expireAt` hota hi nahi, isliye wo kabhi expire nahi hote.
- `pre('save')` **hook** (`order.model.js:164`): agar `orderId` nahi hai to generate karta hai.
  - 5 baar try karta hai, aur har baar `findOne({orderId})` se check karta hai ki ye ID pehle se to nahi hai.
  - Phir har item ko `itemId = BigInt(base) + index` deta hai.
  - `BigInt` isliye, kyunki 18 digit ka number JavaScript ke `Number` mein exactly store nahi ho sakta (`Number.MAX_SAFE_INTEGER` 16 digit ka hai).
- **Indexes:** `orderId` (unique), `products.itemId`, `expireAt` (TTL).
- **Missing indexes:** `user` (har "my orders" query), `products.sellerId` (seller ke orders), `razorpayOrderId`. Abhi data kam hai isliye pata nahi chalta; data badhne pe ye queries poori collection scan karengi.

4.1.7 `cart.model.js` (`Cart`)

- Fields: `user` (→ User, `required`), `items[{ product → Product (required), quantity (default 1, min 1) }]`.
- `user` **pe unique index nahi hai.** Agar ek hi user ki do "add to cart" requests ek saath aayein aur cart abhi exist na kare, to dono `new Cart()` bana denge, aur us user ke 2 cart documents ban jaayenge.
- Fix: `user: { unique: true }`, ya `findOneAndUpdate({user}, ..., {upsert:true})`.

4.1.8 `address.model.js` (`Address`)

- Fields (sab `required` + `trim`): `userId` → User, `name`, `phone`, `address`, `locality`, `city`, `state`, `pincode`.
  - `pincode` pe custom validator hai: `/^\d{6}$/`. Comment mein likha hai ki length check kyun nahi kiya: pincode `0` se shuru ho sakta hai, isliye ise String rakha gaya, Number nahi.
- `isDefault` (default false), `addressType` (enum `home / work / other`), aur `timestamps`.
- `pre('save')` **hook:** agar ye address default ban raha hai, to `updateMany({userId, _id: {$ne: this._id}}, {isDefault:false})` se baaki sab addresses non-default ho jaate hain.
  - Ye transaction ke andar nahi hai, isliye do saath-saath requests mein do default addresses ban sakte hain (rare case).
- **Index:** `{userId:1, isDefault:1}`.
- Dhyan do: `Order.shippingAddress` ke field names (`fullName`, `street`, `pincode`) `Address` ke field names (`name`, `address`, `locality`) se alag hain. Checkout pe frontend inhe map karta hai.

4.1.9 `coupon.model.js` (`Coupon`) aur `couponUsage.model.js` (`couponUsage`)

- **Coupon:**
  - `code` (`required`, `unique`, `uppercase`, `trim`)
  - `type` (enum `flat / percent`, `required`)
  - `value` (`required`)
  - `isActive` (default true), `expiresAt` (null matlab kabhi expire nahi)
  - `maxUsesTotal` (null matlab unlimited), `maxUsesPerUser` (default 1)
  - `usedCount` (default 0), `timestamps`
- **CouponUsage:** `couponId`, `userId`, `orderId` (teeno required ObjectIds), `timestamps`. Index: `{couponId:1, userId:1}`, jo "is user ne ye coupon kitni baar use kiya" wale count ke liye hai.
  - Model ka naam lowercase `'couponUsage'` hai, isliye collection ka naam `couponusages` banta hai.
- **Admin panel mein coupon banane ka koi UI ya API nahi hai.** Coupons seed script ya DB se seedha daalne padte hain.

4.1.10 `deliveryPartner.model.js` (`DeliveryPartner`): rider

Field	Validation	Matlab
`name`, `phone`	`required`	`phone` pe unique index nahi hai. Duplicate phone sirf `registerPartner` ka code check karta hai.
`email`	`required`, `unique`	
`password`	`required`, `select:false`	`pre('save')` mein hash hota hai
`isActive`	default **true**	Admin approval. `registerPartner` khud `false` set karta hai.
`isAvailable`	default false	Rider abhi free hai ya nahi. Isi field se "rider lock" banta hai.
`currentLocation{type:'Point', coordinates:[lng, lat], updatedAt}`	GeoJSON	**Order dhyan do: pehle longitude, phir latitude**

Field	Validation	Matlab
`documents.{aadhar, drivingLicense, vehiclePapers}.{number (select:false), verified, imageUrl}`		Upload ka koi UI ya API nahi hai
`bankDetails{accountNumber (select:false), ifscCode (select:false), accountHolderName, bankName}`		
`currentCity`	`index:true`	
`stats{totalDeliveries, cancelledDeliveries, rating (default 5)}`		Delivered/cancel hone pe `$inc` hota hai
`createdAt`		

- **Indexes:**
  - `{currentLocation: "2dsphere"}` : ye index ke bina `$geoNear` query chalegi hi nahi.
  - `{currentCity:1, isActive:1, isAvailable:1}` : ye index kisi query mein use nahi hota.
  - `email` (unique), `currentCity` .

4.1.11 `deliveryTask.model.js` (`DeliveryTask`)

Ek task = ek order ke andar ek seller ke items. Agar ek order mein 2 sellers ke products hain, to 2 tasks banenge, kyunki pickup do alag jagah se hoga.

Field	Matlab
`order` (→ Order, `required` , index)	`unique: true` comment out hai, kyunki ek order ke kai tasks ho sakte hain
`seller` (→ Seller, `required` , index)	
`items[{itemId (required), product, name, quantity, price, status}]`	Item status enum: `Pending / PickedUp / OutForDelivery / Delivered / Cancelled` . **Ye item-level status kabhi update nahi hota**, sirf task ka `status` update hota hai.
`deliveryPartner` (→ DeliveryPartner, default null)	Kis rider ko mila
`pickupContact{name, phone}` , `dropContact{name, phone, alternatePhone}`	
`pickupAddress` , `dropAddress` `{fullName, street, city, state, pincode}`	
`pickupLocation` , `dropLocation` `{type:'Point', coordinates (required)}`	Abhi hamesha placeholder `[77.209, 28.613]`
`liveLocation`	**Kabhi update nahi hota**
`status` (index)	enum `Pending / Assigned / Accepted / PickedUp / OutForDelivery / Delivered / Failed / Cancelled`

Field	Matlab
`attempts[{deliveryPartner, status (Assigned/Accepted/Cancelled/Failed), assignedBy (System/Seller/Partner), reason, at}]`	Audit log: kis rider ko kab offer hua, kisne accept ya reject kiya
`otp{code, isVerified, verifiedAt}`	**Kahin use nahi hota**
`deliveryFee` (default 0), `cancelledBy` (enum `user / delivery_partner / admin / system / seller`), `cancelReason`, `deliveryRating{score 1–5, comment, createdAt}`	`deliveryRating` bhi kabhi set nahi hota
`timestamps: true`	

Indexes: `pickupLocation` 2dsphere (rider "nearby tasks" query yahi use karti hai), `dropLocation` 2dsphere (use nahi hota), `{status:1, deliveryPartner:1}`, aur `order`, `seller`, `status` ke single-field indexes.

4.1.12 Baaki chhote models

Model (file)	Fields + validation	Notes
`Notification` (`notification.model.js`)	`recipient` (ObjectId, `required`, `refPath: 'recipientModel'`), `recipientModel` (enum `User / Seller / Admin`, `required`), `message` (`required`), `link`, `read` (default false), `timestamps`	**Polymorphic reference:** `refPath` ka matlab hai ki `populate('recipient')` karne pe Mongoose `recipientModel` field padhke tay karta hai ki kis collection mein dhoondhna hai. `recipient` pe index nahi hai.
`Banner` (`banner.model.js`)	`imageUrl` (`required`), `redirectUrl` (`required`), `isActive` (default false), `timestamps`	"Ek time pe ek hi banner active" wala rule schema mein nahi hai, controller ka code ise sambhalta hai
`Blog` (`blog.model.js`)	`title`, `author`, `content`, `image` (sab `required`), `tags[]`, `createdAt`, `updatedAt` (manual; `timestamps` option nahi, isliye `updatedAt` kabhi update nahi hota)	
`Category` (`category.model.js`)	`name` (`required`, `unique`), `subcategories[{name (required), keywords[]}]` with `{_id:false}`, `timestamps`	`_id:false` ka matlab: subdocuments ko apni alag `_id` nahi milti
`Contact` (`contact.model.js`)	`name`, `email`, `subject`, `message` (sab `required`), `status` (enum `Pending / In Progress / Resolved`), `response`, `isFromSeller`, `sellerId` → Seller, `createdAt`	Seller se admin ko support ticket
`ContactUs` (`contactUs.model.js`)	`name`, `email` (lowercase), `message` (sab `required` + trim), `timestamps`	Customer website ka contact form
`Subscriber` (`NlLetterSubs.model.js`)	`email` (`required`, `unique`), `subscribedAt`	Newsletter

4.1.13 Validation kahan-kahan hoti hai (3 layers)

1. **Route layer:** `express-validator` (purane customer routes) ya Zod (`zodValidate` , naye routes). Detail Section 4.6 mein.
2. **Mongoose schema layer:** `required` , `enum` , `min` / `max` , `minlength` , custom `validate` (pincode, images ki limit). Ye `save()` aur `create()` pe chalti hai.
   - `findByIdAndUpdate` jaise query updates pe validators **default mein nahi chalte** (`runValidators: true` dena padta hai).
   - Project mein kai jagah updates bina `runValidators` ke hote hain. `updateSellerProfile` mein to `runValidators: false` likh kar band kiya gaya hai.
3. **Database layer:** unique indexes (duplicate email, phone, coupon code).

4.2 Saare API endpoints

Kaise padhein:

- **Auth column:** `PUBLIC` = koi token nahi chahiye. `User` = `protect` (customer). `Seller` = `protectSeller` . `Admin` = `protectAdmin` . `Rider` = `protectDelivery` . `Optional` = `optionalAuth` .
- **Request:** body / query / params mein kya bhejna hai.
- **Response:** success pe status code + JSON ka shape. Har API ka shape alag hai (koi common format nahi): kahin `{success, data}` , kahin seedha array, kahin `{message}` , kahin `{error}` .
- Total **~140 endpoints**, 25 router files mein, plus `app.js` ke 5 inline routes. Inme se **lagbhag 56 pe koi auth nahi hai** (kuch jaan-boojh ke public hain, jaise product listing; kuch galti se, jaise saare orders).
- Base URL local pe `http://localhost:5000` . Swagger docs `/api-docs` pe hain (`docs/api.swagger.js`), lekin coupons, banners aur `/api/admin/banners` ke docs missing hain.

4.2.1 `app.js` ke inline routes

Method	Path	Auth	Response
GET	`/` aur `/check`	PUBLIC	`"Hello World!"` (health check jaisa)
GET	`/api/document`	PUBLIC	`"Hello document!"`
GET	`/api/server-status`	PUBLIC	`{ bootTime: process.uptime() }` . Naam "bootTime" hai par value uptime (seconds) hai.
GET	`/api-docs`	PUBLIC	Swagger UI

4.2.2 `/users` : customer auth, profile, cart, favorites, blogs

Method	Path	Auth	Request	Response	Notes
POST	`/users/register`	PUBLIC	body `{fullname:{firstname, lastname}, email, password}`	201 `{token, user}`	express-validator. Validation fail → **404** `{errors}` . Response mein password hash leak.
POST	`/users/login`	PUBLIC	`{email, password}`	200 `{token, user}`	404 user nahi, 400 galat password. Hash leak.
GET	`/users/profile`	User	—	200 `{success, user}`	
PUT	`/users/update-profile`	User	`{firstname, lastname, email, phone, country, city, postalCode}` (sab optional)	200 `{success, message, user}`	Email bina verification ke badal sakta hai

Method	Path	Auth	Request	Response	Notes
PUT	`/users/avatar`	User	multipart `avatar` (50 MB, image types)	200 `{success, message, avatar}`	S3 `avatars/`
POST	`/users/forgot-password`	PUBLIC	`{email}`	200 `{message, user, success}`	6-digit OTP email. `email` ka type check nahi, isliye `{"$ne": null}` jaisa operator bhi chal jaata hai.
POST	`/users/verify-otp`	PUBLIC	`{email, otp}`	(sahi chale to) `{token, user}`	**Hamesha 400** (select:false bug)
POST	`/users/reset-password`	PUBLIC	`{email, newPassword, confirmPassword}`	200 `{message, success}`	**Bina OTP ke reset (critical).** `email: {"$ne": null}` bhejo to DB ka pehla user reset ho jaata hai, email jaanna bhi zaroori nahi.
GET	`/users/cart`	User	—	200 `{cart: [ {_id, product (toClientProduct), quantity} ]}`	Cart nahi hai to `{cart: []}`
POST	`/users/cart`	User	`{productId, quantity?}`	200 `{message, cart}`	Approval/stock check nahi. `quantity` validate nahi hoti: string `"2"` aaye to `1 + "2" = "12"` ban jaata hai.
PUT	`/users/cart/:itemId`	User	param = cart line ka `_id` ; body `{quantity}` (number ≥ 1)	200 `{message, cart}`	Ownership: `{user, "items._id"}` filter
DELETE	`/users/cart/:itemId`	User	—	200 `{message, cart}`	
DELETE	`/users/cart`	User	—	200 `{message, cart: []}`	
GET	`/users/favorites`	User	—	200 `{favorites: [products]}`	
POST	`/users/favorites`	User	`{productId}`	200 `{message, favorites: [ids]}`	Product exist karta hai ya nahi, check nahi
DELETE	`/users/favorites/:productId`	User	—	200 `{message, favorites}`	
POST	`/users/blog`	**PUBLIC**	multipart `title, content, author, image`	201 `{blog}`	Koi bhi blog daal sakta hai
GET	`/users/blogs?tag=`	PUBLIC	—	200 `{blogs}` (newest first)	
GET	`/users/blogs/:id`	PUBLIC	—	200 `{blog}`	
POST	`/users/save-address` , `/users/get-address`	**PUBLIC**	`{email, address}`	—	**Dead/broken:** code `savedAddress` likhta hai, schema field `savedAddresses` hai, isliye kuch save nahi hota

4.2.3 `/products` : public catalog + reviews (`routes/product.routes.js`)

Method	Path	Auth	Request	Response	Notes
GET	`/products`	PUBLIC	query: `category` , `subcategory` (comma list, slug → regex), `tag` / `tags` , `store` , `minPrice` , `maxPrice` (offeredPrice pe), `inStock=true` , `minDiscount` , `minRating` , `sort` (`price_asc` , `price_desc` , `rating_desc` , `createdAt_desc`), `page` (default 1), `limit` (default 15, max 50)	200 `{products, totalCount}`	Sirf approved + not deleted. Aggregation pipeline (Section 4.15).

Method	Path	Auth	Request	Response	Notes
GET	`/products/store/:sellerId`	PUBLIC	valid ObjectId	200 `[products]`	404 agar koi product nahi
GET	`/products/shops`	PUBLIC	—	200 `[{storeName, sellerId, sellerName, products (count), rating, img}]`	Rating na ho to nakli **4.5** dikhata hai
GET	`/products/navigation/trending-tags?limit=`	PUBLIC	limit 1–24 (default 8)	200 `{tags: [string]}`	`/:id` se pehle register hai, warna "navigation" ko id samjha jaata
GET	`/products/:id`	PUBLIC	—	200 product (toClientProduct)	Soft-deleted approved product bhi aa jaata hai
POST	`/products/:id/reviews`	User	multipart `stars, text, photos[] (≤6, 10 MB)`	200 **poora product document**	Sirf kharidne wala review kar sakta hai, lekin "kharida" ka matlab koi bhi order hai (pending/cancelled bhi). Response mein fees leak.
GET	`/products/:id/reviews`	PUBLIC	—	200 `[reviews]`	Reviewer ka **email** public
POST	`/products/:productId/reviews/:reviewId/helpful`	User	—	200 `{reviewId, helpfulCount}`	Ek user ek hi baar vote kar sakta hai (`helpfulVoters`). Default `helpfulCount: 1` hai, isliye pehla vote 2 dikhata hai.

4.2.4 `/api/orders` (`routes/order.routes.js`)

Method	Path	Auth	Request	Response	Notes
POST	`/api/orders`	User	`{products?: [{product, quantity}], discountCode?, shippingAddress, paymentMethode?, paymentStatus?, deliveryStatus?}`	201 order	COD order. `paymentStatus` **client se aata hai (critical).** Delivery tasks, emails aur notifications yahin se trigger hote hain.
GET	`/api/orders/my-orders`	User	—	200 `[orders]` (COD ya Paid)	GET request mein bhi `save()` hota hai (`ensureOrderIds`)
GET	`/api/orders/seller/orders`	User (galat middleware)	—	**Hamesha 500** (verified)	`mongoose.Types.ObjectId()` bina `new` ke
GET	`/api/orders/user/:userId`	**PUBLIC**	—	200 `[orders]`	Comment kehta hai "admin", par auth nahi hai
GET	`/api/orders/:orderId`	**PUBLIC**	Mongo `_id` ya `OD…` ID	200 order	Naam, phone, address sab ko dikh jaata hai
PATCH	`/api/orders/:orderId/cancel`	User (owner)	`{cancelReason}`	200 order	Pura order cancel hota hai. Shipped / Out for Delivery / Delivered pe 400. Refund nahi, delivery task cancel nahi.
GET	`/api/orders/eligibility/review/:productId`	User	—	200 `{eligible}`	`$elemMatch` nahi hai, isliye order ka koi bhi delivered item ho to eligible maan leta hai. Frontend ise use nahi karta.
POST	`/api/orders/:orderId/rating`	User (owner)	multipart `stars, text, photos[]` (disk storage)	200 order	Saare items Delivered hone chahiye. Product review sirf **pehle** product pe banta hai.

Method	Path	Auth	Request	Response	Notes
GET	`/api/orders`	**PUBLIC**	—	200 `[saare orders]`	Admin panel isi ko use karta hai. **(verified: bina token ke customers ka data aata hai)**
PUT	`/api/orders/seller/item-status`	Seller	`{orderId (OD…), itemId, status}` (Dispatched / Shipped / Out for Delivery / Delivered / Cancelled)	200 `{message, order}`	Ownership check hai, par transition rule nahi (seller seedha "Delivered" kar sakta hai). Order-level `deliveryStatus` recompute nahi hota.

4.2.5 `/api/payment` (`routes/payment.routes.js`): detail Section 4.9 mein

Method	Path	Auth	Request	Response
POST	`/api/payment/create-order`	User	`{orderData: {products, shippingAddress, discountCode}}`	200 `{orderId (Razorpay), key_id, amount (paise), currency, discountCode, discountAmount}`
DELETE	`/api/payment/cancel-pending/:razorpayOrderId`	User	—	200 `{success, deleted}`
POST	`/api/payment/verify`	User	`{razorpay_order_id, razorpay_payment_id, razorpay_signature, orderData}`	201 (naya Paid) / 200 (pehle se Paid) order
POST	`/api/payment/webhook`	PUBLIC, par HMAC signature se protected	raw Razorpay event	200 `{status:'ok'}`

4.2.6 `/home` (home page data, cache ke saath)

Method	Path	Auth	Request	Response	Notes
GET	`/home/tags?limit=`	PUBLIC	limit ≤ 50	200 `{success, tags: [{tag, avgRating, image, productCount}]}`	3 min cache
GET	`/home/categories?limit=`	PUBLIC	≤ 30	200 `{success, categories: [{category, avgRating, productCount, image, featuredProduct}]}`	`featuredProduct.sellingPrice` (seller ka payout) leak
GET	`/home/products?limit=`	Optional	≤ 50	200 `{success, mode: 'recommended' | 'popular', products}`	**Fees leak (verified)**
GET	`/home/sellers?limit=`	PUBLIC	≤ 30	200 `{success, sellers}`	Bank details aur IDs hata diye jaate hain, par seller ka email, phone, address, GST public

4.2.7 Chhote customer routes

Method	Path	Auth	Request	Response	Notes
GET	`/api/banners/active`	PUBLIC	—	200 `{success, banner | null}`	
GET	`/api/document/invoice/:orderId`	**PUBLIC**	Mongo `_id`	PDF download	"Total: ₹undefined" bug (verified)
GET	`/api/document/label/:orderId`	**PUBLIC**	Mongo `_id`	PDF (A6 label)	
GET	`/api/addresses`	User	—	200 `{success, data: [addresses]}` (default pehle)	Saare address routes pe `router.use(protect)`

Method	Path	Auth	Request	Response	Notes
GET	`/api/addresses/:addressId`	User	—	200 `{success, data}`	
POST	`/api/addresses`	User	`{name (2–50), phone (10 digits), address (5–200), locality, city, state, pincode (6 digits), addressType?}`	201 `{success, message, data}`	Pehla address apne aap default banta hai
PUT	`/api/addresses/:addressId`	User	same	200	
DELETE	`/api/addresses/:addressId`	User	—	200	Default address delete hua to doosra default ban jaata hai
PATCH	`/api/addresses/:addressId/default`	User	—	200	
POST	`/api/coupons/validate`	User	`{code, cartTotal}`	200 `{success, data: {discountCode, discountType, discountValue, discountAmount, finalTotal}}`	Sirf preview ke liye. `code` missing ho to 500.
POST	`/api/contact-us`	PUBLIC	`{name, email, message}`	201 `{success, message, data}`	Kisi bhi email pe acknowledgement mail chala jaata hai (spam risk, rate limit nahi)
POST	`/api/subscribe`	PUBLIC	`{email}`	201 `{msg}` / 400 duplicate	
GET	`/sitemap.xml`	PUBLIC	—	static file	Dynamic route shadowed hai

4.2.8 Contact / support (seller ↔ admin): `routes/contactRoutes.js`, sab PUBLIC

Method	Path	Request	Response	Notes
POST	`/api/contact`	`{name, email, subject, message, isFromSeller?, sellerId?}` (poora `req.body` model mein jaata hai)	201 `{message}`	Admins ko notification. Agar koi admin na ho to **default admin** `admin@test.com` / `admin123` **ban jaata hai**.
GET	`/api/contact?email=`	—	200 `[messages]`	Email na do to **saare** messages
PUT	`/api/contact/:id`	`{response, status}`	200 message	Koi bhi "admin" ban ke reply kar sakta hai
DELETE	`/api/contact/:id`	—	200	

4.2.9 Seller panel APIs

Method	Path	Auth	Request	Response	Notes
POST	`/api/seller/register`	PUBLIC	`{email, password, phone (10 digits)}`	201 `{success, sellerId, message}`	OTP email (5 min). OTP console mein bhi print hota hai.
POST	`/api/seller/verify-otp`	PUBLIC	`{sellerId, emailOtp}`	200 `{success, message, isEmailVerified, token}`	Attempt limit nahi
POST	`/api/seller/resent-otp`	PUBLIC	`{email}`	200	Purana OTP expire hone ke baad hi

Method	Path	Auth	Request	Response	Notes
POST	`/api/seller/onboarding`	Seller	multipart: `sellerName, storeName` (required), `bio`, `businessType`, `gstNumber`, `registrationNumber`, `street, city, state, country, postalCode`, `accountNumber, accountHolderName, ifscCode, upiId`, `aadharNumber, panNumber, addressProof`, files `aadharFile, panFile`	200 `{success, message, sellerId}`	Email verified hona chahiye, aur ek hi baar ho sakta hai. Saare admins ko notification. Client `role` bhi bhej sakta hai.
POST	`/api/seller/login`	PUBLIC	`{email, password}`	200 `{success, message, seller, token, onboardingCompleted}`	Har error pe 400
GET	`/api/seller/profile`	Seller	—	200 `{success, seller}`	
PUT	`/api/seller/profile`	Seller	koi bhi fields (JSON ya multipart)	200 `{success, seller, message}`	**Mass assignment (critical)**
GET	`/api/seller/products`	Seller	—	200 `[products]` (har status, deleted bhi)	"My Products" page
POST	`/api/seller/forgot-password` → `/verify-password-otp` → `/reset-password`	PUBLIC	`{email}` → `{email, otp}` → `{email, newPassword}`	200	Section 4.3.6
POST	`/api/products`	Seller	multipart: `name, description, sellingPrice, discountPercentage, platformFee, rating, inStock, category, subcategory, tags (JSON)`, files `images (≤5)`, `productVideo (≤20 MB)`	201 `{message, product}`	Product `pending` banta hai. Category/subcategory DB mein na ho to apne aap ban jaati hai.
GET	`/api/productinfo/my`	Seller	query (Zod): `category, subcategory, minprice, maxprice, instock, minrating, tag, view (deleted/all)`	200 `{success, count, products}`	Sirf approved products
GET	`/api/productinfo/:id`	Seller (owner)	—	200 product	
PATCH	`/api/productinfo/:id/details`	Seller (owner)	multipart (Zod): `name, description, category, subcategory, tags, rating, inStock, productVideo` + files	200 `{success, message, product}`	Re-approval nahi. `existingImages` Zod strip kar deta hai, isliye nayi images aane pe purani sab replace ho jaati hain.
PATCH	`/api/productinfo/:id/pricing`	Seller (owner)	Zod: `{sellingPrice (>0), discountPercentage (0–100), platformFee (≥0)}`	200 `{success, message, pricing}`	MRP dobara calculate hota hai
DELETE	`/api/productinfo/:id`	Seller (owner)	—	200	Soft delete. Product kisi "active" order mein ho to 400, lekin "Delivered" bhi active list mein hai, isliye ek baar bika product kabhi delete nahi hoga.
GET	`/api/dashboard/stats/:sellerId`	**PUBLIC**	—	200 `{sellerId, sellerName, storeName, totalRevenue, totalOrders, totalProducts, totalCustomers}`	`Order.find({})` (saare orders memory mein)

Method	Path	Auth	Request	Response	Notes
GET	`/api/dashboard/orders/:sellerId`	**PUBLIC**	—	200 `[orders with customer info]`	Customer PII leak. Seller panel ka OrderTable isi ko use karta hai.
GET	`/api/stats/overview`, `/sales-graph?filter=`, `/order-status`, `/category-stats`, `/inventory`, `/inventory-management`	Seller	`filter` = `today` / `week` / `month` (warna all-time)	200 `{success, data}`	Aggregations (`servicesSellerPanel/statsService.js`). `/inventory` ka `totalStockValue` hamesha null (`p.price` field exist nahi karta).
GET	`/api/categories`	PUBLIC	—	200 `[Category]`	
GET	`/api/categories/:categoryName/subcategories`	PUBLIC	—	200 `[subcategories]`	
GET	`/api/seller/delivery/active-deliveries`	Seller	—	200 `{success, count, data}`	"Accepted" status list mein nahi, isliye rider-accepted tasks seller ko nahi dikhte
PUT	`/api/seller/delivery/markAsPickedUp`	Seller	`{taskId}`	200	Sirf "Assigned" (default-rider) tasks pe. Transaction use karta hai.

4.2.10 Admin panel APIs

Method	Path	Auth	Request	Response	Notes
POST	`/api/admin/login`	PUBLIC	`{email, password}`	200 `{_id, email, token}` / 401	Rate limit nahi
GET	`/api/admin/verify`	Admin	—	200 `{message}`	Boot pe token check
GET	`/api/admin/products?status=pending|approved|rejected`	Admin	—	200 `[products + seller info]`	
PUT	`/api/admin/products/:productId/approve`	Admin	—	200 product	Seller ko notification
PUT	`/api/admin/products/:productId/reject`	Admin	`{adminNotes}`	200 product	Seller ko notification
PUT	`/api/admin/products/:productId/badges`	Admin	Zod strict: `{badges: {featured?: bool, ...8 keys}}`	200 product	
DELETE	`/api/admin/products/:productId`	Admin	—	200 `{message}`	**Hard delete**, aur purane orders ka reference toot jaata hai
GET	`/api/admin/sellers`	Admin	—	200 `[sellers + productCount]`	N+1 queries
GET	`/api/admin/sellers/:sellerId`	Admin	—	200 seller	
GET	`/api/admin/sellers/:sellerId/products`	Admin	—	200 `[products]`	
PUT	`/api/admin/sellers/:sellerId/block`	Admin	`{isBlocked: bool}`	200 seller	
DELETE	`/api/admin/sellers/:sellerId`	Admin	—	200	Hard delete; products orphan ho jaate hain
GET	`/api/admin/analytics/sales`	Admin	—	200 `[{name: 'Jan', sales}] × 12`	Saare saalon ke same mahine mila diye jaate hain; unpaid orders bhi gine jaate hain

Method	Path	Auth	Request	Response	Notes
GET	`/api/admin/analytics/users`	Admin	—	200 `[{name, users}]` × 12	
GET	`/api/admin/analytics/weekly-active-users`	Admin	—	**hamesha** `[]`	`lastLogin` field exist nahi karta
GET	`/api/admin/stats/category-stats`	Admin	—	200 `{success, data: [{category, count}]}`	
GET	`/api/admin/riders/pending`	**PUBLIC**	—	200 `{success, count, data}`	"bypassing protectAdmin"
GET	`/api/admin/riders/active`	**PUBLIC**	—	200 `{success, count, data}`	Riders ka phone, email aur location public
PATCH	`/api/admin/riders/:id/status`	**PUBLIC**	Zod: `{isActive: boolean}`	200 `{success, message}`	Koi bhi rider khud ko approve kar sakta hai
POST / GET / PATCH `/:id/toggle` / PUT / DELETE	`/api/admin/banners[/:id]`	Admin (`router.use`)	multipart `image`, `redirectUrl`, `isActive`	`{success, banner(s)}`	Ek active banner baaki sab ko inactive kar deta hai. Purani image S3 se delete nahi hoti.
GET / PUT `/:id/read` / PUT `/read-all` / DELETE `/:id` / DELETE	`/api/notifications`	User (`protect`)	—	`[notifications]`	Seller aur admin ke liye kaam nahi karta (Section 4.13.2)

4.2.11 Delivery (rider) APIs

Method	Path	Auth	Request (Zod)	Response	Notes
POST	`/api/delivery/register`	PUBLIC	`{name (≥3), email, password (≥6), phone (≥10), currentCity (≥2)}`	201 `{success, message, riderId}`	`isActive: false` (admin approval chahiye)
POST	`/api/delivery/login`	PUBLIC	`{email, password}`	200 `{success, token, partner: {id, name, email, phone, isAvailable}}`	Deactivated → 403. Login pe `isAvailable = true`.
GET	`/api/delivery/verify`	Rider	—	200 `{message}`	
GET	`/api/delivery/me`	Rider	—	200 `{success, data: rider}`	Sensitive fields hata ke
PUT	`/api/delivery/update-location`	Rider	`{lng: number, lat: number}`	200	GeoJSON save. `isAvailable` dobara calculate hota hai.
PUT	`/api/delivery/availability`	Rider	`{isAvailable}`	200	Validation nahi. Active task hote hue bhi rider "available" ban sakta hai.
GET	`/api/delivery/nearby-Deliveries`	Rider	—	200 `{success, count, data}` / 400 "Update Location First"	`$geoNear` 5 km, Pending tasks, max 20
POST	`/api/delivery/accept-task`	Rider	`{taskId}`	200 `{success, message, data: task}` / 400	**Transaction + atomic claim** (Section 4.7)
POST	`/api/delivery/reject-task`	Rider	`{taskId, reason?}`	200	Sirf `attempts` mein entry. Kisi aur rider ko dobara broadcast nahi hota.
PUT	`/api/delivery/task-status`	Rider (owner)	`{taskId, status: PickedUp | OutForDelivery | Delivered | Failed | Cancelled}`	200	State machine + order sync (transaction)

Method	Path	Auth	Request (Zod)	Response	Notes
PUT	`/api/delivery/cancel-task`	Rider (owner)	`{taskId, reason (≥3)}`	200	Sirf Assigned/PickedUp; "Accepted" task cancel nahi hota
GET	`/api/delivery/active-deliveries`	Rider	—	200 `{success, data: [tasks + order]}`	
GET	`/api/delivery/orders`	Rider	—	200 `[dashboard-shape tasks]`	History tab
GET	`/api/delivery/tasks/:taskId`	Rider (owner)	—	200 dashboard-shape task	Detail page
POST	`/api/deliverytask/create`	Seller	Zod `{orderId, pickupLocation: {coordinates: [lng, lat]}, dropLocation: {...}}`	201	Manual task (frontend use nahi karta)
POST	`/api/deliverytask/sendRequest/:deliveryTaskId`	Seller	—	200	Riders ko dobara broadcast. Seller panel ka "Send Delivery Request" button.
POST	`/api/deliverytask/task/:id/accept`, `/reject`	Rider	—	—	Purana parallel flow. `AppError` bina `new` ke, aur undefined function call. Frontend use nahi karta.

4.2.12 Mount order aur collisions

- `/users` pe 5 routers mount hain: cart, user, shipAddress, blogs, favorites. Inke paths overlap nahi karte. Cart router **pehle** hai (comment: "Cart routes first so POST/GET /users/cart hit Cart model only").
- `/api` pe 4 routers hain: contact, contact-us, subscribe, coupons. Inme bhi collision nahi.
- Har router ke andar literal routes (`/my-orders`, `/shops`, `/my`) `/:id` jaise param routes se **pehle** register hain, isliye shadowing nahi hoti. Ye ek achhi practice hai.
- `express.static('public')` `/sitemap.xml` route ko shadow kar deta hai.
- **Mount nahi hue (dead):** `routes-seller-panel/seller.routes.js` (isme syntax bug bhi hai), `routes-admin-panel/Newsletter.routes.js`, `Middleware/sellerAuth.js`.

4.3 Authentication aur authorization

Pehle do shabd clear kar lo:

- **Authentication** ka matlab hai "tum kaun ho?" (login karke JWT milna).
- **Authorization** ka matlab hai "tumhe ye kaam karne ki permission hai ya nahi?" (jaise ye order tumhara hai ya nahi).

4.3.1 Project mein kya use hua: JWT, Bearer header mein

- **Sessions ya cookies nahi hain. OAuth / Google login bhi nahi hai.** Sirf JWT (JSON Web Token) use hota hai.
- **JWT kya hai:** teen hisse hote hain, `header.payload.signature`.
  - Payload mein data hota hai (jaise `{ _id, iat, exp }`). Ye sirf base64 encoded hai, encrypted nahi, isliye koi bhi ise padh sakta hai.
  - Signature `JWT_SECRET` se banta hai. Server signature check karke pakka karta hai ki token server ne hi diya tha aur kisi ne badla nahi.
- **Flow:** login → server JWT banata hai → client localStorage mein rakhta hai → har request mein `Authorization: Bearer <token>` header bhejta hai → middleware verify karta hai.
- **Refresh token nahi hai.** Ek hi token 30 din chalta hai.

4.3.2 Chaar roles: kaun kaise login karta hai

	Customer	Seller	Admin	Rider
Login API	`POST /users/login`	`POST /api/seller/login`	`POST /api/admin/login`	`POST /api/delivery/login`
Token banane wala code	`user.model.js:96` `generateAuthToken()`	`sellerPanelService.js:14` `generateToken(id)`	`admin.controller.js` `generateToken(id)`	`delivery.controller.js:10` `generateToken(partner)`
Payload	`{ _id }`	`{ id }`	`{ id }`	`{ id, role: "delivery" }`
Expiry	30d	30d	30d	30d
Secret	ek hi `JWT_SECRET` (chaaron ka)			
Middleware	`protect` (`Middleware/authMiddleware.js`)	`protectSeller` (`MiddlewareSellerPanel/sellerPanelauthMidWar.js`)	`protectAdmin` (`MiddlewareAdminPanel/Adminauth.middleware.js`)	`protectDelivery` (`MiddlewareDeliveryPanel/deliveryAuth.middleware.js`)
Extra checks	—	`isBlocked` → 403	—	`role === "delivery"` , `isActive` (admin approved) → warna 403
`req.user`	User document (password ke bina)	Seller document	`{...admin, role:'admin'}`	DeliveryPartner document
Client storage	`localStorage["user"] = {user, token}`	`localStorage.sellerToken` (+ `sellerId`)	`localStorage.adminToken`	`localStorage.deliveryToken` + `riderId`
Token kaise lagta hai	Har service mein haath se `authHeader(getState)`	Har call mein haath se header	Har call mein haath se header	`axios.create` + **request interceptor** (`deliveryService.js`)
Boot pe check	Nahi (localStorage se seedha state; pehli 401 pe logout)	`GET /api/seller/profile` (`AuthInitializer`)	`GET /api/admin/verify` (loading screen ke saath)	`GET /api/delivery/verify`

Roles alag kaise rehte hain? Har middleware **apne alag collection** mein token ki ID dhoondhta hai. Example:

- Seller ka token customer route pe bhejo, to `protect` `decoded._id` padhega. Seller token mein `_id` hai hi nahi (`id` hai), isliye `User.findById(undefined)` null deta hai aur 401 aata hai.
- Admin ka token seller route pe bhejo, to `Seller.findById(adminId)` null deta hai, aur 401.
- Sirf rider token mein `role` claim hai, aur sirf `protectDelivery` use check karta hai.
- Payload mein role na hone ke bawajood ye kaam karta hai, kyunki alag collections mein ObjectIds practically kabhi same nahi hote. Lekin ye design kamzor hai. Behtar ye hota ki har token mein `role` ho aur har middleware use check kare.

4.3.3 `protect` middleware (customer), line by line

```
const protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
    console.log("req", req.headers.authorization);             // ⚠ poora token log hota hai
    try {
      token = req.headers.authorization.split(' ')[1];          // "Bearer abc.def.ghi" → "abc.def.ghi"
      const decoded = jwt.verify(token, process.env.JWT_SECRET);// signature + expiry check; fail → throw
      req.user = await User.findById(decoded._id).select('-password'); // DB se fresh user
      if (!req.user) return res.status(401).json({ message: "User not found" }); // user delete ho gaya
      next();                                                    // aage controller tak jao
    } catch (error) {
      res.status(401).json({ message: "Not authorized, token failed" }); // galat/expired token
    }
  } else {
    return res.status(401).json({ message: "Not authorized, no token" });
  }
```

```
};
```

- `jwt.verify` do cheezein check karta hai: (1) signature sahi hai, aur (2) `exp` time nikla nahi.
- Har request pe DB se user nikalta hai. Isse ek DB call lagti hai, lekin fayda ye hai ki deleted user ka token turant bekaar ho jaata hai. Blocked seller bhi isi tarah turant ruk jaata hai (`protectSeller`).

4.3.4 `protectDelivery` (sabse saaf likha hua middleware)

```
if (!token) return next(new AppError("Not authorized, token missing", 401));
const decoded = jwt.verify(token, process.env.JWT_SECRET);      // fail → catch → next(err) → globalErrorHandler (prod mein 401)
if (decoded.role !== "delivery") return next(new AppError("Invalid token role", 403));
const deliveryPartner = await DeliveryPartner.findById(decoded.id).select("-password");
if (!deliveryPartner) return next(new AppError("Delivery partner not found", 401));
if (!deliveryPartner.isActive) return next(new AppError("Delivery partner account is deactivated", 403));
req.user = deliveryPartner; next();
```

401 vs 403:

- **401 Unauthorized:** "pehchaan nahi paaye", yaani token nahi hai ya galat hai.
- **403 Forbidden:** "pehchaan liya, par permission nahi hai", jaise blocked account ya galat role.

4.3.5 Customer login aur register (backend)

`loginUser` (`user.controllers.js:41`):

1. `validationResult` check karta hai (fail hone pe 404, jo galat code hai).
2. `userModel.findOne({ email }).select('+password')` : password `select:false` hai, isliye `+password` se explicitly maangna padta hai.
3. User nahi mila → 404 "User not found". Password galat → 400 "Invalid credentials".
   - Security note: alag-alag messages se attacker pata kar sakta hai ki koi email registered hai ya nahi (**user enumeration**). Dono cases mein ek hi message dena behtar hai.
4. `user.comparePassword(password)` : `bcrypt.compare` andar se hash mein se salt nikaal ke compare karta hai.
5. `generateAuthToken()` → `res.json({ token, user })` . Is `user` mein password hash bhi chala jaata hai (Section 4.14).

`registerUser` : validation → `userModel.hashPassword(password)` → `userService.createUser` → turant token. Registration pe email verification nahi hoti.

4.3.6 Password reset aur OTP flows

Customer (toota hua flow):

```
POST /users/forgot-password {email} → 6-digit OTP (Math.random), 15 min expiry, Resend se email
POST /users/verify-otp {email, otp} → user.forgot_password_otp !== otp ?   ← field select:false hai, isliye
                                       hamesha undefined → "Invalid or expired OTP"
                                       (sahi chale to 30-din ka poora login token deta hai)
POST /users/reset-password {email, newPassword, confirmPassword} → bina OTP/token ke password badal deta hai
```

- Frontend `token` bhi bhejta hai, par server use ignore karta hai.
- Frontend ka `/change-password` page render hote hi crash hota hai (`password` aur `showPassword` variables declare nahi hain).
- Fix: `findOne({email}).select('+forgot_password_otp')` , verify-otp ek **short-lived reset token** de (jaise 10 min, `purpose: 'reset'`), aur reset-password sirf wahi token accept kare.

Seller (sahi design kiya gaya flow):

- Registration:
  1. `POST /api/seller/register` : seller ban jaata hai `emailOtp` ke saath (5 min).
  2. `POST /api/seller/verify-otp {sellerId, emailOtp}` : `isEmailVerified = true` , aur token milta hai.
  3. `POST /api/seller/resent-otp` : sirf tab chalta hai jab purana OTP expire ho chuka ho (rate limiting jaisa).
- Forgot password:
  1. `forgot-password` : OTP 15 min ke liye.
  2. `verify-password-otp` : OTP sahi ho to `emailOtp = null` kar do, lekin `otpExpiresAt` waisa hi rehne do. Isse ek "reset window" khuli rehti hai.
  3. `reset-password` : sirf tab chalega jab `emailOtp === null` **aur** window abhi khuli ho. Password set hone ke baad window band (`otpExpiresAt = null`).
- Ye state-based design hai: server ki state se tay hota hai ki reset allowed hai ya nahi. Kami bas ye hai ki window ke dauraan jo bhi email jaanta hai wo reset kar sakta hai, kyunki token nahi maanga jaata.
- Seller login mein `isEmailVerified` ka check comment out hai (`// TODO`), isliye bina verify kiye bhi login ho jaata hai.

Rider: register → `isActive:false` (pending) → admin `PATCH /api/admin/riders/:id/status {isActive:true}` → ab login ho sakta hai. Login hote hi `isAvailable = true` set hota hai.

- **Bug:** login code active task dhoondhta to hai, par use use nahi karta. Isliye agar rider ke paas pehle se ek task chal raha ho aur wo dobara login kare, to `isAvailable` true ho jaata hai aur wo doosra task bhi accept kar sakta hai.

Admin: sirf login hai. Register API nahi hai; admin DB/seed se banta hai (ya contact controller ka default admin backdoor, Section 4.14).

4.3.7 Frontend pe auth (chaaron apps)

FrontEnd (customer):

- **Token store:** `authService` login/register response ko `localStorage.setItem("user", JSON.stringify(data))` mein rakhta hai (`{token, user}`).
- **Page reload pe session:** `authSlice.js:4-14` module load hote hi `localStorage.getItem("user")` padh ke `initialState.user` aur `token` bhar deta hai. Isliye refresh pe login bana rehta hai. Server se verify nahi hota; pehli API call pe 401 aaye to logout ho jaata hai.
- **Token lagana:** koi request interceptor nahi hai. Har service mein ye helper copy hai:

```
const authHeader = (getState) => ({ Authorization: `Bearer ${getState().user.token}` }); // cartService.js:4
```

- **401 handling** (`src/axiosSetup.js`): global axios response interceptor:

```
axios.interceptors.response.use((r) => r, (error) => {
  if (error.response?.status === 401) store.dispatch(logoutUser());
  return Promise.reject(error);
});
```

- **Logout:** `logoutUser` reducer `user/token = null` karta hai aur `localStorage.removeItem("user")` . Do kamiyan:
  - Reducer ke andar side effect hai, jabki reducers pure hone chahiye.
  - Cart, favorites, orders slices reset nahi hote, isliye logout ke baad bhi purana data memory mein rehta hai (navbar badge mein dikhta hai).

- **Protected pages:** `ProtectedRoute` component nahi hai. Har page `useEffect` mein khud check karta hai:

  ```
  useEffect(() => { if (!user || !token) navigate("/login", { replace: true }); }, [user, token]); // Checkout.jsx:46
  ```

  Isse page ek pal ke liye dikh jaata hai, phir redirect hota hai.

Seller-Panel:

- Token `localStorage.sellerToken` mein.
- `AuthInitializer` (`src/App.jsx:51`) app load pe `loadCurrentSeller()` → `GET /api/seller/profile` call karta hai, aur 401/403 aaye to token hata deta hai.
- **Route protection bilkul nahi hai.** Layout bina condition ke render hota hai, aur pages `Bearer null` ke saath API call kar dete hain.
- Do logout buttons hain: Navbar wala Redux bhi saaf karta hai, Sidebar ka "Sign Out" sirf `localStorage.removeItem("sellerToken")` karta hai aur Redux mein seller pada rehta hai.

Admin-Panel:

- `App.jsx` mein `isAdmin` state hai. Boot pe `GET /api/admin/verify` hota hai, aur tab tak "Loading..." dikhta hai.
- Har route pe inline guard: `isAdmin ? <AdminLayout><Page/></AdminLayout> : <Navigate to="/login"/>` .
- `/riders` route pe `AdminLayout` ko `onLogout` prop nahi diya gaya, isliye wahan se logout karo to UI "logged in" hi dikhta rehta hai.
- `DashboardPage.jsx:28` admin token ko console mein print karta hai.

delivery-dashboard:

- `axios.create({ baseURL: VITE_API_URL + '/api/delivery' })` .
- **Request interceptor** har request mein token lagata hai. **Response interceptor** 401 pe token hata deta hai, lekin Redux ka `isDelivery` true hi rehta hai (redirect wali line comment out hai).
- Inline route guards hain: `isDelivery ? ... : <Navigate/>` .
- Logout: `PUT /availability {isAvailable:false}` → `logout()` reducer → `/login` . `riderId` localStorage mein reh jaata hai.

4.3.8 Authorization (ownership) ka summary

Achha	Missing
Order cancel/rating: `order.user === req.user._id`	`GET /api/orders/:orderId` : koi bhi kisi ka order dekh sakta hai
Seller item status: item ka `sellerId === req.user._id`	Invoice / label PDF
Productinfo: `findOne({_id, sellerId})`	`/api/dashboard/stats/:sellerId` : URL mein kisi bhi seller ki ID daalo
Rider task status/cancel: `{_id, deliveryPartner: req.user._id}`	`sendDeliveryRequest` : task kis seller ka hai, check nahi
Address: `{_id, userId}`	Admin rider routes, contact routes (bilkul public)
Payment cancel-pending: `{user, razorpayOrderId}`	Socket `registerRider` : koi bhi riderId

Is kami ko **IDOR** (Insecure Direct Object Reference) kehte hain: sirf ID badal ke doosre ka data mil jaana.

4.3.9 Token storage: localStorage vs httpOnly cookie (interview question)

- **localStorage (is project mein):**
  - Fayde: simple hai, aur CSRF ka khatra nahi, kyunki browser ise khud nahi bhejta.
  - Nuksaan: koi bhi XSS script ise padh sakti hai, aur token 30 din valid hai.
- **httpOnly cookie:**
  - Fayda: JavaScript ise padh hi nahi sakti, isliye XSS se token chori nahi hota.
  - Nuksaan: CSRF protection lagana padta hai (`SameSite` , CSRF token), aur CORS mein `credentials: true` chahiye.
- **Best practice:** short-lived access token (memory mein) + refresh token (httpOnly cookie) + token rotation.

4.4 Frontend state management

4.4.1 Kaunsi app kya use karti hai, aur kyun

App	State tool	Reducers / slices	Context API?
FrontEnd	**Redux Toolkit**	`user` (auth!), `profile` , `products` , `cart` , `favorites` , `addresses` , `orders` , `reviews` , `blogs` , `invoice` (10)	Nahi
Seller-Panel	**Redux Toolkit**	`auth` , `seller` , `dashboard` , `stats` , `categories` , `products` (dead), `test` (dead)	Nahi
Admin-Panel	**Sirf** `useState` **+** `useEffect` har page mein	—	Nahi
delivery-dashboard	**Redux Toolkit**	`auth` , `orders` , `deliveryTasks` (`serializableCheck: false`)	Nahi

Redux kyun (interview answer):

- Cart count Navbar mein, product card pe aur checkout mein, teeno jagah chahiye. Login state har page pe chahiye. Rider ke socket se aaya task popup aur list dono mein chahiye. Aisa **shared global state** prop drilling se nahi sambhalta.
- `createAsyncThunk` se har API call ka `pending / fulfilled / rejected` lifecycle apne aap milta hai, aur saari apps mein ek jaisa pattern rehta hai.
- Admin panel mein har page ka data sirf usi page ka hai (aur kahin share nahi hota), isliye wahan `useState` kaafi hai.

Kya use nahi hua: RTK Query / React Query (inse caching, dedupe aur refetch milta), redux-persist (auth ko haath se localStorage mein rakha gaya hai), Context API.

4.4.2 Redux Toolkit ka pattern (project ka asli code)

```
// FrontEnd/src/Redux/cartSlice.js:66
export const addToCartWithBackendSync = createAsyncThunk(
  'cart/addToCartWithBackendSync',                    // action type ka prefix
  async (item, { getState, rejectWithValue }) => {
    try {
      return await cartService.add(() => getState(), { productId: item.id, quantity: item.quantity ?? 1, price: item.price });
    } catch (error) { return rejectWithValue(error.message); }   // rejected action ka payload
  });

// extraReducers: thunk ke 3 actions ko handle karna
.addCase(addToCartWithBackendSync.pending,   (state) => { state.loading = true; })
.addCase(addToCartWithBackendSync.fulfilled, (state, action) => {
   state.cartItems = mapBackendCartToItems(action.payload);    // server ka cart hi "source of truth"
```

```
    state.totalQuantity = state.cartItems.reduce((t, i) => t + i.quantity, 0);
    state.totalAmount   = state.cartItems.reduce((t, i) => t + i.price * i.quantity, 0);
})
.addCase(addToCartWithBackendSync.rejected,  (state, action) => { state.error = action.payload; })
```

- RTK andar se **Immer** use karta hai, isliye `state.loading = true` jaisa "mutate" karne wala code likhna allowed hai. Immer peeche se naya immutable object bana deta hai.
- Component mein: `dispatch(thunk(args)).unwrap()`. `unwrap()` rejected thunk ko `throw` mein badal deta hai, jisse `try/catch` mein error pakda ja sake.

4.4.3 FrontEnd (customer): har slice ek nazar mein

Slice (`state.x`)	Initial state	Thunks → endpoint	Kahan use
`user` (authSlice)	`{user, token}` localStorage se, `msg, loading, error`	`registerUser`, `loginUser`, `ForgotPassword`, `verifOtp`, `resetPassword` → `/users/*`	Login, Register, guards, sab services (token)
`profile` (userSlice)	`{profile, status, error}`	`fetchUserProfile`, `updateUserProfile`, `updateAvatar`	Profile page
`products`	`products, totalProductCount, shops, filterOptions{…}, sortOption, currentPage, currentProduct…`	`fetchProducts(params)` → `GET /products?…`, `fetchShops`, `fetchProductById`	ShopM, FilterDrawer, ProductDetails, ShopCom
`cart`	`{cartItems, totalQuantity, totalAmount, loading, error}`	fetch / add / update / remove / clear → `/users/cart`	Navbar badge, Cart, Checkout, ProductCard
`favorites`	`{items: [productIds]}`	fetch / add / remove → `/users/favorites`	Heart icons, Navbar count
`addresses`	`{addresses, loading, error, success, message}`	CRUD + setDefault → `/api/addresses`	Checkout, AddressList
`orders`	`{myOrders, currentOrder, loading, error}`	`getMyOrders`, `getOrderById`, `createOrder` (COD), `createRazorpayOrder`, `verifyRazorpayPayment`, `cancelOrder`, `cancelPendingRazorpayOrder`, `submitOrderRating`	Checkout, OrderList, OrderDetails
`reviews`	`{items, loading, submitLoading, error}`	`fetchReviews`, `submitReview` (FormData), `markHelpful`	ProductReview
`blogs`		`fetchBlogs`, `createBlog`	Blog pages
`invoice`	`{loading, error}`	`downloadInvoice`, `downloadLabel` (Blob download)	OrderDetails

Notes:

- **Naming trap:** `state.user` mein auth (user + token) hai, aur `state.profile` mein userSlice. Interview mein confuse mat hona.
- **Server state Redux mein:** cart har response ke baad backend wale cart se **poora replace** hota hai (optimistic update nahi). Isse data hamesha sahi rehta hai, par UI thoda slow lagta hai.
- **Local state kahan hai:** Checkout ka selected address, payment method aur coupon (page-level form state); FilterDrawer ke inputs; FavoritesPage ke poore product objects (Redux mein sirf IDs hain). Home page ke sections khud `fetch` karte hain.

- **Session restore:** `authSlice` module load pe `localStorage["user"]` padh leta hai. `useCartPersistence()` hook (App.jsx) login hote hi cart aur favorites fetch karta hai:

```
useEffect(() => { if (user && token) { dispatch(fetchCartFromBackend()); dispatch(fetchFavoritesFromBackend()); } }, [dispatch,
     user, token]);
```

- **Kamiyan:**
  - Logout pe cart, favorites aur orders reset nahi hote.
  - Kai thunks `error.message` (axios ka generic message) reject karte hain, server ka asli message nahi.
  - Kai components `.unwrap()` ke baad `err.message` padhte hain, jabki rejected value ek string hai. Isliye user ko server ka message nahi dikhta.

4.4.4 Seller-Panel

- `auth`:
  - initial `{seller: null, token: localStorage.sellerToken, isLoading, isSuccess, isError, message}`
  - `loginSeller`, `loadCurrentSeller` (`GET /api/seller/profile`), forgot/verify/reset thunks, `logoutSeller`
- `seller`: profile GET/PUT.
- `dashboard`: `fetchSellerStats` (`/api/dashboard/stats/:sellerId`), `fetchSellerProducts` (`/api/productinfo/my?view=`), `deleteSellerProduct`, `fetchSellerOrders` (`/api/dashboard/orders/:sellerId`). Chaaron ek hi `isLoading` flag share karte hain.
- `stats`: 6 thunks (`/api/stats/*`), jinme se sirf 3 use hote hain (sales-graph, order-status, inventory-management).
- `categories`: `GET /api/categories`, jo AddProductForm ke CreatableSelect dropdowns mein jaata hai.
- **Dead:** `productSlice` (AddProductForm seedha axios use karta hai), `testSlice` (counter scaffold).
- Token kahin Redux se padha jaata hai, kahin localStorage se. Ye inconsistent hai.

4.4.5 Admin-Panel (Redux nahi)

- Har page ka pattern ek jaisa hai:

```
const [data, setData] = useState([]); const [loading, setLoading] = useState(true);
useEffect(() => { /* axios/fetch with Authorization: Bearer localStorage.adminToken */ }, [activeTab]);
```

- Search, sort aur pagination sab **browser mein** hote hain (server poori list bhejta hai).
- **Optimistic update + rollback** (badges, `pages/ProductPage/ProductPage.jsx:129`). Ye interview ka achha example hai:

```
const previousProducts = products;                                          // snapshot
setProducts(cur => cur.map(p => p._id === productId
  ? { ...p, badges: { ...p.badges, [badgeKey]: checked } } : p));           // UI turant badlo
try {
  const response = await axios.put(`${API}/api/admin/products/${productId}/badges`, { badges: { [badgeKey]: checked } }, {headers});
  setProducts(cur => cur.map(p => p._id === productId ? response.data : p)); // server ka data
} catch (err) { setProducts(previousProducts); toast.error('Failed to update product badge.'); } // rollback
```

  Kami: snapshot purana closure hai, isliye do toggles ek saath fail hon to rollback doosre toggle ko bhi undo kar deta hai.

4.4.6 delivery-dashboard

- `auth` : `{isDelivery: !!localStorage.deliveryToken, token, loading, error}` . Thunks: `loginDelivery` , `registerDelivery` , `verifyToken` .
- `deliveryTasks` : `{tasks (socket se aaye), activeTask, nearbyTasks, locationError, loading, actionLoading}` .
  - Thunks: `getNearbyDeliveries` , `updateDriverLocation` (phir nearby refetch), `getActiveTask` , `acceptTask` (phir active refetch), `updateTaskStatus` , `rejectTask` .
  - Reducers `addDeliveryTask` (socket event se, duplicate nahi hone deta) aur `removeDeliveryTask` (popup timeout).
- `orders` : history (Delivered / Cancelled / Failed) + search/filter.
- `addMatcher` se saare thunks ke pending/fulfilled/rejected pe `actionLoading` set hota hai. Matcher doosri slices ke actions bhi pakad leta hai.

4.5 Routing

4.5.1 Backend routing

Mount table (kaunsa prefix kis file mein jaata hai) Appendix B ke Section 1.2 mein hai. Detail endpoints Section 4.2 mein. Yaad rakhne wali baatein:

- **Express Router** ek mini-app hai: `const router = express.Router(); router.get(...); app.use('/prefix', router)` .
- **Order matter karta hai:** Express upar se neeche match karta hai. Isliye literal routes (`/my-orders` , `/shops` , `/my`) `/:id` se pehle likhe gaye hain.
- **Route-level middleware:** `router.post('/x', mw1, mw2, handler)` . **Router-level middleware:** `router.use(protect)` (address aur admin banner routers), jo us router ke har route pe lagta hai.
- Express by default **case-insensitive** match karta hai. Isliye seller panel `/api/deliveryTask/...` bhejta hai aur backend ka `/api/deliverytask` route phir bhi match ho jaata hai.

4.5.2 Frontend routing (React Router 7, `BrowserRouter`)

`BrowserRouter` HTML5 History API (`pushState`) use karta hai, isliye URLs saaf hote hain (`/cart` , `#/cart` nahi). Iske liye host pe **SPA fallback** chahiye: har unknown URL pe `index.html` serve ho. Iska koi config file repo mein nahi hai (Section 4.17).

FrontEnd routes (`src/App.jsx`):

Path	Page
`/`	Homepage
`/products` , `/sale` , `/:category` , `/:category/:subcategory`	Productpage → ShopM (listing)
`/:category/:subcategory/:productName/:productId`	ProductDetails (sirf `productId` padha jaata hai)
`/shops` , `/shop-products?store=`	Shops list / ek shop ke products

Path	Page
`/cart`, `/favorites`, `/order-checkout`, `/order-placed/:orderNumber`, `/OrderList`, `/order_details?order_id=&item_id=`, `/AddressList`, `/profile`	Logged-in pages
`/login`, `/register`, `/forgot-password`, `/verification-otp`, `/password-change-continue`, `/change-password`	Auth
`/blog`, `/blogs/:id`, `/contact`, `/about` + 6 policy pages	Static/content
`/seller`	Legacy form (toota hua)

- **Ranking:** React Router v6/v7 routes ko "specificity" se rank karta hai, likhne ke order se nahi. Isliye `/cart` (static) hamesha `/:category` (dynamic) se jeet jaata hai.
- **404 page nahi hai.** Koi bhi unknown URL (`/faq`, `/orders`, `/shop`) `/:category` mein chala jaata hai aur "No products" dikhata hai. Footer ke kuch links aise hi toote hue hain.
- **Code splitting nahi hai:** koi `React.lazy` / `Suspense` nahi, saare ~45 pages ek bundle mein jaate hain.
- **Guards:** har page apna `useEffect` guard rakhta hai (Section 4.3.7).
- **URL query as state:** ShopM category, tag, price, rating aur discount filters URL se padhta hai, aur FilterDrawer `navigate('?' + params, {replace: true})` se URL update karta hai. Sort aur page sirf Redux mein hain, isliye refresh pe kho jaate hain.
- Buy Now cart ko bypass karta hai: `navigate("/order-checkout", { state: { buyNowProduct, isBuyNow: true } })`. Data **router state** se jaata hai. Isi wajah se `/order-placed` page direct URL se kholo to crash hota hai, kyunki tab state hoti hi nahi.

Seller-Panel: `/` → `/login`. Ek Layout route ke andar `/home`, `/addProduct`, `/orders`, `/statistics`, `/inventory`, `/my-products`, `/products/:id/update-details`, `/products/:id/update-prices`, `/onboarding`, `/seller/profile`, `/contact`, `/my-submissions`, aur 6 support pages hain. `/delivery` Layout ke bahar hai.

- **Guard nahi hai, 404 nahi hai.**
- Bug: kai pages khud bhi `<Layout>` mein wrapped hain, jabki wo pehle se Layout route ke andar hain. Isliye sidebar/navbar do baar render hote hain.

Admin-Panel: `/login`, `/`, `/sellers`, `/seller/:sellerId`, `/sellers/:sellerId/products`, `/products`, `/orders`, `/orders/:orderId`, `/riders`, `/banners`, `/contact-messages`, `/analytics`, `/reports`, `/search?q=`. Har protected route pe inline `isAdmin ? … : <Navigate to="/login"/>` hai.

delivery-dashboard: sirf 4 routes hain: `/login`, `/register`, `/` (DeliveryPanel, jisme 3 tabs hain), aur `/order/:orderId/:itemId` (task detail; `itemId` asal mein taskId hai).

4.6 Error handling aur validation

(Backend ka hissa neeche 4.6.1 aur 4.6.2 mein hai. Frontend ka hissa 4.6.3 mein.)

4.6.1 Backend: errors kaise handle hote hain

Project mein **3 style** ek saath chalte hain, kyunki alag logon ne alag tarah se code likha:

Style	Kahan	Kaise
A. Har route mein apna try/catch	`order.routes.js`, `payment.routes.js`, `product.routes.js`, admin controller, `dashboard.routes.js`	`catch (err) { res.status(500).json({ error: err.message }) }`. Response ka shape har jagah alag hai: kahin `{error}`, kahin `{message}`, kahin `{success:false, message}`.
B. `AppError` **+** `next(err)`	delivery controller, seller service, coupon, zodValidate, user forgot/reset password	Error throw karo, aur `globalErrorHandler` ek jaisa response banata hai
C. Bina try/catch ke async handler	`registerUser`, `loginUser` (`user.controllers.js`)	**Express 5 ki wajah se chal jaata hai**: Express 5 async function ka rejected promise khud pakad ke error handler ko de deta hai. Express 4 mein yahi code request ko hang kar deta (unhandled rejection).

`utils/AppError.js`: custom error class (line by line)

```
class AppError extends Error {
  constructor(message, statusCode, payload = null) {
    super(message);                                  // Error ka message set
    this.statusCode = statusCode;                     // 400, 404...
    this.status = `${statusCode}`.startsWith('4') ? 'fail' : 'error'; // 4xx = client ki galti
    this.isOperational = true;                        // "ye error humne jaan-boojh ke banaya hai"
    this.payload = payload;                           // extra data (jaise Zod issues)
    Error.captureStackTrace(this, this.constructor);   // stack trace mein constructor wali line nahi aati
  }
}
```

`isOperational` **kya hai?** Errors do tarah ke hote hain:

- **Operational:** expected galtiyan, jaise "OTP galat hai" ya "product nahi mila". Inka message user ko dikhana safe hai.
- **Programming:** bugs, jaise `undefined.x`. Production mein inka asli message user ko nahi dikhana chahiye.

`utilsSellerPanel/AppError.js` isi file ki copy hai (duplicate).

`utils/asyncHandler.js`

```
const asyncHandler = (fn) => async (req, res, next) => {
  try { await fn(req, res, next); } catch (error) { next(error); }
};
```

Ye ek **higher-order function** hai: ek function leta hai aur naya function return karta hai. Iska kaam hai har controller mein try/catch likhne se bachana. Express 5 mein ye zaroori nahi hai (Express 5 khud karta hai), par Express 4 ke zamane ka common pattern hai.

`Middleware/globalErrorHandler.js` (line by line)

Express error handler pehchanta kaise hai? **4 parameters** se: `(err, req, res, next)`. Ye `app.js:148` pe sabse aakhir mein mount hai.

```
module.exports = (err, req, res, next) => {
  err.statusCode = err.statusCode || 500;      // AppError nahi hai to 500
  err.status = err.status || 'error';
  if (process.env.NODE_ENV === 'development') return sendDevError(err, res); // poora err + stack bhejo
  else if (process.env.NODE_ENV === 'production') {
    let error = { ...err };  error.message = err.message;   // copy (message enumerable nahi hota, isliye alag se)
    if (err.name === 'CastError') error = handleCastErrorDB(err);         // galat ObjectId → 400 "Invalid _id: abc"
```

```
    if (err.code === 11000) error = handleDuplicateFieldsDB(err);         // duplicate email → 400
    if (err.name === 'ValidationError') error = handleValidationErrorDB(err); // mongoose validation → 400
    if (err.name === 'JsonWebTokenError') error = handleJWTError();       // 401
    if (err.name === 'TokenExpiredError') error = handleJWTExpired();     // 401
    return sendProdError(error, res);  // isOperational ho to message, warna generic "Something went wrong"
  }
  return res.status(err.statusCode).json({ success:false, message: err.message || "Something went wrong" });
};
```

- **Dev mode** mein `stack` bhi jaata hai, jo debugging ke liye achha hai, par production mein security risk hai.
- **Prod mode** mein Mongo aur JWT ke errors ko friendly 400/401 mein badla jaata hai, aur unknown bugs pe sirf generic message jaata hai.
- **Teesra case:** agar `NODE_ENV` set hi nahi hai, to raw message bhej diya jaata hai.

Kamiyan:

- **404 handler nahi hai.** Kisi bhi unknown URL pe Express ka default HTML page "Cannot GET /xyz" aata hai, JSON nahi.
- Style A wale routes errors ko yahan tak aane hi nahi dete, isliye response format consistent nahi hai.
- Multer ke errors (jaise "File too large" ya file filter ka error) bhi isi handler mein aate hain. Wahan `statusCode` nahi hota, isliye ye 500 ban jaate hain (asal mein 400 hona chahiye).
- `admin.controller.js` `new AppError(...)` call karta hai (`updateRiderStatus`) par file mein `AppError` **import hi nahi hai**. Isliye "Rider not found" wala case `ReferenceError` deta hai, aur response 404 ki jagah 500 jaata hai.
- `deliveryTask.controller.js` mein `AppError(...)` **bina** `new` ke call hua hai, aur `class` ko bina `new` call karne pe TypeError aata hai. (Ye routes frontend se use nahi hote.)

4.6.2 Backend: validation

(a) `express-validator` **: purane routes**

```
// routes/user.routes.js:70
router.post("/register", [
  body("email").isEmail().withMessage("Invalid email"),
  body("password").isLength({ min: 6 }),
  body("fullname.firstname").isLength({ min: 3 }),
], userController.registerUser);
```

- Ye rules sirf errors **collect** karte hain, request ko rokte nahi. Controller ko khud `validationResult(req)` check karna padta hai.
- `validationResult` sirf 3 controllers mein check hota hai: `user.controllers.js` (sirf register aur login), `address.controllers.js`, aur `blog.controllers.js`.
- Iska matlab, ye rules **lage hue hain par kuch nahi karte**:
  - customer ka `forgot-password`, `verify-otp`, `reset-password`
  - seller ka `forgot-password`, `verify-password-otp`, `reset-password`
- Register aur login validation fail hone pe **404** bhejte hain (`user.controllers.js:15`). Sahi status code 400 ya 422 hota.

(b) Zod + `Middleware/zodValidate.js` **: naye routes**

```
const zodValidate = (schema, property = "body") => (req, res, next) => {
  const parsed = schema.safeParse(req[property] || {});        // throw nahi karta, result object deta hai
  if (!parsed.success)
```

```
    return next(new AppError(parsed.error.issues[0].message, 400, parsed.error.issues)); // pehla error message
  req[property] = parsed.data;    // saaf data (unknown keys hat jaati hain, transforms apply)
  next();
};
```

- Ye **factory pattern** hai: `zodValidate(schema)` ek naya middleware banake return karta hai.
- Zod object by default **unknown keys strip** kar deta hai. Isliye `registerPartner` mein `DeliveryPartner.create({...req.body})` safe hai: client `isActive: true` bheje to bhi wo hat jaayega.
- **Express 5 ka gotcha (local pe run karke verify kiya):** Express 5 mein `req.query` ek getter hai jiska setter nahi hai. Isliye `req.query = parsed.data` **chupchaap ignore** ho jaata hai.
  - Matlab `GET /api/productinfo/my` pe Zod query validate to karta hai (galat value pe 400 deta hai), par handler ko transform kiya hua data nahi milta, purana raw `req.query` hi milta hai.
  - Ironically isi wajah se `view=deleted` param chalta hai. Wo schema mein nahi hai; agar assignment kaam karta to Zod use strip kar deta.
- **Zod 4 ka gotcha (verify kiya):** schemas Zod 3 wala syntax use karte hain: `z.string({ required_error: "Task ID is required" })`. Zod 4.3 mein `required_error` option ignore hota hai, isliye user ko custom message nahi, default `"Invalid input: expected string, received undefined"` milta hai.
- **Zod kahan lagta hai:**
  - Delivery routes: login, register, update-location, accept, task-status, cancel, reject
  - `deliverytask/create`
  - Seller productinfo: `my` (query), `details`, `pricing`
  - Admin: badges (`.strict()`, yaani unknown badge key bhejo to error), rider status (`isActive: boolean`)

Seekhne wali baat: ek project mein ek hi validation library rakho, aur use har route pe lagao. Abhi bahut se routes (cart, order, payment, admin login, banner, contact, coupon) pe koi validation nahi hai.

4.6.3 Frontend: errors aur validation

	FrontEnd	Seller-Panel	Admin-Panel	delivery-dashboard
Form validation	HTML `required` / `type=email` + haath se regex (Address: phone `^\d{0,10}$`, pincode `^\d{0,6}$`; Profile: phone `^[6-9]\d{9}$`)	Register: phone `/^\d{10}$/`, password `/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).{6,}$/`; AddProduct: required-fields loop + `alert`; UpdateDetails/UpdatePrices: `validate()` functions	Banner form ke basic checks, reject reason ke liye `prompt()`	Register: `alert` checks
Toasts	react-toastify (kaam karta hai)	`react-hot-toast` ka `toast()` call hota hai par `<Toaster/>` **mount hi nahi**, isliye kuch nahi dikhta	react-hot-toast (kaam karta hai)	react-toastify (kaam karta hai)
Error boundary	Sirf reviews section pe (`ReviewsErrorBoundary`)	Nahi	Nahi	Nahi
API error dikhana	`rejectWithValue(error.response?.data?.message)` → Redux `error` → UI. Kai jagah generic message aa jaata hai.	`alert()` / inline message	toast	toast / `alert`

- **Form library (Formik, React Hook Form) ya schema validation (Zod/Yup) frontend pe nahi hai.**
- **App-level ErrorBoundary nahi hai.** Kisi bhi component ka render crash (jaise `/change-password`, ya `/order-placed` bina state ke) poori app ko blank screen kar deta hai.
- Client validation sirf **UX** ke liye hoti hai (user ko jaldi feedback). Security ke liye server validation zaroori hai, kyunki koi bhi Postman/curl se seedha API call kar sakta hai.

4.7 Real-time feature: Socket.io (rider ko delivery request)

4.7.1 Socket.io kyun?

- HTTP mein sirf client poochta hai aur server jawab deta hai. Server khud se rider ko "naya order aaya" nahi bata sakta.
- **WebSocket** ek connection hai jo khula rehta hai, aur dono taraf se message ja sakta hai.
- **Socket.io** WebSocket ke upar library hai. Ye auto-reconnect, rooms aur events deta hai, aur zaroorat pade to long-polling fallback bhi (lekin client `transports: ["websocket"]` set karta hai, isliye fallback band hai).

4.7.2 Server side: `server.js` (line by line)

```
const server = http.createServer(app);                // Express aur Socket.io ek hi HTTP server, ek hi port
const io = new Server(server, { cors: { origin: "*" } });
global.io = io;                                        // services se emit karne ke liye global
global.riderToSocket = new Map();                      // riderId  → socket.id
global.socketToRider = new Map();                      // socket.id → riderId  (disconnect pe ulta lookup)
io.on("connection", (socket) => {                      // har naya client
  socket.on("registerRider", (riderId) => {            // client batata hai "main ye rider hoon"
    riderToSocket.set(riderId, socket.id);
    socketToRider.set(socket.id, riderId);
  });
  socket.on("disconnect", () => {                      // tab band / network gaya
    const riderId = socketToRider.get(socket.id);
    if (riderId) { riderToSocket.delete(riderId); socketToRider.delete(socket.id); }
  });
});
```

- **Do Maps kyun?** Emit karte waqt `riderId → socketId` chahiye, aur disconnect pe sirf `socket.id` milta hai, isliye ulta map bhi chahiye. Dono lookups O(1) hain.
- Ek rider ke 2 tabs khule hon to Map mein sirf aakhri tab ka socket rehta hai. Pehla tab band hua to rider poori tarah "offline" ho jaata hai, jabki doosra tab abhi khula hai.

4.7.3 Server se emit: `services/deliveryBroadcast.service.js`

```
const partners = await DeliveryPartnerModel.aggregate([
  { $geoNear: { near: { type: "Point", coordinates: task.pickupLocation.coordinates },
                distanceField: "distance", maxDistance: 5000, spherical: true,
                query: { isActive: true, isAvailable: true, _id: { $nin: triedPartnerIds } } } },
  { $sort: { distance: 1 } }, { $limit: 10 } ]);
if (!partners.length) { await assignDefaultRider(taskId); return; }        // koi nahi mila → test rider
partners.forEach((p) => {
  const socketId = global.riderToSocket.get(p._id.toString());
  if (socketId) global.io.to(socketId).emit("newDeliveryRequest", { taskId, pickup, drop });   // sirf usi rider ko
});
setTimeout(() => assignDefaultRider(taskId), 15000);                       // 15s mein kisi ne accept nahi kiya
```

- `io.to(socketId).emit()` sirf ek client ko bhejta hai. Isse `io.emit()` (sabko broadcast) se alag samjho.
- Ye **Uber-jaisa dispatch** hai: paas ke riders ko offer jaata hai, aur jo pehle accept kare usko task milta hai.

4.7.4 Client side: `delivery-dashboard`

- `src/services/socket.js` : `io(VITE_API_URL, { autoConnect: false, transports: ["websocket"] })`. Module-level **singleton** hai; `autoConnect:false` ki wajah se login ke baad hi connect hota hai.
- `src/hooks/useRiderSocket.js` (DeliveryPanel mein `useRiderSocket(riderId, token)` call hota hai):

```
useEffect(() => {
  if (!riderId || !token) return;
  socket.connect();
  socket.on("connect", async () => {
    socket.emit("registerRider", riderId);                        // server ke Map mein entry
    await updateLocationAPI(77.209, 28.613);                        // ⚠ placeholder location (har connect pe!)
  });
  socket.on("newDeliveryRequest", (task) => {
    dispatch(addDeliveryTask(task));                                // Redux → DeliveryRequest popup
    dispatch(getNearbyDeliveries());                                // list bhi refresh
  });
  return () => { socket.disconnect(); };                            // ⚠ socket.off() nahi
}, [riderId, token]);
```

- `DeliveryRequest.jsx`: har naye `taskId` ke liye 10 second ka countdown chalta hai. "Accept" dabane pe `POST /api/delivery/accept-task`, aur time khatam hone pe popup hat jaata hai. Server ka fallback 15 second ka hai, isliye rider ke paas 10 second ka window hota hai.
- Socket ke saath **polling** bhi chalti hai: "Nearby Radar" tab pe har 30 second `getNearbyDeliveries()`. Ye ek safety net hai, agar socket message miss ho jaaye.

4.7.5 Accept race condition kaise rokta hai (interview ka favourite topic)

Problem: do riders ek hi task ek saath accept karein.

Solution 1: conditional atomic update. MongoDB ka single-document update atomic hota hai. Filter mein `status:"Pending"` hai, isliye pehla rider task le lega aur doosre ki query ko koi document match hi nahi karega (`null` milega):

```
DeliveryTask.findOneAndUpdate({ _id: taskId, status: "Pending" },
  { status: "Accepted", deliveryPartner: riderId, $push: { attempts: {...} } }, { new: true, session });
```

Solution 2: rider lock. Ek rider ek time pe ek hi active task le sakta hai:

```
DeliveryPartner.findOneAndUpdate({ _id: riderId, isAvailable: true }, { isAvailable: false }, { session });
```

Transaction kyun? Rider lock ho gaya, par task kisi aur ne pehle le liya, to rider ka lock bhi wapas hona chahiye. Transaction ke andar ye dono updates ya to saath hote hain, ya saath rollback hote hain (`abortTransaction`). MongoDB transactions ke liye **replica set** chahiye. Atlas pe ye default hai; locally isi wajah se port 27018 pe replica set banaya gaya (Appendix A).

Status update bhi transaction mein: `updateTaskStatus` task aur order dono update karta hai. `PickedUp` hone pe order items ko "Out for Delivery" karta hai, aur `Delivered` hone pe items Delivered, rider phir se available, aur `stats.totalDeliveries +1`.

4.7.6 Limitations (honestly batao)

1. **Socket auth nahi hai:** koi bhi client `registerRider` mein kisi bhi rider ki ID bhej ke uski requests (pickup/drop address) sun sakta hai. Fix: handshake mein JWT bhejo aur verify karo:

   ```
   io.use((socket, next) => {
     try { socket.data.rider = jwt.verify(socket.handshake.auth.token, JWT_SECRET); next(); }
     catch { next(new Error("unauthorized")); }
   });
   ```

2. **In-memory Map +** `setTimeout` **:** server restart ya multiple instances pe kaam nahi karega. Fix: Redis adapter, aur fallback ke liye job queue.

3. **Placeholder location:** har socket connect pe rider ki location New Delhi placeholder se overwrite ho jaati hai, aur tasks ke coordinates bhi placeholder hain. Isliye asli "5 km" logic abhi kaam nahi karta.
4. **Listener leak:** cleanup mein `socket.off("connect")` / `socket.off("newDeliveryRequest")` nahi hai. Component dobara mount ho to listeners double ho jaate hain, jisse duplicate `registerRider` aur duplicate dispatch hote hain.
5. Reject ya timeout ke baad task kisi **doosre rider ko dobara broadcast nahi hota**. Seedha default rider ko assign ho jaata hai.
6. **Customer ya seller ke liye real-time kuch nahi hai.** Order status dekhne ke liye unhe page refresh karna padta hai, aur notifications bhi push nahi hoti.

4.8 File uploads (Multer → AWS S3 → CloudFront)

4.8.1 Flow

```
Browser: FormData (multipart/form-data) ─► Multer middleware ─► req.file / req.files (memory mein Buffer)
   ─► uploadToS3(buffer, folder, {contentType, originalName}) ─► S3 PutObject
   ─► return { secure_url: CLOUDFRONT_URL/key, key } ─► URL MongoDB mein save
```

- **Multer** `multipart/form-data` parse karta hai (`express.json()` ye kaam nahi kar sakta).
- `memoryStorage()` file ko RAM mein `Buffer` ki tarah rakhta hai, disk pe nahi likhta. Isliye S3 pe seedha bhejna asaan hai.

4.8.2 `utils/s3Upload.js` (line by line)

```
const requiredEnv = ["S3_REGION","AWS_ACCESS_KEY_ID","AWS_SECRET_ACCESS_KEY","S3_BUCKET","CLOUDFRONT_URL"];
function assertConfig() { /* koi env missing ho to Error throw */ }
let client;                        // lazy singleton: S3Client sirf ek baar banta hai, pehli upload pe
function getClient() { assertConfig(); if (!client) client = new S3Client({ region, credentials }); return client; }
function sanitizeFileName(name) { return path.basename(name).replace(/[^a-zA-Z0-9._-]/g, "_").slice(0,120); } // path traversal /
     ajeeb characters hatao
async function uploadToS3(fileBuffer, folder = "uploads", { contentType, originalName } = {}) {
  const key = `${folder}/${Date.now()}-${crypto.randomUUID()}-${safeName}`;   // unique key, overwrite nahi hoga
  await getClient().send(new PutObjectCommand({ Bucket, Key: key, Body: fileBuffer, ContentType: contentType }));
  return { secure_url: `${cloudFrontBase()}/${key}`, key };     // Cloudinary jaisa shape
}
```

- **Adapter pattern:** return value `{ secure_url }` hai, bilkul Cloudinary ke response jaisi. Project pehle Cloudinary pe tha, aur is shape ki wajah se purana code (`result.secure_url`) bina badle chalta raha.
- **CloudFront** AWS ka CDN hai. Files S3 bucket mein rehti hain, par user ko CDN ka URL milta hai. Isse duniya bhar mein file paas wale server se aati hai, fast loading hoti hai, aur bucket ko public rakhne ki zaroorat nahi padti (inference: bucket policy code mein nahi hai).
- `deleteFromS3(urlOrKey)` bana hua hai, lekin **kahin call nahi hota**. Product ki image badlo ya delete karo, to purani file S3 mein padi rehti hai (orphan files).

4.8.3 Har upload route ka config

Route	Multer setup	Size limit	Type check	Kahan jaata hai
`PUT /users/avatar`	`memoryStorage` , `single('avatar')`	50 MB	mimetype list: jpeg, png, webp, gif, heic, heif	S3 `avatars/`
`POST /users/blog` (**auth nahi**)	memory, `single('image')`	15 MB	ext + mime regex `/ jpeg|jpg|png|webp/`	S3 `Blogs/`

Route	Multer setup	Size limit	Type check	Kahan jaata hai
`POST /products/:id/reviews`	memory, `array('photos', 6)`	10 MB / file	extension **aur** mimetype dono regex `/jpeg|jpg|png|webp/`	S3 `ProductReviews/`
`POST /api/orders/:orderId/rating`	`diskStorage` → `uploads/`	10 MB	same regex	**Server ki local disk.** URL `/uploads/<time>-<originalname>`, jo `app.js:73` ka `express.static("uploads")` serve karta hai
`POST /api/products` (seller add product)	memory, `fields([images ×5, productVideo ×1])`	**koi limit nahi**	Images ka koi type check nahi. Video: mimetype `mp4/webm/quicktime` aur size ≤ 20 MB, lekin ye check images upload hone ke **baad** hota hai	S3 `SellerProducts/`
`PATCH /api/productinfo/:id/details`	memory, same fields	koi limit nahi	video check (20 MB), images ≤ 5	S3 `SellerProducts/`
`POST /api/seller/onboarding`, `PUT /api/seller/profile`	memory, `fields([aadharFile, panFile])`	**koi limit nahi**	**koi check nahi**	S3 `SellerVerification/Aadhar` aur `/PAN`
`POST/PUT /api/admin/banners`	memory, `single('image')`	koi limit nahi	koi check nahi	S3 `Banners/`

4.8.4 Problems aur sudhaar

1. **Size limit na hona + memoryStorage = memory DoS ka risk.** Koi 2 GB file bheje to poori file server ki RAM mein aayegi. Fix: har multer config mein `limits: { fileSize, files }` lagao.
2. **Order galat hai:** pehle S3 upload hota hai, phir validation. Video invalid nikla, ya Mongoose validation fail hui (jaise `name` missing), to images S3 pe pahunch chuki hoti hain aur product nahi banta. Fix: pehle saari validation, phir upload.
3. **Mimetype client bhejta hai, use spoof kiya ja sakta hai.** Asli check ke liye file ke "magic bytes" padhne padte hain (jaise `file-type` library se).
4. **KYC documents (Aadhar/PAN)** CloudFront ke public URL pe hain. Jiske paas URL ho wo dekh sakta hai. Sensitive documents ke liye private bucket + **pre-signed URL** (thodi der ke liye valid link) behtar hai.
5. **Rating photos local disk pe jaati hain.** Render jaise host pe disk temporary hoti hai (redeploy pe files ud jaati hain), aur multiple servers mein disk share nahi hoti. Baaki sab uploads S3 pe hain, ye ek inconsistency hai. Saath mein `originalname` filename mein seedha use hota hai.
6. **Migration script:** `scripts/migrateCloudinaryToS3.js` (`npm run migrate:s3`, dry-run ke liye `migrate:s3:dry`). Ye DB mein saved Cloudinary URLs download karke S3 pe upload karta hai aur DB mein CloudFront URL likh deta hai. Isse dikhta hai ki project ne storage provider beech mein badla.

4.9 Payments: Razorpay aur COD (line by line)

4.9.1 Pehle concept samjho: Razorpay ka 3-step flow

1. **Server pe order banao:** `razorpay.orders.create({amount, currency})`. Isse `order_xxx` ID milti hai. Amount **server** decide karta hai, client nahi.
2. **Browser mein checkout popup:** `new window.Razorpay({key, order_id, handler})`. User yahan pay karta hai, aur Razorpay browser ko 3 cheezein deta hai: `razorpay_order_id`, `razorpay_payment_id`, `razorpay_signature`.

3. **Server pe signature verify karo:** `HMAC_SHA256(order_id + "|" + payment_id, KEY_SECRET)` ki value `signature` ke barabar honi chahiye. Secret sirf server ke paas hai, isliye client nakli signature nahi bana sakta.

Backup: Webhook. Agar user payment ke baad tab band kar de, to step 3 kabhi nahi chalega. Iske liye Razorpay khud server ko `POST /api/payment/webhook` bhejta hai. Is request pe Razorpay ka apna signature hota hai (`x-razorpay-signature` header), jo `RAZORPAY_WEBHOOK_SECRET` se banta hai.

4.9.2 Server-side pricing: client ke price pe bharosa nahi

Dono flows (COD aur Online) mein price ka calculation server pe hi hota hai:

- `utils/buildPaymentOrderItems.js`:
  - Items `req.body.orderData.products` ya `req.body.products` se aate hain, aur agar body mein nahi hain to user ke **Cart** se.
  - Phir `Product.find({_id: {$in: productIds}})` se DB ka asli product nikalta hai, aur `price = toClientProduct(prod).offeredPrice` set karta hai.
  - Client ka bheja hua price kabhi use nahi hota. Sirf `quantity` client se aati hai (`raw.quantity || 1`).
- `utils/resolveOrderPricing.js`:

```
subtotal  = Σ price × quantity                      // toFixed(2)
gstAmount = subtotal × 0.05                         // GST_RATE, pricingConstants.js
coupon    = Coupon.findOne({ code: code.toUpperCase(), isActive: true })
// valid tabhi hai jab: expire nahi hua, usedCount < maxUsesTotal, aur user ka usage count < maxUsesPerUser
discount  = flat ? value : subtotal × value / 100;   discount = min(discount, subtotal)
total     = subtotal + gst + 0 (shipping) + 0 (delivery) − discount
```

  - Coupon invalid ho to error nahi aata; bas discount 0 lag jaata hai (silent ignore).
  - Note: GST discount se **pehle** wale subtotal pe lagta hai.

Kamiyan (interview mein honestly bata sakte ho):

- Products ka `approvalStatus`, `isDeleted` aur `inStock` check nahi hota. Matlab koi client body mein kisi pending/deleted product ki ID bhej ke usko order kar sakta hai, aur out-of-stock product bhi order ho jaata hai (Section 2.6 mein local run se verify hua tha).
- `quantity` ki upper limit nahi hai, aur `0` bheja to `|| 1` ki wajah se 1 ho jaata hai.
- `POST /api/coupons/validate` (`coupon.controller.js`) `usedCount > maxUsesTotal` check karta hai, jabki order wala code `>=` check karta hai. Ye off-by-one inconsistency hai. Aur `cartTotal` client se aata hai, lekin ye sirf UI preview ke liye hai; asli discount order banate waqt dobara calculate hota hai.

4.9.3 `POST /api/payment/create-order` (`payment.routes.js:96`)

```
router.post('/create-order', protect, async (req, res) => {
  const items = await buildPaymentOrderItems(req);           // DB se price
  if (!items || items.length === 0) return 400 'No Product to create order';
  const pricing = await resolvePricingAndCoupon(req, items);  // GST + coupon
  const amountPaise = Math.round(pricing.totalAmount * 100);  // Razorpay paise mein leta hai (₹1 = 100 paise)
  const razorpayOrder = await razorpayInstance.orders.create({ amount: amountPaise, currency: 'INR', receipt: `rcpt_${Date.now()}`
     });
  await Order.create({ ..., paymentMethode: 'Online', paymentStatus: 'Pending',
                       razorpayOrderId: razorpayOrder.id,
                       expireAt: new Date(Date.now() + 30 * 60 * 1000) });   // 30 min TTL
  res.json({ orderId: razorpayOrder.id, key_id: process.env.RAZORPAY_KEY_ID, amount: amountPaise, currency: 'INR', discountCode,
     discountAmount });
});
```

- DB mein ek **Pending** order pehle hi ban jaata hai. Agar user pay na kare, to MongoDB ka TTL monitor ise 30 minute baad apne aap delete kar deta hai. Koi cron job likhne ki zaroorat nahi padi.

- `key_id` public key hai, browser ko bhejna safe hai. `key_secret` kabhi browser ko nahi jaata.
- `if (!razorpayInstance)` wala check bekaar hai, kyunki `config/razorpay.js` instance hamesha bana deta hai. Key na ho to SDK server start hote hi crash kar deta hai (ye local setup mein dekha gaya tha).

4.9.4 `DELETE /api/payment/cancel-pending/:razorpayOrderId`

User popup band kar de (frontend ka `ondismiss`), to ye route Pending order ko turant delete kar deta hai, 30 minute ka wait nahi hota. Query mein `user: req.user._id` aur `paymentStatus:'Pending'` dono hain, isliye koi doosre user ka order ya paid order delete nahi kar sakta.

4.9.5 `POST /api/payment/verify` (`payment.routes.js:185`)

1. Body se `razorpay_order_id`, `razorpay_payment_id`, `razorpay_signature` lo. Teeno na hon to 400.
2. Signature check:

```
const expectedSignature = crypto.createHmac('sha256', secret)
  .update(`${razorpay_order_id}|${razorpay_payment_id}`).digest('hex');
if (expectedSignature !== razorpay_signature) return 400 'Invalid payment signature';
```

3. `Order.findOne({ razorpayOrderId })`:
   - **Already Paid hai** (webhook pehle aa gaya tha) → populated order lautao. Ye **idempotency** hai: same request dobara aaye to dobara kaam nahi hota.
   - **Order mila hi nahi** (TTL ne 30 min baad delete kar diya, par user ne pay kar diya tha) → `buildPaymentOrderItems` + `resolveOrderPricing` se naya order banao, seedha `paymentStatus:'Paid'` ke saath. Achhi recovery hai. Shipping address `orderData.shippingAddress` se aata hai; fallback `req.user.defaultAddress` hai, jo User schema mein exist hi nahi karta.
   - **Pending order mila** → `markRazorpayOrderPaid(order, paymentId)`: `paymentStatus='Paid'`, `expireAt = undefined` (TTL hata do, warna paid order bhi delete ho jaata!), save, phir `applyCouponUsageIfNeeded`.
4. Agar order is request mein Pending se Paid hua (`transitionedToPaid`):
   - cart khaali karo (sirf tab jab order cart se bana ho)
   - `createDeliveryTasksForOrder(order)` (rider broadcast)
   - `notifySellersNewOrder` (har seller ko ek DB notification)
   - response `201` + order (`orderWithClientProducts` se fees chhupa ke)

`applyCouponUsageIfNeeded`**:** pehle check karta hai ki is order ke liye `CouponUsage` pehle se to nahi hai, phir create karta hai aur `usedCount` ko `$inc` karta hai. Iska maksad hai ki verify aur webhook dono chalein to bhi coupon do baar count na ho.

4.9.6 `POST /api/payment/webhook` (`payment.routes.js:330`)

- **Raw body kyun?** HMAC exact bytes pe banta hai. Agar `express.json()` body ko parse karke object bana de, to bytes badal jaate hain aur signature match nahi hoga. Isliye:
  - `app.js:61-67`: `if (req.originalUrl === '/api/payment/webhook') next(); else express.json()(req,res,next);`, yaani sirf is URL ke liye JSON parser skip.
  - Route khud `express.raw({ type: "application/json" })` lagata hai, jisse `req.body` ek **Buffer** milta hai.
- Steps:
  1. `x-razorpay-signature` header nahi hai → 400.
  2. `createHmac('sha256', WEBHOOK_SECRET).update(req.body)` match nahi hua → 400.
  3. `JSON.parse(req.body)`.

4. `payment.captured`: order dhundho; Paid nahi hai to `markRazorpayOrderPaid`, cart khaali, delivery tasks, seller notifications.
5. `payment.failed`: user ko "Payment Failed" email (`userPaymentFailed` template).
6. Hamesha `200 {status:"ok"}` lautao, kyunki non-2xx milne pe Razorpay webhook dobara bhejta hai (retry).

4.9.7 COD flow: `POST /api/orders` (`order.routes.js:69`)

1. Items `req.body.products` ya Cart se bante hain. Har product `Product.findById` se aata hai (loop mein ek-ek query, N+1 pattern), aur price `toClientProduct(...).offeredPrice` se.
2. `resolveOrderPricing` (wahi GST + coupon logic).
3. Order save hota hai. Coupon laga ho to `CouponUsage.create` + `usedCount $inc`.
4. `createDeliveryTasksForOrder(saved)`: COD mein rider ko **turant** request jaati hai.
5. `sendOrderEmails(saved, user)`: customer aur admin ko email. `await` **nahi hai** (fire-and-forget), taaki email slow ho to bhi response na ruke.
6. Cart khaali, sellers ko notifications, response `201`.

Serious bug (code padh ke mila, run nahi kiya):

```
paymentMethode: req.body.paymentMethode || "COD",
paymentStatus:  req.body.paymentStatus  || "Pending",   // client se aata hai!
deliveryStatus: req.body.deliveryStatus || "Pending",
```

Client body mein `"paymentStatus": "Paid", "paymentMethode": "Online"` bhej de, to bina pay kiye "Paid" order ban jaata hai. Ye **mass assignment** ki galti hai. Fix: COD route mein ye teeno values server pe hardcode karo.

4.9.8 Race conditions aur missing cheezein

- **Verify aur webhook ek saath aa jaayein**, to dono `findOne` mein order ko Pending dekhenge, dono `markRazorpayOrderPaid` chalayenge, aur dono delivery tasks banayenge. (`createDeliveryTaskForSeller` ka `existing` check thoda bachata hai, par wo bhi atomic nahi hai.)
  - Fix: Pending se Paid ka transition ek atomic conditional update se karo, jaise `acceptTask` mein kiya gaya hai:

    ```
    const updated = await Order.findOneAndUpdate(
      { _id: order._id, paymentStatus: 'Pending' },
      { $set: { paymentStatus: 'Paid', razorpayPaymentId }, $unset: { expireAt: 1 } },
      { new: true });
    if (!updated) return; // kisi aur ne pehle hi kar diya
    ```

- **Verify ka "rebuild" branch amount check nahi karta (code padh ke, run nahi kiya):**
  1. Attacker ek sasta order banata hai aur uska payment karta hai.
  2. `/verify` call karne se pehle `DELETE /cancel-pending/:id` se Pending order delete kar deta hai.
  3. Ab `/verify` mein signature to sahi hai (asli payment hai), par DB mein order nahi milta. Server `orderData.products` se **naya** order banata hai, jisme client mehenge products bhej sakta hai, aur use "Paid" mark kar deta hai.
  - Asal mein kitna pay hua, ye compare nahi hota. Fix: Razorpay API se `payments.fetch(payment_id)` karke `amount` match karo, ya rebuild branch hata do.
  - Saath mein `findOne({razorpayOrderId})` mein `user: req.user._id` filter bhi nahi hai.
- Signature compare `!==` se hota hai. Best practice `crypto.timingSafeEqual` hai (timing attack se bachav).

- **Refund nahi hai:** online paid order cancel karo to sirf status "Cancelled" hota hai, Razorpay refund API call nahi hoti.
- Order cancel hone pe `DeliveryTask` cancel nahi hota (`order.routes.js:406`), isliye rider cancelled order bhi deliver kar sakta hai.
- `payment.failed` ke baad bhi Pending order TTL tak pada rehta hai.
- `paymentStatus` sirf `Pending/Paid` hai. "Failed/Refunded" states nahi hain.
- Webhook mein `RAZORPAY_WEBHOOK_SECRET` missing ho to `createHmac` throw karta hai, aur response 500 jaata hai.

4.9.9 Client side (`FrontEnd/src/components/Order/Checkout/Checkout.jsx`)

Poora sequence Section 3.2.4 ke diagram mein hai. Code-level points:

- `index.html` mein `<script src="https://checkout.razorpay.com/v1/checkout.js">` har page pe load hota hai, aur isi se `window.Razorpay` milta hai. Checkout pe pehla check: `if (!window.Razorpay) toast.error("Payment service not loaded…")`.
- `options` object: `key: rzpOrder.key_id` (key server se aati hai, env se nahi), `amount` (paise), `order_id`, `prefill {name, email, contact}`. `user.name` exist nahi karta (`fullname.firstname` hai), isliye naam prefill nahi hota.
- `handler(response)`: `verifyRazorpayPayment({...response, orderData}).unwrap()`, phir `createOrderSection()` (ETA calculate karke `/order-placed/:id` pe jaata hai). Fail ho to message: "If money is debited, contact support".
- `modal.ondismiss`: `cancelPendingRazorpayOrder(rzpOrder.orderId)`. User usi page pe rehta hai, taaki dobara try kar sake.
- `rzp.on("payment.failed")`: cancel-pending + error toast + `rzp.close()`. Close karne se Razorpay ka apna "retry" option bhi chala jaata hai.
- **Client ke totals sirf display ke liye hain:** `subtotal`, `GST 5%` (`utils/pricingConstants.js` backend ki copy hai) aur coupon discount. Asli amount server decide karta hai.
- "Place Order" button request ke dauraan disable nahi hota, isliye double-click pe do orders ban sakte hain.
- COD path mein client ek nakli "transaction ID" (`TXN` + random number) bana ke success page pe dikhata hai. `OrderPlaced` page online payment ke liye hardcoded fake card "**** 3456 Visa" dikhata hai.

4.10 Email

4.10.1 `utilsSellerPanel/sendEmail.js` (Resend), jo poore project mein use hota hai

```
const resend = new Resend(process.env.RESEND_API_KEY);          // key na ho to SDK boot pe hi crash karta hai
const sendEmail = async ({ to, subject, text, html }) => {
  const mode = process.env.EMAIL_MODE || "production";
  let finalTo = to;
  if (mode === "test") finalTo = "<developer ka gmail>";          // test mode: saari mails ek developer inbox mein
  const { data, error } = await resend.emails.send({
    from: `${EMAIL_FROM_NAME || "Aestheticommerce"} <${EMAIL_FROM_ADDRESS || "onboarding@resend.dev"}>`,
    to: finalTo, subject, text, html });
  if (error) throw new Error(error.message);     // fail hone pe throw, caller ko pata chalta hai
  return data;
};
```

- `EMAIL_MODE=test` **kyun?** Development mein galti se asli customers ko mail na chala jaaye. Resend ka free/testing sender (`onboarding@resend.dev`) waise bhi sirf apne verified address pe hi mail bhej sakta hai **(Resend ka general rule hai, code mein likha nahi)**.

- Pehle nodemailer + Gmail tha. Wo code isi file mein comment out pada hai.

4.10.2 Emails kahan se jaati hain

Trigger	File	`await` hai?	Fail hone pe kya hota hai
Seller register OTP	`sellerPanelService.js:62`	haan	**Seller DB mein ban chuka hai**, phir bhi error response jaata hai. Dobara register karo to "already exists" (409) aata hai, isliye `resent-otp` hi raasta bachta hai.
Seller forgot-password OTP, resend OTP	`sellerPanelService.js`	haan	error response
Customer forgot-password OTP	`user.controllers.js:104`	haan	OTP DB mein save ho chuka, par response 500
COD order confirm (customer + admin)	`orderEmailService.js`, call `order.routes.js:216`	**nahi** (fire-and-forget)	sirf log hota hai, order pe asar nahi
Razorpay `payment.failed`	`payment.routes.js:389`	haan	webhook 500 deta hai, aur Razorpay retry karta hai
Contact Us form	`contactUs.controller.js`	haan	
Newsletter	`utilsAdminPanel/NewsLetterEmail.js` (nodemailer + Gmail SMTP)	—	route mount hi nahi hai, dead code

- Online (Razorpay) orders pe confirmation email **nahi** jaata. `sendOrderEmails` sirf COD route mein call hota hai.
- Templates: `utils/forgotPasswordTemplet.js`, `utilsSellerPanel/sellerVerifyTemplate.js`, `utils/userPaymentFailed.js`. Ye HTML strings hain jinme `${}` se values daali jaati hain.
- `orderEmailService` mein `productName` bina escape kiye HTML mein jaata hai. Agar product name mein HTML ho, to email ka layout bigad sakta hai. Risk kam hai, kyunki naam seller deta hai aur product admin approve karta hai.
- Admin ka email `aestheticommerce@gmail.com` code mein **hardcoded** hai (`orderEmailService.js:39`).

4.11 Caching

Backend: `services/home.services.js` mein in-memory TTL cache

```
const responseCache = new Map();                                    // process ki memory mein
const CACHE_TTL_MS = { tags: 180_000, categories: 180_000, sellers: 300_000, popularProducts: 120_000 };
async function withTtlCache(key, ttlMs, fetcher) {
  const hit = responseCache.get(key);
  if (hit && hit.expiresAt > Date.now()) return hit.value;          // cache hit: DB query nahi
  const value = await fetcher();                                    // cache miss: DB se lao
  responseCache.set(key, { value, expiresAt: Date.now() + ttlMs });
  return value;
}
```

- Isko **cache-aside** pattern kehte hain. Key mein `limit` bhi hota hai (`tags:30`), kyunki alag limit ka alag result hota hai.
- Sirf **anonymous (sabke liye same)** data cache hota hai: tags, categories, top sellers, popular products. Personalised recommendations cache nahi hoti, kyunki wo har user ke liye alag hain.
- **Kamiyan:**

- **Invalidation nahi hai.** Admin koi product approve kare, to wo home page pe 2–5 minute baad dikhega.
- Map ka size limit nahi hai, aur expired entries delete nahi hoti. Abhi keys kam hain, isliye problem nahi hai.
- Har server instance ki apni cache hoti hai. Scale karne pe Redis lagana padega.
- **Cache stampede:** cache expire hote hi 100 requests ek saath aayein, to sab DB pe jaayengi.

Baaki

- **HTTP caching headers** (`Cache-Control`, `ETag`) ka koi custom use nahi hai. Express static files ke liye default ETag deta hai.
- **Redis ya koi external cache nahi hai.**
- Frontend pe caching ke liye Section 4.4 dekho (RTK Query ya React Query use nahi hui).

4.12 Cron job: badge automation

`jobs/badgeCron.js`

```
cron.schedule("0 0 * * *", async () => {          // minute hour day month weekday: roz 00:00
  try { await runBadgeAutomation(); } catch (e) { console.error(...); }
}, { scheduled: true, timezone: "UTC" });           // 00:00 UTC = subah 5:30 IST
```

`server.js:13` mein DB connect hone ke baad `initBadgeCron()` call hota hai.

`services/badgeAutomationService.js: runBadgeAutomation()`

1. Saare `approved` + not deleted products load karo (poore Mongoose documents, `.lean()` nahi).
2. Teen aggregations chalao:
   - 90 din ke Delivered units per product (`$unwind products`, phir `$match status Delivered`, phir `$group` sum quantity)
   - 7 din ke Delivered units
   - favorites count: `User.aggregate($unwind favorites, $group)`
3. Har product ke liye rules apply karo:

Badge	Rule
`bestSeller`	90 din mein ≥ 15 units delivered
`trending`	7 din mein ≥ 3 units delivered
`mostPopular`	≥ 2 users ki wishlist mein
`newArrival`	Product 0–15 din purana: false ("observation period"). 16–30 din: true, agar kam se kam 1 unit bika ho. 30 din ke baad: false.
`topRated`	rating ≥ 4.5 **aur** ≥ 3 reviews
`onSale`	`discountPercentage > 0` **ya** `mrp > sellingPrice`
`limitedStock`	0 < stock ≤ 10
`featured`	**Cron isse nahi chhoota**, ye sirf admin ka manual badge hai

4. Saare updates ek `Product.bulkWrite([...updateOne])` mein jaate hain, yaani har product ke liye alag DB call nahi, ek hi round trip.

Dhyan dene wali baatein:

- `onSale` ka rule lagbhag har product pe true hoga, kyunki `mrp` hamesha `sellingPrice + fees` se bada hota hai (MRP formula, Section 4.1.5).
- Maps `reduce((acc, x) => ({...acc, [k]: v}), {})` se bante hain. Har step pe poora object copy hota hai, isliye ye $O(n^2)$ hai. Hazaaron products pe slow hoga; `new Map()` ya `Object.fromEntries` behtar hai.
- **Multiple server instances** chalein to cron har instance pe chalega. Fix: ek alag worker process, ya DB lock, ya BullMQ/Agenda jaisa job queue.
- Server theek 00:00 pe down ho, to us din ka job skip ho jaata hai (catch-up nahi hota).
- Testing ke liye "server start pe turant ek baar chalao" wali line comment out hai (`badgeCron.js:11`).

4.13 Baaki features: PDF, notifications, geo, SEO

4.13.1 PDF invoice aur shipping label (PDFKit)

- Routes: `GET /api/document/invoice/:orderId` aur `GET /api/document/label/:orderId` (`routes/invoice.routes.js`). **Auth nahi hai**: jiske paas order ki `_id` hai, wo kisi ka bhi invoice (naam, address, phone) download kar sakta hai.
- Flow: `findOrder()` (populate products + user, `.lean()`), phir ek `safeOrder` object banta hai, phir headers set hote hain (`Content-Type: application/pdf`, `Content-Disposition: attachment`), phir `streamInvoicePDF(safeOrder, res)`.
  - `streamInvoicePDF`: `doc.pipe(res)`. Matlab PDF memory mein poori banti nahi, seedha response mein **stream** hoti hai.
- **Bug (local pe verify kiya):** `invoice.routes.js` mein

  ```
  totalAmount: typeof order.totalAmount === "number" ? order.totalPrice : ...
  ```

  `totalAmount` number hai, isliye `order.totalPrice` liya jaata hai, jo exist hi nahi karta. Local invoice PDF mein ye line chhapi: `Total Amount: ₹undefined`. Saath hi PDFKit ke default Helvetica font mein `₹` symbol nahi hai, isliye wo bhi galat character banke chhapta hai.
- Invoice mein GST, discount ya seller ka GSTIN nahi hai, isliye ye legal tax invoice nahi hai.
- Shipping label: A6 size PDF (`utils/shippingLabelGenerator.js`). Seller panel se download hota hai.

4.13.2 In-app notifications (DB-based, real-time nahi)

- **Create hoti hain:**
  - naya order (har seller ko)
  - product approve/reject (seller ko)
  - seller onboarding complete (saare admins ko)
  - seller ka support message (saare admins ko)
- **Padhne ke routes:** `/api/notifications` (`routes-admin-panel/notification.routes.js`) customer wala `protect` use karte hain. Controller `req.user.role` dekh ke `recipientModel` chunta hai:

  ```
  if (req.user.role === 'admin') recipientModel = 'Admin';
  else if (req.user.role === 'seller') recipientModel = 'Seller';
  else recipientModel = 'User';
  ```

  - Par `protect` sirf **User** collection mein dhoondhta hai, aur seller/admin tokens mein `_id` nahi, `id` hota hai. Isliye seller aur admin ki saari notifications **DB mein banti hain par kabhi padhi nahi ja sakti**.

    - Aur customer ke liye koi notification banti hi nahi. Feature practically adhoora hai.
- Real-time push (socket) notifications ke liye use nahi hota. Seller panel ka bell (`Seller-Panel/src/components/Notifications/Notifications.jsx`) sirf mount pe ek baar `GET /api/notifications` karta hai, `sellerToken` ke saath, bina polling ke. Upar wali wajah se jawab hamesha 401 aata hai, aur bell khaali list dikhata hai.

4.13.3 Geospatial (location) features

- **GeoJSON Point:** `{ type: "Point", coordinates: [longitude, latitude] }`. Order hamesha **lng pehle** hota hai. Google Maps "lat, lng" dikhata hai, isliye log yahan aksar galti karte hain.
- **2dsphere index:** prithvi ko gola (sphere) maan ke distance nikalta hai. `$geoNear` ke liye ye index zaroori hai.
- `$geoNear` **aggregation stage:** pipeline ka pehla stage hona chahiye. Ye distance ke hisaab se sorted documents deta hai aur har document mein `distanceField: "distance"` (meters mein) jod deta hai.
    - `maxDistance: 5000` = 5 km.
    - `query: {...}` se extra filter lagta hai (jaise `status: "Pending"`).
    - `key: "pickupLocation"` : collection mein do 2dsphere indexes hain (pickup aur drop), isliye batana padta hai ki kaunsa use karna hai.
- Do jagah use hota hai:
    1. `broadcastToNearbyRiders` : task ke pickup point ke 5 km mein active + available riders (max 10).
    2. `getNearByDeliveries` : rider ki location ke 5 km mein Pending tasks (max 20).
- **Asli limitation:** pickup/drop coordinates hamesha placeholder `[77.209, 28.613]` (New Delhi) hain, kyunki address se coordinates nikalne ka (geocoding) code nahi hai. Isliye abhi "5 km" ka matlab hai "sab riders jinki location bhi New Delhi placeholder pe hai".

4.13.4 SEO (backend side)

- `public/robots.txt` , `public/sitemap.xml` : static files.
- `routes/sitemap.js` : dynamic `/sitemap.xml` . Lekin `express.static('public')` pehle mount hai, isliye static file hi serve hoti hai, dynamic route tak request pahunchti hi nahi.
- `generateSiteMap.js` : alag se chalane wali script, jisme DB credentials hardcoded hain (Part 0).

4.14 Security measures: kya hai, kya nahi hai

4.14.1 Jo achha kiya gaya hai (interview mein confidently bolo)

Measure	Kahan	Kya karta hai
Password hashing (bcrypt, 10 salt rounds)	`user.model.js` (`hashPassword`), `sellerPanelService.js` , admin aur rider ke `pre('save')` hooks	Plain password kabhi DB mein nahi jaata. bcrypt jaan-boojh ke slow hai aur har hash mein random salt hota hai, isliye brute force mehenga padta hai.
`select: false`	User/Seller/Rider ka password, OTPs, rider ke document numbers aur bank numbers	Galti se API response mein sensitive data na chala jaaye
JWT auth + role-wise middleware	`protect` , `protectSeller` , `protectAdmin` , `protectDelivery`	Har panel ke routes alag collection se user verify karte hain
Server-side price calculation	`buildPaymentOrderItems` , `resolveOrderPricing` , `toClientProduct`	Client ka bheja price kabhi use nahi hota

Measure	Kahan	Kya karta hai
Razorpay HMAC signature verify	`payment.routes.js`	Nakli "payment success" nahi bheja ja sakta
Webhook raw-body HMAC	`app.js:61` + `express.raw`	Webhook bhi verified hai
Ownership checks (kai jagah)	order cancel/rating (`order.user === req.user._id`), productinfo (`{_id, sellerId}`), delivery tasks (`deliveryPartner: req.user._id`), seller item-status (item ka `sellerId`), address CRUD (`userId`)	Ek user doosre ka data nahi badal sakta
Regex escaping (ReDoS se bachav)	`escapeRegex()` (`product.routes.js:18`), blog tag filter	User input ko seedha `new RegExp()` mein nahi daala jaata
NoSQL operator injection se bachav	seller login: `Seller.findOne({ email: { $eq: email } })`	`$eq` ki wajah se attacker `{"$ne": null}` jaisa object bheje to bhi wo operator nahi, value ki tarah treat hoga
Zod strips unknown keys	delivery register, product details update	Mass assignment se bachav
Blocked seller check	`protectSeller` → 403	
Customer ko internal fees nahi dikhte	`toClientProduct` , listing pipeline ka `$project`	Business data private rehta hai
Secrets env mein	`.env` + `.gitignore`	(Exception: `generateSiteMap.js` ka hardcoded URl)
Filename sanitize + UUID S3 keys	`s3Upload.js`	Path traversal aur overwrite nahi ho sakta
CORS	`app.use(cors())`	Configured hai, par open hai (neeche dekho)

4.14.2 Jo missing hai (honest gaps)

Missing	Asar	Fix
Rate limiting (`express-rate-limit` nahi hai)	Login, OTP verify aur forgot-password pe brute force possible hai. 6 digit OTP mein sirf 10 lakh combinations hote hain.	Login/OTP routes pe rate limit, aur galat attempts ki ginti
`helmet` (security headers)	`X-Content-Type-Options` , HSTS, frame options jaise headers nahi lagte	`app.use(helmet())`
CORS `*`	`cors()` bina options ke, aur Socket.io `origin: "*"` . Koi bhi website API call kar sakti hai. (Token localStorage mein hai, cookie nahi, isliye CSRF ka khatra kam hai.)	`cors({ origin: [4 frontend URLs] })`
Request body size limit	`express.json()` default 100 KB theek hai, par multer pe limits nahi hain (Section 4.8)	
Input sanitization / `sanitizeFilter`	Kai `findOne({ email })` raw body se hote hain (admin login, rider login without Zod...). Mongoose ka `sanitizeFilter` option on nahi hai.	`mongoose.set('sanitizeFilter', true)` ya Zod everywhere
Token revocation / refresh token	JWT 30 din valid rehta hai. Logout sirf client pe hota hai, server pe token cancel nahi hota. Password badlo tab bhi purana token chalta rehta hai.	Short access token (15 min) + refresh token (httpOnly cookie), ya `tokenVersion` field
Socket.io auth	`registerRider` event mein koi bhi client koi bhi `riderId` bhej sakta hai aur us rider ki delivery requests sun sakta hai	Socket handshake mein JWT verify karo (`io.use(...)`)

Missing	Asar	Fix
Logging/monitoring	Sirf `console.log`. Kai jagah token, OTP aur poora user object log hota hai (`authMiddleware.js:8` authorization header print karta hai, `sellerPanelService.js:59` OTP print karta hai)	Structured logger (pino/winston) + secrets redact
OTP randomness	`Math.random()` cryptographically secure nahi hai	`crypto.randomInt(100000, 1000000)`
HTTPS	Code mein nahi; hosting (Render/Vercel) deta hai **(inference)**	

4.14.3 Security bugs jo is deep dive mein mile (severity ke hisaab se)

Jo cheezein **local pe run karke verify** hui hain, unpe "(verified)" likha hai. Baaki code padh ke mili hain.

Critical / High

1. **Customer password reset bina OTP proof ke** (`user.controllers.js:159`): `POST /users/reset-password` sirf `email + newPassword + confirmPassword` leta hai. Jo bhi kisi ka email jaanta hai, wo uska password badal sakta hai, yaani **account takeover**. Saath mein `verify-otp` hamesha fail hota hai (OTP `select:false` hai, `user.forgot_password_otp` undefined aata hai). To asli flow bhi toota hai aur shortcut khula hai.
2. **COD order mein `paymentStatus` client se** (`order.routes.js:191-193`): bina pay kiye "Paid" order ban sakta hai (Section 4.9.7).
3. **Seller profile update mass assignment** (`sellerPanelController.js`, `PUT /api/seller/profile`): `{ $set: req.body }` + `runValidators: false`. Ek pending seller body mein `status: "approved"`, `verificationStatus: "verified"`, `isEmailVerified: true` bhej ke khud ko approve kar sakta hai, ya `password` field pe plain text likh ke apna login tod sakta hai. Fix: allow-list fields (Zod schema).
4. **Bina auth wale admin/sensitive routes:**
   - `GET /api/orders` (verified: bina token ke saare orders, customers ke naam, phone, address aa gaye)
   - `GET /api/orders/:orderId`, `GET /api/orders/user/:userId`
   - `/api/document/invoice|label/:orderId`
   - `/api/dashboard/stats|orders/:sellerId`
   - `/api/contact` (GET/PUT/DELETE: koi bhi support tickets padh, edit aur delete kar sakta hai)
   - admin ke rider routes: koi bhi kisi bhi rider ko activate ya deactivate kar sakta hai (`admin.routes.js:59` ka comment: "right now bypassing protectAdmin")
   - `POST /users/blog`: koi bhi blog post kar sakta hai
5. **Default admin backdoor** (`contact.controller.js`): agar DB mein ek bhi admin nahi hai, aur koi seller-type contact message aaye (ye route khud public hai), to code `admin@test.com / admin123` wala admin **bana deta hai**. Fresh deployment pe ye known password wala admin account hai.
6. **Password hash response mein leak:** `loginUser` aur `registerUser` poora `user` document bhejte hain. Login mein `.select('+password')` hai, aur `create()` se mila document bhi password field rakhta hai. Isliye response JSON mein bcrypt hash jaata hai.

7a. **NoSQL operator injection** (customer `forgot-password` / `reset-password`): `email` ka type check nahi hai, aur express-validator rules ka result check nahi hota. Isliye body mein `{"email": {"$ne": null}, ...}` bhejne pe `findOne` DB ka **pehla user** de deta hai, aur reset-password us user ka password badal deta hai. Email jaanna bhi zaroori nahi. Fix: `typeof email === 'string'` check, Zod, ya `mongoose.set('sanitizeFilter', true)`.

Medium 7. **Public reviews API reviewers ke email dikhati hai:** `GET /products/:id/reviews` mein `.populate("reviews.user", "fullname email")` hai, isliye har reviewer ka email public ho jaata hai (privacy issue). 8. `/home/products` **internal fees leak karta hai (verified):** response mein `sellingPrice`, `platformFee`, `logisticsFee`, `transactionFee` raw jaate hain, aur `offeredPrice` hai hi nahi. Baaki saari APIs `toClientProduct` use karti hain, ye nahi karti. Review POST ka response (`res.json(product)`) bhi poora product

(fees ke saath) bhejta hai. 9. **Product edit ke baad re-approval nahi:** `PATCH /api/productinfo/:id/details` approved product ka naam, description, images badal deta hai, aur `approvalStatus` "approved" hi rehta hai. Admin ki quality-control wali baat bypass ho jaati hai. 10. **Seller khud rating aur platform fee set kar sakta hai:** `addProduct.routes.js` mein `rating` aur `platformFee` body se aate hain, aur `updateDetails` schema mein bhi `rating` allowed hai. 11. `sendDeliveryRequest` (`deliveryTask.controller.js:159`): task seller ka hai ya nahi, ye check nahi hota. 12. **KYC docs public CDN URL pe** (Section 4.8.4).

- **Public rider data:** `GET /api/admin/riders/active` (bina auth) riders ka naam, phone, email aur location deta hai. `/home/sellers` sellers ka email, phone, address aur GST public karta hai.
- **Contact-us email abuse:** `POST /api/contact-us` kisi bhi email address pe acknowledgement mail bhejta hai, aur `${name}` / `${message}` bina escape kiye HTML mein jaate hain. Rate limit bhi nahi hai, isliye spam/phishing ke liye use ho sakta hai.

Low 13. Admin `pre('save')` double-hash bug (latent). 14. Bank details aur Aadhar/PAN numbers plain text mein (encryption-at-rest nahi). 15. `protectAdmin` mein `Authorization: Bearer` header ho par token khaali ho, to do baar response bhejne ki koshish hoti hai ("headers already sent" error).

4.14.4 Frontend security

- **Tokens localStorage mein hain** (chaaron apps). Kai jagah console mein print hote hain: `Loginsection.jsx:25`, `addressSlice.js`, `userSlice.js:9`, Admin `DashboardPage.jsx:28`.
- `dangerouslySetInnerHTML`**:** sirf ek jagah hai, `FrontEnd/src/utils/SEO.jsx` mein JSON-LD ke liye: `JSON.stringify(schema)`.
  - `JSON.stringify` `</script>` ko escape nahi karta. Abhi ye client-side render hai aur risk kam hai, par SSR lagaya to product ya blog text se XSS ho sakta hai.
  - Baaki teen apps mein ye use nahi hota.
  - Seller panel ki packing slip `document.write` use karti hai, lekin har value `escapeHtml()` se guzarti hai, jo sahi practice hai.
- React by default `{text}` ko escape karta hai, isliye normal rendering XSS-safe hai.
- **Hardcoded URLs** jo production mein toot jaayenge:
  - FrontEnd: `BlogDetails.jsx:12`, `Contactus.jsx:25`, `SellerForm.jsx` (`http://localhost:5000`)
  - Admin: `ContactMessagesPage.jsx` (`http://localhost:5001`, galat port bhi)
  - Seller: `OrderTable.jsx` (fallback `localhost:5000`), `Sidebar.jsx` (fallback `localhost:5173`)
- `VITE_*` **env variables public hote hain:** Vite unhe build ke waqt bundle mein daal deta hai, isliye wahan kabhi secret mat rakho. Is project mein sirf API URLs hain, jo theek hai.
- Undocumented env vars: `VITE_APP_URL` (FrontEnd SEO aur Seller links) `env.example` mein nahi hai, aur `VITE_DEV_API_URL` bhi nahi.
- `.env` files: root `.gitignore` mein `**/.env` hai, isliye saari apps ki `.env` ignore hoti hain. (delivery-dashboard ki apni `.gitignore` mein env pattern nahi hai, par root wala cover kar leta hai.) Admin ka `env.txt` kisi pattern mein nahi aata, lekin usme koi secret nahi hai.
- Google Analytics (gtag `G-QST2WZW56J`) chaaron apps mein hai. Ye third-party script hai jo localStorage wale tokens ke saath same page pe chalti hai.

4.15 Performance optimizations

4.15.1 Backend: jo achha kiya gaya hai

Optimization	Kahan	Kyun faydemand
Compound indexes	`product.model.js:175–182`	Home/listing ki query `{approvalStatus, isDeleted, inStock}` + `rating` sort ek hi index se chal jaati hai
Multikey index on `tags`	Product	Recommendations ki `tags: {$in: [...]}` query
2dsphere indexes	DeliveryPartner, DeliveryTask	`$geoNear` ke liye zaroori, aur fast
TTL index	`Order.expireAt`	Unpaid orders bina cron ke apne aap saaf ho jaate hain
Aggregation + `$facet`	`GET /products`	Ek hi DB call mein total count **aur** current page dono aa jaate hain (`count: [{$count}]` , `products: [$sort, $skip, $limit]`)
Derived field DB mein calculate	`offeredPrice` pipeline mein (`$add` , `$multiply` , `$round`)	Price ke hisaab se sort aur filter DB hi kar deta hai. Warna saare products laake JS mein sort karne padte.
Server-side pagination + limit cap	`/products` (max 50), home APIs (`clampLimit`), trending tags (max 24)	Koi `limit=100000` bhej ke server nahi gira sakta
`.lean()`	home services, `buildPaymentOrderItems` , delivery task lists	Mongoose document ki jagah plain JS object milta hai, jo kam memory leta hai aur fast hota hai
Field projection	`.select('name sellingPrice …')` , `SAFE_SELLER_PROJECTION`	Kam data network pe aata hai
`Promise.all` **(parallel)**	Recommendation ke 3 signals (wishlist/order/cart), multiple S3 uploads	Teen queries ek saath chalti hain, ek ke baad ek nahi
`bulkWrite`	Badge cron	Saare products ka update ek DB round-trip mein
Working-set cap	Tag aggregation mein `$limit: 200` before `$unwind`	Aggregation ka kaam simit rehta hai
In-memory TTL cache	`home.services.js`	Home page ki repeat requests DB tak nahi jaati
Connection pool	`db.js` : `maxPoolSize: 50`	Har request pe naya connection nahi banta
Fire-and-forget email	COD order (`sendOrderEmails` bina await)	Response email ka wait nahi karta
Streaming PDF	`doc.pipe(res)`	Poori file memory mein nahi banti
CDN	CloudFront	Images aur video paas ke edge server se aate hain

4.15.2 Backend: performance problems

1. `/api/dashboard/stats|orders/:sellerId` : `Order.find({}).populate("products.product")` **saare orders** memory mein laata hai, phir JS mein filter karta hai. Har dashboard load pe poori collection scan. Fix: `$match: {"products.sellerId": id}` wali aggregation (jaisi `servicesSellerPanel/statsService.js` mein hai).
2. **N+1 queries:**
   - `getAllSellers` : har seller ke liye alag `countDocuments` . Fix: ek `$group` aggregation.
   - COD order: har product ke liye alag `findById` . Fix: `find({_id: {$in}})` , jaisa `buildPaymentOrderItems` mein kiya gaya hai.

3. **Missing indexes:** `Order.user`, `Order.products.sellerId`, `Order.razorpayOrderId`, `Cart.user` (unique bhi hona chahiye), `Notification.recipient`, `Product.category`.
4. **GET mein DB write:** `my-orders` aur `/:orderId` har purane order pe `ensureOrderIds` + `save()` chalate hain.
5. **Listing pipeline mein `$lookup` pagination se pehle:** seller ka join **har matching product** pe hota hai, sirf current page ke 15 pe nahi. Fix: `$lookup` ko `$facet` ke `products` branch mein `$limit` ke baad rakho.
6. **Regex filters** (`category`, `tags` case-insensitive regex) index ka poora fayda nahi le paate. Fix: tags pehle se lowercase hain, to exact match use karo; category ke liye slug field.
7. **Compression nahi hai:** `compression` middleware nahi hai. Reverse proxy (Render/Vercel) shayad gzip karte hon **(inference)**.
8. **Bahut zyada `console.log`:** hot paths mein poore objects print hote hain (`"check1"…"check6"` order route mein). Production mein logging I/O slow karti hai.
9. Admin/seller ki saari lists **poori** aati hain (client-side pagination). Data badhne pe slow hoga.
10. Badge cron mein `reduce` + spread $O(n^2)$ hai, aur saare products poore documents ki tarah load hote hain.

4.15.3 Frontend performance

Achha	Problem
Products ka server-side pagination (15 per page)	**Code splitting zero:** koi `React.lazy` nahi, sab ek bundle mein. `@mdxeditor/editor` (bhaari) sirf description dikhane ke liye load hota hai.
`useMemo` kuch jagah (OrderDetails, ProductReview, Admin OrderDetails, Seller ProductPage/OrderTable)	`React.memo` kahin nahi; delivery app mein koi memoization nahi
Kuch images pe `loading="lazy"` (ProductCard, blogs, shops)	`FrontEnd/src/Images` **~69 MB** (64 files, kuch 3–4 MB ki). Image optimization nahi hai.
Admin ke polling intervals (30s) cleanup ke saath	Caching nahi (RTK Query nahi). Har page ka Navbar dobara categories aur shops fetch karta hai.
ScrollToTop ka scroll save 150 ms throttle	**Duplicate requests:** home pe `/home/tags` 2 baar, `/products/shops` 2 baar; ShopM har URL change pe double fetch; seller `/orders` page pe orders 2 baar; nested Layout se notifications 2 baar
	Har product detail view pe user ke **saare orders** fetch hote hain (review eligibility client pe check hoti hai)
	Razorpay script aur Google Fonts har page pe render-blocking load hote hain
	Har `ProductCard` apna `resize` listener aur `setInterval` lagata hai
	Debounce nahi hai (search submit-based hai, isliye abhi problem nahi)

Interview line: "*Main sabse pehle route-level code splitting (`React.lazy`) karunga, images ko WebP mein compress karke responsive `srcset` dunga, aur server data ke liye RTK Query lagaunga taaki caching aur dedupe mil jaaye.*"

4.16 Testing setup

Poore project mein ek bhi test nahi hai:

- Backend `package.json` ka `"test"` script default wala hai: `echo "Error: no test specified" && exit 1`.
- Chaaron frontends mein `test` script nahi, Jest/Vitest nahi, koi `*.test.*` ya `*.spec.*` file nahi.

- Sirf **ESLint** hai (har frontend mein `eslint.config.js`, `npm run lint`). Backend mein linter bhi nahi.
- Testing ke naam pe sirf manual testing hui (Postman/Swagger). Code mein `EMAIL_MODE=test` aur test rider `delivery@test.com` iske saboot hain.

Interview mein agar pooche "kaise test karoge?" (concrete plan):

1. **Unit tests (Jest):** pure functions, jinme DB nahi lagta:
   - `roundToPriceEndingIn9(742) === 739`, `roundToPriceEndingIn9(748) === 749`
   - `toClientProduct()`: fees hat jaayein aur `offeredPrice` sahi aaye
   - `resolveOrderPricing()`: GST 5%, flat/percent coupon, discount subtotal se zyada na ho
   - `getEstimatedDelivery()`: IST 10–20 baje +2 ghante, warna +7 din
   - `syncOrderStatusFromTask` ka status derive karne wala logic
2. **API integration tests (Jest + Supertest + `mongodb-memory-server`, replica-set mode taaki transactions chalein):**
   - auth: galat token pe 401, blocked seller pe 403
   - **race test:** do riders `Promise.all` se ek hi task accept karein → ek ko 200, doosre ko 400
   - payment: galat signature pe 400; webhook raw body ke saath
   - authorization: user A, user B ka order cancel na kar sake
3. **Frontend:** Vitest + React Testing Library (slices ke reducers, Checkout ke totals), aur Playwright/Cypress se E2E (login → cart → COD order).
4. **CI:** har PR pe lint + test + build (Section 4.17).

4.17 Deployment, hosting aur CI/CD

Repo mein deployment ka koi config nahi hai: `Dockerfile`, `docker-compose`, `.github/workflows`, `vercel.json`, `netlify.toml`, `_redirects`, `render.yaml`, `Procfile`, kuch nahi.

Hosting (inference, code ke hints se):

- Backend **Render** pe tha. `server.js:15` ka comment hai: `// Start server (Render will provide PORT)`, aur code `const PORT = process.env.PORT || 3000` likhta hai. Render jaise platforms PORT env variable se dete hain.
- Frontends shayad **Vercel** pe the, kyunki root `.gitignore` mein `.vercel` folder ka pattern hai **(inference)**. Sitemap/robots mein domain `https://aestheticommerce.com` hai.
- Database **MongoDB Atlas** pe (`mongodb+srv://` URI), media **AWS S3 + CloudFront** pe.

Build/run commands:

- Backend: `npm install` → `npm start` (`node server.js`). Build step nahi hai, kyunki plain CommonJS JavaScript hai.
- Har frontend: `npm run build` (`vite build` → `dist/` mein static files) → kisi bhi static host pe deploy.

Production mein kya-kya configure karna padta (code ki zaroorat ke hisaab se):

1. **Backend env variables:**
   - Core: `PORT`, `NODE_ENV`, `DB_CONNECT`, `JWT_SECRET`
   - Storage: `S3_REGION`, `S3_BUCKET`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `CLOUDFRONT_URL`
   - Payment: `RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET`
   - Email: `RESEND_API_KEY`, `EMAIL_MODE`, `EMAIL_FROM_NAME`, `EMAIL_FROM_ADDRESS`, `EMAIL`, `MAIL_PASS`, `SMTP_USER`, `SMTP_PASS`

- Sitemap: `DB_NAME`
- Migration script: `MONGO_URI` / `MONGODB_URI`
- `env.example` mein inme se Razorpay, Resend, `EMAIL_MODE/FROM_*` aur `DB_NAME` missing hain.

2. **Frontend env variables** (build time pe bundle mein jaate hain): `VITE_API_URL` (sab), `VITE_APP_URL` (FrontEnd SEO, Seller links), `VITE_SELLER_API_URL` (FrontEnd ka "Become a Seller" link).
3. **SPA rewrite:** BrowserRouter ki wajah se host pe "har path → `/index.html`" ka rule chahiye. Warna `/cart` refresh karne pe 404 aayega. Ye rule repo mein nahi hai; shayad hosting dashboard pe set kiya gaya ho **(inference)**.
4. **Razorpay dashboard** mein webhook URL `https://<backend>/api/payment/webhook` aur webhook secret set karna padta hai.
5. **WebSocket support:** host ko WebSockets allow karne honge (Render karta hai). Multiple instances ke liye sticky sessions ya Redis adapter chahiye.
6. **Single instance hi chalana:** cron, `setTimeout` aur in-memory Maps ki wajah se abhi scale-out safe nahi hai (Section 3.1).

Local dev: `local-dev/start.sh` / `stop.sh` (Appendix A), jo is revision ke liye banaya gaya tha. Ye project ka original hissa nahi hai.

Ek simple CI/CD jo add kiya ja sakta hai (GitHub Actions):

```
on: [pull_request]
jobs:
  backend:  { steps: [checkout, setup-node, "npm ci", "npm test"] }              # Backend/
  frontend: { strategy: { matrix: { app: [FrontEnd, Seller-Panel, Admin-Panel, delivery-dashboard] } },
              steps: [checkout, setup-node, "npm ci", "npm run lint", "npm run build"] }
# main pe merge → Render/Vercel ka auto-deploy (Git integration)
```

4.18 Step 3 + 4 ke naye findings: quick list (Section 2.6 ki list ke alawa)

Ye cheezein is deep dive mein nayi mili hain. Interview mein inhe "done" claim mat karna. Agar interviewer code dikha ke pooche, to fix bata dena.

Local pe verified:

1. `GET /api/orders` bina token ke saare orders deta hai, customers ke naam, phone aur address ke saath.
2. `/home/products` internal fees (`sellingPrice` , `platformFee` , …) public kar deta hai.
3. Invoice PDF mein "Total Amount: ₹undefined" chhapta hai.
4. `GET /api/orders/seller/orders` hamesha 500 deta hai (`ObjectId` bina `new`).
5. Express 5 mein `req.query = parsed.data` ignore hota hai (Zod ka query output kabhi apply nahi hota).
6. Zod 4 `required_error` ke custom messages ignore karta hai.

Code padh ke mile (run nahi kiye):

- **Security:**
  - COD order mein `paymentStatus: "Paid"` client se set ho sakta hai.
  - Seller profile update mass assignment (khud ko approve/unblock kar sakta hai).
  - Default admin `admin@test.com/admin123` public contact route se ban sakta hai.
  - Customer reset-password mein NoSQL injection (`{"$ne": null}`).
  - `/verify` ka rebuild branch paid amount check nahi karta.
  - Reviewers ke email public.
  - Rider approval routes public.

- **Logic:**
  - Rider "Accepted" task cancel nahi kar sakta.
  - Seller ko rider-accepted tasks nahi dikhte.
  - Rider login active task hote hue bhi `isAvailable = true` kar deta hai.
  - Order cancel karne pe delivery task cancel nahi hota.
  - Product edit ke baad re-approval nahi hoti.
  - MRP formula inflated hai.
  - Admin ka "weekly active users" hamesha khaali, aur sales analytics saare saalon ko mila deta hai.
  - Seller aur admin notifications kabhi padhi nahi ja sakti.
- **Frontend:**
  - Seller-Panel ke toasts dikhte hi nahi (`<Toaster/>` mount nahi).
  - Seller-Panel mein route guard nahi, aur nested Layout ki wajah se sidebar/navbar do baar.
  - Admin ka contact page `localhost:5001` pe hardcoded hai.
  - Rider app har socket connect pe location placeholder se overwrite kar deti hai, aur socket listeners leak hote hain.
  - FrontEnd mein 404 page nahi hai, code splitting nahi hai, aur images ~69 MB ki hain.

Step 5: Design decisions, sabse mushkil problems, kamiyan aur scaling

> Step 5 aur Step 6 2026-10-14 ko likhe gaye: Steps 1–4 ke notes, aur kuch core files dobara padh ke (list: Part 0.3). Is step mein jo baatein **nayi mili** hain, unpe **(naya)** likha hai. Inme se koi bhi run karke verify nahi hui, sirf code padh ke mili hain. Attribution ("I" vs "we") ke liye Part 0.2 dekho.

5.1 Key design decisions aur unke trade-offs

Har decision ke liye 4 cheezein: **kya decide kiya**, **code mein kahan dikhta hai**, **fayda**, **nuksaan (trade-off)**. Aakhir mein ek English line hai jo interview mein bol sakte ho.

> Code mein kahin likha nahi hai ki koi decision *kyun* liya gaya. "Fayda" wale points standard engineering reasons hain jo code ke design se match karte hain. Unhe apne words mein confirm kar lena.

D1. Ek backend (monolith) + 4 alag React apps

- **Kya:** customer, seller, admin, rider ke liye 4 alag Vite apps (`FrontEnd/` , `Seller-Panel/` , `Admin-Panel/` , `delivery-dashboard/`), aur sab ek hi Express backend (`Backend/app.js`) se baat karte hain.
- **Code mein:** `app.js` ka mount table (`/users` , `/api/seller` , `/api/admin` , `/api/delivery` …) aur backend ke panel-wise folders (`routes-seller-panel/` , `controllersAdminPanel/` , …).
- **Fayda:**
  - Har role ka bundle chhota rehta hai. Customer ko admin ka code download nahi karna padta.
  - Alag team alag app pe kaam kar sakti hai (folder names isi taraf ishara karte hain).
  - Backend ek hai, to ek hi deploy, ek hi DB, aur "rider ne status badla → customer ko dikha" bina message queue ke ho jaata hai.
- **Nuksaan:**
  - Common code chaaron apps mein copy hai: auth handling, axios setup, `BackButton` , toasts.
  - Backend mein duplicates ban gaye: do `AppError` (`utils/` aur `utilsSellerPanel/`), do `statsService.js` , purana `Middleware/sellerAuth.js` (dead).
  - Monolith ka ek hissa crash ho (jaise Razorpay SDK bina key ke boot pe crash karta hai), to poora backend gir jaata hai.
- **Interview line:** "*I kept one backend so all four apps share the same data and business rules, and split the frontends by role so each bundle stays small and teams could work independently.*"

D2. MongoDB + Mongoose

- **Code mein:** `Backend/models/` ke 17 schemas, `db/db.js` (`maxPoolSize: 50`).
- **Fayda (code se saboot ke saath):**
  - Flexible data: `Product.badges` (8 booleans), embedded `reviews[]` , `Order.products[]` .
  - **Geo queries built-in:** `DeliveryPartner.currentLocation` pe `2dsphere` index aur `$geoNear` (`deliveryBroadcast.service.js`). SQL mein iske liye PostGIS lagta.
  - **TTL index:** `order.model.js:139` `{ expireAt: 1 }, { expireAfterSeconds: 0 }` . Unpaid orders bina cron ke delete ho jaate hain.

    - Aggregation pipeline se listing, analytics aur stats ban jaate hain.
- **Nuksaan:**
    - Orders aur payments jaise "paisa wale" data ke liye SQL ke foreign keys, constraints aur joins zyada strong hote hain. Yahan wo guarantees code ko khud deni padti hain (aur kai jagah nahi di gayi, jaise Cart pe unique index nahi).
    - Transactions ke liye **replica set** chahiye (Atlas pe default hai; local pe alag setup karna pada tha).
- **Interview line:** "*MongoDB fit well because I needed geospatial queries for riders, TTL cleanup for unpaid orders, and flexible documents for products. The honest trade-off is that for payments a relational database gives stronger constraints out of the box.*"

D3. Order ke andar data ki copy (snapshot / denormalization)

- **Kya:** order banate waqt har item mein `productName`, `sellerName`, `storeName`, `price` copy karke save hote hain (`order.model.js` ka `products[]`).
- **Fayda:** product ka price ya naam baad mein badle, tab bhi purana order waisa hi dikhta hai jaisa khareedte waqt tha. Invoice aur history sahi rehti hai.
- **Nuksaan:** data do jagah hota hai. Store ka naam badle to purane orders mein purana naam rahega. Order history ke liye ye sahi behaviour hai, isliye yahan ye nuksaan nahi, jaan-boojh ke liya gaya decision hai.
- **Interview line:** "*I snapshot the price and product name into each order item, so later price changes never rewrite order history.*"

D4. Item-level status + har seller ke liye alag delivery task

- **Kya:**
    - `Order.products[].status` har item ka apna status hai (`Pending / Dispatched / Shipped / Out for Delivery / Delivered / Cancelled`).
    - `createDeliveryTasksForOrder()` (`services/deliveryTask.service.js`) order ke items ko seller ke hisaab se group karta hai, aur **har seller ke liye ek** `DeliveryTask` banata hai:

```
order.products.forEach((item) => {
  const SID = item.sellerId.toString();
  if (!sellerGroups[SID]) sellerGroups[SID] = true;
});
for (const sellerId of Object.keys(sellerGroups)) {
  await createDeliveryTaskForSeller(order, sellerId);
}
```

- **Kyun:** ek order mein 2 sellers ke products ho sakte hain jo do alag shehron se pick honge. Ek rider dono jagah nahi jaayega. Isliye pickup alag, task alag, aur item status bhi alag.
- **Order-level status kaise banta hai:** `syncOrderStatusFromTask()` (`delivery.controller.js:285`) us seller ke items update karta hai, phir saare items dekh ke order ka `deliveryStatus` nikaalta hai:

```
if (statuses.every((s) => s === "Delivered")) order.deliveryStatus = "Delivered";
else if (statuses.some((s) => s === "Out for Delivery")) order.deliveryStatus = "Out for Delivery";
// ... Shipped, Dispatched, Cancelled, Pending
```

- **Nuksaan:**
    - Customer ka "cancel order" poora order cancel karta hai, item-wise cancel nahi.
    - Order cancel hone pe delivery task cancel nahi hota (Step 4.9.8).
    - `DeliveryTask.items[].status` kabhi update nahi hota, sirf task ka `status` update hota hai.

- **Interview line:** "*Because one cart can contain items from several sellers, I track status per item and create one delivery task per seller, then derive the overall order status from the items.*"

D5. Price hamesha server pe calculate hota hai, aur fees customer se chhupi rehti hain

- **Kya:**
  - Product mein seller ka `sellingPrice` + `platformFee` + `logisticsFee` (60) + `transactionFee` (10) store hote hain.
  - Customer ko sirf `offeredPrice` dikhta hai: `(sellingPrice + fees) × (1 − discount%)`, phir `roundToPriceEndingIn9()` (`utils/productForClient.js`):

```
function roundToPriceEndingIn9(price) {
  const p = Math.round(price);
  const base = Math.floor(p / 10) * 10;
  const low = base - 1, high = base + 9;
  return p - low <= high - p ? low : high;   // 742 → 739, 748 → 749
}
```

  - Order ke waqt `buildPaymentOrderItems` price **DB se** nikalta hai, aur `resolveOrderPricing` GST (5%) aur coupon khud lagata hai. Client ka bheja `price` ignore hota hai.
- **Fayda:** koi DevTools/Postman se price nahi badal sakta. Business ke internal fees private rehte hain (`toClientProduct` unhe response se hata deta hai).
- **Nuksaan (important):** price ka formula **3 jagah** likha hai:
  1. JavaScript mein `toClientProduct()` (cart, checkout, order)
  2. MongoDB aggregation mein (`product.routes.js:212` ke aas-paas `rawOfferedPriceExpr`, `$floor`, `$cond`), kyunki listing mein price se sort aur filter DB mein karna tha
  3. Frontend mein display ke liye (`FrontEnd/src/utils/pricingConstants.js` GST ki copy)
  - Formula badla aur ek jagah update karna bhool gaye, to listing aur checkout mein alag price dikhega.
  - **(naya) Ek chhota edge case:** JS ka `Math.round(744.5)` 745 deta hai, jabki MongoDB ka `$round` "round half to even" use karta hai aur 744 deta hai. Agar raw price theek `xx4.5` pe aaye, to listing 739 dikha sakti hai aur checkout 749. Ye rare hai aur run karke verify nahi kiya, par "logic do jagah = drift ka risk" ka achha example hai.
  - `/home/products` `toClientProduct` use nahi karta, isliye wahan fees leak hoti hain (verified, Step 4.14.3).
- **Behtar design:** product save hote waqt `offeredPrice` ek baar calculate karke DB mein field ki tarah store karo (index bhi lag jaayega). Phir formula sirf ek jagah rahega.
- **Interview line:** "*The client never decides the price. The server re-reads every product from the database and computes GST and coupons itself. The trade-off was that the price formula ended up in both JavaScript and the aggregation pipeline, so today I'd store the computed price on the product instead.*"

D6. Razorpay: pehle Pending order, phir verify + webhook, aur TTL cleanup

- **Kya (`routes/payment.routes.js`):**
  1. `POST /create-order` (line 96): Razorpay order banao, aur DB mein `paymentStatus: 'Pending'` order save karo, `expireAt = now + 30 min` ke saath.
  2. `POST /verify` (line 185): HMAC-SHA256 signature check, phir order Paid.
  3. `POST /webhook` (line ~331): agar user ne tab band kar diya aur verify nahi chala, to Razorpay khud server ko batata hai.
  4. Pay nahi kiya → MongoDB TTL index 30 min baad Pending order delete kar deta hai.

- **Idempotency:** `markRazorpayOrderPaid()` (line 50) pehle check karta hai ki order already Paid to nahi:

```
async function markRazorpayOrderPaid(order, razorpayPaymentId) {
  if (order.paymentStatus === 'Paid') return false;   // dobara kaam mat karo
  order.paymentStatus = 'Paid';
  order.razorpayPaymentId = razorpayPaymentId;
  order.expireAt = undefined;                          // TTL hatao, warna paid order bhi delete hota
  await order.save();
  await applyCouponUsageIfNeeded(order);
  return true;
}
```

- **Fayda:** cleanup ke liye cron nahi likhna pada. Verify aur webhook dono raaste hain, to payment "kho" nahi jaata.
- **Nuksaan:**
  - Ye check memory mein padhe gaye document pe hai, **atomic nahi hai**. Verify aur webhook bilkul ek saath aayein to dono "Pending" dekh sakte hain aur dono delivery tasks bana sakte hain (Step 4.9.8).
  - `/verify` ka "order nahi mila to naya bana do" wala branch paid amount match nahi karta (Step 4.9.8, security issue).
  - Refund, `Failed` / `Refunded` status nahi hain.
- **Interview line:** "*I create a pending order before payment, confirm it either through the signature check or the webhook, and let a TTL index delete abandoned orders after 30 minutes, so I didn't need a cleanup job.*"

D7. Stateless JWT, har role ka alag middleware

- **Kya:** 4 login APIs, 4 middlewares (`protect` , `protectSeller` , `protectAdmin` , `protectDelivery`). Ek hi `JWT_SECRET` , 30 din expiry, token `localStorage` mein.
- **Fayda:** server pe session store (Redis) nahi chahiye. Har middleware DB se fresh user laata hai, isliye blocked seller (`isBlocked` → 403) ya deactivated rider (`isActive` → 403) turant ruk jaata hai.
- **Nuksaan:**
  - Logout sirf client pe hota hai. Token 30 din tak valid rehta hai, password badalne ke baad bhi.
  - Sirf rider ke token mein `role` hai. Baaki roles alag isliye rehte hain kyunki har middleware alag collection mein ID dhoondhta hai. Ye kaam karta hai par kamzor design hai.
  - Customer token mein `_id` , baaki mein `id` . Isi wajah se seller aur admin ki notifications kabhi nahi padhi ja sakti (Step 4.13.2).
  - localStorage XSS ke saamne kamzor hai.
- **Interview line:** "*I used stateless JWTs so four different clients could share one backend without a session store. The trade-off is revocation, so the next step is short-lived access tokens with a refresh token in an httpOnly cookie, and a role claim in every token.*"

D8. Rider dispatch: geo query + Socket.io push + "jo pehle accept kare"

- **Kya:**
  - `broadcastToNearbyRiders()` (`deliveryBroadcast.service.js`): `$geoNear` se 5 km (`RADIUS_METERS = 5000`) ke andar ke max 10 (`MAX_PARTNERS`) active + available riders, aur har ek ke socket pe `newDeliveryRequest` .
  - `acceptTask()` (`delivery.controller.js:229`): transaction + do conditional updates. Pehla rider jeet-ta hai.
  - 15 second (`ASSIGNMENT_TIMEOUT_MS = 15000`) mein koi accept na kare, to `assignDefaultRider()` task test rider `delivery@test.com` ko de deta hai.

- **Fayda:** Uber/Swiggy jaisa model. Rider ko poll nahi karna padta, aur ek task do riders ko kabhi nahi milta.
- **Nuksaan:**
  - `global.riderToSocket` ek in-memory `Map` hai, aur 15 second ka timer `setTimeout` hai. Server restart ya 2 instances pe dono toot jaate hain.
  - Pickup/drop coordinates placeholder hain (`PLACEHOLDER_COORDS = [77.209, 28.613]`), isliye 5 km wala logic abhi asli nahi hai.
  - Socket pe auth nahi hai (`registerRider` mein koi bhi riderId).
  - Reject ya timeout ke baad kisi doosre rider ko dobara offer nahi jaata, seedha default rider ko jaata hai.
- **Interview line:** "*I push each task over Socket.io to the nearest available riders found with a geospatial query, and the first rider to accept wins, guaranteed by a conditional update inside a transaction.*"

D9. Har product admin approve karta hai (quality gate)

- **Kya:** product `approvalStatus: 'pending'` ke saath banta hai (`addProduct.routes.js`). Customer ko sirf approved dikhte hain, jaise `home.services.js:7` mein:

```
const ACTIVE_PRODUCT_FILTER = { approvalStatus: "approved", isDeleted: false, inStock: { $gt: 0 } };
```

- **Fayda:** handmade marketplace mein quality aur fake listings pe control.
- **Nuksaan:** admin bottleneck ban sakta hai. Aur approve hone ke baad seller naam, description, images badal sakta hai bina re-approval ke (`PATCH /api/productinfo/:id/details`), to gate bypass ho jaata hai. Seller ka khud ka approval (`seller.status`) karne ka koi endpoint hi nahi hai.

D10. Product ka soft delete

- **Kya:** seller delete kare to `isDeleted: true` + `deletedAt` (`productInfo.routes.js`). Product DB mein rehta hai.
- **Fayda:** purane orders, reviews aur invoices product ko refer karte hain, wo nahi toot-te.
- **Nuksaan:** har query mein `isDeleted: false` yaad rakhna padta hai. Aur admin ka `DELETE /api/admin/products/:id` **hard delete** karta hai, jo isi design ke khilaaf hai (inconsistency).

D11. Media S3 + CloudFront pe, Cloudinary jaise "shape" ke saath

- **Kya:** `utils/s3Upload.js` `{ secure_url, key }` return karta hai, bilkul Cloudinary jaisa. `scripts/migrateCloudinaryToS3.js` purani files move karta hai (`--dry-run` ke saath).
- **Fayda:** provider badla par purana code (`result.secure_url`) bina badle chala (Adapter pattern). Multer `memoryStorage` se buffer seedha S3 jaata hai, disk pe temp file nahi.
- **Nuksaan:** zyaadatar multer configs mein size limit nahi hai, aur memory storage ke saath badi file RAM bhar sakti hai. KYC documents public CDN URL pe hain. Order-rating photos abhi bhi local disk pe jaati hain.

D12. Validation: Zod (naye routes) + express-validator (purane routes)

- **Fayda (Zod):** `zodValidate(schema, 'body')` ek factory hai, aur Zod unknown keys strip kar deta hai, jo mass assignment se bachata hai (jaise `registerPartner` mein client `isActive: true` bheje to hat jaata hai).
- **Nuksaan:** do libraries, aur kai routes pe validation hai hi nahi (cart, order, payment, admin login, contact). express-validator ke kuch rules lagaye gaye par unka result kabhi check nahi hota (customer forgot/reset password). Express 5 mein `req.query = parsed.data` chupchaap ignore hota hai (verified).

D13. Caching: sirf home page ka anonymous data, process ki memory mein

- **Kya:** `withTtlCache()` (`home.services.js:36`), TTL 2–5 min, sirf tags, categories, top sellers aur popular products ke liye. Personalised recommendations cache nahi hoti.
- **Fayda:** sabse zyada hit hone wale page ke repeat DB queries bach jaate hain, bina Redis ke.
- **Nuksaan:** invalidation nahi (approve kiya product 2–5 min baad dikhega), har instance ki alag cache, aur cache stampede possible.

D14. Background kaam usi process mein: cron, timer, email

- **Kya:** `jobs/badgeCron.js` (roz 00:00 UTC, `bulkWrite` se saare badges), `setTimeout` wala rider fallback, aur COD order ki email bina `await` ke (fire-and-forget).
- **Fayda:** koi alag worker, queue ya infra nahi chahiye. Chhote project ke liye simple.
- **Nuksaan:** 2 instances pe cron do baar chalega. Restart pe timer kho jaata hai. Email fail ho to retry nahi hota.

D15. Frontend: Redux Toolkit + service layer, server hi "source of truth"

- **Kya:** `xyzService.js` (sirf axios) + `xyzSlice.js` (`createAsyncThunk` + reducers). Cart har response ke baad backend wale cart se **poora replace** hota hai (`cartSlice.js`). Admin panel mein Redux nahi, sirf `useState`. Listing ke filters URL query mein (`ShopM.jsx`).
- **Fayda:** cart hamesha server se match karta hai. Filters wala URL share karo to wahi page khulta hai. Admin ka data kahin share nahi hota, isliye wahan Redux overkill hota.
- **Nuksaan:** RTK Query/React Query nahi, isliye caching aur dedupe nahi (home pe duplicate requests). Optimistic update nahi (sirf admin badges mein hai), isliye UI thoda slow lagta hai.

D16. Recommendations: rule-based tag scoring (ML nahi)

- **Kya:** `getPersonalisedProducts()` (`home.services.js:234`) teen signals se tags ko score deta hai: wishlist (1.2 × 1.2 = 1.44), orders (1.0), cart (0.8). Top 20 tags se products laata hai.
- **Fayda:** samajhne aur explain karne mein asaan, koi training data nahi chahiye, aur naye user ke liye "popular" fallback hai.
- **Nuksaan:** sirf tags pe chalta hai, quality tags pe depend karti hai. **(naya)** `getOrderSignals` sirf `paymentStatus: "Paid"` orders padhta hai, aur COD orders kabhi "Paid" nahi hote (poore backend mein koi code COD ko delivery pe Paid mark nahi karta). Isliye sirf COD se khareedne wale customer ki order history recommendations mein count hi nahi hoti.

5.2 Sabse mushkil 5 technical problems ("the hardest problem I solved")

Har problem ke liye: **problem simple bhasha mein**, **asli code**, **narration** (English, ~60–90 second, interview mein bolne ke liye), **follow-up ke liye taiyaari**, aur **honest caveat**.

> Narration sirf usi problem ki bolo jo tumne khud solve ki ho, ya jisme tumhara hissa ho. Nahi to "we" use karo.

H1. Do riders ek hi task accept na kar paayein (race condition)

Problem simple bhasha mein:

- Ek naya order aaya, 10 riders ko popup gaya. Do riders ne same second mein "Accept" dabaya.
- Naive code aisa hota: pehle task padho, `if (task.status === "Pending")` check karo, phir save karo. Dono requests ek saath "Pending" padh lengi, aur dono save kar dengi. Ek task, do riders.

- Doosri problem: ek rider ek waqt pe ek hi delivery le sakta hai. Rider ko "busy" mark kiya, aur task kisi aur ko mil chuka tha, to rider galti se busy reh jaayega.

Asli code (`delivery.controller.js:229`, chhota kiya hua):

```
session = await mongoose.startSession();
session.startTransaction();
// 1. Rider lock: sirf tab update hoga jab rider abhi free ho
const lockedRider = await DeliveryPartner.findOneAndUpdate(
  { _id: rider._id, isAvailable: true }, { isAvailable: false }, { new: true, session });
if (!lockedRider) throw new AppError("You already have an active delivery", 400);
// 2. Task claim: sirf tab update hoga jab task abhi bhi Pending ho
const task = await DeliveryTask.findOneAndUpdate(
  { _id: taskId, status: "Pending" },
  { status: "Accepted", deliveryPartner: req.user._id, $push: { attempts: {...} } },
  { new: true, session });
if (!task) throw new AppError("Task already assigned or not available", 400);
await session.commitTransaction();
// catch: await session.abortTransaction()  → rider ka lock bhi wapas
```

- **Trick:** condition (`status: "Pending"`) filter ke **andar** hai. MongoDB ek document ka update atomic karta hai, isliye doosre rider ki query ko koi document match hi nahi karta aur `null` milta hai.
- **Transaction kyun:** step 1 ho gaya aur step 2 fail hua, to `abortTransaction()` step 1 ko bhi undo kar deta hai. Rider busy nahi phansta.

Narration (English):

> "The hardest problem in the delivery module was a race condition. When a new order comes in, up to ten nearby riders get a popup at the same time, so two of them can tap Accept within milliseconds. A simple read-check-save would let both of them get the same task. I solved it in two layers. First, I moved the check into the update itself: `findOneAndUpdate` with `status: 'Pending'` in the filter, so MongoDB's single-document atomicity guarantees only one request matches and the other gets null. Second, a rider should only have one active delivery, so I also lock the rider with a conditional update on `isAvailable: true`. Because those are two documents, I wrapped both in a MongoDB transaction, so if the task was already taken, the rider's lock is rolled back too. I tested it by sending two accept requests for the same task, and the second one got a 400 'Task already assigned'. One thing I learned is that transactions need a replica set, so I had to run MongoDB as a single-node replica set locally."

Follow-up ke liye taiyaari:

- "*Transaction ke bina kaam chalta?*" Task claim akela atomic hai, wo chal jaata. Transaction rider lock ko rollback karne ke liye hai.
- "*Optimistic vs pessimistic locking?*" Ye optimistic style hai: lock pehle se nahi lete, update ke waqt condition check karte hain.
- "*Standalone MongoDB pe kya hoga?*" `startTransaction` ke baad pehli query error degi, kyunki transactions sirf replica set ya mongos pe chalte hain.

Honest caveat:

- Local run pe test hua tha ("doosra accept reject", Step 4 ka verified flow). Automated test nahi hai.
- Narration ki line "I tested it by sending two accept requests" tabhi bolo jab tumne khud ye kiya ho. Is revision mein ye test 2026-10-10 ke local smoke run mein hua tha. Original developer ne kaise test kiya, ye code se pata nahi chalta.
- **(naya)** `assignDefaultRider()` (`deliveryBroadcast.service.js`) is pattern ko follow **nahi** karta. Wo `findById` → `if (task.status !== "Pending") return` → `task.save()` karta hai, conditional update nahi.

Agar 15 second wala timer aur kisi rider ka accept bilkul same moment pe chalein, to default rider wala save rider ke accept ko overwrite kar sakta hai. (Code padh ke; Mongoose ka exact behaviour test karke confirm karna padega.) Fix: `findOneAndUpdate({ _id: taskId, status: "Pending" }, {...})`.

- **(naya)** `assignDefaultRider` default rider ka `isAvailable` check ya set nahi karta, isliye test rider ko unlimited tasks mil sakte hain.

H2. Payment kabhi "khoye" nahi, aur nakli payment accept na ho

Problem simple bhasha mein:

- Razorpay mein paisa browser mein katta hai, par hamara server ko kaise pata chale ki sach mein kata?
- Teen mushkil cases:
  1. Koi nakli "payment success" request bheje.
  2. User pay kare aur turant tab band kar de, to browser `/verify` call hi nahi karega.
  3. User popup khole aur pay hi na kare, to DB mein kachra (Pending orders) jama hoga.

Solution aur code:

1. **Nakli payment:** `/verify` mein HMAC signature:

   ```
   const expectedSignature = crypto.createHmac('sha256', secret)
     .update(`${razorpay_order_id}|${razorpay_payment_id}`).digest('hex');
   if (expectedSignature !== razorpay_signature) return 400;
   ```

   Secret sirf server ke paas hai, isliye client sahi signature bana hi nahi sakta.
2. **Tab band:** Razorpay ka webhook (`payment.captured`). Iska signature **raw body** pe banta hai, isliye `app.js:60–62` mein sirf is URL ke liye JSON parser skip hota hai:

   ```
   // Bypass the JSON parser ONLY for the webhook route
   if (req.originalUrl === '/api/payment/webhook') { next(); } else { express.json()(req, res, next); }
   ```

   aur route `express.raw({ type: "application/json" })` lagata hai.
3. **Kachra:** `expireAt = now + 30 min` + TTL index (`order.model.js:139`). Paid hone pe `expireAt = undefined`.
4. **Dono (verify + webhook) chalein:** `markRazorpayOrderPaid` already-Paid pe `false` lautata hai, aur `applyCouponUsageIfNeeded` pehle `CouponUsage.findOne({ orderId })` check karta hai, taaki coupon do baar count na ho.

Narration (English):

> "Another tricky part was making online payments reliable. The browser can't be trusted, so after Razorpay returns the payment ID and signature, my server recomputes an HMAC-SHA256 of order ID and payment ID with the secret key and only then marks the order paid. But if the user closes the tab right after paying, verify never runs, so I also handle Razorpay's `payment.captured` webhook. That one is signed over the raw request body, so I had to skip the JSON parser just for that route and use `express.raw`. Since both paths can fire, I made the 'mark as paid' step idempotent: if the order is already paid it does nothing, and coupon usage is only recorded once per order. And for users who open checkout but never pay, I create the pending order with an `expireAt` 30 minutes ahead and a TTL index, so MongoDB deletes abandoned orders on its own."

Follow-up ke liye taiyaari:

- "*TTL exactly 30 min pe delete karta hai?*" Nahi, MongoDB ka TTL monitor ~60 second mein ek baar chalta hai, to thodi der ho sakti hai.
- "*Agar TTL ne order delete kar diya aur phir payment aaya?*" `/verify` order na mile to cart/body se naya Paid order bana deta hai (recovery). Isme amount match nahi hota, ye ek known gap hai.
- "*Webhook hamesha 200 kyun?*" Non-2xx pe Razorpay retry karta hai.

Honest caveat: "already Paid" check atomic nahi hai (verify + webhook ek saath = double delivery tasks possible). Fix: `Order.findOneAndUpdate({ _id, paymentStatus: 'Pending' }, { $set: { paymentStatus: 'Paid' }, $unset: { expireAt: 1 } })`, wahi pattern jo H1 mein hai. Signature compare `!==` se hai; `crypto.timingSafeEqual` behtar hai. Refund nahi hai. Local pe Razorpay keys fake thi, to ye flow run nahi kiya gaya, sirf code padha gaya.

H3. Chhupi fees ke saath price pe filter, sort aur pagination

Problem simple bhasha mein:

- DB mein `offeredPrice` field hai hi nahi. Wo `sellingPrice + fees`, discount aur "9 pe khatam" rounding se banta hai.
- Customer kehta hai "₹500–₹1000 ke products, sasta pehle, page 3". Agar saare products JS mein laa ke price nikaalo, phir sort karo, to hazaaron products pe bahut slow hoga.
- Saath mein total count bhi chahiye (pagination ke "Page 3 of 12" ke liye).

Solution (`routes/product.routes.js`, `GET /products`, line 56 se):

1. `$match`: approved + not deleted + category/tag filters (user input `escapeRegex()` se escape hota hai, line 18).
2. `$addFields`: `rawOfferedPrice` (line 212), phir `base`, phir `offeredPrice` (rounding to 9) **MongoDB expressions** mein (`$floor`, `$divide`, `$cond`).
3. `$match` on `offeredPrice` (min/max price).
4. `$facet` (line 289): ek hi query mein do results:

```
$facet: {
  count:    [{ $count: "total" }],
  products: [sortStage, { $skip: skip }, { $limit: limitNum }, projectOutFeesAndTemp],
}
```

5. `projectOutFeesAndTemp` fees aur temp fields hata deta hai, taaki customer ko fees na dikhein.
6. `limit` max 50 tak cap hota hai.

Narration (English):

> "On the listing page, customers filter and sort by the final price, but that price doesn't exist in the database. It's computed from the seller's price plus platform, logistics and transaction fees, minus discount, rounded to end in 9. Loading everything into Node to compute and sort would not scale. So I moved the formula into the aggregation pipeline: `$addFields` computes the offered price, then I match on the price range, and a `$facet` stage returns both the total count and the current page in a single query. A final `$project` strips the internal fees so they never reach the client. That gave me server-side filtering, sorting and pagination with one round trip, and I capped the page size at 50 so nobody can request everything at once."

Follow-up: "*Ye formula do jagah hai, to?*" Haan, JS (`toClientProduct`) aur pipeline dono mein. Isse drift ka risk hai (D5 wala `$round` edge case). Fix: `offeredPrice` write time pe store karo aur us pe index lagao.

Honest caveat:

- `store` filter wale pipeline mein `$lookup` (sellers join) pagination se **pehle** hota hai, to har matching product pe join hota hai (Step 4.15.2).
- Regex filters index ka poora fayda nahi lete.
- `$skip` badi pages pe slow hota hai; bahut data pe cursor-based pagination behtar hai.

H4. Ek order, kai sellers: status ko sync rakhna

Problem simple bhasha mein:

- Order mein Seller A ka item Delhi se aur Seller B ka item Jaipur se. Do alag riders, do alag timelines.
- Rider A "Delivered" kare to customer ko dikhna chahiye ki ek item aa gaya, ek abhi raste mein hai. Poora order tabhi "Delivered" jab saare items delivered hon.
- Aur ye update aadha nahi hona chahiye: task Delivered ho gaya par order update fail, ye nahi chalega.

Solution:

- Har seller ka alag `DeliveryTask` (D4).
- `updateTaskStatus()` (`delivery.controller.js:316`) transaction mein:

```
const allowedTransitions = {
  Assigned: ["PickedUp"], Accepted: ["PickedUp"],
  PickedUp: ["OutForDelivery"], OutForDelivery: ["Delivered"],
};
if (allowedTransitions[task.status] && !allowedTransitions[task.status].includes(status))
  throw new AppError("Invalid status transition", 400);
task.status = status;
if (status === "PickedUp")  await syncOrderStatusFromTask(task.order, task.seller, "Out for Delivery", session);
if (status === "Delivered") {
  await syncOrderStatusFromTask(task.order, task.seller, "Delivered", session);
  await DeliveryPartner.findByIdAndUpdate(req.user._id,
    { isAvailable: true, $inc: { "stats.totalDeliveries": 1 } }, { session });
}
await task.save({ session }); await session.commitTransaction();
```

- `syncOrderStatusFromTask` sirf usi seller ke items badalta hai, phir order-level status saare items se derive karta hai.

Narration (English):

> "Orders can contain items from several sellers, so I had to keep delivery and order status consistent across them. Each seller's items become a separate delivery task. When a rider moves a task forward, a small state machine only allows valid steps, like PickedUp to OutForDelivery to Delivered. In the same transaction I update only that seller's items in the order and then recompute the overall order status from all items: it's Delivered only when every item is delivered. When a task is delivered, the rider becomes available again and their delivery count goes up. Because the task, the order and the rider are three documents, all of that runs in one transaction, so the customer never sees a half-updated state."

Honest caveat:

- Task `PickedUp` hote hi item seedha "Out for Delivery" ban jaata hai, "Shipped" step skip hota hai (verified, Step 4).
- **(naya) State machine mein ek chhed:** `allowedTransitions` mein `Delivered` , `Cancelled` , `Failed` ki keys hi nahi hain. Code `if (allowedTransitions[task.status] && ...)` hai, to inme se kisi state pe check **skip** ho jaata hai. Matlab Delivered task pe rider dobara `PickedUp` bhej sakta hai, aur order item wapas "Out for Delivery" ban jaayega. Fix: unknown/final state pe default reject (`const allowed = allowedTransitions[task.status] || []; if (!allowed.includes(status)) throw ...`).

- **(naya)** Isi wajah se Zod schema `Failed` aur `Cancelled` allow karta hai, par active states (Accepted, PickedUp…) se wahan pahunchne ka raasta hi nahi hai. `Failed` normal flow mein unreachable hai.
- Seller ka `PUT /api/orders/seller/item-status` is state machine ko follow nahi karta (seller seedha "Delivered" kar sakta hai).

H5. Home page ki personalised feed, fast bhi

Problem simple bhasha mein:

- Logged-in user ko uski pasand ke products dikhane hain, logged-out user ko popular products. Aur home page sabse zyada khulta hai, to fast hona chahiye.

Solution (`services/home.services.js`):

- `GET /home/products` pe `optionalAuth` hai: token ho to user milta hai, na ho to bhi request chalti hai.
- `getPersonalisedProducts()` (line 234):

```
const [wishlistSignals, orderSignals, cartSignals] = await Promise.all([   // 3 queries parallel
  getWishlistSignals(userId), getOrderSignals(userId), getCartSignals(userId) ]);
const scoreMap = buildTagScoreMap([
  { tags: wishlistSignals.tags, weight: WEIGHTS.WISHLIST, multiplier: 1.2 },  // 1.2 × 1.2
  { tags: orderSignals.tags,    weight: WEIGHTS.ORDER },                       // 1.0
  { tags: cartSignals.tags,     weight: WEIGHTS.CART },                        // 0.8
]);
const rankedTags = topTagsByScore(scoreMap);          // top 20 tags
```

  Phir in tags wale active products, jo cart ya pichle orders mein pehle se nahi hain, aur jinka `sellingPrice` user ke average ka 2 guna se zyada nahi. Kuch na mile to `null`, aur controller popular products pe fallback karta hai.
- Popular products, tags, categories, top sellers sabke liye same hain, isliye `withTtlCache` (2–5 min) mein cache hote hain.

Narration (English):

> "For the home feed I built a simple, explainable recommender. For a logged-in user I fetch three signals in parallel with `Promise.all`: wishlist, past orders and cart. Each product tag gets a weighted score, with wishlist weighted highest, and I take the top 20 tags. Then I query active products with those tags, exclude things already in the cart or already bought, and keep prices within about twice the user's average spend. If there's no signal, like a brand-new user, it falls back to popular products. Data that's the same for everyone, such as popular products and categories, sits in a small in-memory cache with a two-to-five-minute TTL, so the busiest page doesn't hit the database on every request."

Honest caveat: in-memory cache multi-instance pe share nahi hoti aur invalidate nahi hoti. **(naya)** COD orders `paymentStatus: "Pending"` hi rehte hain, aur `getOrderSignals` sirf `"Paid"` padhta hai, to COD order history signal nahi banti. ML nahi hai, ye rule-based hai.

> **Backup story (agar interviewer kuch aur pooche):** Cloudinary → S3 migration. `scripts/migrateCloudinaryToS3.js` DB ke Cloudinary URLs download karke S3 pe daalta hai, `--dry-run` support karta hai, failures report file mein likhta hai, aur `uploadToS3` Cloudinary jaisa `{ secure_url }` lautata hai taaki baaki code na badle. Ye Step 6 ke behavioral section mein bhi use hua hai.

5.3 Kamiyan, bugs, aur "what would you do differently?"

Saare bugs ki detail Step 4.14 aur "Step 3 + 4 ke naye findings" list mein hai. Yahan unhe **priority** ke hisaab se lagaya gaya hai, saath mein Step 5 ke naye findings.

A. Security (sabse pehle fix karne wale)

#	Problem	Kahan	Fix ek line mein
1	Customer password reset bina OTP proof ke, aur `{"$ne": null}` se kisi ka bhi	`user.controllers.js` (`resetPasswordController`)	verify-otp ek short-lived reset token de; reset sirf wahi token le; `typeof email === 'string'`
2	COD order mein `paymentStatus` client se	`order.routes.js:191–193`	COD route mein `paymentStatus: 'Pending'` server pe hardcode
3	Seller profile update mass assignment (`$set: req.body`)	`sellerPanelController.js`	Zod allow-list schema
4	Bina auth ke routes: saare orders (verified), order by id, invoice, seller dashboard, contact, admin rider routes, blog create	`order.routes.js` , `invoice.routes.js` , `dashboard.routes.js` , `contactRoutes.js` , `admin.routes.js:59`	sahi middleware + ownership check
5	Default admin `admin@test.com/admin123` public route se ban sakta hai	`contact.controller.js`	ye code hatao; admin sirf seed script se
6	Login/register response mein password hash	`user.controllers.js`	response se `password` hatao
7	`/verify` rebuild branch amount check nahi karta	`payment.routes.js`	`razorpay.payments.fetch()` se amount match, ya branch hatao
8	Rate limit, helmet, CORS whitelist nahi; socket pe auth nahi	`app.js` , `server.js`	`express-rate-limit` , `helmet()` , `cors({ origin: [...] })` , `io.use()` JWT check

B. Logic / correctness bugs

- Stock check nahi aur stock kam nahi hota (out-of-stock product ka order verified).
- Order cancel → delivery task cancel nahi, refund nahi.
- Verify + webhook race (H2), `assignDefaultRider` non-atomic **(naya)**, state machine final states pe check skip karta hai **(naya)**.
- COD orders kabhi "Paid" nahi hote **(naya)**, jiska asar recommendations pe padta hai. Admin sales analytics unpaid orders bhi ginta hai aur saare saalon ke same mahine mila deta hai.
- Product edit ke baad re-approval nahi. MRP formula inflated (`mrp = base × (1 + d%)` , sahi `base / (1 − d%)`).
- Seller/admin notifications kabhi padhi nahi ja sakti (`_id` vs `id`).
- Rider login active task ke saath bhi `isAvailable = true` kar deta hai.
- Invoice pe "₹undefined" (verified), `GET /api/orders/seller/orders` hamesha 500 (verified).

C. Code quality aur architecture

- Business logic route files mein: `product.routes.js` (749 lines), `order.routes.js` (697). Controller/service layer nahi.
- 3 error-handling styles, 2 validation libraries, response shape har API ka alag (`{error}` , `{message}` , `{success, data}`).
- Duplicates aur dead code: do `AppError` , `seller.routes.js` (mount nahi + syntax bug), `Newsletter.routes.js` , `Middleware/sellerAuth.js` , frontend ke `data/` , `seo.js` , `BlogManager/` .
- **Zero tests**, CI nahi, backend pe linter nahi, TypeScript nahi.
- Bahut saare `console.log` , kuch mein token aur OTP.
- Missing indexes: `Order.user` , `Order.products.sellerId` , `Order.razorpayOrderId` , `Cart.user` (unique).

D. Frontend

- Code splitting nahi (`React.lazy` zero), `FrontEnd/src/Images` ~69 MB.
- Route guard component nahi (har page ka apna `useEffect`), 404 page nahi, app-level ErrorBoundary nahi.
- Seller-Panel mein `<Toaster/>` mount nahi (toasts dikhte hi nahi), nested Layout se sidebar do baar.
- Duplicate API calls, RTK Query nahi. Hardcoded `localhost` URLs (Admin ka `localhost:5001`).
- `ChangePassword.jsx` render pe crash.

E. Adhoore features (stubs) — inhe "done" mat bolna

- Geocoding nahi: coordinates placeholder `[77.209, 28.613]` , isliye 5 km matching aur live tracking asli nahi.
- Delivery OTP field hai, use nahi hota.
- Seller approve karne ka endpoint nahi. Coupon banane ka admin UI nahi.
- Refunds, returns, seller payouts nahi.
- Customer aur seller ke liye real-time updates nahi.

"What would you do differently?" — bolne wala answer (English)

> "Looking back, I'd change four things. First, security from day one: one auth middleware that reads a role claim, an ownership check on every resource, rate limiting on login and OTP routes, and a strict validation schema on every route, because I later found routes like the all-orders endpoint without auth and a COD route that trusted `paymentStatus` from the client. Second, consistency: one validation library, one error format through a central error handler, and thin route files with logic in services, since the panels were built separately and drifted apart. Third, tests: at least integration tests for payments and the rider accept race, with Supertest and an in-memory MongoDB replica set, plus CI on every pull request. And fourth, I'd store the computed customer price on the product instead of calculating it in two places, and move the in-memory socket map, timers and cron into Redis and a job queue so the backend can run on more than one instance."

Hinglish mein yaad rakhne ka tareeka: *Security, Consistency, Tests, Scale* — 4 shabd. Har ek ke saath project ka ek asli example do.

Agar interviewer pooche "pehle 2 hafte mein kya fix karoge?" (priority order)

1. Upar table A ke 1–7 (security holes). Ye sab chhote code changes hain.
2. Stock check + atomic stock decrement (B ka pehla point).
3. Payment transition atomic + refund on cancel.
4. Missing indexes + `/api/dashboard` ka "saare orders memory mein" wala query aggregation se.

5. Payment aur rider-accept ke integration tests + GitHub Actions CI.

5.4 Project ko scale aur extend kaise karein

5.4.1 Abhi ka design kahan toot-ta hai (single instance assumptions)

Agar backend ke 2 copies (instances) load balancer ke peeche chalayein, to ye cheezein galat ho jaayengi:

Cheez	Kahan	2 instances pe kya hoga	Fix
`global.riderToSocket` Map	`server.js`	Rider instance A pe connected, order instance B pe aaya → B ko rider ka socket nahi milega	`@socket.io/redis-adapter`, aur rider ko `rider:<id>` room mein join karao
15s `setTimeout`	`deliveryBroadcast.service.js`	Restart pe timer gaya	BullMQ delayed job (Redis)
`node-cron` badge job	`jobs/badgeCron.js`	Har instance pe chalega	Alag worker process, ya BullMQ repeatable job, ya DB lock
In-memory cache	`home.services.js`	Har instance ki alag, aur stale	Redis cache + approve/edit pe invalidate
Order-rating photos local disk pe	`order.routes.js` (`diskStorage`)	Ek instance ki disk doosre ko nahi dikhti; Render jaise host pe redeploy pe files ud jaati hain	S3 pe bhejo, baaki uploads ki tarah

5.4.2 Step-by-step scaling plan

> Repo mein koi load test ya traffic data nahi hai, isliye neeche "kitne users pe" wale numbers nahi diye. Ye order hai jisme kaam karna chahiye.

1. **Stage 0: pehle sahi karo.** Security fixes, stock, atomic payment transition, indexes, tests. Galat system ko scale karna bekaar hai.
2. **Stage 1: zyada instances.**
   - Upar wali table ke fixes (Redis adapter, BullMQ, Redis cache, S3 uploads).
   - Emails, notifications aur rider broadcast ko queue mein daalo, taaki API response fast rahe aur fail hone pe retry ho.
   - `compression`, `helmet`, structured logging (pino) + monitoring.
3. **Stage 2: database.**
   - Missing indexes, aur slow queries ko `explain()` se check karna.
   - Admin/seller lists ko server-side pagination (abhi poori list browser mein aati hai).
   - Full-text search ke liye MongoDB Atlas Search (abhi sirf tag/regex search hai).
   - Reviews ko `Product.reviews[]` se alag collection mein (16 MB document limit aur badi documents se bachne ke liye).
   - Purane orders ko archive, read replicas analytics ke liye.
4. **Stage 3: frontend.**
   - Route-level code splitting, images WebP + `srcset`, RTK Query se caching.
   - Customer store ke SEO ke liye Next.js (SSR), kyunki abhi ye pure client-side SPA hai.
   - Chaaron apps ka common code ek shared package mein (monorepo, npm workspaces).
5. **Stage 4: services alag karna (sirf zaroorat pe).**

- Payments aur delivery ko alag services bana sakte hain jo events (`order.paid` , `task.delivered`) se baat karein.
- Ye tabhi karo jab team aur traffic bade hon. Chhote project mein microservices complexity zyada laati hain.

5.4.3 Features jo add kiye ja sakte hain (extend)

Feature	Kaise (project ke code se jod ke)
Asli location + live tracking	Address save pe geocoding (Google/Mapbox) se `pickupLocation` / `dropLocation` ; rider app `navigator.geolocation.watchPosition` se location bheje; `DeliveryTask.liveLocation` (field already hai) update karo aur customer ko socket se push
Delivery OTP	`DeliveryTask.otp` field already hai: order pe OTP banao, customer ko dikhao, rider `Delivered` se pehle verify kare
Inventory sahi	Order pe `Product.updateOne({ _id, inStock: { $gte: qty } }, { $inc: { inStock: -qty } })` transaction mein; online order ke liye reservation jo TTL/cancel pe wapas ho
Refunds + returns	Razorpay Refunds API, `paymentStatus` mein `Refunded` / `Failed` , `refund.processed` webhook, cancel pe delivery task bhi cancel
Seller payouts	Razorpay Route se seller ko `sellingPrice` aur platform ko fees (fees ka data already product mein hai)
Customer/seller ke liye real-time	Socket rooms `user:<id>` , `seller:<id>` ; order status change pe emit; notifications bell live
Admin tools	Coupon CRUD, seller approve/verify, review moderation
Better search + recommendations	Atlas Search (typo tolerance), "customers also bought" co-purchase aggregation, baad mein ML
Mobile app	React Native; backend REST API waisa hi reuse ho jaayega
Engineering	TypeScript, shared Zod schemas frontend + backend dono pe, tests + CI, Docker

Interview line (English): "*To scale it, I'd first move everything that lives in process memory, like the socket map, the 15-second timer, the cron job and the cache, into Redis and a job queue, so I can run several instances behind a load balancer. Then I'd add the missing indexes and server-side pagination, and only split out payments or delivery as separate services if the team and traffic actually needed it.*"

Step 6: Interview questions aur model answers

> **Kaise use karein:**
>
> - Har sawaal ke neeche ek **model answer** hai, English mein, first person mein, bolne layak simple language mein. Interview zyaadatar English mein hota hai, isliye answers English mein rakhe gaye hain (jaise Step 2.4 ka pitch).
> - Har answer ke neeche **Code:** line hai. Ye revision ke liye hai, taaki tum file khol ke dekh sako. Interview mein file names bolna achha impression deta hai, par line numbers ratne ki zaroorat nahi.
> - ⚠ wali line Hinglish mein hai. Wahan koi trap ya honest caveat hai. Use zaroor padhna.
> - Answers ko ratta mat maaro. Pehle samjho, phir apne words mein bolo. 30–60 second ka answer kaafi hai; interviewer aur poochega to detail do.
> - Saare answers sirf is codebase pe based hain. Jo cheez code mein nahi hai (jaise production traffic, team size, timeline), uske liye `[ ]` mein blank chhoda hai.
> - Attribution yaad rakho (Part 0.2): jo module tumne nahi banaya, wahan "we" bolo.

6.1 Basic questions (15): stack aur project kya hai

B1. Tell me about this project.

> "Aestheticommerce is a multi-vendor marketplace for handmade art and crafts, built for India. It has four React apps: a customer store, a seller dashboard, an admin panel and a delivery-rider app, all talking to one Node.js and Express backend with MongoDB. Customers browse and filter products, add them to a cart, apply coupons and pay with Razorpay or cash on delivery. Sellers sign up with email OTP and KYC, list products with images and video, and every product goes live only after an admin approves it. When an order is placed, nearby riders get a real-time request over Socket.io, and the first one to accept gets the delivery."

Code: `Backend/app.js` (saare routers), `FrontEnd/`, `Seller-Panel/`, `Admin-Panel/`, `delivery-dashboard/`, `Backend/data/categories.json` (6 craft categories).

B2. What was your role in the project?

> "This was a team project of [N] people over [duration]. The work was split by panel, which you can still see in the backend folder names like `routes-seller-panel` and `controllersDeliveryPanel`. I mainly worked on [module, e.g. the payment and order flow / the delivery module / the seller panel], and I also [integration work, e.g. merging the panels into one backend, the S3 migration]."

⚠ Ye template hai. Code se pata nahi chalta ki tumne kya banaya (git history nahi hai, comments mein kai naam hain). Blanks sach se bharo, aur interview se pehle tay kar lo. Baaki saare answers isi choice ke hisaab se "I" ya "we" mein bolo.

B3. Who are the users, and why four separate frontends?

> "There are four roles: customer, seller, admin and delivery rider, and each has a very different job. A customer needs a fast storefront, a seller needs forms and tables, an admin needs approval queues and analytics, and a rider needs a simple mobile-friendly screen with live requests. Keeping them as separate apps means each bundle only contains what that role needs, and each part of the team could work on its own app. They all share one backend, so the data and business rules stay in one place."

Code: 4 apps ke `src/App.jsx`; backend ke panel-wise folders.

⚠ Nuksaan bhi bata sakte ho: auth, axios setup aur `BackButton` jaisa common code chaaron apps mein copy hai.

B4. Walk me through the tech stack.

> "On the frontend I used React 19 with Vite, React Router, and Redux Toolkit for shared state like auth and cart. The seller panel uses Material UI for tables, and charts are done with nivo in the seller panel and recharts in the admin panel. The backend is Node.js with Express 5 and Mongoose on MongoDB Atlas. Auth is JWT with bcrypt-hashed passwords. Images and videos go to AWS S3 and are served through CloudFront. Payments use Razorpay, emails use Resend, real-time rider requests use Socket.io, invoices are generated with PDFKit, and a node-cron job updates product badges every night. The API is documented with Swagger at `/api-docs`."

Code: har app ka `package.json`; `Backend/swagger.js`, `Backend/docs/api.swagger.js`.

B5. Why Node.js and Express for the backend?

> "JavaScript on both sides meant the whole team could move between frontend and backend without switching languages. Express is minimal, so it was easy for each panel to add its own routers and middleware. We're on Express 5, which also forwards errors from async route handlers to the error handler automatically. And the npm ecosystem had everything we needed: Razorpay, AWS SDK, Socket.io, Multer and PDFKit."

Code: `Backend/app.js`, `Backend/server.js`.

⚠ Alternative pooche to: NestJS zyada structured hota par boilerplate zyada; Django/Spring ke liye alag language.

B6. Why MongoDB instead of a SQL database?

> "Three features of MongoDB fit this project directly. First, geospatial queries: rider locations are GeoJSON points with a 2dsphere index, and `$geoNear` finds riders within 5 km. Second, TTL indexes: unpaid online orders get an `expireAt` and MongoDB deletes them after 30 minutes without any cron job. Third, flexible documents: products have a badges object and embedded reviews, and orders embed their line items. The aggregation pipeline also gave us filtering and analytics. The honest trade-off is that for orders and payments, a relational database would give stronger constraints and joins by default."

Code: `models/deliveryPartner.model.js` (2dsphere), `models/order.model.js:139` (TTL), `services/deliveryBroadcast.service.js` (`$geoNear`).

B7. Why React with Vite?

> "React has the biggest ecosystem and a component model that worked well for things like product cards, filters and order timelines that repeat across pages. I used Vite because Create React App is deprecated, and Vite's dev server and hot reload are much faster. The trade-off is that it's a client-side SPA, so for SEO on the customer store, something like Next.js with server-side rendering would be better. I partly covered that with react-helmet-async for meta tags and JSON-LD."

Code: har app ka `vite.config.js`; `FrontEnd/src/utils/SEO.jsx`.

B8. Why Redux Toolkit, and why doesn't the admin panel use it?

> "In the customer store, the same data is needed in many places: the cart count in the navbar, product cards and checkout, and the login state on every page. In the rider app, a task that arrives over the socket shows up in a popup and in the list. That's shared global state, so Redux Toolkit made sense, and `createAsyncThunk` gave every API call the same pending, fulfilled and rejected pattern. In the admin panel, each page's data is only used on that page, so plain `useState` and `useEffect` were enough and Redux would have been overkill."

Code: `FrontEnd/src/Store/store.js` (10 reducers), `FrontEnd/src/Redux/cartSlice.js`, `delivery-dashboard/src/Redux/deliveryTaskSlice.js`, `Admin-Panel/pages/*`.

B9. How is the backend code organised?

> "It's layered: routes, then middleware for auth, validation and file upload, then controllers, services and Mongoose models. Each panel has its own folders, like `routes-seller-panel`, `controllersAdminPanel` and `MiddlewareDeliveryPanel`, so teams didn't step on each other. Shared things like models and utilities are common. All routers are mounted in `app.js`, and a global error handler sits at the very end."

Code: `Backend/app.js` (mount table, `app.use(globalErrorHandler)` line 148).

⚠ Honest raho: delivery panel achha layered hai, lekin `product.routes.js` (749 lines) aur `order.routes.js` (697) mein saari logic route file ke andar hi hai.

B10. How does login and authentication work?

> "It's JWT-based. When a user logs in, the server checks the password with bcrypt and signs a token that's valid for 30 days. The client stores it and sends it as `Authorization: Bearer <token>` on every request. Each role has its own middleware: `protect` for customers, `protectSeller`, `protectAdmin` and `protectDelivery`. Each one verifies the token and loads the user from that role's own collection, so a seller token can't be used on a customer route. The seller middleware also blocks blocked sellers with a 403, and the rider middleware checks that the admin has approved the rider."

Code: `Middleware/authMiddleware.js`, `MiddlewareSellerPanel/sellerPanelauthMidWar.js`, `MiddlewareAdminPanel/Adminauth.middleware.js`, `MiddlewareDeliveryPanel/deliveryAuth.middleware.js`, `models/user.model.js:96` (`generateAuthToken`).

B11. How do you store passwords?

> "Passwords are never stored in plain text. They're hashed with bcrypt using 10 salt rounds. bcrypt is deliberately slow and adds a random salt to every hash, so two users with the same password get different hashes and brute force is expensive. On the schema, the password field has `select: false`, so it isn't returned by normal queries; you have to ask for it explicitly with `.select('+password')` during login."

Code: `models/user.model.js` (`hashPassword`, `comparePassword`), `models/admin.model.js` aur `models/deliveryPartner.model.js` ke `pre('save')` hooks.

⚠ Interviewer code dekhe to: customer login response mein poora `user` object jaata hai jisme hash bhi hai (kyunki `+password` select kiya tha). Pooche to maan lo aur fix batao: response se `password` hatao.

B12. Where are images and videos stored, and why?

> "Uploads go through Multer with memory storage, so the file stays in memory as a buffer and goes straight to AWS S3 without touching the server's disk. Each file gets a unique key with a timestamp and UUID, and we save the CloudFront URL in MongoDB, so images load from a CDN edge close to the user. We actually started on Cloudinary and moved to S3 and CloudFront; there's a migration script for that, and the upload helper returns the same `secure_url` shape as Cloudinary so the rest of the code didn't have to change."

Code: `utils/s3Upload.js` (`uploadToS3`, `sanitizeFileName`), `scripts/migrateCloudinaryToS3.js`.

⚠ Order-rating photos abhi bhi local `uploads/` disk pe jaati hain, aur zyaadatar multer configs mein size limit nahi hai.

B13. Why Razorpay, and which payment options do you support?

> "Razorpay is the standard gateway in India and supports UPI, cards and netbanking, which matters for our users. It gives an Orders API, a signed response that I verify on the server, and webhooks as a backup. We support two methods: online payment through Razorpay and cash on delivery."

Code: `routes/payment.routes.js`, `routes/order.routes.js` (COD), `config/razorpay.js`.

B14. What is Socket.io used for?

> "It's used for one thing: pushing new delivery requests to riders in real time. With plain HTTP the server can't start a conversation, so the rider app would have to keep polling. With Socket.io, when a rider logs in the app connects and registers its rider ID, and the server keeps a map from rider ID to socket ID. When an order is placed, the server finds nearby available riders and emits `newDeliveryRequest` only to their sockets. The rider sees a popup with a 10-second countdown to accept."

Code: `Backend/server.js` (`registerRider`, two Maps), `services/deliveryBroadcast.service.js`, `delivery-dashboard/src/hooks/useRiderSocket.js`, `delivery-dashboard/src/components/DeliveryRequest/`.

⚠ Customer aur seller ke liye socket use nahi hota. Unhe order status dekhne ke liye refresh karna padta hai.

B15. How is the project deployed?

> "There are five deployables. The backend is a plain Node app started with `node server.js`, with no build step, and it reads its config from environment variables like the Mongo URI, JWT secret, AWS, Razorpay and Resend keys. Each React app is built with `vite build` into static files that any static host can serve. The database is on MongoDB Atlas and media on S3 with CloudFront. [We hosted the backend on Render and the frontends on Vercel.]"

Code: `Backend/package.json` (`start` script), `Backend/server.js:15` (comment "Render will provide PORT"), root `.gitignore` (`.vercel`).

⚠ Repo mein deployment ka koi config nahi hai (Dockerfile, CI, `vercel.json`). Render/Vercel sirf code ke hints se andaaza hai, isliye bracket mein hai. Sach pata ho to wahi bolo.

6.2 Intermediate questions (15): features kaise kaam karte hain

I1. Walk me through what happens when a customer places a COD order.

> "The checkout page dispatches a `createOrder` thunk, which posts to `/api/orders` with the JWT. The `protect` middleware loads the user. Then the route builds the line items: for each product it reads the product from the database and uses the server-computed offered price, ignoring any price from the client. `resolveOrderPricing` adds 5% GST and re-validates the coupon. The order is saved; a pre-save hook generates a readable order ID like `OD…` . If a coupon was used, a coupon-usage record is created and the coupon's count is incremented. Then delivery tasks are created per seller and broadcast to nearby riders, the confirmation emails are sent without awaiting so they don't slow the response, sellers get notifications, and the API returns 201. The frontend clears the cart and navigates to the order-placed page."

Code: `FrontEnd/src/components/Order/Checkout/Checkout.jsx` , `routes/order.routes.js:69` , `utils/resolveOrderPricing.js` , `services/deliveryTask.service.js` , `servicesSellerPanel/orderEmailService.js` .

⚠ Is route mein `paymentStatus` client se aata hai (`order.routes.js:191–193`) aur stock check nahi hota. Pooche to maan lo aur fix batao.

I2. How does the Razorpay online payment flow work?

> "It's three steps. First, `/api/payment/create-order` recalculates the amount on the server, creates a Razorpay order in paise, and saves a pending order in MongoDB with an `expireAt` 30 minutes ahead. Second, the browser opens Razorpay checkout with that order ID, and after payment Razorpay returns the order ID, payment ID and a signature. Third, `/api/payment/verify` recomputes HMAC-SHA256 of `order_id|payment_id` with the secret and compares it with the signature. If it matches, the order becomes Paid, the TTL is removed, and delivery tasks and seller notifications are created. If the user closes the tab after paying, the `payment.captured` webhook does the same job. If they close the popup without paying, the frontend deletes the pending order, and if they just leave, the TTL index deletes it."

Code: `routes/payment.routes.js` (`/create-order` line 96, `/verify` line 185, `/webhook` ~331), `FrontEnd/src/components/Order/Checkout/Checkout.jsx` (`handler` , `ondismiss` , `payment.failed`).

I3. How do you make sure a customer can't change the price?

> "The server never trusts a price from the client. In both COD and online flows, I take only product IDs and quantities from the request, read each product from the database, and compute the price there. The offered price is the seller's price plus platform, logistics and transaction fees, minus the discount, rounded to end in 9. Then GST and the coupon are applied on the server too. The client calculates totals only for display. Even the cart stores just product IDs and quantities, not prices."

Code: `utils/buildPaymentOrderItems.js` , `utils/productForClient.js` (`toClientProduct`), `utils/resolveOrderPricing.js` , `controllers/cart.controllers.js` .

⚠ Price safe hai, lekin `approvalStatus` , `isDeleted` aur `inStock` check nahi hote, to pending ya out-of-stock product bhi order ho sakta hai.

I4. How does the product listing with filters, sorting and pagination work?

> "It's one aggregation pipeline in `GET /products`. It first matches approved, non-deleted products plus category, tag, store, rating and discount filters, and user input is regex-escaped. Then `$addFields` computes the offered price inside MongoDB, so I can filter and sort by the real customer price. A `$facet` stage returns both the total count and the current page, using sort, skip and limit, in a single query, and a final `$project` removes the internal fee fields. Page size defaults to 15 and is capped at 50. On the frontend, filters live in the URL query string, so a filtered page can be shared or refreshed."

Code: `routes/product.routes.js` (`router.get("/")` line 56, `escapeRegex` line 18, `rawOfferedPriceExpr` line 212, `facetStage` line 289), `FrontEnd/src/components/Product/Testing/ShopM/Product/ShopM.jsx`.

I5. How do coupons work?

> "A coupon has a code, a type of flat or percent, a value, an optional expiry, a total usage limit and a per-user limit. There's a preview endpoint, `/api/coupons/validate`, that checks all of that and returns the discount for the checkout UI. But the real discount is calculated again when the order is created, in `resolveOrderPricing`, so the preview can't be abused. The discount can never exceed the subtotal. Usage is recorded in a separate `CouponUsage` collection with coupon, user and order IDs, which is how the per-user limit is counted, and the payment flow checks that usage isn't recorded twice for the same order."

Code: `models/coupon.model.js`, `models/couponUsage.model.js`, `controllers/coupon.controller.js`, `utils/resolveOrderPricing.js`, `payment.routes.js:31` (`applyCouponUsageIfNeeded`).

⚠ Invalid coupon pe order fail nahi hota, discount chupchaap 0 ho jaata hai. Admin panel mein coupon banane ka UI nahi hai (seed/DB se bante hain). Preview `>` aur order `>=` check karta hai (off-by-one).

I6. How does a rider get a delivery request in real time?

> "When the rider app logs in, it connects a Socket.io client and emits `registerRider` with the rider's ID. The server stores that in a `riderToSocket` map, and the reverse map so it can clean up on disconnect. When an order is placed or paid, a delivery task is created per seller, and `broadcastToNearbyRiders` runs a `$geoNear` aggregation to find up to 10 active and available riders within 5 km of the pickup point. For each one that's online, it emits `newDeliveryRequest` to that socket only. The rider sees a popup with a 10-second timer. If nobody accepts within 15 seconds, the system assigns a fallback rider so the order doesn't get stuck."

Code: `server.js`, `services/deliveryBroadcast.service.js` (`RADIUS_METERS`, `MAX_PARTNERS`, `ASSIGNMENT_TIMEOUT_MS`), `delivery-dashboard/src/hooks/useRiderSocket.js`.

⚠ **Zaroor bolo agar pooche:** pickup/drop coordinates abhi placeholder `[77.209, 28.613]` hain (geocoding nahi hai), aur rider app connect hote hi location bhi placeholder pe set karti hai. To "5 km" ka logic bana hua hai par asli locations ke saath nahi chalta. Fallback rider ek hardcoded test account `delivery@test.com` hai.

I7. When a rider updates the status, how does the customer's order change?

> "The rider app calls `PUT /api/delivery/task-status`. Inside a transaction, the controller loads the task, checks it belongs to that rider, and checks the transition against a small state machine: Accepted or Assigned to PickedUp, then OutForDelivery, then Delivered. Then `syncOrderStatusFromTask` updates only that seller's items in the order and recomputes the order-level status from all items. On Delivered, the rider is marked available again and their delivery count goes up. Everything commits together."

Code: `controllersDeliveryPanel/delivery.controller.js` (`updateTaskStatus` line 316, `syncOrderStatusFromTask` line 285).

⚠ PickedUp pe item seedha "Out for Delivery" ho jaata hai ("Shipped" skip). Final states (Delivered, Cancelled) pe state machine ka check skip ho jaata hai (Step 5, H4).

I8. How do you handle an order with products from multiple sellers?

> "Each item in the order stores its own seller, a snapshot of the price and name, and its own status. When the order is confirmed, I group the items by seller and create one delivery task per seller, because they'll be picked up from different places. Each seller sees only their items, and each task moves independently. The overall order status is derived from the items, so it's Delivered only when every item is delivered."

Code: `models/order.model.js` (`products[]`), `services/deliveryTask.service.js` (`createDeliveryTasksForOrder`, `createDeliveryTaskForSeller`).

I9. How do the four authentication middlewares differ?

> "They all verify the same JWT signature, but each one looks the user up in its own collection. `protect` reads `_id` from the token and loads a User. `protectSeller` reads `id`, loads a Seller, and returns 403 if the seller is blocked. `protectAdmin` loads an Admin and sets the role to admin on the request. `protectDelivery` is the strictest: the rider token carries `role: 'delivery'`, and it checks the role, that the rider exists, and that the admin has activated them. 401 means we couldn't identify you, 403 means we know who you are but you're not allowed."

Code: 4 middleware files (B10 dekho), `controllersDeliveryPanel/delivery.controller.js:10` (`generateToken` with role).

⚠ Sirf rider token mein `role` hai, aur customer token mein `_id` jabki baaki mein `id`. Isi mismatch ki wajah se seller/admin notifications kaam nahi karti (`/api/notifications` customer wala `protect` use karta hai).

I10. How does a file upload work end to end?

> "The frontend sends `multipart/form-data` with a `FormData` object. On the route, Multer with memory storage parses it, so `req.file` or `req.files` holds the file as a buffer. Then `uploadToS3` sanitises the original filename, builds a unique key like `folder/timestamp-uuid-name`, uploads with `PutObjectCommand` and the right content type, and returns the CloudFront URL, which I save in MongoDB. The S3 client is a lazy singleton, created once on the first upload, and the helper checks that all required environment variables exist before uploading."

Code: `utils/s3Upload.js`, `routes-seller-panel/addProduct.routes.js` (5 images + 1 video), `routes/user.routes.js` (`/avatar`).

⚠ Seller product, KYC aur banner uploads pe size/type limit nahi hai, aur product route mein validation upload ke **baad** hoti hai. `deleteFromS3` bana hai par kahin call nahi hota, to purani files S3 mein padi rehti hain.

I11. How does error handling work in the backend?

> "There's a custom `AppError` class with a status code and an `isOperational` flag, which marks expected errors like 'OTP is invalid' whose message is safe to show. Controllers throw it or call `next(err)`, and a global error handler, registered last in `app.js`, formats the response. In development it returns the stack trace. In production it converts known errors into friendly ones, like an invalid ObjectId into a 400, a duplicate-key error 11000 into a 400, Mongoose validation errors into 400, and JWT errors into 401, and hides the message of unexpected bugs behind a generic one."

Code: `utils/AppError.js`, `Middleware/globalErrorHandler.js`, `app.js:148`, `utils/asyncHandler.js`.

⚠ Honest: project mein 3 styles hain. Purane routes apna try/catch karke `{ error: err.message }` bhejte hain, isliye response format consistent nahi hai. JSON 404 handler bhi nahi hai.

I12. How do you validate request data?

> "Newer routes use Zod through a small middleware factory, `zodValidate(schema, 'body')`, which returns a middleware. It runs `safeParse`; on failure it passes a 400 `AppError` with the first message, and on success it replaces the body with the parsed data. That's useful because Zod strips unknown keys, so a client can't sneak in fields like `isActive: true` when registering a rider. The admin badges schema is `.strict()`, so unknown badge names are rejected. Older customer routes use express-validator. Mongoose schemas add a second layer with required fields, enums and custom validators like the 6-digit pincode, and unique indexes are the last layer."

Code: `Middleware/zodValidate.js`, `validators/` folder, `routes/user.routes.js:70`, `models/address.model.js` (pincode).

⚠ Do gotchas jo local pe verify hue: Express 5 mein `req.query = parsed.data` chupchaap ignore hota hai, aur Zod 4 `required_error` wale custom messages ignore karta hai. Aur kai routes (cart, order, payment) pe validation hai hi nahi.

I13. How does the nightly badge job work?

> "A node-cron job runs every day at midnight UTC, which is 5:30 AM IST. It loads all approved products and runs three aggregations: units delivered in the last 90 days, in the last 7 days, and how many users have each product in their wishlist. Then it applies rules, for example best seller is 15 or more units in 90 days, trending is 3 or more in 7 days, top rated is a rating of at least 4.5 with at least 3 reviews, and limited stock is between 1 and 10. All updates go to the database in one `bulkWrite`, so it's one round trip instead of one query per product. The 'featured' badge is left alone because only the admin sets it."

Code: `jobs/badgeCron.js`, `services/badgeAutomationService.js` (`runBadgeAutomation`), `server.js` (`initBadgeCron()`).

⚠ Multiple instances pe cron har instance pe chalega. `onSale` lagbhag har product pe true hoga (MRP formula ki wajah se). Maps `reduce` + spread se bante hain jo O(n²) hai.

I14. How are personalised recommendations on the home page built?

> "The home products endpoint uses optional auth. If there's no user, it returns popular products. If there is, it fetches three signals in parallel: tags from the wishlist, from paid orders and from the cart. Each tag gets a weighted score, wishlist highest at 1.44, orders 1.0 and cart 0.8, and I take the top 20 tags. Then I query active products with those tags, excluding items already in the cart or already bought, and limit price to about twice the user's average. If that returns nothing, it falls back to popular products. The response includes a `mode` field, 'recommended' or 'popular', so the UI knows which one it got."

Code: `controllers/home.controllers.js` (`getProducts` line 41), `services/home.services.js` (`WEIGHTS` line 127, `getPersonalisedProducts` line 234), `Middleware/optionalAuth.js`.

⚠ COD orders "Paid" nahi hote, to unki history signal nahi banti. `/home/products` ka response `toClientProduct` se nahi guzarta, isliye fees leak hoti hain (verified).

I15. How does the cart work in the frontend state?

> "The cart lives on the server, and Redux holds a copy. When you click Add to Cart, `ProductCard` dispatches an async thunk, which calls a service function that posts to `/users/cart` with the token. The backend adds or increments the item and returns the whole cart with client-safe prices. On fulfilled, the reducer replaces the Redux cart with what the server sent and recomputes the total quantity and amount, so the navbar badge updates. I chose to replace instead of updating optimistically, so the UI always matches the server. When the user logs in, a `useCartPersistence` hook fetches the cart and favourites, and a global axios interceptor logs the user out on any 401."

Code: `FrontEnd/src/components/Product/Testing/ShopM/ProductCard/`, `FrontEnd/src/Redux/cartSlice.js:66`, `FrontEnd/src/Redux/cartService.js`, `FrontEnd/src/hooks/useCartPersistence.js`, `FrontEnd/src/axiosSetup.js`.

⚠ Guest cart nahi hai (login zaroori). Logout pe cart/favorites slices reset nahi hote.

6.3 Advanced questions (15): security, scaling, edge cases, trade-offs

A1. How do you stop two riders from accepting the same task, and why a transaction?

> "The accept endpoint does two conditional updates inside a MongoDB transaction. First it locks the rider with `findOneAndUpdate({ _id, isAvailable: true }, { isAvailable: false })`, so a rider can only hold one active delivery. Then it claims the task with `findOneAndUpdate({ _id: taskId, status: 'Pending' }, { status: 'Accepted', deliveryPartner })`. Because the condition is in the filter and single-document updates are atomic, only one request can match; the other gets null and a 400. The transaction is there because those are two documents: if the task was already taken, aborting the transaction also releases the rider's lock. This is optimistic concurrency: no lock up front, just a check at write time."

Code: `controllersDeliveryPanel/delivery.controller.js:229` (`acceptTask`).

⚠ Fallback `assignDefaultRider()` read-then-save karta hai, conditional update nahi (Step 5, H1). Pooche "koi aur jagah race hai?" to ye aur payment verify/webhook wala bata do.

A2. What happens if the verify call and the webhook arrive at the same time?

> "Both paths call `markRazorpayOrderPaid`, which returns early if the order is already Paid, and coupon usage is recorded only once per order. But I have to be honest: that check reads the document first and then saves, so it isn't atomic. If both requests read the order as Pending at the same moment, both could create delivery tasks and notifications. The fix is the same pattern I used for rider accept: do the transition as one conditional update, `findOneAndUpdate({ _id, paymentStatus: 'Pending' }, { $set: { paymentStatus: 'Paid' }, $unset: { expireAt: 1 } })`, and only the request that gets the document back runs the side effects. A unique index on order plus seller in delivery tasks would be a second safety net."

Code: `routes/payment.routes.js:50` (`markRazorpayOrderPaid`), `services/deliveryTask.service.js` (`existing` check, jo atomic nahi hai).

A3. What security issues does the project have, and how would you fix them?

> "When I reviewed it, the biggest issues were authorization gaps. A few routes had no auth, including the endpoint that lists all orders, the invoice download and the seller dashboard, so I'd add the right middleware and an ownership check on each, which also fixes the IDOR problem of guessing another user's order ID. Second, mass assignment: the COD route accepts `paymentStatus` from the request body, and the seller profile update does `$set: req.body`, so a seller could approve themselves. The fix is an allow-list with a Zod schema on every write route. Third, the customer password reset doesn't require proof of the OTP, so I'd have OTP verification issue a short-lived reset token that the reset endpoint requires. And then the basics: rate limiting on login and OTP, `helmet`, a CORS whitelist instead of `*`, JWT auth on the socket handshake, and `crypto.randomInt` instead of `Math.random` for OTPs."

Code: Step 4.14.2 aur 4.14.3 ki tables; Step 5.3 table A.

⚠ Ye answer tabhi achha lagta hai jab tum ownership lo: "I found / we had", aur har problem ke saath fix bolo. Defensive mat bano.

A4. Your JWT is stored in localStorage. What are the risks, and what would you change?

> "localStorage is simple and safe from CSRF, because the browser doesn't send it automatically, but any XSS script can read it, and our token lives for 30 days. There's also no server-side revocation: logout only clears the client, and changing the password doesn't invalidate old tokens. I'd move to a short-lived access token, around 15 minutes, kept in memory, plus a refresh token in an httpOnly, Secure, SameSite cookie with rotation. To revoke tokens, I'd add a `tokenVersion` field on the user, put it in the token, and bump it on logout or password change. And I'd put a `role` claim in every token, not just the rider's."

Code: `models/user.model.js:96` (`expiresIn: "30d"`), `FrontEnd/src/Redux/authSlice.js` (localStorage), Step 4.3.9.

A5. How would you scale the backend to multiple instances?

> "Right now a few things live in process memory, so I'd fix those first. The socket map from rider to socket ID breaks across instances, so I'd use the Socket.io Redis adapter and put each rider in a room. The 15-second fallback is a `setTimeout`, which is lost on restart, so it becomes a delayed job in BullMQ. The nightly cron would run on every instance, so it moves to a single worker or a repeatable queue job. The home-page cache becomes Redis with invalidation, and the rating photos that go to local disk move to S3. After that the API is stateless thanks to JWT, so I can put several instances behind a load balancer with sticky sessions for the WebSocket connections."

Code: `server.js` (`global.riderToSocket`), `services/deliveryBroadcast.service.js` (`setTimeout`), `jobs/badgeCron.js`, `services/home.services.js` (`responseCache`), `routes/order.routes.js` (`diskStorage`).

A6. Which indexes do you have, which are missing, and how would you find slow queries?

> "Products have a compound index on approval status, deleted flag, stock and rating, which matches the home and listing query, plus a seller index and a multikey index on tags for recommendations. Riders and delivery tasks have 2dsphere indexes for the geo queries, and orders have a TTL index on `expireAt`. What's missing are indexes on `Order.user` for 'my orders', `Order.products.sellerId` for seller views, `Order.razorpayOrderId`, which verify and the webhook search by, and a unique index on `Cart.user`, which would also prevent duplicate carts. To find slow queries I'd use `explain('executionStats')` to check for collection scans, and Atlas's performance advisor and the profiler for slow operations."

Code: `models/product.model.js:175–182`, `models/deliveryPartner.model.js`, `models/deliveryTask.model.js`, `models/order.model.js:139`.

A7. Why a TTL index for pending orders, and what are its edge cases?

> "It let me clean up abandoned checkouts without writing a cron job: the pending order gets `expireAt` 30 minutes ahead, MongoDB's TTL monitor deletes it, and when payment succeeds I unset `expireAt` so the order survives. Edge cases: the TTL monitor runs about once a minute, so deletion isn't exact. More importantly, a user might pay after the order was already deleted. For that, verify rebuilds the order from the request and marks it Paid, which is good for recovery, but it doesn't compare the amount actually paid. I'd fetch the payment from Razorpay and match the amount before rebuilding. COD orders never get `expireAt`, so they never expire."

Code: `models/order.model.js:139`, `routes/payment.routes.js` (`/create-order`, `/verify` rebuild branch).

A8. What would change if we moved from MongoDB to PostgreSQL?

> "Orders and payments would get stronger guarantees for free: foreign keys, unique constraints and transactions without needing a replica set. The data would be normalised into tables like orders, order_items, reviews and cart_items, instead of embedded arrays. The geo search would need the PostGIS extension, using `ST_DWithin` instead of `$geoNear`. The TTL cleanup would become a scheduled job, since Postgres has no TTL index. The listing pipeline would become SQL with a computed price column, and the product badges object could be a JSONB column. Mongoose would be replaced by an ORM like Prisma. I'd only do it if relational integrity became a real pain point, because the current features map well to MongoDB."

⚠ Ye "what if" sawaal hai, code mein iska koi hissa nahi. Balanced answer do, kisi ek ko "best" mat bolo.

A9. What breaks first if the catalogue grows to a million products?

> "A few things. In the listing pipeline, the store filter joins sellers with `$lookup` before pagination, so it joins every matching product instead of just the page; I'd move the lookup after skip and limit. Category and tag filters use case-insensitive regex, which doesn't use indexes well, so I'd store normalised values or slugs and match exactly. Deep pages with `$skip` get slow, so I'd switch to cursor-based pagination. Search would move to Atlas Search for full-text and typo tolerance. Reviews are embedded in the product document, which is capped at 16 MB, so popular products would need a separate reviews collection. And the nightly badge job loads all products as full documents, so it should use `.lean()`, process in batches, and build lookup maps with `Map` instead of spreading objects in a reduce."

Code: `routes/product.routes.js` (store pipeline), `models/product.model.js` (`reviews[]`), `services/badgeAutomationService.js`.

A10. Stock never decreases today. How would you implement inventory correctly?

> "I'd decrement stock atomically with a condition, inside the same transaction as order creation: `Product.updateOne({ _id, inStock: { $gte: qty } }, { $inc: { inStock: -qty } })`. If any item returns zero modified documents, I abort the whole transaction and tell the customer which item is out of stock. That way two customers can't buy the last unit. For online payments, I'd reserve stock when the pending order is created and release it if the payment fails, the user cancels, or the order expires. And cancellations before dispatch would add the stock back."

⚠ Ye abhi project mein **nahi** hai: stock check bhi nahi, decrement bhi nahi (out-of-stock product ka order local pe verify hua tha). Ise "planned improvement" ki tarah bolo, done ki tarah nahi.

A11. A customer cancels a paid online order. What happens, and what should happen?

> "Today, the order status becomes Cancelled, but no refund is triggered and the delivery task isn't cancelled, so a rider could still deliver it. What should happen: if the order was paid online, call Razorpay's refund API and move `paymentStatus` to a new 'Refund initiated' state, then confirm it with the `refund.processed` webhook. In the same transaction I'd cancel the related delivery tasks and free the rider, put the stock back, and notify the seller. I'd also allow item-level cancellation, since items from different sellers move independently."

Code: `routes/order.routes.js` (`PATCH /:orderId/cancel`), `models/order.model.js` (`paymentStatus` enum sirf `Pending/Paid`).

A12. How reliable and secure is the Socket.io part?

> "There are three weaknesses I'd fix. Security: any client can emit `registerRider` with any rider ID and receive that rider's requests, so I'd verify the JWT in an `io.use` middleware during the handshake and take the rider ID from the token, not from the client. Reliability: socket messages can be missed if a rider is briefly offline, so the rider app also polls nearby tasks every 30 seconds as a safety net. Ideally, every request would also be stored, so the app can fetch missed offers on reconnect. And a frontend bug: the hook disconnects on cleanup but doesn't remove its listeners, so listeners can stack up; I'd call `socket.off` for each event. Also, if a rider has two tabs open, the map keeps only the last socket, which rooms would solve."

Code: `server.js`, `delivery-dashboard/src/hooks/useRiderSocket.js`, `delivery-dashboard/src/components/DeliveryPanel/` (30s polling).

A13. Why compute the offered price at query time instead of storing it?

> "Computing it at read time meant the fees and discount could change and every response would automatically use the latest formula, and the internal fees never had to be duplicated. The cost was that the formula exists twice: in JavaScript for cart and checkout, and in the aggregation pipeline for listing, because sorting and filtering by price had to happen in the database. Two copies can drift; for example JavaScript's `Math.round` and MongoDB's `$round` handle .5 differently. If I did it again, I'd compute `offeredPrice` once when the product is saved or its pricing is updated, store it, and index it. Then listing is a simple indexed sort, and there's a single source of truth. If the fee rules change, a migration recomputes it."

Code: `utils/productForClient.js` (`roundToPriceEndingIn9`, `toClientProduct`), `routes/product.routes.js:212` (`rawOfferedPriceExpr`).

⚠ `$round` wala edge case sirf code padh ke nikla hai, run nahi kiya. "Can drift" bolo, "it is broken" nahi.

A14. The notifications feature doesn't work for sellers and admins. Why, and how would you redesign it?

> "The data model is fine: one `Notification` collection with a polymorphic reference, `recipient` plus `recipientModel` with `refPath`, so a notification can belong to a User, Seller or Admin. The bug is in auth: the notifications routes use the customer `protect` middleware, which reads `_id` from the token and looks only in the User collection, while seller and admin tokens carry `id`. So seller and admin requests always get a 401. I'd fix it with one auth middleware that reads a role claim and loads from the right collection, add an index on recipient, and push new notifications in real time with Socket.io rooms like `seller:<id>`, instead of the bell fetching only once on mount."

Code: `models/notification.model.js`, `routes-admin-panel/notification.routes.js`, `controllersAdminPanel/notification.controller.js`, `Seller-Panel/src/components/Notifications/Notifications.jsx`.

A15. What happens if the email provider (Resend) is down?

> "It depends on the flow. For COD order confirmation, the email is fire-and-forget, so the order succeeds and only an error is logged. For the `payment.failed` webhook, the email is awaited, so the webhook returns an error and Razorpay retries it, which is acceptable. The worst case is seller registration: the seller is saved first and then the OTP email fails, so the user gets an error, but registering again says 'already exists', and they have to use resend-OTP. I'd fix it with an outbox pattern: save the email job in the database or a queue in the same step as the main write, and have a worker send it with retries."

Code: `utilsSellerPanel/sendEmail.js`, `servicesSellerPanel/sellerPanelService.js:62`, `routes/order.routes.js` (`sendOrderEmails` without await), `routes/payment.routes.js` (`payment.failed`).

6.4 Tricky follow-ups (10): "kya tumne sach mein banaya?"

Ye sawaal interviewer isliye poochta hai taaki pata chale tumne code andar se dekha hai ya nahi. Inke answers chhote aur specific rakho.

T1. Why does the customer token have `_id` but the seller and admin tokens have `id`? Does it matter?

> "They were written by different parts of the team: the customer model's `generateAuthToken` signs `{ _id }`, while the seller and admin helpers sign `{ id }`. It mostly works because each middleware reads its own key and looks in its own collection. But it does matter in one place: the notifications routes use the customer middleware, so seller and admin tokens fail there. That's why I'd standardise on one payload with `id` and `role` for every user type."

Code: `models/user.model.js:96`, `servicesSellerPanel/sellerPanelService.js:14`, `controllersAdminPanel/admin.controller.js` (`generateToken`).

T2. Why is the JSON body parser skipped for only one route?

> "That's the Razorpay webhook. Razorpay signs the exact raw bytes of the body, and if `express.json()` parses it first, I can't reproduce those bytes reliably, so the HMAC won't match. So in `app.js` I skip the JSON parser when the URL is `/api/payment/webhook`, and the route uses `express.raw({ type: 'application/json' })` to get a Buffer. I verify the signature on that Buffer and only then `JSON.parse` it."

Code: `app.js:60-62`, `routes/payment.routes.js` (`/webhook` line ~331).

T3. In your geo data, does latitude or longitude come first? And does the 5 km matching work with real locations today?

> "GeoJSON is longitude first, then latitude: `[lng, lat]`. That's a common mistake because Google Maps shows lat, lng. Our placeholder is `[77.209, 28.613]`, which is New Delhi: 77 is longitude, 28 is latitude. And to be honest, the matching logic is built, the `$geoNear` query with a 5 km radius and 2dsphere indexes, but real coordinates aren't wired in yet. We don't geocode addresses, so tasks use that placeholder, and the rider app also sets the rider's location to it on connect. The next step is geocoding addresses on save and sending real GPS from the rider app."

Code: `services/deliveryTask.service.js` (`PLACEHOLDER_COORDS`), `delivery-dashboard/src/hooks/useRiderSocket.js` (`updateLocationAPI(77.209, 28.613)`).

⚠ Ye sabse important honesty point hai. Agar tum ise "working" bologe aur interviewer code dekh le, to poore project pe bharosa toot jaayega.

T4. Why is there a `BigInt` in the order model?

> "Each order item gets an `itemId` that's the numeric order ID plus the item's index. That number is 18 digits long, 13 from the timestamp and 5 random, and JavaScript's normal Number can only represent integers exactly up to about 16 digits, `Number.MAX_SAFE_INTEGER`. Adding 1 to an 18-digit Number can silently give the wrong value, so I convert it to `BigInt`, add the index, and store it as a string."

Code: `models/order.model.js:156–157`, pre-save hook line 164.

T5. What does `select: false` do, and did it ever cause a problem?

> "It hides a field from query results by default, which is why passwords and OTPs don't leak in normal reads; you have to opt in with `.select('+password')`. It did cause a bug: the customer verify-OTP controller reads `user.forgot_password_otp`, but that field is `select: false` and the query doesn't add it, so the value is always undefined and verification always fails. It works in the other direction too: login selects `+password` and then returns the whole user object, so the hash ends up in the response. Both are one-line fixes."

Code: `models/user.model.js` (`password`, `forgot_password_otp`), `controllers/user.controllers.js` (`verifyOtpController`, `loginUser`).

T6. Why did you need a replica set just to run the project locally?

> "Because the rider accept, status update, cancel and seller pickup endpoints use MongoDB transactions, and transactions only work on a replica set or a sharded cluster. On a standalone `mongod` the first operation in the transaction fails with an error saying transaction numbers are only allowed on a replica set member. Atlas clusters are replica sets by default, so production was fine, but locally I ran MongoDB as a single-node replica set."

Code: `controllersDeliveryPanel/delivery.controller.js` (`startSession`, `startTransaction` lines 232, 319, 381, 438), `controllersSellerPanel/sellerDeliveryController.js` (`markAsPickedUp`).

T7. Why does the product model have a separate `pre('findOneAndUpdate')` hook for tags?

> "Tags are normalised by a setter, lowercase, trimmed and de-duplicated, so 'Diwali' and 'diwali ' become one tag. But Mongoose setters run when you assign on a document and save, not on query-style updates like `findOneAndUpdate`. Sellers edit products through update queries, so I added a pre hook for `findOneAndUpdate`, `updateOne` and `updateMany` that normalises `update.tags` and `update.$set.tags` too. Without it, edited products could get un-normalised tags and stop matching the tag filters."

Code: `models/product.model.js` (`normalizeTagsArray`, hook line 165).

T8. You're on Express 5. Did anything behave differently from Express 4?

> "Two things. The good one: Express 5 catches rejected promises from async handlers and forwards them to the error handler, so a missing try/catch doesn't hang the request like it would in Express 4. The surprising one: in Express 5, `req.query` is a getter, so our Zod middleware's `req.query = parsed.data` is silently ignored. The query is still validated, invalid values get a 400, but the handler receives the raw query, not the transformed one. I'd store the parsed result on something like `req.validatedQuery` instead."

Code: `Middleware/zodValidate.js`, `routes-seller-panel/productInfo.routes.js` (`GET /my`), `controllers/user.controllers.js` (`registerUser` without try/catch).

⚠ Ye local pe run karke verify hua tha (Step 4.6.2). Ye answer dikhata hai ki tumne framework ko andar se samjha, isliye achha follow-up hai.

T9. Why two maps, `riderToSocket` and `socketToRider`? What if a rider opens two tabs?

> "When I emit a request, I have a rider ID and need the socket ID, so I need rider-to-socket. But on disconnect, Socket.io only gives me the socket ID, so I need the reverse map to know which rider to remove. Both lookups are O(1). The two-tab case is a real limitation: the map keeps only the last socket, so if the newer tab closes, the rider looks offline even though the first tab is still open. Using a Socket.io room per rider fixes that, because a room can hold multiple sockets and `io.to('rider:<id>')` reaches all of them."

Code: `server.js` (`registerRider`, `disconnect`).

T10. How did you test this project? Are there automated tests?

> "Honestly, there are no automated tests in the repo; the backend test script is still the npm default. Testing was manual: Swagger at `/api-docs` and Postman for the APIs, a test email mode that redirects all emails to a developer inbox, a seeded test rider, and running full flows end to end, like placing an order, accepting it as a rider and moving it to Delivered. If I added tests now, I'd start with unit tests for pure functions like the price rounding and pricing calculation, then Supertest integration tests with an in-memory MongoDB replica set for payments and the rider-accept race, and run them in GitHub Actions on every pull request."

Code: `Backend/package.json` (`"test": "echo \"Error: no test specified\" && exit 1"`), `utilsSellerPanel/sendEmail.js` (`EMAIL_MODE=test`), `seedFile/seedDeliveryPartner.js`.

⚠ "We had 80% coverage" jaisa kuch kabhi mat bolna. Repo mein ek bhi test file nahi hai.

6.5 Behavioral / STAR questions (5)

> **STAR** = **S**ituation (kya haalat thi), **T**ask (tumhara kaam kya tha), **A**ction (tumne kya kiya), **R**esult (kya nateeja nikla).
> **Bahut zaroori:** neeche ke technical facts asli code se hain. Lekin "kab hua, kisne kiya, team kitni badi thi, kitna time laga" code se pata nahi chalta. Wo `[ ]` mein blank hai. Sirf wahi story bolo jo sach mein hui; agar koi story tumhare saath nahi hui, to use mat bolo, ya teammate ki story ki tarah "we" mein bolo. Interviewer "uske baad kya hua?" pooch ke jhooth pakad leta hai.

S1. Tell me about the most challenging technical problem you faced.

- **S:** "In our delivery module, a new order is pushed to up to ten nearby riders at once, and [during testing / in a demo] we realised two riders could accept the same task at nearly the same time."

- **T:** "I [was responsible for the accept flow] and had to guarantee that each task goes to exactly one rider, and that a rider can only hold one active delivery."
- **A:** "I replaced the read-then-save logic with conditional atomic updates: claim the task only if its status is still Pending, and lock the rider only if they're still available. Because those are two documents, I wrapped them in a MongoDB transaction so a failed claim also releases the rider. That needed a replica set, so I set MongoDB up as a single-node replica set locally."
- **R:** "Sending two accept requests for the same task now gives one success and one clean 400, 'Task already assigned'. [Add any real outcome.] It also taught me to look for check-then-act patterns elsewhere, and I later spotted the same pattern in the payment verify and webhook paths, which I'd fix the same way."

Code: `delivery.controller.js:229`. Detail ke liye Step 5, H1.

S2. Tell me about a time you had to make a trade-off under a deadline.

- **S:** "Before [the demo / the internship deadline], the delivery flow wasn't usable end to end, because we had no geocoding for addresses and very few real riders online."
- **T:** "We needed the full flow, order to rider to delivered, working in time, without blocking on a maps integration."
- **A:** "We used placeholder coordinates in New Delhi for pickup and drop points, and added a fallback that assigns a test rider if nobody accepts within 15 seconds, so an order never gets stuck. We kept the real pieces in place, the 2dsphere indexes and the `$geoNear` query with a 5 km radius, so real coordinates can be plugged in later."
- **R:** "The complete flow worked for the demo [result]. The lesson I took is to mark shortcuts clearly, because a few of them stayed longer than planned, like that placeholder, and an admin route comment that literally says it's bypassing auth for now. Today I'd track each one as a ticket with an owner."

Code: `services/deliveryTask.service.js` (`PLACEHOLDER_COORDS`), `services/deliveryBroadcast.service.js` (`assignDefaultRider`), `routes-admin-panel/admin.routes.js:59` (comment "bypassing protectAdmin").

⚠ Ye story tabhi bolo jab sach mein deadline ki wajah se ye shortcut liye gaye the. Code mein sirf shortcut dikhte hain, unki wajah nahi.

S3. Tell me about working in a team, or a disagreement or inconsistency you had to resolve.

- **S:** "Our team built the four panels in parallel, [N people], each with its own backend folders, and later merged everything into one Express app."
- **T:** "[I was involved in the merge / I had to integrate my module with the others.]"
- **A:** "Merging surfaced inconsistencies: two validation libraries, two copies of the AppError class, two different files both named `statsService.js`, different error response formats, and even different JWT payloads per role. We handled clashes by mounting each panel under its own URL prefix and agreeing on route ordering, for example registering literal routes before `/:id` routes, and the cart router before the user router on `/users`. [Describe the real conversation or decision.]"
- **R:** "The merged backend worked with all four apps. Looking back, I'd agree on shared conventions on day one, one validation library, one error format and one auth middleware, and enforce them with linting and pull-request reviews, because fixing drift later is harder."

Code: `utils/AppError.js` + `utilsSellerPanel/AppError.js`, `app.js` (comment "Cart routes first…"), panel-suffixed folders, nivo vs recharts, react-toastify vs react-hot-toast.

S4. Tell me about a time you had to learn something new quickly.

- **S:** "I hadn't worked with payment gateways before, and the project needed online payments through Razorpay."
- **T:** "I had to integrate it so that payments are verified securely and nothing is lost if the user closes the browser."
- **A:** "I read the Razorpay docs and learned the order, checkout and signature flow. I implemented HMAC-SHA256 verification on the server, and when I learned about webhooks, I added the `payment.captured` handler as a

backup. That's where I learned that the signature is over the raw body, so I had to skip the JSON parser for that one route. I also used a TTL index so abandoned checkouts clean themselves up."

- **R:** "[Result, e.g. the online checkout worked in test mode.] More generally, I learned to read a provider's failure cases, like closed tabs and retries, not just the happy path."

Code: `routes/payment.routes.js`, `app.js:60–62`, `models/order.model.js:139`.

⚠ Agar payments tumne nahi banaye, to isi template mein apni asli nayi cheez daalo, jaise MongoDB geo queries, Socket.io, ya Cloudinary → S3 migration (`scripts/migrateCloudinaryToS3.js`, `--dry-run`, failure report).

S5. Tell me about a mistake you made, or something you would do differently.

- **S:** "While reviewing the project for this interview, I went through every route and found security gaps that we had missed."
- **T:** "I wanted to understand how serious they were and what the right fixes would be."
- **A:** "For example, the endpoint that lists all orders had no auth, and I confirmed it by calling it without a token and getting customer names and addresses back. The COD order route also accepted `paymentStatus` from the client. I traced both to the same root causes: we added auth route by route instead of by default, and we trusted request bodies. I wrote down fixes: an ownership check on every resource, an allow-list schema for every write, and rate limiting on login and OTP."
- **R:** "The lesson I took is secure by default: deny unless a route explicitly allows access, never trust the client for anything that affects money or permissions, and have a reviewer look at auth on every pull request."

Code: `routes/order.routes.js` (`GET /` aur lines 191-193). Step 4.14.3 dekho.

⚠ "Interview ke liye review karte waqt mila" bilkul sach hai (2026-10-10 ke local run mein verify hua tha), isliye ye story safe hai. Agar ye bug tumhare module mein tha, to ownership lo; nahi tha, to "we missed" bolo.

6.6 Quick revision: numbers aur facts jo yaad rakhne hain

Cheez	Value	Kahan
Apps	4 frontends + 1 backend	root folders
Models	17 Mongoose schemas	`Backend/models/`
Endpoints	~140 (lagbhag 56 bina auth)	Step 4.2
Rider search radius	5 km (`RADIUS_METERS = 5000`)	`deliveryBroadcast.service.js`
Riders per broadcast	max 10	same
Fallback timer	15 s server, 10 s popup client pe	same, `DeliveryRequest/`
Nearby polling	har 30 s	`DeliveryPanel/`
Pending online order TTL	30 min	`payment.routes.js`, `order.model.js:139`
GST	5%	`utils/pricingConstants.js`
Fees	logistics ₹60, transaction ₹10, platform 15% (seller form calculate karta hai)	`product.model.js`, `AddProductForm/`
Price rounding	9 pe khatam (742 → 739)	`utils/productForClient.js`
JWT expiry	30 din	4 token helpers
bcrypt	10 salt rounds	models / seller service
OTP	6 digit; customer 15 min, seller register 5 min	controllers / seller service

Cheez	Value	Kahan
Listing page	15 per page, max 50	`product.routes.js`
Cache TTL	tags/categories 3 min, sellers 5 min, popular products 2 min	`home.services.js:29`
Badge cron	roz 00:00 UTC (5:30 AM IST)	`jobs/badgeCron.js`
Product media	max 5 images, video ≤ 20 MB	`addProduct.routes.js`
Recommendation weights	wishlist 1.44, order 1.0, cart 0.8, top 20 tags	`home.services.js:127`
Mongo pool	`maxPoolSize: 50`	`db/db.js`

6.7 Interview mein ye cheezein "done" mat bolna

1. Real rider location / live tracking / asli 5 km matching (coordinates placeholder hain).
2. Stock management (check aur decrement dono nahi).
3. Refunds, returns, seller payouts.
4. Delivery OTP (field hai, logic nahi).
5. Seller approval by admin (sirf product approval hai).
6. Seller/admin notifications (DB mein banti hain, padhi nahi ja sakti).
7. Automated tests, CI/CD, Docker.
8. Real-time updates for customers or sellers (socket sirf riders ke liye).
9. Coupon management UI (coupons seed/DB se).

Inme se koi bhi pooche to formula: **"That part isn't finished. Here's what exists today, and here's exactly how I'd complete it."** Ye answer "haan, wo kaam karta hai" se zyada strong lagta hai.

Step 7: Key concepts jo is project mein use hue

> Step 7 2026-10-14 ko likha gaya. Har concept ke liye do cheezein hain: **Kya hai** (2–3 line mein simple explanation) aur **Project mein** (asli file, function aur snippet). Snippets Steps 1–6 ke notes se liye gaye hain. Jo snippets naye hain (jaise `optionalAuth.js`, `deliveryEta.js`, `ScrollToTop.jsx`, `documentService.js`, `statsService.js` ka `getSalesOverTime`, `DeliveryPanel.jsx` ki polling), wo is step ke liye code khol ke dobara check kiye gaye. Is step mein kuch bhi run nahi kiya gaya. Jahan detail pehle ke kisi step mein hai, wahan section number diya hai, jaise "(detail: 4.7.5)". ⚠ wali line trap ya honest caveat hai.

7.0 Sabse pehle ye concepts pakke karo

Interview mein sabse zyada inhi pe sawaal aate hain. Har ek ki detail neeche us section mein hai.

Concept	Section	Ek line mein (is project ke hisaab se)
async / await	7.1.2	`server.js` pehle `await connectDB()`, phir server start
Environment variables	7.1.13	secrets `.env` mein, `dotenv.config()` se `process.env`
REST API	7.2.2	`GET/POST/PUT/DELETE /users/cart`, ~140 endpoints
CORS	7.2.6	4 apps alag origin pe hain, isliye `app.use(cors())`
Webhook	7.2.9	Razorpay khud `POST /api/payment/webhook` bhejta hai
Middleware	7.3.2	`protectDelivery → zodValidate → acceptTask` chain
JWT	7.4.3	`jwt.sign({ _id }, JWT_SECRET, { expiresIn: "30d" })`
bcrypt hashing	7.4.5	`bcrypt.hash(password, 10)`, login pe `bcrypt.compare`
HMAC signature	7.4.10	`createHmac('sha256', secret)` se payment verify
Indexes	7.5.11	Product ka compound index, `tags` multikey
TTL index	7.5.12	unpaid online order 30 min baad apne aap delete
Aggregation pipeline	7.5.13	listing ka `$addFields` + `$facet`, seller stats ka `$group`
GeoJSON + `$geoNear`	7.5.14	5 km ke andar ke riders, `[lng, lat]` order
Transactions	7.5.15	rider accept ke 2 updates "sab ya kuch nahi"
Race condition + atomic update	7.5.16	`findOneAndUpdate({ _id, status: "Pending" })`
Socket.io	7.6.2	`io.to(socketId).emit("newDeliveryRequest")`
S3 + CDN	7.7.2, 7.7.3	Multer buffer → S3 → CloudFront URL DB mein
Caching (TTL)	7.9.3	`withTtlCache()` home page data 2–5 min
Cron job	7.9.1	`"0 0 * * *"` roz raat badges update
useEffect	7.10.3	30 second polling + `clearInterval` cleanup
Redux Toolkit + thunks	7.11.2, 7.11.4	`createAsyncThunk` pending/fulfilled/rejected
React Router	7.12.1, 7.12.3	`useParams`, URL query filters, `navigate` state
Axios interceptors	7.13.2	401 pe logout, rider app mein token apne aap

7.1 JavaScript aur Node.js basics

7.1.1 Node.js (runtime)

- **Kya hai:** Node.js browser ke bahar JavaScript chalane ka runtime hai (Chrome ka V8 engine). Ye ek hi thread pe chalta hai, lekin DB aur network jaise I/O kaam non-blocking hote hain, isliye ek saath bahut saari requests sambhal leta hai.
- **Project mein:** poora `Backend/` Node pe chalta hai: `npm start` → `node server.js` (`Backend/package.json`). Build step nahi hai, kyunki plain JavaScript hai.

7.1.2 async / await aur Promise

- **Kya hai:** Promise ek "baad mein milne wali value" hai, jaise DB query ka result. `async` function hamesha Promise lautata hai, aur `await` us Promise ke poora hone tak sirf us function ko rokta hai (baaki server chalta rehta hai). Errors `try/catch` se pakde jaate hain.
- **Project mein:** `server.js` ka `startServer()` pehle DB connect hone ka wait karta hai, phir hi port pe listen karta hai, taaki DB ke bina koi request na aaye:

```
async function startServer(){
  await connectDB();
  initBadgeCron();
  const PORT = process.env.PORT || 3000;
  const server = http.createServer(app);
  // ... Socket.io setup ...
  server.listen(PORT, () => { console.log(`Server running on port ${PORT}`); });
}
```

- Lagbhag har controller `async (req, res) => { ... await Model.find(...) }` jaisa hai.

7.1.3 `Promise.all` (kai kaam ek saath)

- **Kya hai:** kai independent Promises ek saath shuru karta hai aur sabke khatam hone ka wait karta hai. Teen 100 ms ki queries ek-ek karke 300 ms leti hain; `Promise.all` se lagbhag 100 ms.
- **Project mein:**
  - `services/home.services.js:235` : recommendation ke teen signals (`getWishlistSignals`, `getOrderSignals`, `getCartSignals`) ek saath.
  - `routes-seller-panel/addProduct.routes.js:67` aur `productInfo.routes.js:215` : saari product images S3 pe parallel upload.
- ⚠ Ek bhi Promise fail ho to poora `Promise.all` fail (reject) ho jaata hai.

7.1.4 Fire-and-forget (bina `await` ke async call)

- **Kya hai:** async function call karo par uska `await` mat karo. Kaam background mein chalta rehta hai aur response turant chala jaata hai. Fayda speed hai; nuksaan ye ki fail hone pe sirf log hota hai, retry nahi.
- **Project mein:** COD order route (`routes/order.routes.js`, line ~216) mein `sendOrderEmails(saved, user);` bina `await` ke hai, taaki email slow ho to bhi order ka response na ruke. (detail: 4.10.2)

7.1.5 Event loop, `setTimeout` aur `setInterval`

- **Kya hai:** Node aur browser dono mein ek "event loop" hota hai jo timers aur network responses jaise kaam ek queue se utha ke chalata hai. `setTimeout(fn, ms)` ek baar baad mein chalata hai, `setInterval(fn, ms)` baar-baar. Timer process ki memory mein rehta hai, to process restart hua to timer bhi gaya.

- **Project mein:**
  - Backend: `services/deliveryBroadcast.service.js` mein `setTimeout(() => assignDefaultRider(taskId), 15000)`: 15 second mein koi rider accept na kare to fallback rider.
  - Rider app: `DeliveryPanel.jsx:321` har 30 second `getNearbyDeliveries()` (polling), aur `DeliveryRequest.jsx:22` popup ka 10 second countdown. Dono `clearInterval` se cleanup karte hain (7.10.3).

7.1.6 CommonJS vs ES Modules

- **Kya hai:** JavaScript files ko aapas mein jodne ke do tareeke hain. CommonJS mein `require()` aur `module.exports` (Node ka purana default). ES Modules mein `import` aur `export` (naya standard, jo browser bhi samajhta hai).
- **Project mein:**
  - Backend CommonJS hai: `const express = require("express")`, `module.exports = connectDB` (`db/db.js`). `Backend/package.json` mein `"type"` field nahi hai.
  - Frontends ES Modules hain: `FrontEnd/package.json` aur `Seller-Panel/package.json` mein `"type": "module"`, aur har file mein `import`.

7.1.7 Destructuring, spread aur rest

- **Kya hai:** `const { a, b } = obj` object se values nikaalta hai. `{ ...obj }` (spread) object ki copy banata hai. `const { a, ...rest } = obj` (rest) "a ke alawa baaki sab" alag nikaal deta hai.
- **Project mein:** customer se internal fees chhupane ka asli tareeka yahi hai (`utils/productForClient.js:22`):

```
const { sellingPrice: _s, platformFee: _p, logisticsFee: _l, transactionFee: _t, ...rest } = raw;
return { ...rest, offeredPrice };
```

- Matlab chaaron fees hat jaati hain, aur baaki product ke saath naya `offeredPrice` customer ko jaata hai.

7.1.8 Higher-order function aur closure

- **Kya hai:** higher-order function wo hai jo function le, ya function lautaye. Closure ka matlab hai ki andar wala function bahar ke variables yaad rakhta hai, bahar wala function khatam hone ke baad bhi.
- **Project mein:**
  - `utils/asyncHandler.js`: `(fn) => async (req, res, next) => { try { await fn(req, res, next); } catch (e) { next(e); } }`.
  - `Middleware/zodValidate.js`: `zodValidate(schema, 'body')` ek naya middleware lautata hai, aur andar ka function `schema` ko closure se yaad rakhta hai.
  - `FrontEnd/src/components/ScrollToTop.jsx`: `isThrottled` variable closure mein rehta hai (7.13.5).
- ⚠ Closure ka trap: Admin `pages/ProductPage/ProductPage.jsx:129` ka rollback `previousProducts` naam ka purana snapshot yaad rakhta hai. Do toggles ek saath fail hon to rollback doosre toggle ko bhi undo kar deta hai. (detail: 4.4.5)

7.1.9 `Map` (key-value store)

- **Kya hai:** `new Map()` key → value rakhta hai, aur `get`, `set`, `delete` O(1) mein hote hain. Jab keys dynamic hon (jaise IDs), tab plain object se behtar hai.
- **Project mein:** `server.js` mein `global.riderToSocket` aur `global.socketToRider` (rider ↔ socket), aur `services/home.services.js` ka `responseCache` (cache). (detail: 4.7.2, 4.11)

7.1.10 `BigInt`

- **Kya hai:** JavaScript ka normal `Number` sirf lagbhag 16 digit tak ke integers exactly rakh sakta hai (`Number.MAX_SAFE_INTEGER` = 9007199254740991). Usse bade integers ke liye `BigInt` hai.
- **Project mein:** `models/order.model.js` (lines 156-157) 18 digit ke order number mein item ka index jodne ke liye `BigInt(base) + index` karta hai, aur result string mein save hota hai. (detail: 4.1.6, T4)

7.1.11 Regular expressions (regex) aur escaping

- **Kya hai:** regex text ka pattern hai, jaise `/^\d{6}$/` matlab "exactly 6 digits". User ka input seedha `new RegExp(input)` mein daalo to `.`, `*`, `+` jaise characters pattern ban jaate hain, aur ek bura pattern server ka CPU atka sakta hai (ReDoS, 7.4.13).
- **Project mein:** `models/address.model.js` mein pincode validator `/^\d{6}$/`. Search input escape karne ke liye `routes/product.routes.js:18`:

```
function escapeRegex(str) {
  if (typeof str !== "string") return "";
  return str.replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
}
```

7.1.12 Date aur timezone

- **Kya hai:** server aam taur pe UTC mein time rakhta hai, lekin business rules local time pe hote hain (India = IST = UTC + 5:30). Timezone convert kiye bina "subah 10 baje" ka matlab galat nikal sakta hai.
- **Project mein:** `utils/deliveryEta.js` IST ka ghanta nikaalta hai: `new Date(date).toLocaleString("en-IN", { timeZone: "Asia/Kolkata", hour: "numeric", hour12: false })`. 10 AM–8 PM ke beech order ho to ETA +2 ghante, warna +7 din. Cron `timezone: "UTC"` mein hai, yaani 00:00 UTC = 5:30 AM IST (7.9.1).

7.1.13 Environment variables (`process.env`, dotenv)

- **Kya hai:** secrets aur config (DB URL, API keys) code mein likhne ki jagah environment se aate hain. Isse har environment (local, production) mein alag values ho sakti hain, aur secrets git mein nahi jaate. `dotenv` package `.env` file padh ke values `process.env` mein daal deta hai.
- **Project mein:** `app.js` aur `server.js` dono `dotenv.config()` karte hain. `db/db.js` mein `DB_CONNECT` na mile to `process.exit(1)`. Saari `.env` files root `.gitignore` (`**/.env`) se ignore hoti hain.
- ⚠ Exception: `generateSiteMap.js` mein Atlas URI password ke saath hardcoded hai (Part 0). Frontend ke env variables ke liye 7.16.4 dekho.

7.2 Web aur HTTP basics

7.2.1 Client-server model aur SPA

- **Kya hai:** browser (client) request bhejta hai aur server response deta hai. SPA (Single Page Application) mein browser ek hi baar HTML + JavaScript load karta hai; uske baad page badalne pe sirf JSON data aata hai aur React screen badal deta hai. Ise Client-Side Rendering (CSR) kehte hain.
- **Project mein:** chaaron apps Vite SPAs hain, aur backend sirf JSON deta hai (koi HTML template nahi). (detail: 3.1)
- ⚠ CSR mein search engine ko content der se milta hai. Customer store ke SEO ke liye SSR (Next.js) behtar hota (7.15).

7.2.2 REST API

- **Kya hai:** REST ek style hai jisme har "resource" ka ek URL hota hai, aur HTTP method batata hai ki kya karna hai: GET padhna, POST banana, PUT/PATCH badalna, DELETE hatana. Server stateless hota hai: har request apne saath poori jaankari (jaise token) laati hai.
- **Project mein:** cart ek resource hai: `GET /users/cart`, `POST /users/cart`, `PUT /users/cart/:itemId`, `DELETE /users/cart/:itemId` (`routes/cart.routes.js`). Poore backend mein ~140 endpoints hain (4.2).
- ⚠ Kuch jagah REST ke rules toot-te hain: `GET /api/orders/my-orders` GET hote hue bhi DB mein `save()` karta hai (4.15.2), aur har API ka response shape alag hai.

7.2.3 HTTP status codes

- **Kya hai:** status code batata hai ki request ka kya hua. 2xx = success (200 OK, 201 Created). 4xx = client ki galti (400 galat input, 401 pehchaan nahi, 403 permission nahi, 404 nahi mila, 409 conflict). 5xx = server ki galti (500).
- **Project mein:** COD order banne pe 201 (`order.routes.js`); seller dobara register kare to 409 "already exists"; blocked seller pe 403 (`protectSeller`); galat token pe 401. (401 vs 403: 4.3.4)
- ⚠ Kuch codes galat hain: customer register/login ka validation fail hone pe 404 jaata hai (`user.controllers.js:15`), jabki 400 ya 422 hona chahiye.

7.2.4 HTTP headers

- **Kya hai:** request aur response ke saath jaane wala metadata. `Authorization` token le jaata hai, `Content-Type` batata hai ki body kis format mein hai, aur `Content-Disposition: attachment` browser ko file download karne ko kehta hai.
- **Project mein:** har protected call `Authorization: Bearer <JWT>` bhejti hai. Invoice route `Content-Type: application/pdf` aur `Content-Disposition: attachment` set karta hai (4.13.1). Razorpay webhook ke saath `x-razorpay-signature` header aata hai (4.9.6).

7.2.5 JSON aur body parsing

- **Kya hai:** JSON ek text format hai jisme data jaata hai. Server ko us text ko JavaScript object mein badalna padta hai (parsing). Express mein ye kaam `express.json()` middleware karta hai.
- **Project mein:** `app.js` ka ek custom middleware har route pe `express.json()` chalata hai, **sirf** `/api/payment/webhook` ko chhod ke, kyunki wahan raw body chahiye (7.8.4). Form data ke liye `express.urlencoded({ extended: true })`.

7.2.6 CORS (Cross-Origin Resource Sharing)

- **Kya hai:** ye browser ka security rule hai. Ek origin (jaise `localhost:5173`) ka page doosre origin (`localhost:5000`) ki API tabhi call kar sakta hai jab server `Access-Control-Allow-Origin` header se permission de. Ye rule sirf browser pe lagta hai, Postman ya curl pe nahi.
- **Project mein:** chaaron apps alag ports/domains pe hain, isliye CORS zaroori hai. `app.js` mein `app.use(cors())` (bina options = har origin allowed), aur Socket.io mein `new Server(server, { cors: { origin: "*" } })`.
- ⚠ Behtar: `cors({ origin: [chaaron frontend URLs] })` (4.14.2).

7.2.7 `multipart/form-data` aur `FormData`

- **Kya hai:** files bhejne ka HTTP format. Body kai "parts" mein bati hoti hai: text fields aur binary files. Browser mein ye `FormData` object se banta hai, aur server pe Multer jaisa parser chahiye.

- **Project mein:** `Seller-Panel/src/components/Register/Onboarding.jsx:116` : `const fd = new FormData(); fd.append("storeName", ...)` (KYC files ke saath). Review photos, avatar aur product images bhi aise hi jaate hain. Server side: 7.7.1.

7.2.8 Idempotency

- **Kya hai:** koi operation idempotent hai agar use do baar chalane pe bhi result ek baar jaisa hi rahe. Payments mein ye bahut zaroori hai, kyunki network retry ya do alag raaste (verify + webhook) same kaam dobara bhej sakte hain.
- **Project mein:** `markRazorpayOrderPaid()` (`payment.routes.js:50`) order pehle se Paid ho to `return false` kar deta hai, aur `applyCouponUsageIfNeeded` pehle check karta hai ki `CouponUsage` already bani to nahi.
- ⚠ Ye check atomic nahi hai (4.9.8, A2).

7.2.9 Webhook

- **Kya hai:** ek "ulta API call": koi third-party service (yahan Razorpay) koi event hone pe khud tumhare server ko HTTP request bhejti hai. Tumhe baar-baar poochna nahi padta.
- **Project mein:** `POST /api/payment/webhook` (`payment.routes.js` , line ~331): `payment.captured` pe order Paid, `payment.failed` pe user ko email. Ye hamesha 200 lautata hai, kyunki non-2xx milne pe Razorpay webhook dobara bhejta hai. (detail: 4.9.6)

7.2.10 Polling vs push

- **Kya hai:** polling mein client har kuch second poochta hai "kuch naya hai?". Push mein server khud bata deta hai (WebSocket, SSE). Polling simple hai, par bahut saari faltu requests jaati hain.
- **Project mein:** dono use hue hain. Push: Socket.io ka `newDeliveryRequest` (7.6). Polling: rider app ka Nearby tab har 30 second, aur Admin analytics ka optional 30 second polling. Rider app mein polling ek safety net hai, agar socket message miss ho jaaye.

7.2.11 Health check endpoint

- **Kya hai:** ek simple URL jo bataye ki server chal raha hai. Hosting platform aur monitoring tools ise check karte rehte hain.
- **Project mein:** `GET /` aur `GET /check` → `"Hello World!"` , aur `GET /api/server-status` → `{ bootTime: process.uptime() }` (naam "bootTime" hai par value uptime seconds mein hai). (4.2.1)

7.3 Express backend ke concepts

7.3.1 Express app aur Router

- **Kya hai:** Express Node ka chhota sa web framework hai jo URL + method ko ek function se jodta hai. `express.Router()` ek mini-app hai jise kisi URL prefix pe mount kiya jaata hai.
- **Project mein:** `app.js` saare routers mount karta hai, jaise `app.use("/api/orders", orderRoutes)` . Ek hi prefix pe kai routers bhi lag sakte hain (`/users` pe 5 hain). Mount table Section 1.2 mein hai, detail 4.5.1 mein.

7.3.2 Middleware aur `next()`

- **Kya hai:** middleware ek function `(req, res, next)` hai jo request aur final handler ke beech chalta hai. Wo ya to `next()` call karke request aage bhejta hai, ya khud response bhej ke wahin rok deta hai. Kai middleware milke ek chain banate hain.

- **Project mein:** `router.post('/accept-task', protectDelivery, zodValidate(schema), acceptTask)` : pehle auth, phir validation, phir controller. Global chain `app.js` mein: `cors()` → JSON parser → `urlencoded` → static files → routers → `globalErrorHandler` . (detail: 3.2.1)

7.3.3 Route-level vs router-level middleware

- **Kya hai:** route-level middleware sirf ek route pe lagta hai (`router.get('/x', protect, handler)`). Router-level middleware us router ke har route pe lagta hai (`router.use(protect)`).
- **Project mein:** cart routes har route pe alag se `protect` lagate hain (`router.post('/cart', protect, cartController.addToCart)`). `routes/address.routes.js` aur admin ka `banner.routes.js` `router.use(...)` se ek hi baar lagate hain.

7.3.4 Route params, query string aur route order

- **Kya hai:** `/:id` URL ka variable hissa hai (`req.params.id`), aur `?page=2` query string hai (`req.query.page`). Express routes ko upar se neeche match karta hai, isliye literal route (`/my-orders`) ko `/:id` se pehle likhna padta hai, warna "my-orders" ko hi id samajh liya jaayega.
- **Project mein:** `order.routes.js` mein `/my-orders` `/:orderId` se pehle hai, aur `product.routes.js` mein `/navigation/trending-tags` `/:id` se pehle. Listing filters query string mein aate hain: `GET /products?category=…&sort=price_asc&page=2` . (4.2.12)

7.3.5 Static file serving

- **Kya hai:** server ek folder ki files (images, `robots.txt`) seedha URL pe de deta hai, bina koi route likhe.
- **Project mein:** `app.use("/uploads", express.static("uploads"))` (order-rating photos) aur `app.use(express.static('public'))` (`robots.txt` , `sitemap.xml`).
- ⚠ Static `public/sitemap.xml` dynamic `/sitemap.xml` route ko shadow kar deta hai (4.13.4).

7.3.6 Error-handling middleware (central error handler)

- **Kya hai:** Express mein 4 parameters `(err, req, res, next)` wala middleware error handler maana jaata hai. Kahin bhi `next(err)` hua, error yahan pahunchta hai, aur saare errors ka response ek jaisa ban jaata hai.
- **Project mein:** `Middleware/globalErrorHandler.js` , jo `app.js:148` pe sabse aakhir mein mount hai. Production mein `CastError` → 400, duplicate key `11000` → 400, `ValidationError` → 400, aur JWT errors → 401. (detail: 4.6.1)

7.3.7 Custom error class aur operational errors

- **Kya hai:** `class AppError extends Error` se apna error type banta hai jisme status code bhi hota hai. "Operational" error ek expected galti hai (jaise OTP galat), jiska message user ko dikhana safe hai. Programming bug ka asli message production mein chhupaya jaata hai.
- **Project mein:** `utils/AppError.js` (`statusCode` , `status` , `isOperational` , `payload`). Use aise hota hai: `throw new AppError("Task already assigned or not available", 400)` (`acceptTask` mein).

7.3.8 Express 5 vs Express 4

- **Kya hai:** Express 5 async handler ka rejected Promise khud error handler tak bhej deta hai; Express 4 mein aisi request hang ho jaati thi. Express 5 mein `req.query` ek getter hai, use naye object se replace nahi kar sakte.
- **Project mein:** `registerUser` aur `loginUser` bina try/catch ke bhi safe hain. Lekin `zodValidate` ka `req.query = parsed.data` chupchaap ignore ho jaata hai (verified, 4.6.2, T8).

7.3.9 API documentation (Swagger / OpenAPI)

- **Kya hai:** OpenAPI ek standard format hai jisme har endpoint ka input aur output likha jaata hai. Swagger UI usse ek web page bana deta hai jahan se API try bhi kar sakte ho.
- **Project mein:** `swagger.js` (swagger-jsdoc), `docs/api.swagger.js` (~3000 lines ke `@swagger` comments), aur `app.js:87` : `app.use("/api-docs", swaggerUi.serve, swaggerUi.setup(swaggerSpec))` .
- ⚠ Coupons aur banners ke docs missing hain.

7.4 Authentication aur security

7.4.1 Authentication vs Authorization

- **Kya hai:** authentication ka matlab "tum kaun ho?" (login). Authorization ka matlab "tumhe ye kaam karne ki permission hai?" (jaise ye order tumhara hai ya nahi).
- **Project mein:** authentication 4 middlewares karte hain: `protect` , `protectSeller` , `protectAdmin` , `protectDelivery` . Authorization ownership checks se hoti hai, jaise order cancel mein `order.user === req.user._id` . (4.3, 4.3.8)

7.4.2 Stateless auth vs sessions

- **Kya hai:** session mein server yaad rakhta hai ki kaun logged in hai (memory ya Redis mein session ID). Stateless (JWT) mein server kuch yaad nahi rakhta; har request ka token khud saboot hota hai. Stateless ko scale karna asaan hai, par token ko beech mein cancel karna mushkil.
- **Project mein:** koi session ya auth cookie nahi hai, sirf JWT. Isliye "logout" sirf client pe token delete karta hai (D7).

7.4.3 JWT (JSON Web Token)

- **Kya hai:** JWT ke teen hisse hote hain: `header.payload.signature` . Payload (jaise `{ _id, iat, exp }`) sirf base64url encoded hai, encrypted nahi, isliye koi bhi use padh sakta hai. Signature secret key se banta hai, isliye koi payload badle to `jwt.verify` fail ho jaata hai.
- **Project mein:** customer token `user.model.js:96` mein banta hai:

```
userSchema.methods.generateAuthToken = async function () {
  const token = jwt.sign({ _id: this._id }, process.env.JWT_SECRET, { expiresIn: "30d" });
  return token;
};
```

- Rider ka token `{ id, role: "delivery" }` hai (`delivery.controller.js:10`). Chaaron roles ka ek hi `JWT_SECRET` , 30 din expiry, aur refresh token nahi. (4.3.1, 4.3.2)

7.4.4 Bearer token

- **Kya hai:** "jiske paas token hai, wahi andar aayega" (bearer = le jaane wala). Header ka format: `Authorization: Bearer <token>` .
- **Project mein:** `protect` middleware `req.headers.authorization.split(' ')[1]` se token nikaalta hai. Customer app ki har service mein `authHeader(getState)` helper hai (`cartService.js:4`), aur rider app mein axios interceptor token lagata hai (7.13.2).

7.4.5 Hashing, salt aur bcrypt

- **Kya hai:** hashing one-way hai: password se hash ban jaata hai, par hash se password wapas nahi nikalta. Salt ek random value hai jo har hash mein milayi jaati hai, isliye same password ke do hash alag bante hain. bcrypt jaan-boojh ke slow hai, taaki brute force mehenga pade. (Encryption two-way hota hai: key se wapas khul jaata hai.)
- **Project mein:** `userSchema.statics.hashPassword` → `bcrypt.hash(password, 10)` (10 salt rounds), aur login pe `comparePassword` → `bcrypt.compare`. Admin aur rider ke `pre('save')` hooks bhi password hash karte hain.
- ⚠ Bank account aur Aadhar/PAN numbers na hash hote hain na encrypt; plain text mein hain (4.14.3).

7.4.6 `select: false` (sensitive fields chhupana)

- **Kya hai:** Mongoose ka field option, jisse wo field normal queries ke result mein nahi aata. Chahiye ho to `.select('+field')` se maangna padta hai.
- **Project mein:** User/Seller/Rider ka `password`, OTP fields, aur rider ke document aur bank numbers.
- ⚠ Isi se 2 bugs hain: `verifyOtpController` OTP padh hi nahi paata, aur login `+password` select karke poora user lautata hai, isliye hash response mein leak hota hai (T5).

7.4.7 Role-based access control (RBAC)

- **Kya hai:** user ke role (customer, seller, admin, rider) ke hisaab se permission dena. Aam taur pe token mein `role` claim hota hai aur middleware use check karta hai.
- **Project mein:** har role ka alag collection aur alag middleware hai. Sirf rider token mein `role` hai: `protectDelivery` `decoded.role !== "delivery"` pe 403 deta hai. Baaki roles isliye alag rehte hain kyunki har middleware apne hi collection mein ID dhoondhta hai. (4.3.2)

7.4.8 Optional authentication

- **Kya hai:** token ho to user ko pehchano, na ho to bhi request chalne do. Isse ek hi page logged-in aur guest dono ke liye kaam karta hai.
- **Project mein:** `Middleware/optionalAuth.js` `/home/products` pe lagta hai. Token na ho ya galat ho, to bas `next()`:

```
if (!authHeader || !authHeader.startsWith("Bearer ")) return next();
// ... jwt.verify fail ho to bhi return next() ...
const user = await User.findById(decoded._id).select("_id").lean();
if (user) req.user = user;
next();
```

- Controller user mile to "recommended" products deta hai, warna "popular".

7.4.9 OTP (One-Time Password)

- **Kya hai:** thodi der ke liye valid ek random code, jo email ya SMS pe bheja jaata hai, ye saabit karne ke liye ki email tumhara hi hai. Expiry aur galat attempts ki limit zaroori hoti hai.
- **Project mein:** 6-digit OTP: `Math.floor(100000 + Math.random() * 900000)` (`user.controllers.js:94`, `sellerPanelService.js:50`). Customer reset ka OTP 15 min, seller registration ka 5 min.
- ⚠ `Math.random` cryptographically secure nahi hai (`crypto.randomInt` behtar), attempt limit nahi hai, aur customer reset flow toota hua hai (4.3.6).

7.4.10 HMAC (signature se message verify karna)

- **Kya hai:** HMAC = message + secret key se bana hash. Sahi HMAC sirf wahi bana sakta hai jiske paas secret hai, isliye isse pakka hota hai ki message asli sender se aaya hai aur raste mein badla nahi gaya.
- **Project mein:** Razorpay verify (`payment.routes.js:185`):

```
const expectedSignature = crypto.createHmac('sha256', secret)
  .update(`${razorpay_order_id}|${razorpay_payment_id}`).digest('hex');
if (expectedSignature !== razorpay_signature) return 400; // (chhota kiya hua)
```

- Webhook bhi raw body pe HMAC check karta hai (`RAZORPAY_WEBHOOK_SECRET` se).
- ⚠ Compare `!==` se hota hai; `crypto.timingSafeEqual` behtar hai (timing attack, 4.9.8).

7.4.11 IDOR (Insecure Direct Object Reference)

- **Kya hai:** sirf URL mein ID badal ke kisi aur ka data mil jaana, kyunki server ye check hi nahi karta ki "ye cheez tumhari hai ya nahi".
- **Project mein:** achha example: productinfo `findOne({ _id, sellerId })` , rider task `{ _id, deliveryPartner: req.user._id }` . Missing: `GET /api/orders/:orderId` , invoice/label PDF, `/api/dashboard/stats/:sellerId` (4.3.8).

7.4.12 Mass assignment

- **Kya hai:** client ki poori body seedha DB update mein daal dena. Attacker extra fields (jaise `status: "approved"`) bhej ke apni permission khud badal leta hai. Fix: allow-list, yaani sirf chune hue fields accept karo.
- **Project mein:** bugs: `PUT /api/seller/profile` mein `{ $set: req.body }` + `runValidators: false` , aur COD route `paymentStatus` body se leta hai. Bachav ka example: Zod unknown keys strip kar deta hai (`registerPartner`). (4.14.3)

7.4.13 Injection: NoSQL operator injection aur ReDoS

- **Kya hai:** NoSQL injection mein string ki jagah object bheja jaata hai, jaise `{"email": {"$ne": null}}` . Query ka matlab ban jaata hai "jiska email null nahi hai", aur DB ka pehla user mil jaata hai. ReDoS mein user ka regex pattern itna complex hota hai ki server ka CPU uspe atak jaata hai.
- **Project mein:** seller login `Seller.findOne({ email: { $eq: email } })` safe hai. Customer forgot/reset password mein `email` ka type check nahi hai (bug 7a, Section 4.14.3). ReDoS se bachav `escapeRegex()` se hota hai (7.1.11).

7.4.14 XSS aur CSRF

- **Kya hai:** XSS mein attacker ka JavaScript tumhare page pe chal jaata hai (aur localStorage ka token padh sakta hai). CSRF mein koi doosri website tumhare browser se tumhare naam pe request bhej deti hai (kyunki cookies apne aap jaati hain). React text ko default escape karta hai, isliye normal rendering mein XSS mushkil hai.
- **Project mein:** token localStorage mein hai (XSS ka risk, CSRF ka nahi). `dangerouslySetInnerHTML` sirf `FrontEnd/src/utils/SEO.jsx` ke JSON-LD mein hai. Seller panel ki packing slip `escapeHtml()` use karti hai. (4.14.4, 4.3.9)

7.4.15 Token storage: localStorage vs httpOnly cookie, aur refresh token

- **Kya hai:** localStorage ko JavaScript padh sakti hai, isliye XSS mein token chori ho sakta hai. httpOnly cookie ko JavaScript nahi padh sakti, par phir CSRF protection lagani padti hai. Best practice: short-lived access token (memory mein) + refresh token (httpOnly cookie) + rotation.

- **Project mein:** chaaron apps localStorage use karti hain (`user` , `sellerToken` , `adminToken` , `deliveryToken`), token 30 din ka hai, aur refresh token nahi. (4.3.9, A4)

7.4.16 Rate limiting, helmet, user enumeration, timing attack (ye sab missing hain)

- **Kya hai:**
  - Rate limiting: ek IP ya user ki requests per minute ki limit, jo brute force rokti hai.
  - helmet: security headers (HSTS, `X-Content-Type-Options` jaise) lagane wala middleware.
  - User enumeration: alag-alag error messages se pata chal jaana ki koi email registered hai.
  - Timing attack: compare karne mein lage time se secret ka andaaza lagana.
- **Project mein:** koi `express-rate-limit` ya `helmet` nahi hai. Customer login "User not found" (404) aur "Invalid credentials" (400) alag deta hai (4.3.5). Signature compare `!==` se hai. (4.14.2)

7.4.17 Secrets aur pre-signed URLs

- **Kya hai:** secret (API key secret, DB password) sirf server ke env mein rehna chahiye. Pre-signed URL ek private file ka thodi der ke liye valid link hota hai, taaki sensitive file public na rahe.
- **Project mein:** Razorpay ka `key_id` browser ko jaata hai (ye public key hai, safe hai), lekin `key_secret` kabhi nahi.
- ⚠ KYC documents (Aadhar/PAN) public CloudFront URL pe hain; pre-signed URL behtar hota (4.8.4).

7.5 MongoDB aur Mongoose

7.5.1 NoSQL document database

- **Kya hai:** MongoDB data ko JSON jaise "documents" mein rakhta hai, tables aur rows mein nahi. Documents "collections" mein rehte hain, har document ki ek unique `_id` (ObjectId) hoti hai, aur ek document ke andar arrays aur nested objects bhi ho sakte hain.
- **Project mein:** 17 collections hain (users, products, orders, deliverytasks…). Ek order document ke andar `products[]` array mein saare items hain. Production mein MongoDB Atlas (cloud), aur local pe replica set (Appendix A). (D2)

7.5.2 ODM, Schema aur Model (Mongoose)

- **Kya hai:** Mongoose ek ODM (Object Data Modeling) library hai. Schema data ka shape aur rules batata hai; Model us schema se bani class hai jisse queries chalti hain (`User.findById`).
- **Project mein:** `Backend/models/` ki 17 files, jaise `mongoose.model('User', userSchema)` . Connection `db/db.js` mein: `mongoose.connect(DB_CONNECT, { serverSelectionTimeoutMS: 5000, socketTimeoutMS: 45000, maxPoolSize: 50 })` .

7.5.3 Schema validation

- **Kya hai:** `required` , `enum` , `min` , `minlength` aur custom `validate` jaise rules `save()` aur `create()` pe chalte hain. Query updates (`findByIdAndUpdate`) pe ye default mein nahi chalte; uske liye `runValidators: true` dena padta hai.
- **Project mein:** `Order.products[].quantity` pe `min: 1` , `Product.images` pe custom validator `val.length <= 5` , Address pe pincode regex.
- ⚠ `updateSellerProfile` mein `runValidators: false` likh ke validation band ki gayi hai (4.1.13).

7.5.4 Setters

- **Kya hai:** field pe value set hote waqt chalne wala function, jo data ko ek jaisa (normalize) kar deta hai.
- **Project mein:** `Product.tags` pe `set: normalizeTagsArray`: har tag lowercase + trim, aur duplicates hat jaate hain.
- ⚠ Setter query updates pe nahi chalta, isliye alag hook lagaya gaya hai (7.5.5).

7.5.5 Mongoose middleware (hooks)

- **Kya hai:** `pre('save')` ya `pre('findOneAndUpdate')` jaise hooks kisi operation se pehle (ya baad) apne aap chalte hain. Inme hashing, ID banana, data normalize karna jaise kaam hote hain.
- **Project mein:**
  - `order.model.js:164` `pre('save')`: `orderId` ("OD…") aur har item ka `itemId` banata hai.
  - `address.model.js` `pre('save')`: naya default address baaki addresses ko non-default kar deta hai.
  - `deliveryPartner.model.js` aur `admin.model.js` `pre('save')`: password hash. ⚠ Admin wale hook mein `return next()` missing hai (double-hash bug, 4.1.4).
  - `product.model.js:165` `pre(["findOneAndUpdate","updateOne","updateMany"])`: updates mein bhi tags normalize (T7).

7.5.6 Instance methods vs static methods

- **Kya hai:** instance method ek document pe chalta hai (`user.comparePassword()`). Static method poore model pe chalta hai (`User.hashPassword()`), jab document abhi bana bhi na ho.
- **Project mein:** `user.model.js` mein `userSchema.methods.generateAuthToken`, `userSchema.methods.comparePassword` aur `userSchema.statics.hashPassword`. Admin ka `matchPassword()` bhi instance method hai.

7.5.7 `timestamps` option

- **Kya hai:** schema mein `{ timestamps: true }` dene se Mongoose `createdAt` aur `updatedAt` apne aap set aur update karta hai.
- **Project mein:** User, Address, Coupon, CouponUsage, Category, Banner, Notification aur DeliveryTask mein hai. Product, Order, Seller aur Blog mein nahi hai (wahan manual `createdAt` ya `dateJoined` hai, aur Blog ka `updatedAt` kabhi update nahi hota, 4.1.12).

7.5.8 References aur `populate()`

- **Kya hai:** ek document doosre document ki sirf `ObjectId` rakhta hai (`ref: 'Product'`), aur `populate()` query ke waqt asli document laake jod deta hai. Ye SQL join jaisa hai, par andar se alag query chalti hai.
- **Project mein:** `controllers/cart.controllers.js:21`:

```
const cart = await Cart.findOne({ user: req.user._id })
  .populate({ path: 'items.product', select: PRODUCT_POPULATE_SELECT });
```

- Invoice ke liye order ke `products.product` aur `user` bhi populate hote hain.

7.5.9 Polymorphic reference (`refPath`)

- **Kya hai:** ek hi field alag-alag collections ko point kar sake, aur ek doosra field bataye ki kis model mein dhoondhna hai.

- **Project mein:** `notification.model.js`: `recipient` (`refPath: 'recipientModel'`) + `recipientModel` (enum `User / Seller / Admin`). (4.1.12, A14)

7.5.10 Embedded vs referenced, denormalization aur soft delete

- **Kya hai:**
  - Embedded: child data parent document ke andar (ek query mein sab aata hai, update atomic hota hai, par document ki 16 MB limit hai).
  - Referenced: alag collection, sirf ID store hoti hai.
  - Denormalization: jaan-boojh ke data copy karna (snapshot).
  - Soft delete: document delete karne ki jagah `isDeleted: true` set karna.
- **Project mein:** embedded `Order.products[]`, `Product.reviews[]`, `Cart.items[]`. Snapshot: order item mein `price`, `productName`, `storeName` (D3). Soft delete: `Product.isDeleted` + `deletedAt` (D10).
- ⚠ Admin ka product delete hard delete hai (D10).

7.5.11 Indexes (single, compound, unique, multikey)

- **Kya hai:** index ek sorted lookup structure hai, kitaab ke index jaisa, jisse query poori collection scan kiye bina jaldi chalti hai. Compound index kai fields ka ek saath hota hai (fields ka order matter karta hai). Unique index duplicate value rokta hai (error code 11000). Multikey index array field pe banta hai.
- **Project mein:** `product.model.js:175–182`: `{approvalStatus:1, isDeleted:1, inStock:1, rating:-1}`, `{sellerId:1, rating:-1, createdAt:-1}`, aur `{tags:1}` (multikey). Unique: `User.email`, `Seller.email` aur `phone`, `Coupon.code`.
- ⚠ Missing: `Order.user`, `Order.products.sellerId`, `Order.razorpayOrderId`, aur `Cart.user` (A6).

7.5.12 TTL index

- **Kya hai:** Date field pe ek special index, jisse MongoDB documents ko unka time nikalne ke baad khud delete kar deta hai. Iska background monitor lagbhag har 60 second mein chalta hai.
- **Project mein:** `order.model.js:139`: `orderSchema.index({ expireAt: 1 }, { expireAfterSeconds: 0 })`. Online Pending order pe `expireAt = now + 30 min` lagta hai; Paid hone pe `expireAt = undefined` se field hat jaata hai aur order bach jaata hai. (A7)

7.5.13 Aggregation pipeline

- **Kya hai:** data pe stages ki ek chain: `$match` (filter), `$unwind` (array ke har item ko alag document banana), `$group` (sum/count), `$lookup` (join), `$addFields` (naya calculated field), `$sort` / `$skip` / `$limit`, `$project` (fields chunna), `$facet` (ek input se kai alag results). Saara kaam DB ke andar hota hai, isliye network pe kam data aata hai.
- **Project mein:** seller ka sales graph (`servicesSellerPanel/statsService.js`, `getSalesOverTime`):

```
const results = await Order.aggregate([
  { $unwind: "$products" },
  { $match: { "products.sellerId": new mongoose.Types.ObjectId(sellerId),
              createdAt: { $gte: start, $lte: end } } },
  { $group: { _id: { $dateToString: { format: "%Y-%m-%d", date: "$createdAt" } },
              totalSales: { $sum: { $multiply: ["$products.price", "$products.quantity"] } } } },
  { $sort: { _id: 1 } },
]);
```

- Product listing mein `$addFields` se `offeredPrice` aur `$facet` se count + current page ek hi query mein (H3). Badge cron ke 90/7 din ke sales aur home page ke popular tags bhi aggregation se.

7.5.14 GeoJSON, 2dsphere index aur `$geoNear`

- **Kya hai:** GeoJSON Point = `{ type: "Point", coordinates: [lng, lat] }`, yaani **longitude pehle**. 2dsphere index prithvi ko gola maan ke distance nikaalta hai. `$geoNear` aggregation ka pehla stage hota hai, jo paas-paas ke documents distance ke saath sorted deta hai.
- **Project mein:** `DeliveryPartner.currentLocation` aur `DeliveryTask.pickupLocation` / `dropLocation` pe 2dsphere index. `broadcastToNearbyRiders` mein `maxDistance: 5000` (5 km), `query: { isActive: true, isAvailable: true }`, aur max 10 riders. (4.13.3)
- ⚠ Coordinates abhi placeholder `[77.209, 28.613]` hain (T3).

7.5.15 Transactions, sessions aur replica set (ACID)

- **Kya hai:** transaction kai document updates ko "sab ho, ya ek bhi na ho" bana deta hai (ACID: Atomic, Consistent, Isolated, Durable). MongoDB mein: `session.startTransaction()` → saare updates `{ session }` ke saath → `commitTransaction()`, ya error pe `abortTransaction()`. Transactions sirf replica set (ya sharded cluster) pe chalte hain.
- **Project mein:** `acceptTask`, `updateTaskStatus`, `cancelTask` (`controllersDeliveryPanel/delivery.controller.js`, lines 232/319/381/438 pe `startSession` / `startTransaction`) aur seller ka `markAsPickedUp`. Atlas replica set hi hota hai; local pe port 27018 pe single-node replica set chalaya gaya (T6).

7.5.16 Race condition aur atomic conditional update (optimistic concurrency)

- **Kya hai:** race condition tab hoti hai jab do requests ek saath "padho → check karo → likho" karein aur dono jeet jaayein. Fix: check ko update ke filter ke andar daal do. MongoDB ka single-document update atomic hai, to sirf ek request match karegi. Ise optimistic concurrency kehte hain: pehle se lock nahi, likhte waqt check.
- **Project mein:** `acceptTask` mein `DeliveryTask.findOneAndUpdate({ _id: taskId, status: "Pending" }, { status: "Accepted", ... }, { session })`, aur rider lock `{ _id, isAvailable: true }` (H1).
- ⚠ `assignDefaultRider` aur `markRazorpayOrderPaid` ye pattern follow nahi karte (A1, A2).

7.5.17 `bulkWrite`, `.lean()`, projection aur connection pool

- **Kya hai:**
  - `bulkWrite`: kai updates ek hi DB round-trip mein.
  - `.lean()`: Mongoose document ki jagah plain JavaScript object (kam memory, tez, par methods nahi milte).
  - Projection: sirf zaroori fields mangwana (`.select('name price')`).
  - Connection pool: pehle se khule DB connections jo har request pe reuse hote hain.
- **Project mein:** badge cron `Product.bulkWrite(bulkOps)` (`services/badgeAutomationService.js:139`); `optionalAuth` mein `.select("_id").lean()`; `db/db.js` mein `maxPoolSize: 50`. (4.15.1)

7.5.18 N+1 query problem aur `$in`

- **Kya hai:** list ke har item ke liye ek alag query (1 list query + N item queries). Fix: ek hi query mein saare IDs mangwao (`find({ _id: { $in: ids } })`), ya aggregation use karo.
- **Project mein:** achha: `buildPaymentOrderItems` mein `Product.find({ _id: { $in: productIds } })`. N+1: COD route har product pe alag `findById`, aur admin `getAllSellers` har seller pe alag `countDocuments` (4.15.2).

7.5.19 Pagination (offset-based)

- **Kya hai:** data ko pages mein dena: `skip = (page − 1) × limit`, phir `limit` jitne items. Bahut aage ke pages pe `$skip` slow ho jaata hai; tab cursor-based pagination ("is ID ke baad ke 15") behtar hai.
- **Project mein:** `GET /products` default 15, max 50 (`$facet` ke andar `$skip` + `$limit`). Home APIs mein `clampLimit(raw, def, max)` (`controllers/home.controllers.js:10`).
- ⚠ Admin aur seller ki lists browser mein paginate hoti hain, yaani server poori list bhejta hai.

7.5.20 Common Mongo errors: E11000, CastError, ValidationError

- **Kya hai:** `E11000` (code 11000) = unique index pe duplicate value. `CastError` = galat format ki ID (jaise `"abc"` ko ObjectId banane ki koshish). `ValidationError` = schema ka koi rule toota.
- **Project mein:** `globalErrorHandler` production mein teeno ko friendly 400 message mein badalta hai. Newsletter controller duplicate email (`11000`) pe 400 lautata hai.

7.6 Real-time: WebSocket aur Socket.io

7.6.1 WebSocket

- **Kya hai:** ek connection jo khula rehta hai, aur client aur server dono kabhi bhi message bhej sakte hain. HTTP mein sirf client poochta hai; WebSocket mein server khud bhi bol sakta hai.
- **Project mein:** rider app ka client `transports: ["websocket"]` use karta hai, yaani long-polling fallback band hai (`delivery-dashboard/src/services/socket.js`).

7.6.2 Socket.io: connection, events, `emit` / `on`

- **Kya hai:** WebSocket ke upar ek library. Isme naam wale events hote hain (`socket.emit("x", data)`, `socket.on("x", fn)`), auto-reconnect, aur rooms. `io.to(socketId).emit()` sirf ek client ko bhejta hai, `io.emit()` sabko.
- **Project mein:** `server.js` usi `http.createServer(app)` pe Socket.io lagata hai, yaani REST aur socket ek hi port pe. Events: client → server `registerRider`, server → client `newDeliveryRequest`, aur built-in `connection` / `disconnect`. (4.7)

7.6.3 Rooms aur multi-server scaling (Redis adapter)

- **Kya hai:** room ek naam wala group hai jisme kai sockets ho sakte hain (`socket.join("rider:42")`). Kai server instances hon to sockets alag-alag servers pe hote hain; Redis adapter un servers ko aapas mein jodta hai.
- **Project mein:** rooms use **nahi** hote. In-memory `Map` (riderId → ek socketId) hai, isliye 2 tabs aur 2 instances dono pe problem hai (T9, A5).

7.6.4 Socket authentication (missing)

- **Kya hai:** socket connect hote waqt (handshake mein) token verify karna, taaki koi aur kisi ki identity na le sake. Socket.io mein ye `io.use((socket, next) => ...)` middleware se hota hai.
- **Project mein:** ye nahi hai. Koi bhi client `registerRider` mein kisi bhi rider ki ID bhej ke uski requests sun sakta hai (4.7.6).

7.6.5 Client socket: singleton, `autoConnect` aur cleanup

- **Kya hai:** poori app mein ek hi socket connection (singleton). `autoConnect: false` ka matlab: login ke baad hi `socket.connect()`. Component hatne (unmount) pe listeners hatana (`socket.off`) zaroori hai, warna handlers double ho jaate hain.

- **Project mein:** `services/socket.js` singleton hai. `hooks/useRiderSocket.js` connect hone pe `emit("registerRider", riderId)` karta hai, aur `on("newDeliveryRequest")` pe Redux mein task daalta hai.
- ⚠ Cleanup mein sirf `disconnect()` hai, `socket.off` nahi (listener leak, 4.7.6).

7.7 File upload aur media

7.7.1 Multer (memoryStorage vs diskStorage) aur Buffer

- **Kya hai:** Multer Express ka middleware hai jo `multipart/form-data` parse karta hai. `memoryStorage` file ko RAM mein `Buffer` (raw bytes) ki tarah rakhta hai; `diskStorage` file server ki disk pe likhta hai. `limits.fileSize` se size ki limit lagti hai.
- **Project mein:** `addProduct.routes.js:9` `multer.memoryStorage()` + `upload.fields([...])` (5 images + 1 video). Order-rating photos `diskStorage` se `uploads/` folder mein jaati hain.
- ⚠ Kai configs mein size limit nahi hai (memory DoS ka risk, 4.8.3, 4.8.4).

7.7.2 Object storage (AWS S3)

- **Kya hai:** files ko ek "bucket" mein "key" (path jaisa naam) ke saath rakhna. Ye sasta aur durable hai, aur server ki disk pe nirbhar nahi, isliye kai servers ke saath bhi kaam karta hai.
- **Project mein:** `utils/s3Upload.js` : `PutObjectCommand({ Bucket, Key, Body: fileBuffer, ContentType })` . Key ka format `folder/<timestamp>-<uuid>-<safeName>` (`crypto.randomUUID()`), aur `sanitizeFileName` ajeeb characters aur path traversal hata deta hai. S3 client ek lazy singleton hai. (4.8.2)

7.7.3 CDN (CloudFront)

- **Kya hai:** Content Delivery Network: duniya bhar ke edge servers file ki copy rakhte hain, isliye user ko file paas wale server se jaldi milti hai, aur original storage (S3) pe load kam padta hai.
- **Project mein:** `uploadToS3` `{ secure_url: CLOUDFRONT_URL/key, key }` lautata hai, aur wahi CloudFront URL MongoDB mein save hota hai.

7.7.4 MIME type aur file validation

- **Kya hai:** MIME type batata hai ki file kya hai (`image/png` , `video/mp4`). Ye value client bhejta hai, isliye nakli bhi ho sakti hai. Asli check ke liye file ke shuru ke bytes ("magic bytes") padhe jaate hain.
- **Project mein:** avatar pe mimetype list, reviews pe extension + mimetype regex `/jpeg|jpg|png|webp/` , aur product video `mp4/webm/quicktime` ≤ 20 MB.
- ⚠ Product video ka check images upload hone ke **baad** hota hai, aur KYC/banner uploads pe koi check nahi (4.8.3).

7.7.5 Data migration script (dry run)

- **Kya hai:** ek baar chalne wali script jo purana data naye format ya naye provider mein le jaaye. "Dry run" ka matlab: bina kuch badle dikhao ki kya-kya badlega.
- **Project mein:** `scripts/migrateCloudinaryToS3.js` (`npm run migrate:s3` , aur dry run ke liye `migrate:s3:dry`), jo failures ki report file mein likhti hai. `seedFile/migrateProducts.js` purane `price` / `originalPrice` fields ko `sellingPrice` / `mrp` mein badalta hai.

7.7.6 Streaming (PDF)

- **Kya hai:** poori file memory mein banane ki jagah thoda-thoda data seedha response mein bhejte jaana (`stream.pipe(res)`). Isse memory kam lagti hai aur download jaldi shuru hota hai.

- **Project mein:** `utils/invoiceGenerator.js` PDFKit se `doc.pipe(res)` karta hai; shipping label A6 size ka hai (`utils/shippingLabelGenerator.js`). (4.13.1)

7.7.7 Blob download (browser side)

- **Kya hai:** Blob browser mein binary data (jaise PDF) ka object hai. `URL.createObjectURL(blob)` se ek temporary link banta hai, jisse file download karwayi jaati hai.
- **Project mein:** `FrontEnd/src/Redux/documentService.js`: axios `responseType: "blob"`, phir `window.URL.createObjectURL(new Blob([data]))` (invoice aur label download).

7.8 Payments

7.8.1 Payment gateway aur Razorpay ka order flow

- **Kya hai:** payment gateway card ya UPI payment ko bank tak le jaata hai. Razorpay ka flow: server pe order banao (amount server tay kare) → browser mein checkout popup → server pe signature verify.
- **Project mein:** `/api/payment/create-order` → `razorpay.orders.create({ amount: amountPaise, currency: 'INR', ... })` → `Checkout.jsx` mein `new window.Razorpay(options).open()` → `/api/payment/verify`. (4.9, I2)

7.8.2 Paise (sabse chhoti currency unit)

- **Kya hai:** gateways amount sabse chhoti unit mein lete hain (₹1 = 100 paise), taaki decimal ki galtiyaan na hon.
- **Project mein:** `const amountPaise = Math.round(pricing.totalAmount * 100)` (`payment.routes.js`, create-order).

7.8.3 Server-side pricing (client pe bharosa nahi)

- **Kya hai:** price, discount aur tax hamesha server DB se calculate kare; client ka bheja price sirf dikhane ke liye hota hai.
- **Project mein:** `utils/buildPaymentOrderItems.js` (price DB se), `utils/resolveOrderPricing.js` (5% GST + coupon), `utils/productForClient.js` (`offeredPrice`). (D5, I3)

7.8.4 Raw body aur webhook signature

- **Kya hai:** webhook ka signature request body ke exact bytes pe banta hai. JSON parse karke wapas string banao to bytes badal sakte hain, isliye webhook route ko raw (bina parse ki) body chahiye.
- **Project mein:** `app.js` webhook URL pe `express.json()` skip karta hai, aur route `express.raw({ type: "application/json" })` (`payment.routes.js:332`) se Buffer leta hai (T2).

7.8.5 COD aur payment states

- **Kya hai:** Cash on Delivery mein paisa delivery ke waqt milta hai, aur order turant confirm ho jaata hai. Payment ke states (Pending → Paid → Refunded/Failed) saaf define hone chahiye.
- **Project mein:** `paymentMethode` enum `Online / COD`, aur `paymentStatus` sirf `Pending / Paid`.
- ⚠ COD order kabhi Paid mark nahi hota, refund nahi hai, aur COD route `paymentStatus` body se le leta hai (4.9.7, A11).

7.9 Background jobs, caching aur email

7.9.1 Cron job aur cron syntax

- **Kya hai:** tay time pe apne aap chalne wala kaam. Cron expression ke 5 hisse hote hain: minute, hour, day-of-month, month, day-of-week. `"0 0 * * *"` matlab roz 00:00 baje.
- **Project mein:** `jobs/badgeCron.js`: `cron.schedule("0 0 * * *", ..., { scheduled: true, timezone: "UTC" })` (node-cron), aur `server.js` DB connect hone ke baad `initBadgeCron()` call karta hai.
- ⚠ Kai server instances pe ye har instance pe chalega (4.12).

7.9.2 Rule-based automation (badges)

- **Kya hai:** data dekh ke fixed rules ke hisaab se label lagana. Isme koi ML ya training nahi hoti.
- **Project mein:** `runBadgeAutomation()`: bestSeller = 90 din mein ≥ 15 units, trending = 7 din mein ≥ 3, topRated = rating ≥ 4.5 aur ≥ 3 reviews, limitedStock = stock 1–10. `featured` sirf admin set karta hai. (4.12)

7.9.3 Caching: cache-aside aur TTL

- **Kya hai:** baar-baar maange jaane wale data ki copy kisi tez jagah (memory ya Redis) mein rakhna. Cache-aside: pehle cache dekho; miss ho to DB se lao aur cache mein rakh do. TTL batata hai ki copy kitni der baad purani maani jaayegi.
- **Project mein:** `services/home.services.js` mein `withTtlCache(key, ttlMs, fetcher)` aur `responseCache = new Map()`. TTL: tags/categories 3 min, sellers 5 min, popular products 2 min. Sirf sabke liye same (anonymous) data cache hota hai. (4.11)

7.9.4 Cache invalidation aur cache stampede

- **Kya hai:** invalidation = data badalne pe purani cache hata dena. Stampede = cache expire hote hi saari requests ek saath DB pe pahunch jaana.
- **Project mein:** invalidation nahi hai (admin ka approve kiya product home page pe 2–5 min baad dikhega), stampede se bachav nahi hai, aur har instance ki apni alag cache hai (4.11).

7.9.5 Transactional email: API vs SMTP

- **Kya hai:** transactional email system ki automatic mail hoti hai (OTP, order confirmation). Provider ki API (jaise Resend) ko HTTP call karke bhejte hain. SMTP purana email protocol hai (jaise nodemailer + Gmail), jisme sending limits hoti hain.
- **Project mein:** `utilsSellerPanel/sendEmail.js` Resend use karta hai; `EMAIL_MODE=test` ho to saari mails ek developer inbox pe jaati hain. Newsletter ka nodemailer wala code dead hai (route mount nahi). (4.10)

7.9.6 HTML email templates

- **Kya hai:** email ka HTML ek string hota hai jisme `${}` se values daali jaati hain. User ka input daalne se pehle use escape karna chahiye.
- **Project mein:** `utils/forgotPasswordTemplet.js`, `utilsSellerPanel/sellerVerifyTemplate.js`, `utils/userPaymentFailed.js`.
- ⚠ Contact-us aur order emails mein values escape nahi hoti (4.10.2, 4.14.3).

7.10 React ke concepts

7.10.1 Component, JSX aur props

- **Kya hai:** component ek function hai jo UI lautata hai. UI JSX mein likhte hain (JavaScript ke andar HTML jaisa syntax). Props parent component se child ko data dene ka tareeka hai, jisse ek component kai jagah reuse hota hai.
- **Project mein:** `ProductCard` (listing ka har card), `BackButton`, Admin ka `Pagination` aur `ConfirmationModal`. Admin `AdminLayout` ko `onLogout` prop milta hai.
- ⚠ `/riders` route pe `onLogout` prop diya hi nahi gaya (4.3.7).

7.10.2 State (`useState`)

- **Kya hai:** component ki apni yaad, jaise form input ya loading flag. `setX(...)` call hone pe React component dobara render karta hai.
- **Project mein:** Admin-Panel ka har page: `const [data, setData] = useState([]); const [loading, setLoading] = useState(true);` (4.4.5). Checkout ka chuna hua address, payment method aur coupon bhi local state mein hain.

7.10.3 `useEffect`: side effects, dependencies aur cleanup

- **Kya hai:** render ke baad chalne wala code, jaise API call, timer ya event listener. Dependency array batata hai ki effect dobara kab chalega. Effect jo function return karta hai wo "cleanup" hai, jo component hatne ya effect dobara chalne se pehle chalta hai.
- **Project mein:** rider app ki polling (`delivery-dashboard/src/components/DeliveryPanel/DeliveryPanel.jsx:319-325`):

```
useEffect(() => {
  if (mainTab !== "Nearby Radar") return;
  const intervalId = setInterval(() => {
    dispatch(getNearbyDeliveries());
  }, 30000);
  return () => clearInterval(intervalId);
}, [mainTab, dispatch]);
```

- Page guard bhi `useEffect` se: `useEffect(() => { if (!user || !token) navigate("/login", { replace: true }); }, [user, token])` (`Checkout.jsx:46`).

7.10.4 Custom hooks

- **Kya hai:** `use` se shuru hone wala apna function, jo React hooks (useEffect, useDispatch) ko jod ke reusable logic banata hai.
- **Project mein:** `FrontEnd/src/hooks/useCartPersistence.js` (login hote hi cart aur favorites fetch) aur `delivery-dashboard/src/hooks/useRiderSocket.js` (socket connect aur events sunna).

7.10.5 `useMemo` aur `React.memo` (memoization)

- **Kya hai:** `useMemo` ek mehenga calculation tabhi dobara karta hai jab uske inputs badlein. `React.memo` component ko same props pe dobara render hone se bachata hai.
- **Project mein:** `useMemo` FrontEnd ke `OrderDetails`, `ProductReview`, `OrderReview`, `Scrollsection`, `LatestBlogSection`, `shopsection` mein; Seller ke `ProductPage`, `OrderTable`, `ProductTable` mein; aur Admin ke `OrderDetailsPage` mein hai. `React.memo` kahin nahi hai (grep se check kiya).

7.10.6 `StrictMode`

- **Kya hai:** development ka ek wrapper jo extra checks karta hai. React 18+ mein dev mode mein ye components ko mount → unmount → dobara mount karta hai, isliye effects do baar chalte hain (bugs pakadne ke liye). Production build pe iska koi asar nahi.
- **Project mein:** `FrontEnd/src/main.jsx` mein `HelmetProvider > StrictMode > Provider(store) > App`. **(inference)** Dev mode mein dikhne wali kuch duplicate API calls ka ek kaaran ye ho sakta hai, lekin sabka nahi: jaise Seller-Panel ka nested Layout ek alag code-level kaaran hai (4.15.3).

7.10.7 Conditional rendering aur route guards

- **Kya hai:** condition ke hisaab se alag UI dikhana: `cond ? <A/> : <B/>` ya `cond && <A/>`. Route guard ka matlab: login nahi hai to login page pe bhejo.
- **Project mein:** Admin `App.jsx`: `isAdmin ? <AdminLayout><Page/></AdminLayout> : <Navigate to="/login"/>`; rider app mein `isDelivery ? ... : <Navigate/>`. FrontEnd mein `ProtectedRoute` component nahi hai (har page ka apna `useEffect` guard), aur Seller-Panel mein guard hai hi nahi. (4.3.7)

7.10.8 Error boundary

- **Kya hai:** ek class component jo apne andar ke components ke render errors pakad leta hai aur fallback UI dikhata hai, taaki poori app blank na ho jaaye.
- **Project mein:** sirf reviews section pe hai (`ReviewsErrorBoundary`). App-level boundary nahi hai, isliye `/change-password` ka crash poori app ko blank kar deta hai (4.6.3).

7.10.9 Optimistic UI aur rollback

- **Kya hai:** server ka jawab aane se pehle hi UI badal do (app tez lagti hai); request fail ho to purani state wapas le aao.
- **Project mein:** sirf Admin badges mein (`Admin-Panel/pages/ProductPage/ProductPage.jsx:129`): snapshot → turant `setProducts` → `axios.put` → error pe `setProducts(previousProducts)` (4.4.5). Cart optimistic **nahi** hai; wo server ke response se replace hota hai.

7.10.10 Code splitting aur lazy loading

- **Kya hai:** `React.lazy(() => import('./Page'))` + `Suspense` se har page ka code alag file mein jaata hai, aur zaroorat pe hi download hota hai. Images pe `loading="lazy"` lagane se image tabhi load hoti hai jab screen pe aane wali ho.
- **Project mein:** `React.lazy` kahin nahi hai (grep se check kiya), saare pages ek hi bundle mein. `loading="lazy"` kuch images pe hai (ProductCard, blogs, shops). (4.15.3)

7.10.11 Forms aur client-side validation

- **Kya hai:** controlled input mein input ki value state mein rehti hai aur `onChange` se update hoti hai. Client-side validation sirf UX ke liye hai (jaldi feedback); security ke liye server validation zaroori hai, kyunki koi bhi seedha API call kar sakta hai.
- **Project mein:** Seller register: phone `/^\d{10}$/`, password `/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).{6,}$/`; customer Address form: pincode `^\d{0,6}$`. Koi form library (Formik, React Hook Form) nahi hai. (4.6.3)

7.11 Redux Toolkit (state management)

7.11.1 Global state, store aur prop drilling

- **Kya hai:** jo data kai alag-alag components ko chahiye (login, cart count), use ek central store mein rakhte hain. Warna har level pe props aage pass karne padte hain, jise prop drilling kehte hain.
- **Project mein:** `FrontEnd/src/Store/store.js` mein `configureStore` + 10 reducers (`user`, `cart`, `orders` ...). Seller aur rider app ke apne stores hain. Admin mein Redux nahi, kyunki har page ka data sirf usi page ka hai. (4.4.1, B8)

7.11.2 Slice, reducer, action aur dispatch

- **Kya hai:** slice ek feature ka state aur uske reducers ek file mein (`createSlice`). Reducer ek function hai: (purani state, action) → nayi state. Action "kya hua" batane wala object hai, aur `dispatch(action)` use store tak bhejta hai.
- **Project mein:** `FrontEnd/src/Redux/cartSlice.js`, `authSlice.js` (`logoutUser`, `restoreUserFromSession` reducers), aur rider app ka `deliveryTaskSlice.js` (`addDeliveryTask`, `removeDeliveryTask`).

7.11.3 `useSelector` aur `useDispatch`

- **Kya hai:** `useSelector(state => ...)` store se data padhta hai, aur wo data badle to component re-render karta hai. `useDispatch()` actions bhejne ka function deta hai.
- **Project mein:** `FrontEnd/src/components/Navbar/Navbar.jsx:56`: `const cartCount = useSelector((state) => state.cart.totalQuantity);` (cart badge), aur line 57 pe favorites count.

7.11.4 `createAsyncThunk` (pending / fulfilled / rejected)

- **Kya hai:** API call jaise async kaam ke liye thunk. RTK uske liye apne aap teen actions banata hai, jinhe `extraReducers` mein `addCase` se handle karte hain. `rejectWithValue` error ka payload deta hai, aur component mein `.unwrap()` rejected thunk ko `throw` mein badal deta hai.
- **Project mein:** `cartSlice.js:66` ka `addToCartWithBackendSync` (poora snippet 4.4.2 mein). Checkout mein `dispatch(createOrder(...)).unwrap()`.

7.11.5 `addMatcher`

- **Kya hai:** ek reducer jo ek condition function ke hisaab se kai action types pe chale (jaise "jo bhi action `/pending` pe khatam ho").
- **Project mein:** `delivery-dashboard/src/Redux/deliveryTaskSlice.js:195`: `action.type.endsWith('/pending')` pe `state.actionLoading = true`, aur `/fulfilled` ya `/rejected` pe `false` (`getActiveTask` aur `getNearbyDeliveries` ko chhod ke).
- ⚠ Ye doosri slices ke actions (jaise login) bhi pakad leta hai (4.4.6).

7.11.6 Immer aur immutability

- **Kya hai:** Redux mein state ko seedha badalna mana hai; har baar naya object banana padta hai. RTK andar se Immer use karta hai, isliye `state.loading = true` jaisa code likh sakte ho, aur Immer peeche se naya immutable object bana deta hai.
- **Project mein:** har slice ke reducers, jaise `state.cartItems = mapBackendCartToItems(action.payload)`.
- ⚠ Reducers pure hone chahiye, lekin `logoutUser` reducer ke andar `localStorage.removeItem` (side effect) hai (4.3.7).

7.11.7 Serializable state check

- **Kya hai:** Redux state mein sirf plain data (string, number, array, object) rakhna chahiye, functions ya class instances nahi. RTK dev mode mein iski warning deta hai.
- **Project mein:** rider app ke store mein ye check band kiya gaya hai: `serializableCheck: false` (`delivery-dashboard/src/Redux/store.js:14`). Kyun band kiya, code mein likha nahi hai.

7.11.8 Service layer aur "server hi source of truth"

- **Kya hai:** API calls alag `*Service.js` files mein, taaki components ko URL ki parwaah na karni pade. Server ka data Redux mein copy rehta hai, par har response ke baad server wala data hi sahi maana jaata hai.
- **Project mein:** `cartService.js` (sirf axios) + `cartSlice.js`, jisme fulfilled pe `state.cartItems = mapBackendCartToItems(action.payload)`. RTK Query ya React Query (automatic caching) use nahi hui (D15).

7.11.9 Session restore (refresh pe login bachana)

- **Kya hai:** page refresh hone pe Redux state khaali ho jaati hai, isliye login ko localStorage se wapas bharna padta hai.
- **Project mein:** `authSlice.js:4–14` module load hote hi `localStorage.getItem("user")` se `user` aur `token` bharta hai. Seller app ka `AuthInitializer` boot pe `GET /api/seller/profile`, aur Admin `GET /api/admin/verify` call karta hai (4.3.7).

7.12 Frontend routing (React Router 7)

7.12.1 Client-side routing aur `BrowserRouter`

- **Kya hai:** URL badalne pe page reload nahi hota; React Router URL dekh ke sahi component dikha deta hai. `BrowserRouter` HTML5 History API (`pushState`) use karta hai, isliye URLs saaf hote hain (`/cart`, `#/cart` nahi).
- **Project mein:** chaaron apps ke `App.jsx` mein `BrowserRouter` aur `<Route>`s; customer store mein ~30 routes (4.5.2).

7.12.2 SPA fallback (rewrite rule)

- **Kya hai:** `/cart` pe refresh karo to browser server se `/cart` naam ki file maangta hai, jo hai hi nahi. Isliye hosting pe "har path pe `index.html` do" wala rule chahiye.
- **Project mein:** repo mein ye rule nahi hai (`vercel.json`, `_redirects` jaisi koi file nahi); shayad hosting dashboard pe set kiya gaya ho **(inference, 4.17)**.

7.12.3 URL params, query string aur route state

- **Kya hai:** `useParams()` path ke variables deta hai. `useLocation().search` + `URLSearchParams` se query string padhte hain. `navigate(path, { state })` URL mein dikhaye bina data bhejta hai, par wo refresh pe kho jaata hai.
- **Project mein:**
  - `ProductDetails.jsx:58`: `const { productId } = useParams();`
  - `ShopM.jsx:49–50`: `const searchParams = new URLSearchParams(location.search);` (filters URL mein hain, isliye link share ho sakta hai).
  - Buy Now: `navigate("/order-checkout", { state: { buyNowProduct, isBuyNow: true } })` (`ProductDetails.jsx`, line ~172).

- ⚠ `/order-placed` page direct URL se kholo to state nahi hoti aur page crash hota hai (4.5.2).

7.12.4 Route ranking aur 404 page

- **Kya hai:** React Router v6/v7 routes ko specificity se rank karta hai, isliye static `/cart` dynamic `/:category` se jeet jaata hai. Unknown URLs ke liye catch-all `*` route se 404 page banta hai.
- **Project mein:** `/:category` lagbhag catch-all ban gaya hai, aur koi `*` (404) route nahi hai. Isliye unknown URL pe "No products" dikhta hai.

7.13 HTTP client aur browser APIs

7.13.1 Axios vs fetch

- **Kya hai:** dono HTTP request bhejte hain. Axios JSON apne aap parse karta hai, non-2xx response pe reject karta hai, aur interceptors deta hai. `fetch` browser mein built-in hai, par ye sab haath se karna padta hai.
- **Project mein:** zyaadatar axios; kuch jagah `fetch` (home page ke kuch sections, Admin ke kuch pages). Base URL `import.meta.env.VITE_API_URL` se aata hai.

7.13.2 Interceptors aur axios instance

- **Kya hai:** interceptor har request ya response pe chalne wala common code hai (jaise token lagana, 401 pe logout). `axios.create({ baseURL })` ek configured instance banata hai.
- **Project mein:** `FrontEnd/src/axiosSetup.js` : response interceptor 401 pe `store.dispatch(logoutUser())` . `delivery-dashboard/src/services/deliveryService.js` : `axios.create({ baseURL: API/api/delivery })` , request interceptor Bearer token lagata hai, aur response interceptor 401 pe token hata deta hai. (4.3.7)

7.13.3 localStorage vs sessionStorage

- **Kya hai:** dono browser mein key-value strings rakhte hain. localStorage tab band hone ke baad bhi rehta hai; sessionStorage sirf us tab ke khule rehne tak.
- **Project mein:** tokens localStorage mein hain. `ScrollToTop.jsx` har route ki scroll position `sessionStorage.setItem(`scrollPosition-${pathname}`, window.scrollY)` se yaad rakhta hai, aur `PasswordChangeOrContinue.jsx:19` `sessionStorage.getItem("tempUser")` padhta hai.

7.13.4 Geolocation API aur reverse geocoding

- **Kya hai:** `navigator.geolocation.getCurrentPosition()` user ki permission lekar GPS ka lat/lng deta hai. Reverse geocoding = coordinates se address. Forward geocoding = address se coordinates.
- **Project mein:** customer `Address.jsx:81-86` ka "Use My Location" coordinates leke OpenStreetMap Nominatim (`https://nominatim.openstreetmap.org/reverse?format=json&lat=…&lon=…`) se address bharta hai. Rider `NearbyDeliveries.jsx:25` ka "Share My Location".
- ⚠ Forward geocoding nahi hai, isliye delivery task ke coordinates placeholder hain (T3).

7.13.5 Throttling aur debouncing

- **Kya hai:** throttle = ek event ko har X ms mein zyada se zyada ek baar chalao (scroll jaise). Debounce = event rukne ke X ms baad hi chalao (search typing jaise).
- **Project mein:** `ScrollToTop.jsx` ka scroll handler 150 ms throttle hai (`isThrottled` flag + `setTimeout`), aur listener `{ passive: true }` ke saath lagta hai. Debounce kahin nahi hai (search submit karne pe chalta hai, isliye abhi zaroorat nahi).

7.13.6 Deep links: `tel:` aur map links

- **Kya hai:** special URLs jo phone ka dialer ya maps app khol dete hain.
- **Project mein:** rider app ke `ActiveDelivery/` mein `tel:` call link aur Google Maps search link (Section 1.6).

7.14 UI aur styling libraries

7.14.1 Styling ke tareeke: global CSS, CSS Modules, `sx`, styled-components

- **Kya hai:** global CSS har jagah lagti hai, isliye class names aapas mein takra sakte hain. CSS Modules class names ko us file tak simit (scope) kar dete hain. MUI ka `sx` prop theme ke saath inline style deta hai. styled-components CSS ko JavaScript ke andar likhne ka tareeka hai.
- **Project mein:** FrontEnd mein ~60 global `.css` files; Seller-Panel mein 6 `*.module.css` + MUI `sx`; FrontEnd ke `ui/dialog` mein styled-components.

7.14.2 Component libraries: MUI aur Radix (shadcn-style)

- **Kya hai:** ready-made, accessible UI components (tables, dialogs, side drawers), taaki sab kuch zero se na banana pade.
- **Project mein:** Seller-Panel mein MUI 7 (`OrderTable` , `Confirm` dialog); FrontEnd ke `src/ui/` mein Radix primitives (`sheet` jo FilterDrawer ka drawer hai, `checkbox` , `radio-group`).

7.14.3 Toast notifications

- **Kya hai:** chhota sa temporary message, jaise "Added to cart!". Library ka container component (`<ToastContainer/>` ya `<Toaster/>`) app mein mount hona zaroori hai.
- **Project mein:** FrontEnd aur rider app mein react-toastify, Admin mein react-hot-toast.
- ⚠ Seller-Panel `toast()` call karta hai par `<Toaster/>` mount nahi hai, isliye toast dikhta hi nahi (4.6.3).

7.14.4 Charts

- **Kya hai:** data ko graph mein dikhane wali React libraries.
- **Project mein:** Seller-Panel mein nivo (bar, line), Admin mein recharts (monthly sales, signups). Data backend ki aggregations se aata hai (7.5.13).

7.14.5 Rich text / Markdown editor

- **Kya hai:** formatting wala text editor, jo text ko Markdown mein save karta hai.
- **Project mein:** `@mdxeditor/editor` : seller product description likhta hai, aur customer use read-only mode mein dekhta hai.
- ⚠ Sirf dikhane ke liye bhi poora bhaari editor load hota hai (4.15.3).

7.14.6 Animations aur creatable dropdown

- **Kya hai:** animation library UI mein movement deti hai; creatable select mein list se option chuno ya naya type karke bana do.
- **Project mein:** framer-motion (product gallery, footer); react-select ka `CreatableSelect` seller ke category aur tags fields mein.

7.15 SEO aur analytics

7.15.1 Meta tags, canonical URL aur Open Graph

- **Kya hai:** `<title>` aur `<meta name="description">` search results mein dikhte hain. Canonical tag batata hai ki page ka "asli" URL kaunsa hai (duplicate content se bachav). Open Graph/Twitter tags social media pe share karne pe preview banate hain.
- **Project mein:** `FrontEnd/src/utils/SEO.jsx` (react-helmet-async ka `<Helmet>`): title, description, `<link rel="canonical">`, OG aur Twitter tags. `main.jsx` mein `HelmetProvider`.

7.15.2 Structured data (JSON-LD)

- **Kya hai:** page ke content ka machine-readable description (Product, BlogPosting), jisse Google search mein price aur rating jaise rich results dikha sake.
- **Project mein:** `SEO.jsx` mein `<script type="application/ld+json">` (`JSON.stringify(schema)`), aur ProductDetails ka Product schema.
- ⚠ `JSON.stringify` `</script>` ko escape nahi karta (4.14.4).

7.15.3 `sitemap.xml` aur `robots.txt`

- **Kya hai:** sitemap search engine ko site ke saare URLs batata hai; robots.txt batata hai ki kya crawl karna hai aur kya nahi.
- **Project mein:** `Backend/public/` aur `FrontEnd/public/` mein static files; `routes/sitemap.js` (dynamic, par static file ki wajah se shadowed) aur `generateSiteMap.js` script (`sitemap` package). (4.13.4)

7.15.4 Analytics

- **Kya hai:** website pe users ka traffic aur behaviour track karne wala script.
- **Project mein:** har app ke `index.html` mein Google Analytics (gtag) script hai (4.14.4).

7.16 Tooling, build aur deployment

7.16.1 npm, `package.json` scripts aur lockfiles

- **Kya hai:** npm packages install karta hai. `package.json` mein dependencies aur scripts (`npm start`, `npm run build`) hote hain. Lockfile exact versions fix kar deti hai, taaki har machine pe same versions install hon.
- **Project mein:** Backend scripts: `start`, `migrate:s3`, `migrate:s3:dry` (test script npm ka default hai). Har frontend mein `bun.lock` aur `package-lock.json` dono hain, yaani do package managers use hue. Monorepo/workspaces setup nahi hai.

7.16.2 Vite: dev server, HMR aur build

- **Kya hai:** Vite ek modern build tool hai. Dev mode mein server turant start hota hai aur HMR (Hot Module Replacement: code save karte hi browser update, bina poora page reload kiye) milta hai. `vite build` production ke liye minified static files `dist/` mein banata hai.
- **Project mein:** chaaron apps ka `vite.config.js`; FrontEnd mein Vite 6, baaki teen mein Vite 7. FrontEnd ke config mein `/api/subscribe` ka dev proxy hai (jo practically use nahi hota).

7.16.3 Linting (ESLint)

- **Kya hai:** code ko chalaye bina galtiyan aur bure patterns pakadne wala tool.

- **Project mein:** har frontend mein `eslint.config.js` (ESLint 9 flat config) aur `npm run lint`. Backend mein koi linter nahi hai.

7.16.4 Frontend env variables (`VITE_` prefix)

- **Kya hai:** Vite sirf `VITE_` se shuru hone wale variables `import.meta.env` mein deta hai, aur build ke waqt unki values bundle ke andar likh deta hai. Isliye ye public hote hain; inme kabhi secret mat rakho.
- **Project mein:** `Checkout.jsx:20`: `const API_URL = import.meta.env.VITE_API_URL || "http://localhost:5000";`.
- ⚠ Asli example: dead component `Blog/Blogsection/Blogsection.jsx:3` `import.meta.env.API_URL` padhta hai (bina `VITE_` ke), jo hamesha `undefined` hoga.

7.16.5 Hosting: PaaS, static hosting aur managed database

- **Kya hai:** PaaS (jaise Render) pe code push karo, wo server chala deta hai aur `PORT` env variable deta hai. Static host (jaise Vercel) build ki hui files CDN se serve karta hai. Managed database (Atlas) backup aur replica set khud sambhalta hai.
- **Project mein:** `server.js` mein `const PORT = process.env.PORT || 3000` aur comment "Render will provide PORT"; root `.gitignore` mein `.vercel` **(inference)**; `mongodb+srv://` URI ka matlab Atlas. (4.17)

7.16.6 CI/CD, Docker aur automated testing (ye sab NAHI hain)

- **Kya hai:**
  - CI: har push ya pull request pe lint + test + build apne aap chalna. CD: sab pass ho to apne aap deploy.
  - Docker: app ko uske poore environment ke saath ek container image mein pack karna.
  - Testing ke types: unit (ek function), integration (API + DB saath mein), E2E (browser se poora user flow).
- **Project mein:** inme se kuch nahi hai: koi `.github/workflows`, `Dockerfile` ya test file nahi, aur Backend ka `"test"` script default wala hai. Plan ke liye 4.16 aur 4.17 dekho (Jest, Supertest, `mongodb-memory-server`, Playwright, GitHub Actions).

7.17 Architecture aur design patterns

7.17.1 Monolith vs microservices

- **Kya hai:** monolith = poora backend ek hi deployable app. Microservices = har feature ki alag service, alag deploy, aur services network pe baat karti hain. Microservices bade traffic aur badi teams ke liye achhe hain, par complexity bahut badha dete hain.
- **Project mein:** ek monolith backend + 4 frontends (D1). Payments ya delivery ko alag service banana sirf zaroorat pe (5.4.2, Stage 4).

7.17.2 Layered architecture (MVC jaisa)

- **Kya hai:** code ko layers mein baantna: route → controller → service → model, aur har layer ka ek hi kaam (separation of concerns). MVC mein View bhi hota hai; yahan View ka kaam React karta hai.
- **Project mein:** delivery panel saaf layered hai (route → Zod → controller → model); seller auth route → controller → service.
- ⚠ `product.routes.js` (749 lines) aur `order.routes.js` (697) mein saari logic route file ke andar hi hai (3.3).

7.17.3 DTO / mapper pattern

- **Kya hai:** DB document ko seedha bhejne ki jagah client ke liye uska shape banana: sensitive fields hatao, calculated fields jodo.
- **Project mein:** `toClientProduct()`, `orderWithClientProducts()`, `taskToDashboardShape()`.
- ⚠ `/home/products` isko use nahi karta, isliye wahan fees leak hoti hain (verified, 4.14.3).

7.17.4 Adapter, factory aur singleton

- **Kya hai:** adapter = naya system purane interface jaisa dikhe. Factory = function jo configured object ya function bana ke de. Singleton = poori app mein ek hi instance.
- **Project mein:** adapter: `uploadToS3` Cloudinary jaisa `{ secure_url }` lautata hai. Factory: `zodValidate(schema)`. Singleton: S3 client (`getClient()`) aur rider app ka socket client (`services/socket.js`). (3.4)

7.17.5 State machine

- **Kya hai:** kisi object ke allowed states aur allowed transitions pehle se tay hote hain, aur galat jump reject ho jaata hai.
- **Project mein:** `updateTaskStatus` ka `allowedTransitions` (`Accepted → PickedUp → OutForDelivery → Delivered`).
- ⚠ Final states (Delivered, Cancelled, Failed) pe check skip ho jaata hai (H4).

7.17.6 Push, scheduled jobs aur in-process background work

- **Kya hai:** push = server khud client ko message bheje. Scheduled job = tay time pe chalne wala kaam. In-process background work = response ke baad bhi usi server process mein chalne wala kaam.
- **Project mein:** push `io.to(socketId).emit("newDeliveryRequest")`; scheduled `jobs/badgeCron.js`; background mein 15 second ka `setTimeout` fallback aur fire-and-forget email. Ye sab usi ek process mein chalte hain (D14).

7.17.7 Statelessness aur horizontal scaling

- **Kya hai:** horizontal scaling = load balancer ke peeche server ki kai copies chalana. Iske liye server stateless hona chahiye: koi zaroori cheez sirf ek process ki memory mein na ho, balki shared Redis/DB mein ho. WebSockets ke liye sticky sessions ya Redis adapter bhi chahiye.
- **Project mein:** JWT ki wajah se HTTP API stateless hai, lekin `riderToSocket` Map, 15 second ka `setTimeout`, cron, in-memory cache aur local disk wale uploads stateful hain (5.4.1). Fix: Redis adapter, BullMQ, Redis cache, aur S3.

7.17.8 Job queue aur outbox pattern (sirf suggestion)

- **Kya hai:** job queue (jaise BullMQ + Redis) mein kaam daala jaata hai, aur ek alag worker use retry ke saath chalata hai. Outbox pattern mein main DB write ke saath hi "ye email bhejna hai" ka record save hota hai, aur worker baad mein bhejta hai, taaki koi kaam chhoote nahi.
- **Project mein:** abhi nahi hai. Scaling aur email reliability ke fix ke roop mein suggest kiya gaya hai (A5, A15).

7.18 Concepts jo project mein NAHI hain (interviewer pooche to sach bolo)

Concept	Project mein	Pooche to kya bolo
TypeScript	Nahi, plain JavaScript	"It's plain JavaScript; I'd add TypeScript and share Zod schemas between frontend and backend."
Automated tests	Nahi (ek bhi test file nahi)	Plan: unit tests + Supertest integration tests (4.16)
CI/CD, Docker	Nahi	GitHub Actions: lint + test + build (4.17)
Redis, message queue	Nahi	Socket map, timer, cron aur cache ko Redis/BullMQ pe le jaana (5.4.1)
Rate limiting, helmet	Nahi	`express-rate-limit` login/OTP pe, `app.use(helmet())`
Refresh token, OAuth / Google login	Nahi, sirf 30 din ka JWT	Short access token + httpOnly refresh cookie (A4)
SSR / Next.js	Nahi, sab client-side SPA	Customer store ke SEO ke liye Next.js
GraphQL	Nahi, sirf REST	REST hi kaafi tha, har screen ke fixed endpoints hain
RTK Query / React Query	Nahi	Caching aur duplicate requests hatane ke liye (4.15.3)
Code splitting (`React.lazy`)	Nahi	Route-level lazy loading pehla frontend fix
Context API (apna)	Nahi, sirf libraries ke providers	Redux Toolkit use hua
Full-text search	Nahi, sirf tag/regex search	MongoDB Atlas Search
Geocoding, live tracking	Nahi, coordinates placeholder	Address save pe geocoding, `watchPosition` se live location (5.4.3)
Microservices	Nahi, monolith	Sirf zaroorat pe (7.17.1)

Formula (6.7 jaisa): "*We didn't use X. Here's what we used instead, and here's when X would make sense.*"

Appendix A: Local pe kaise chalayein (revision ke liye)

Poora project is Mac pe set up ho chuka hai: dependencies installed, `.env` files bani hui, demo data seeded.

> Status 2026-10-14: `local-dev/` folder, paanchon apps ki `.env` files aur `node_modules` maujood hain (sirf `ls` se check kiya; app dobara start karke test nahi kiya).

A.1 Start aur stop

```
cd ~/Downloads/Aestheticommerce-merged
./local-dev/start.sh     # MongoDB + Backend + 4 frontends start (pehli baar ke baad ~5 second)
./local-dev/stop.sh      # sab band (tumhara apna MongoDB on 27017 nahi chhoota)
```

Logs `local-dev/logs/` mein milte hain. Mac restart ke baad bas `start.sh` dobara chala do.

A.2 URLs aur demo logins

App	URL	Login
Customer store	http://localhost:5173	`customer@demo.local` / `Customer@123`
Seller panel	http://localhost:5174	`seed.littlethings@example.com` / `Seller@123`
Admin panel	http://localhost:5175	`admin@demo.local` / `Admin@123`
Rider app	http://localhost:5176	`rider@demo.local` / `Rider@123`
API docs (Swagger)	http://localhost:5000/api-docs	—

Coupons: `WELCOME10` (10%), `FLAT100` (₹100 off).

A.3 Setup mein kya kiya gaya

- **MongoDB:** project ka apna single-node **replica set** port **27018** pe chalta hai, data `local-dev/mongo-data/` mein rehta hai, DB ka naam `aestheticommerce_local` hai.
  - Replica set isliye zaroori hai kyunki rider ka accept-task aur status-update flow MongoDB transactions use karta hai, aur transactions standalone MongoDB pe nahi chalte.
- `.env` **files:** paanchon apps mein bani hain. Teeno `.gitignore` mein covered hain, isliye commit nahi hongi.
  - Razorpay aur Resend ki keys **fake placeholders** hain. Bina kisi key ke dono SDK server start hote hi crash kar dete hain.
- **Seed data:**
  - Project ki apni scripts se: 6 categories, 3 demo shops ke 54 products, aur test rider `delivery@test.com`.
  - `local-dev/seed-demo-accounts.js` se: customer, admin aur rider ke login, seller ka hashed password, 2 pending products, coupons, ek banner, 2 blogs.
  - Ye extra script isliye bani kyunki project ki seed script sellers ka password plain text mein save karti hai, aur bcrypt-hashed password ke bina login nahi hota.
- **Pura** `local-dev/` **folder delete kar do to ye saara local setup hat jaata hai** (`.env` files aur `node_modules` alag se hatane honge).

A.4 Local pe kya kaam nahi karega (expected)

- **Online payment (Razorpay):** key fake hai. Checkout mein **Cash on Delivery** choose karo.
- **Emails:** OTP, order confirmation, contact form. Backend log mein "API key is invalid" aayega.
  - Isliye seller registration aur forgot-password ka OTP step UI se poora nahi hoga. Demo accounts se login karo.
- **Image/video upload** (naya product, avatar, banner) kaam nahi karega, kyunki AWS S3 ke credentials nahi hain. Seed products ki images Unsplash se aati hain.

A.5 Revision ke liye suggested walkthrough (yahi interview wala flow hai)

1. **Rider app** (5176) mein login karo. Login hote hi rider "available" ho jaata hai aur socket se register hota hai.
2. **Customer store** (5173) mein login karo, koi product cart mein daalo, `WELCOME10` lagao, aur COD se order karo.
   - Rider app mein ~10 second ka "new delivery request" popup aana chahiye (Socket.io).
   - Agar 15 second mein koi accept na kare, to server fallback rider `delivery@test.com` ko assign kar deta hai.
3. **Rider app** mein accept karo, phir status aage badhao: PickedUp → OutForDelivery → Delivered. Customer ke "Orders" page pe status update dikhega.
4. **Seller panel** (5174) mein revenue, orders aur delivery tasks dekho.
5. **Admin panel** (5175) mein pending product approve karo; wo product phir store pe dikhne lagega. Badges toggle karke bhi dekho.

Verified on 2026-10-10: API se poora flow chalaya gaya (login → cart → coupon → COD order → rider nearby → transaction accept → doosra accept reject → Delivered → order sync → seller stats → admin approve), aur chaaron UIs ke screenshots liye gaye.

- Ek dekhi hui baat: task `PickedUp` hote hi order item seedha "Out for Delivery" ban jaata hai, "Shipped" step skip ho jaata hai (`syncOrderStatusFromTask`).

Appendix B: Folder tree aur har file ka kaam (Step 1)

1.1 Big picture: ek folder, 5 apps

```
Aestheticommerce-merged/
├── Backend/              # Node + Express REST API + Socket.io server. Saari business logic aur DB yahin hai.
├── FrontEnd/             # Customer website (React): browse, cart, checkout, orders, reviews
├── Seller-Panel/         # Seller dashboard (React): signup + KYC, products add/edit, orders, delivery request, stats
├── Admin-Panel/          # Admin dashboard (React): products approve/reject, badges, sellers, riders, banners, analytics
├── delivery-dashboard/   # Delivery rider app (React): nearby tasks, real-time request, accept, status update
├── package.json          # Galti se root pe installed 3 packages (@aws-sdk/client-s3, react-select, react-select-v1). Koi app inhe
    root se use nahi karta.
├── package-lock.json     # Upar wale ka lock file
└── .gitignore            # node_modules, dist, .env* ignore
```

Ye cheezein project mein hain hi nahi: Dockerfile, CI (`.github/`), koi test file, TypeScript, root README, `vercel.json` , aur workspace tooling (npm workspaces / turborepo).

Har app alag install hoti hai, aur har frontend mein `bun.lock` aur `package-lock.json` dono hain, matlab do package managers use hue.

Paanchon apps aapas mein kaise jude hain:



1.2 `Backend/` (Node.js + Express): poore system ka dimaag

Naming convention (samajhna zaroori hai)

Har layer ke panel-wise alag folders hain:

- `routes/` , `routes-seller-panel/` , `routes-admin-panel/` , `routes-delivery-panel/`
- `controllers*` , `Middleware*` , `services*` , `utils*` bhi isi tarah bante hain

Matlab har panel ki team ne apna code alag folder mein rakha, aur baad mein sab ek backend mein merge hua. **(inference: folder ka naam "merged" bhi isi taraf ishara karta hai)**

Entry aur config files

File	Kaam
`server.js`	App start karta hai: `await connectDB()` , phir `initBadgeCron()` , phir `http.createServer(app)` ke upar Socket.io server. `global.io` , `global.riderToSocket` , `global.socketToRider` naam ke Maps banata hai. `registerRider` event aane pe riderId → socketId save karta hai, aur `disconnect` pe hata deta hai.
`app.js`	Express app. `cors()` lagata hai, phir JSON parser, **lekin** `/api/payment/webhook` **ke liye JSON parsing skip** karta hai (Razorpay signature check ke liye raw body chahiye). Static `/uploads` aur `public/` serve karta hai, `/api-docs` pe Swagger UI deta hai, saare routers mount karta hai, aur aakhir mein `globalErrorHandler` lagata hai.

File	Kaam
`db/db.js`	`mongoose.connect(DB_CONNECT, { serverSelectionTimeoutMS: 5000, socketTimeoutMS: 45000, maxPoolSize: 50 })`. Connection ke error/disconnect events log karta hai.
`config/razorpay.js`	`new Razorpay({ key_id, key_secret })` banata hai, values env se aati hain.
`swagger.js`	swagger-jsdoc se OpenAPI 3 spec banata hai. Server URL `http://localhost:5000` hai.
`docs/api.swagger.js`	~3000 lines ke JSDoc `@swagger` comments: har endpoint ka documentation.
`package.json`	Scripts: `start`, `migrate:s3`, `migrate:s3:dry`. **Test script nahi hai.**
`env.example`	`PORT`, `NODE_ENV`, `DB_CONNECT`, `JWT_SECRET`, `S3_REGION`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `S3_BUCKET`, `CLOUDFRONT_URL`, `EMAIL`, `MAIL_PASS`, `SMTP_USER`, `SMTP_PASS`
(missing env vars)	Code ye variables use karta hai jo `env.example` mein nahi hain: `RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET`, `RESEND_API_KEY`, `EMAIL_MODE`, `EMAIL_FROM_NAME`, `EMAIL_FROM_ADDRESS`, `DB_NAME`
`readme.md`	Purana README. Sirf `/users/register` aur `/users/login` document karta hai.

URL mounting (`app.js`): kaunsa URL kis file mein jaata hai

```
/users            → cart.routes, user.routes, shipAddress.routes, blogs.routes, favorites.routes
/products         → product.routes            /home           → home.routes
/api/orders       → order.routes              /api/payment    → payment.routes
/api/addresses    → address.routes            /api/document   → invoice.routes
/api/banners      → banner.routes (public)    /api            → contactRoutes, contactUs.routes, NLSubs.routes, coupon.routes
/api/products     → addProduct.routes (seller)           /api/seller          → SellerPanel.routes
/api/dashboard    → dashboard.routes                     /api/productinfo     → productInfo.routes
/api/stats        → stats.routes                         /api/categories      → category.routes
/api/seller/delivery → routes-seller-panel/delivery.routes
/api/admin        → admin.routes      /api/admin/banners → routes-admin-panel/banner.routes
/api/notifications → notification.routes
/api/delivery     → delivery.routes   /api/deliverytask  → deliveryTask.router
/                 → sitemap.js        /api-docs          → Swagger UI
```

`models/`: 17 Mongoose schemas (database ka structure)

File	Kya store karta hai
`user.model.js`	Customer: `fullname{firstname,lastname}`, `email` (unique), `password` (`select:false`, yaani default query mein nahi aata), `phone`, `address`, `avatar`, `forgot_password_otp/expiry`, `favorites[ProductId]`, `savedAddresses` (legacy). Methods: `generateAuthToken()` (JWT `{_id}`, 30 din), `comparePassword()`. Static method: `hashPassword()` (bcryptjs, 10 rounds).
`seller.model.js`	`sellerName`, `storeName`, `email`, `phone` (unique), `password`, `status` (pending/approved/suspended/rejected), `isBlocked`, `businessType`, `gstNumber`, `businessAddress`, `bankDetails`, Aadhar/PAN number + file URL, `verificationStatus`, `emailOtp`, `otpExpiresAt`, `isEmailVerified`, `onboardingCompleted`.
`admin.model.js`	Sirf `email` + `password`. `pre('save')` hook mein bcrypt hashing, aur `matchPassword()` method.
`product.model.js`	`name`, `description`, `sellingPrice`, `platformFee`, `logisticsFee` (default 60), `transactionFee` (default 10), `discountPercentage`, `mrp`, `images` (max 5), `productVideo`, `category`, `subcategory`, `tags` (setter se lowercase + duplicates hataye jaate hain), `badges` (8 booleans), `rating`, `inStock`, `reviews[]` (embedded), `sellerId`, `approvalStatus` (pending/approved/rejected), `adminNotes`, `isDeleted` + `deletedAt` (soft delete). Compound indexes: `{approvalStatus, isDeleted, inStock, rating}`, `{sellerId, rating, createdAt}`, `{tags}`.

File	Kya store karta hai
`order.model.js`	`orderId` (`"OD"` + 13-digit timestamp + 5 random digits), `user`, aur `products[]`. Har item mein `itemId`, `product`, seller info, `quantity`, `price`, **per-item** `status`, `trackingInfo`, cancel info. Saath mein `paymentMethode` (Online/COD), `razorpayOrderId/PaymentId`, `totalAmount`, `pricingBreakdown`, `paymentStatus`, `deliveryStatus`, `shippingAddress`, `rating`. `expireAt` pe **TTL index** hai, isliye unpaid online order 30 min baad apne aap delete ho jaata hai. `pre('save')` hook IDs generate karta hai.
`cart.model.js`	`user` + `items[{product, quantity}]`
`address.model.js`	Har user ke multiple addresses. Pincode regex se 6 digit check hota hai. `isDefault` (`pre('save')` baaki addresses ko false kar deta hai), `addressType` (home/work/other). Index: `{userId, isDefault}`.
`coupon.model.js`	`code` (uppercase, unique), `type` flat/percent, `value`, `isActive`, `expiresAt`, `maxUsesTotal`, `maxUsesPerUser`, `usedCount`
`couponUsage.model.js`	`couponId` + `userId` + `orderId`. Isse "is user ne ye coupon kitni baar use kiya" count hota hai.
`deliveryPartner.model.js`	Rider: `name`, `email`, `phone`, `password`, `isActive` (admin approval), `isAvailable` (online/free), `currentLocation` (GeoJSON Point + **2dsphere index**), `documents`, `bankDetails`, `currentCity`, `stats`. Password `pre('save')` mein hash hota hai.
`deliveryTask.model.js`	Har (order, seller) pair ke liye ek task. `items`, `deliveryPartner`, pickup/drop contact aur address, `pickupLocation` / `dropLocation` (GeoJSON + 2dsphere), `liveLocation`, `status` (Pending → Assigned/Accepted → PickedUp → OutForDelivery → Delivered, ya Failed/Cancelled), `attempts[]`, `otp` (model mein hai, kahin use nahi hota), `deliveryFee`, cancel info, `deliveryRating`.
`notification.model.js`	`recipient` + `recipientModel` (`refPath`: User/Seller/Admin), `message`, `link`, `read`. Ye ek polymorphic reference hai: ek hi field kisi bhi teen model ko point kar sakta hai.
`banner.model.js`	`imageUrl`, `redirectUrl`, `isActive`. Ek waqt mein ek hi banner active rehta hai.
`blog.model.js`	`title`, `author`, `content`, `image`, `tags`
`category.model.js`	`name` (unique) + `subcategories[{name, keywords[]}]`
`contact.model.js`	Seller se admin ko support message: `subject`, `message`, `status` (Pending/In Progress/Resolved), `response`, `isFromSeller`, `sellerId`
`contactUs.model.js`	Customer website ka "Contact Us" form: `name`, `email`, `message`
`NlLetterSubs.model.js`	Newsletter subscriber email (unique)

Auth middleware (har panel ka alag gatekeeper)

File	Kaam
`Middleware/authMiddleware.js`	`protect` (customer ke liye): Bearer JWT verify karta hai, phir `User.findById(decoded._id)`, aur result `req.user` mein daal deta hai.
`Middleware/optionalAuth.js`	Token ho to user attach karta hai, na ho to bhi request aage jaane deta hai. Home page ke personalised products ke liye use hota hai.
`Middleware/zodValidate.js`	Generic validator: `zodValidate(schema, 'body'|'query'|'params')`. `safeParse` fail ho to `AppError(400)`. Pass ho to `req[property] = parsed.data`, jisse unknown fields hat jaate hain.
`Middleware/globalErrorHandler.js`	Central error handler. Dev mode mein stack trace bhejta hai. Prod mode mein `CastError`, duplicate key `11000`, `ValidationError` aur JWT errors ko friendly message mein badalta hai.
`Middleware/sellerAuth.js`	Purana seller `protect`. **Kahin import nahi hota (dead code).**

File	Kaam
`MiddlewareSellerPanel/sellerPanelauthMidWar.js`	`protectSeller`: JWT `{id}` se Seller dhundhta hai, aur `isBlocked` ho to 403 deta hai.
`MiddlewareAdminPanel/Adminauth.middleware.js`	`protectAdmin`: JWT `{id}` se Admin dhundhta hai, aur `req.user.role = 'admin'` set karta hai.
`MiddlewareDeliveryPanel/deliveryAuth.middleware.js`	`protectDelivery`: check karta hai ki `decoded.role === 'delivery'` ho, rider exist kare, aur `isActive` true ho (yaani admin ne approve kiya ho).

Note: chaaron roles ke liye **ek hi** `JWT_SECRET` hai. Customer token mein `_id` hota hai, baaki teen mein `id`.

`routes/`: customer side aur shared APIs

File	Endpoints aur kaam
`user.routes.js`	`/users/register`, `/login`, `/profile`, `/forgot-password`, `/verify-otp`, `/reset-password`, `/avatar` (multer memory se S3), `/update-profile`. Validation `express-validator` se hoti hai.
`cart.routes.js`	`/users/cart`: GET, POST, PUT `/:itemId`, DELETE `/:itemId`, DELETE (sab `protect` ke peeche)
`favorites.routes.js`	`/users/favorites`: GET, POST, DELETE `/:productId`
`shipAddress.routes.js`	`/users/save-address`, `/get-address`. **Legacy code:** auth nahi hai, aur `savedAddress` field likhta hai jo schema mein hai hi nahi. Practically dead hai.
`blogs.routes.js`	`POST /users/blog` (image S3 pe jaati hai), `GET /users/blogs[?tag=]`, `GET /users/blogs/:id`
`product.routes.js`	Logic route file ke andar hi hai, alag controller nahi. Endpoints: `GET /products` (filters, sort, pagination ke saath aggregation pipeline; `offeredPrice` server pe calculate hota hai), `/store/:sellerId`, `/shops` (seller-wise group), `/navigation/trending-tags`, `/:id`, reviews (POST sirf us user ke liye jisne product khareeda ho; photos S3 pe), aur `/reviews/:reviewId/helpful`.
`order.routes.js`	`POST /api/orders` (COD order), `/my-orders`, `/seller/orders`, `/user/:userId`, `/:orderId`, `PATCH /:orderId/cancel`, `/eligibility/review/:productId`, `POST /:orderId/rating` (disk upload), `GET /` (saare orders), `PUT /seller/item-status`
`payment.routes.js`	`/api/payment/create-order` (Razorpay order + DB mein Pending order), `/cancel-pending/:id`, `/verify` (HMAC-SHA256 signature check), `/webhook` (`payment.captured` / `payment.failed`)
`address.routes.js`	`/api/addresses`: CRUD + `PATCH /:id/default` (sab `protect` ke peeche, express-validator ke saath)
`home.routes.js`	`/home/tags`, `/home/categories`, `/home/products` (`optionalAuth`), `/home/sellers`
`invoice.routes.js`	`/api/document/invoice/:orderId`, `/api/document/label/:orderId`: PDFKit se PDF stream hota hai
`coupon.routes.js`	`POST /api/coupons/validate` (`protect`)
`banner.routes.js`	`GET /api/banners/active` (public)
`contactRoutes.js`	`/api/contact`: POST, GET, PUT, DELETE (seller aur admin ke beech support messages)
`contactUs.routes.js`	`POST /api/contact-us` (customer form; user ko acknowledgement email bhi jaata hai)
`NLSubs.routes.js`	`POST /api/subscribe` (newsletter)
`sitemap.js`	Dynamic `GET /sitemap.xml`. Lekin `express.static('public')` pehle mount hai, isliye practically static `public/sitemap.xml` hi serve hota hai. Products aur categories mein `slug` field bhi nahi hai.

`controllers/` (customer)

File	Functions
`user.controllers.js`	`registerUser`, `loginUser`, `userProfile`, `forgotPasswordController` (6-digit OTP, 15 min validity, email Resend se), `verifyOtpController`, `resetPasswordController`, `updateAvatar` (S3), `updateUserProfile`
`cart.controllers.js`	`getCart`, `addToCart` (product pehle se ho to quantity badhata hai), `updateCartItem` (positional `items.$.quantity`), `removeCartItem` (`$pull`), `clearCart`. Response `toClientProduct` se jaata hai.
`favorites.controller.js`	`getFavorites` (populate + `toClientProduct`), `addFavorite`, `removeFavorite`
`shipAddress.controller.js`	`saveUserAddress`, `getUserAddress` (legacy, upar dekho)
`address.controllers.js`	`getUserAddresses` (default address pehle), `createAddress` (pehla address apne aap default), `updateAddress`, `deleteAddress` (default delete hua to doosra address default ban jaata hai), `setDefaultAddress`, `getAddressById`
`blog.controllers.js`	`createBlog` (S3), `getBlogs` (tag filter ke liye escaped regex), `getBlog`
`home.controllers.js`	`getTags`, `getCategories`, `getProducts` (login ho to "recommended", warna "popular"), `getSellers`. `clampLimit()` limit ko bounds mein rakhta hai.
`coupon.controller.js`	`validateCoupon` : active hai? expire to nahi? total limit? per-user limit? Phir flat ya percent discount nikalta hai, jo subtotal se zyada nahi ho sakta.
`contactUs.controller.js`	`createContactUs` : DB mein save, user ko email, aur org (`process.env.EMAIL`) ko email
`newsletter.controller.js`	`subscribeToNewsletter` : email normalize karta hai; duplicate key `11000` aaye to 400

`services/`

File	Kaam
`user.services.js`	`createUser()`
`blog.services.js`	`createBlog`, `getAllBlogs`, `getBlogById`, `updateBlog`, `deleteBlog`
`home.services.js`	Home page ka data. Popular tags aur top categories aggregation se nikalte hain. **Personalised recommendations** tag-scoring se: wishlist ke tags ka weight 1.2×1.2, order ka 1.0, cart ka 0.8. Top 20 tags se products laata hai, cart ya order mein pehle se maujood products hata deta hai, aur price ko average order price ke 2x tak limit karta hai. Popular products aur top sellers bhi yahin se. Saath mein **in-memory TTL cache** (`withTtlCache`, 2–5 min).
`deliveryTask.service.js`	`createDeliveryTasksForOrder(order)` : order ke har seller ke liye `createDeliveryTaskForSeller()`, jo task banata hai aur phir `broadcastToNearbyRiders()` call karta hai. Pickup aur drop coordinates abhi **placeholder** `[77.209, 28.613]` **(New Delhi)** hain.
`deliveryBroadcast.service.js`	`broadcastToNearbyRiders(task)` : MongoDB `$geoNear` se 5 km ke andar ke active + available riders dhundhta hai (max 10, jinhe pehle try kiya ja chuka unhe chhod kar). Har rider ke socket pe `io.to(socketId).emit("newDeliveryRequest", …)` bhejta hai. 15 second ke baad `assignDefaultRider()` chalta hai, jo task abhi bhi Pending ho to hardcoded test rider `delivery@test.com` ko assign kar deta hai.
`badgeAutomationService.js`	`runBadgeAutomation()` : bestSeller (90 din mein 15+ delivered), trending (7 din mein 3+), mostPopular (2+ favorites), newArrival (product 15–30 din purana ho aur kam se kam 1 sale ho), topRated (rating ≥4.5 aur ≥3 reviews), onSale, limitedStock (stock ≤10). Sab updates ek `Product.bulkWrite()` mein jaate hain. `featured` badge ko ye nahi chhoota, wo admin manually set karta hai.
`statsService.js`	`getProductsByCategory()` : admin reports ke liye category-wise product count

`jobs/badgeCron.js` mein `initBadgeCron()` hai, jo node-cron se `"0 0 * * *"` pe chalta hai, yaani roz raat 00:00 UTC.

`routes-seller-panel/`, `controllersSellerPanel/`, `servicesSellerPanel/`, `utilsSellerPanel/`

File	Kaam
`SellerPanel.routes.js`	`/api/seller`: `register`, `verify-otp`, `onboarding` (multer se Aadhar/PAN file S3 pe), `resent-otp`, `login`, `profile` GET/PUT, `products`, `forgot-password`, `verify-password-otp`, `reset-password`
`seller.routes.js`	Purana duplicate. `app.js` mein mount nahi hota (comment out hai), aur isme `upload.fields[...]` jaisa syntax bug bhi hai. **Dead code.**
`addProduct.routes.js`	`POST /api/products`: max 5 images + 1 video (mp4/webm/mov, ≤20 MB) S3 pe jaate hain. Category/subcategory DB mein na ho to apne aap ban jaati hai (tags subcategory ke keywords mein merge hote hain). MRP calculate hota hai. Product `approvalStatus: pending` ke saath banta hai.
`productInfo.routes.js`	`/api/productinfo`: `GET /my` (Zod se query validate; `view=deleted/all` ke saath), `GET /:id`, `PATCH /:id/details` (multipart + Zod), `PATCH /:id/pricing` (MRP dobara calculate), `DELETE /:id` (soft delete; product kisi active order mein ho to block)
`dashboard.routes.js`	`/api/dashboard/stats/:sellerId`, `/orders/:sellerId`. **Auth nahi hai**, aur `Order.find({})` se saare orders memory mein load karke filter karta hai.
`stats.routes.js`	`/api/stats/overview`, `/sales-graph`, `/order-status`, `/category-stats`, `/inventory`, `/inventory-management` (sab `protectSeller` ke peeche)
`category.routes.js`	`GET /api/categories`, `/api/categories/:categoryName/subcategories` (public)
`delivery.routes.js`	`/api/seller/delivery/active-deliveries`, `PUT /markAsPickedUp`
`sellerPanelController.js`	Upar ke routes ke thin controllers. `updateSellerProfile` JSON strings parse karta hai, store name unique hai ya nahi check karta hai, aur `$set: req.body` karta hai.
`statsController.js`	Stats routes ko `servicesSellerPanel/statsService.js` se jodta hai
`categoryController.js`	`getAllCategories`, `getSubcategories`
`sellerDeliveryController.js`	`markAsPickedUp` (MongoDB transaction ke andar), `getActiveDeliveries`
`sellerPanelService.js`	`registerSellerService` (bcrypt hash, 6-digit OTP 5 min ke liye, email), `verifyOtpSellerService` (verify karke JWT deta hai), `onboardingCompletedSellerService` (business, bank aur KYC details save, `status=pending`, saare admins ko notification), `loginSellerService`, `forgotPasswordSellerService`, `verifyOtpService`, `resetPasswordSellerService` (sirf tab chalega jab `emailOtp === null` ho aur reset window abhi khuli ho), `resentOtpService`, `getSellerProfileService`, `updateSellerProfileService` (ye function use nahi hota)
`statsService.js` (servicesSellerPanel)	Aggregations: `getSellerStats`, `getSalesOverTime` (date-wise `$dateToString`), `getOrderStatusDistribution`, `getOrdersByCategory` (`$lookup` products), `getInventoryMetrics`, `getInventoryManagement` (alert cards + stock status)
`orderEmailService.js`	`sendOrderEmails`: customer ko "Order Confirmed" aur admin ko "New Order" email. Sirf COD flow mein call hota hai.
`sendEmail.js`	**Resend** API se email bhejta hai. `EMAIL_MODE=test` ho to saari mails ek developer address pe redirect ho jaati hain. Purana nodemailer code comment out hai.
`sellerVerifyTemplate.js`	OTP email ka HTML template
`AppError.js`	`utils/AppError.js` ka duplicate
`dateUtils.js`	`getDateRange('today'|'week'|'month')`. Kuch aur do to all-time range.

`routes-admin-panel/`, `controllersAdminPanel/`, `utilsAdminPanel/`

File	Kaam
`admin.routes.js`	`/api/admin`: `login`, `products?status=`, approve/reject, `products/:id/badges` (strict Zod schema), sellers, seller ke products, delete product, `verify`, analytics (sales/users/weekly-active), seller delete/block, category stats, riders `pending` / `active` / `:id/status`. **Rider wale 3 routes pe** `protectAdmin` **nahi hai**; code comment khud kehta hai "bypassing".
`banner.routes.js`	`/api/admin/banners`: CRUD + toggle (`router.use(protectAdmin)`, image S3 pe)
`notification.routes.js`	`/api/notifications`: get, mark read, read-all, delete. Ye customer wala `protect` use karta hai, isliye seller aur admin ke token yahan kaam nahi karte.
`Newsletter.routes.js`	`POST /NLsend`: sab subscribers ko newsletter. `app.js` **mein mount nahi hai (dead code).**
`admin.controller.js`	`loginAdmin`, `getPendingProducts`, `approveProduct` / `rejectProduct` (dono seller ko notification bhejte hain), `updateProductBadges`, `getAllSellers` (har seller ka approved product count), `getSalesAnalytics` / `getUserAnalytics` (month-wise), `getWeeklyActiveUsers`, `deleteSeller`, `blockSeller`, `getSellerById`, `getCategoryStats`, `getPendingRiders`, `getAllActiveRiders`, `updateRiderStatus`
`banner.controller.js`	`createBanner` (naya active banner baaki sab ko inactive kar deta hai), `getAllBanners`, `toggleBannerActive`, `updateBanner`, `deleteBanner`, `getActiveBanner` (public)
`contact.controller.js`	Support messages ka CRUD. Message seller se aaya ho to saare admins ko notification. Agar DB mein koi admin na ho to ek **default admin bana deta hai**.
`notification.controller.js`	`getNotifications`, `markNotificationAsRead`, `markAllNotificationsAsRead`, `deleteNotification`, `deleteAllNotifications`
`NewsLetterEmail.js`	nodemailer + Gmail SMTP transporter

`routes-delivery-panel/`, `controllersDeliveryPanel/`

File	Kaam
`delivery.routes.js`	`/api/delivery`: `login`, `verify`, `orders`, `tasks/:taskId`, `register`, `nearby-Deliveries`, `me`, `update-location`, `accept-task`, `task-status`, `active-deliveries`, `cancel-task`, `reject-task`, `availability`. Zyaadatar routes pe Zod validation hai.
`deliveryTask.router.js`	`/api/deliverytask`: `create` (seller), `task/:id/accept` aur `task/:id/reject` (rider), `sendRequest/:id` (seller, riders ko broadcast)
`delivery.controller.js`	Rider ka main logic: • `loginDelivery`: `isActive` check, login hote hi `isAvailable = true`, aur JWT `{id, role:"delivery"}` • `registerPartner`: rider `isActive:false` ke saath banta hai, admin approve karega • `updateLocation`: GeoJSON Point save karta hai • `getNearByDeliveries`: `$geoNear` 5 km, sirf Pending tasks • `acceptTask`: MongoDB transaction mein do conditional `findOneAndUpdate`: pehle rider ko `isAvailable:true→false` se lock karo, phir task `Pending→Accepted` karo. Isse ek task do riders ko nahi milta. • `updateTaskStatus`: transaction + `allowedTransitions` state machine; `syncOrderStatusFromTask` order items ka status bhi update karta hai • `cancelTask`, `rejectTask` • `getMyTasks` / `getTaskById`: `taskToDashboardShape` se response banta hai • `getProfile`, `getMyActiveTask`, `updateAvailability`, `verifyToken`
`deliveryTask.controller.js`	`createDeliveryTask`, `acceptDeliveryTask`, `rejectDeliveryTask`, `sendDeliveryRequest`. Accept aur reject wale functions mein bugs hain: `AppError` ko bina `new` ke call kiya gaya hai, aur `assignNearestPartnerBySystem` naam ka function define hi nahi hai.

`validators/` (Zod schemas)

- `deliveryPanelValidators/` : `createDeliveryTask.schema.js` (coordinates `[lng,lat]` length 2), `deliveryAccept` , `deliveryCancel` (reason ≥3), `deliveryLogin` , `deliveryRegister` , `deliveryUpdateLocation` (`lng` , `lat` numbers), `deliveryupdateStatus` (enum), `rejectTask.Schema.js`
- `sellerPanelValidators/` :
  - `helper.js` (`toNumber` : string ko number banata hai)
  - `productValidator.js` (har field ka reusable schema)
  - `myProductQuery.js` , `updateDetails.js` , `updatePricing.js` (har ek mein `.refine` se "kam se kam ek field chahiye")
- `adminPanelValidators/` : `productBadges.schema.js` (8 badges, `.strict()`), `upateRiderStatus.schema.js` (`isActive: boolean`)

`utils/`

File	Kaam
`AppError.js`	Custom Error class: `statusCode` , `status` , `isOperational` , `payload`
`asyncHandler.js`	Async function ko wrap karta hai; error aaye to `next(err)`
`productForClient.js`	`toClientProduct()` : formula `offeredPrice = (sellingPrice + platformFee + logisticsFee + transactionFee) × (1 − discount%)` . Phir `roundToPriceEndingIn9()` price ko 9 pe khatam karta hai (742 → 739). Customer ko internal fees kabhi nahi dikhte.
`pricingConstants.js`	`GST_PERCENT = 5` , `GST_RATE = 0.05`
`resolveOrderPricing.js`	**Server-side pricing:** subtotal, 5% GST, coupon dobara validate (expiry, total limit, per-user limit), discount, total. Shipping aur delivery charge 0.
`buildPaymentOrderItems.js`	Cart ya request body se line items banata hai, lekin **price DB se nikalta hai**. Client ke bheje price pe bharosa nahi karta.
`orderWithClientProducts.js`	Order response mein har product ko `toClientProduct` shape mein badalta hai
`deliveryEta.js`	IST mein 10 AM–8 PM ke beech order ho to ETA = +2 ghante, warna +7 din
`s3Upload.js`	`uploadToS3(buffer, folder, {contentType, originalName})` → `PutObjectCommand` . Key ka format: `folder/<timestamp>-<uuid>-<safeName>` . Return karta hai `{secure_url: CloudFront URL, key}` ; ye shape Cloudinary jaisi hai taaki purana code bina badle chale. Saath mein `deleteFromS3()` bhi.
`invoiceGenerator.js`	PDFKit se invoice banakar seedha response pe stream karta hai
`shippingLabelGenerator.js`	PDFKit se A6 size ka shipping label
`forgotPasswordTemplet.js` , `userPaymentFailed.js`	HTML email templates

Scripts, seed data, aur baaki files

File / Folder	Kaam
`scripts/migrateCloudinaryToS3.js`	One-time migration: DB mein saved Cloudinary URLs download karta hai, S3 pe upload karta hai, aur DB mein CloudFront URL likh deta hai. `--dry-run` support hai, aur failure report file mein likhta hai. Ye dikhata hai ki project **pehle Cloudinary pe tha, baad mein S3 + CloudFront pe shift hua**.
`seedFile/categories.js`	`data/categories.json` se Category collection dobara seed karta hai
`seedFile/seedDeliveryPartner.js`	Test rider `delivery@test.com` banata hai. Ye wahi default rider hai jise fallback mein task assign hota hai.

File / Folder	Kaam
`seedFile/seedProducts.js`	`FrontEnd/src/data/mockData.json` se products insert karta hai
`seedFile/seedShopsAndProducts.js`	3 demo sellers aur unke product variants (browsing aur pagination test karne ke liye)
`seedFile/migrateProducts.js`	Schema migration: purane `price` / `originalPrice` fields ko `sellingPrice` / `mrp` mein badalta hai, aur missing fees fill karta hai
`data/categories.json`	6 categories: Visual Arts, Paper Crafts, Traditional & Cultural Crafts, Sculptural & 3D Crafts, Textile Arts, Decorative Arts & Mixed Media. Har subcategory ke saath occasion keywords hain (Diwali, Eid, Christmas, birthday, wedding…).
`generateSiteMap.js`	Standalone sitemap script. **Isme DB credentials hardcoded hain** (Part 0 dekho).
`public/robots.txt` , `public/sitemap.xml`	Static SEO files
`uploads/`	9 purani images, us zamane ki jab files disk pe save hoti thi. Order-rating photos abhi bhi yahin jaati hain.

1.3 `FrontEnd/` : customer website (React 19 + Vite 6)

```
FrontEnd/
├── index.html          # Google Fonts, GA4 tag, Razorpay checkout.js (window.Razorpay yahin se aata hai). <title> khaali hai.
├── vite.config.js      # react() plugin + /api/subscribe ka dev proxy (practically use nahi hota)
├── eslint.config.js    # ESLint 9 flat config
├── package.json        # name "infinart" (purana naam). Scripts: dev, build, lint, preview. Test nahi.
├── env.example         # VITE_API_URL (backend URL), VITE_SELLER_API_URL ("Become a Seller" link ka Seller-Panel URL)
│                       #   Code mein use hote hain par example mein nahi: VITE_APP_URL (SEO canonical), VITE_DEV_API_URL
├── README.md           # Vite template, project ki koi info nahi
├── public/             # robots.txt, sitemap.xml (static, 5 URLs), placeholder.jpg/svg
├── pages/              # (src ke BAHAR) har route ka patla wrapper: Navbar + component + Footer
└── src/                # asli code (neeche)
```

`pages/` :

- Main pages: `homepage.jsx` , `productpage.jsx` (URL ke category se SEO title banata hai), `shoppage.jsx` , `cartpage.jsx` , `ordersPage.jsx` , `profilepage.jsx` , `AddressListPage.jsx` , `blogpage.jsx` (BlogHome ↔ BlogForm toggle), `BlogDetailsPage.jsx` , `contactpage.jsx` , `aboutpage.jsx`
- Auth pages: `Loginpage.jsx` , `Registerpage.jsx` , `ForgotPasswordpage.jsx` , `VerificationOtpPage.jsx` , `PasswordChangeOrContinuePage.jsx` , `changepasswordPage.jsx`
- Policy pages: `PrivacyPolicy.jsx` , `TermsPage.jsx` , `SellerPolicyPage.jsx` , `BuyerPolicy.jsx` , `PrivacyPolicyAddendum.jsx` , `RefundPolicy.jsx` , plus `PoliciesSelector/PolicySelector.jsx` (in 6 policies ke beech tab bar)

`src/` **ka core:**

File	Kaam
`main.jsx`	Pehle `setupAxiosInterceptor(store)` , phir `HelmetProvider > StrictMode > Provider(store) > App`
`App.jsx`	`BrowserRouter` + saare ~30 routes, `<ScrollToTop/>` , toast containers, `useCartPersistence()`
`axiosSetup.js`	Global axios response interceptor: 401 aaye to `dispatch(logoutUser())`
`Store/store.js`	`configureStore` , 10 reducers: `user` (auth), `blogs` , `profile` , `products` , `favorites` , `cart` , `addresses` , `orders` , `reviews` , `invoice`

File	Kaam
`api/navigationApi.js`	`fetchCategories`, `fetchTrendingTags`, `fetchTopSellers`, `slugify`, mega-menu ke columns banana
`hooks/useCartPersistence.js`	Login hote hi `fetchCartFromBackend()` + `fetchFavoritesFromBackend()`
`utils/SEO.jsx`	Helmet component: title, description, canonical, OG/Twitter tags, JSON-LD
`utils/deliveryEta.js`	Backend wala ETA logic, client pe (express 2 ghante vs standard 7 din)
`utils/pricingConstants.js`	GST 5%, backend ki copy
`utils/seo.js`	**Dead + buggy**, kahin import nahi hota
`ui/`	shadcn/ui jaisa chhota component kit, Radix primitives ke upar: `Button`, `Label`, `checkbox`, `radio-group`, `sheet` (FilterDrawer ka side drawer), `dialog` (styled-components), `toast` / `toaster` / `use-toast` (mount hai par koi call nahi karta)
`data/`	`mockData.js/.json`, `product.json`, `productData.js`. **Sab dead**, sirf seed script `mockData.json` padhti hai.
`Images/`	~60 local images + order-success `.webm` video (~69 MB total; kuch files 3–4 MB ki hain)
`App.css`, `assets/react.svg`	Template leftovers

`src/Redux/`: `*Service.js` files mein axios calls hain, aur `*Slice.js` files mein `createAsyncThunk` + reducers.

File(s)	Kaam
`authService.js` + `authSlice.js`	register, login, forgot-password, verify-otp, reset-password. Session `localStorage["user"]` mein `{user, token}` ke roop mein. Reducers: `restoreUserFromSession`, `logoutUser`.
`userSlice.js`	Profile GET/PUT, avatar upload (axios seedha slice mein)
`productSlice.js`	`fetchProducts` (server-side filters, sort, page), `fetchShops`, `fetchProductById` + filter/sort/page ka UI state
`cartService.js` + `cartSlice.js`	Server pe cart CRUD. Har response ke baad poora cart backend wale se replace hota hai. Guest cart nahi hai.
`favoritesService.js` + `favoritesSlice.js`	Wishlist, sirf product IDs store hote hain
`addressSlice.js`	Address CRUD (axios inline). `addressService.js` iska dead duplicate hai.
`orderService.js` + `orderSlice.js`	my-orders, order by id, COD create, Razorpay create/verify, cancel, cancel-pending, rating
`reviewsService.js` + `reviewsSlice.js`	Reviews fetch/submit, helpful vote
`BlogService.js` + `blogSlice.js`	Blogs list/create
`documentService.js` + `documentSlice.js`	Invoice/label PDF ko Blob ki tarah download karta hai

`src/components/`:

- **Layout:**
  - `Navbar/` (mega menu backend categories se, cart/wishlist badges, mobile bottom nav)
  - `Footersection/`
  - `ScrollToTop.jsx` (har route ki scroll position sessionStorage mein yaad rakhta hai)
  - `categoryBannerImages.js` (tag ke naam se local image map karta hai)

- **Home page:**
  - `SearchBar/` (tag-based search + trending tags; full-text search nahi hai)
  - `Herosection/` (rotating banners)
  - `Scrollsection/` (`/home/tags`)
  - `Shopsection/` (occasions)
  - `BestSeller/` (`/products/shops`)
  - `Salesection/` (admin ka active banner)
  - `Coustomerreview/` (hardcoded testimonials)
  - `Blogsection/` (andar `LatestBlogSection` render karta hai)
  - `NewsletterSection/`
- **Auth:** `LoginSection/` , `RegisterSection/` , `ForgotPassword/` , `VerificationOtp/` (6 OTP boxes + countdown), `PasswordChangeOrContinue/` , `ChangePassword/`
- **Products** (folder ka naam `Product/Testing/` hai, **lekin ye production code hai**):
  - `ProductDetails/` (gallery + video, zoom, MDXEditor read-only description, Buy Now, Product JSON-LD)
  - `ShopM/Product/ShopM.jsx` (listing page; URL query params hi "source of truth" hain)
  - `ShopM/ProductCard/` , `ShopM/FilterDrawer/` , `ShopM/SortDropdown/` , `ShopM/Pagination/` , `ShopM/Breadcrumb/`
  - **Dead:** `ShopM/FilterDrawer/temp.jsx` , `ShopM/BuyNowDialog/` , `ShopM/ImageGallery/`
  - `Product/RatingStars/`
- **Shops:** `ShopCom/` (saari shops), `ShopDetails/` (ek shop ke products; ShopM ka copy-paste hai)
- **Reviews:** `Review/ProductReview/` (summary, sort, error boundary, review form sirf khareedne wale ko), `Review/ReviewCard/` (helpful vote, image lightbox), `Review/ReviewInput/` (stars, text, 5 photos tak), `Review/RatingBreakdown/`
- **Cart, wishlist, address:** `CartSec/Cart.jsx` , `Favorites/FavoritesPage.jsx` , `Address/AddressUI/Address.jsx` (form validation; "Use My Location" button `navigator.geolocation` + OpenStreetMap Nominatim reverse geocoding use karta hai), `Address/AddressList/`
- **Orders:**
  - `Order/Checkout/` (address choose karo, COD ya Razorpay, coupon, totals)
  - `Order/OrderPlaced/` (success page)
  - `Order/OrderList/` (har item ek card)
  - `Order/OrderDetails/` (timeline, cancel, invoice/label download)
  - `Order/CancelOrderModal/`
  - `Order/OrderReview/`
- **Profile:** `ProfileSec/profilesec.jsx` (avatar upload, naam/phone/address edit, sign out, "Become a Seller" link)
- **Blog:**
  - `Blog/BlogHome/` (hero slides + search) + `LatestBlogSection.jsx` (carousel)
  - `Blog/BlogDetails/`
  - `Blog/BlogForm/` (blog create)
  - `Blog/BlogManager/` (**dead**; jo file import karta hai wo exist hi nahi karti)
  - `Blog/Blogsection/` (**dead + broken**)
- **Static:** `About/` , `ContactUs/` , `PrivacyPolicySection/` , `TermsOfUse/` , `SellerPolicy/` , `BuyerPolicyComponent/` , `PrivacyAddendum/` , `RefundPolicy/`

- **Common:** `Common/BackButton/`, `Common/DeliveryEstimateBanner/`
- **Legacy:** `SellerPages/Handler.jsx`, `ProductListing/SellerForm.jsx`, `ProductForm.jsx`. Purana `/seller` form hai jo hardcoded `localhost:5000/seller/...` pe jaata hai. Wo backend route mount nahi hai, isliye 404 aata hai.
- **CSS:** har component ke saath usi naam ki plain global `.css` file (~60 files). CSS Modules nahi hain.

Routes (`App.jsx`):

- `/`, `/products`, `/:category`, `/:category/:subcategory`, `/:category/:subcategory/:productName/:productId` (product page)
- `/shops`, `/shop-products?store=`, `/sale`
- `/cart`, `/favorites`, `/order-checkout`, `/order-placed/:orderNumber`, `/OrderList`, `/order_details?order_id=&item_id=`, `/AddressList`, `/profile`
- `/blog`, `/blogs/:id`, `/contact`, `/about`
- Auth wale 6 routes aur 6 policy routes
- `/seller` (legacy)
- 404 route nahi hai. `ProtectedRoute` component bhi nahi hai: har page `useEffect` mein khud check karta hai `if (!user) navigate("/login")`.

1.4 `Seller-Panel/`: seller dashboard (React 19 + Vite 7 + Redux Toolkit + MUI 7)

```
Seller-Panel/
├── index.html, vite.config.js, eslint.config.js, package.json, README (template)
├── env.example          # VITE_API_URL. Code mein VITE_APP_URL bhi use hota hai ("Switch to Customer" link), jo example mein nahi.
├── public/              # placeholder images
├── pages/               # (src ke bahar)
│   ├── Loginpage/LoginPage.jsx, RegisterPage.jsx, ForgotPasswordPage.jsx, VerificationOtpPage.jsx,
│   │   PasswordChangeOrContinuePage.jsx, ChangePasswordPage.jsx      # auth wrappers
│   ├── Mainpage.jsx (Dashboard), Stats.jsx, OrdersPage.jsx, Inventory.jsx, Settings.jsx (sirf UI, kaam nahi karta)
│   ├── CustomerInsights.jsx       # poora hardcoded mock data
│   ├── ContactPage.jsx            # admin ko support message (POST /api/contact)
│   ├── MySubmissionsPage/         # apne support tickets ki list (product submissions NAHI)
│   ├── AddProductPage.jsx
│   ├── MyProducts/                # saare products + approvalStatus + rejection ke adminNotes
│   ├── ProductPage/               # product dashboard (insight cards + table)
│   ├── ProductViewpage/ProductDetailPage.jsx   # read-only product view
│   ├── ProductUpdate/updateDetails/UpdateDetails.jsx   # naam, description, category, tags, stock, images, video edit
│   ├── ProductUpdate/updatePrices/UpdatePrices.jsx     # price + discount edit, fees/MRP live recalculation
│   ├── DeliveryPage/ActiveDeliveryPage.jsx  # delivery tasks; "Send Delivery Request" (riders ko broadcast), "Mark as Picked Up"
│   ├── ProfilePage.jsx (yahi route hota hai), SellerProfilePage.jsx (dead duplicate)
│   └── seller-support/            # HelpCenter, Faq, ShippingPolicy, TermsOfService, PrivacyPolicy, RefundPolicy (static)
└── src/
    ├── main.jsx, App.jsx          # Provider + BrowserRouter + AuthInitializer (mount pe loadCurrentSeller) + routes (Layout ke
     andar)
    ├── Store/store.js             # reducers: test, auth, products, seller, dashboard, stats, categories
    ├── Redux/
    │   ├── authService.js + authSlice.js       # login, forgot/verify/reset password, logout, loadCurrentSeller. Token:
     localStorage.sellerToken
    │   ├── sellerService.js + sellerSlice.js   # profile GET/PUT
    │   ├── dashboardService.js + dashboardSlice.js  # stats, products (/api/productinfo/my), delete, orders
    │   ├── statsService.js + statsSlice.js     # /api/stats/* (6 thunks, sirf 3 use hote hain)
    │   ├── categoryService.js + categorySlice.js
    │   ├── productSlice.js                     # dead (addProduct thunk kabhi dispatch nahi hota)
    │   └── testSlice.js                        # counter scaffold, dead
    └── components/
        ├── Register/Register.jsx      # Step 1: email + phone + password → email OTP verify
        ├── Register/Onboarding.jsx    # 4-step KYC wizard: basic → business → bank → Aadhar/PAN (FormData multipart)
        ├── AddProductForm/            # product form: MDXEditor description, CreatableSelect category/tags, images + video,
        │                              #   live price calc: platformFee 15%, logistics ₹60, transaction ₹10
        ├── Dashboard/, Statistics/, DashboardCards/, Charts/{BarChart, LineChart (nivo), PieChart, CategoryPieChart (unused)}
        ├── OrderTable/                # MUI table: har row ek item; status Pending→Dispatched; label PDF download; print packing
     slip (escapeHtml)
        ├── Orders/, ProductTable/, Inventory/, CustomerInsights/, SellerProfile/, Settings/{SettingsPage, SettingsSection}
        ├── Layouts/Layout.jsx, Sidebar/, Navbar/, Footer/ (framer-motion), Notifications/ (bell dropdown)
        ├── LoginForm/, ForgotPassword/, VerificationOtp/, PasswordChangeOrContinue/, ChangePassword/
```

```
        └── Common/BackButton/, Confirm/ (MUI delete dialog), ScrollToTop.jsx
```

Styling teen tarah se hoti hai: zyaadatar plain global CSS, 6 CSS Modules (`*.module.css`), aur MUI ka `sx` prop.

1.5 `Admin-Panel/`: admin dashboard (React 19 + Vite 7, Redux nahi)

```
Admin-Panel/
├── index.html, vite.config.js, eslint.config.js, package.json, README (template), public/vite.svg
├── env.example        # VITE_API_URL=http://localhost:5000
├── env.txt            # Wahi variable, value khaali (placeholder; koi secret nahi)
├── pages/             # (src ke bahar)
│   ├── LoginPage/            # POST /api/admin/login → localStorage.adminToken
│   ├── DashboardPage/        # 4 stat cards + latest pending products + approve/reject/delete
│   ├── ProductPage/          # pending/approved/rejected tabs; 8 badge checkboxes (optimistic update + error pe rollback)
│   ├── SellerPage/           # sellers table (search, sort, pagination)
│   ├── SellerDetailsPage/    # seller info, KYC links, bank details, block/unblock, delete
│   ├── SellerProductsPage/   # ek seller ke saare products
│   ├── OrdersPage/, OrderDetailsPage/   # saare orders + read-only order detail
│   ├── RidersPage/Riders.jsx  # pending riders approve, active riders deactivate
│   ├── BannerPage/           # homepage sale banner CRUD (image S3 pe)
│   ├── ContactMessagesPage/  # seller support messages + reply (MUI; URL hardcoded localhost:5001)
│   ├── GlobalSearchPage/     # products + sellers + riders ki browser-side search
│   ├── AnalyticsPage/        # recharts: monthly sales, monthly signups, weekly active (optional 30s polling)
│   └── ReportsPage/          # 8 cards + 6 charts, sab browser mein compute
└── src/
    ├── main.jsx
    ├── App.jsx               # Boot pe GET /api/admin/verify, isAdmin state, har route pe inline guard (isAdmin ? page :
     <Navigate/>), react-hot-toast <Toaster/>
    ├── layouts/AdminLayout.jsx   # collapsible sidebar + TopBar
    └── component/
        ├── TopBar/               # global search box + avatar menu (logout)
        ├── Pagination/           # reusable, ellipsis wali pagination
        ├── ConfirmationModal/    # generic confirm modal
        ├── Common/BackButton/
        └── Notifications/        # DEAD: kabhi render nahi hota; jo packages import karta hai (react-icons, date-fns) wo install bhi
     nahi hain
```

1.6 `delivery-dashboard/`: rider app (React 19 + Vite 7 + Redux Toolkit + Socket.io client)

```
delivery-dashboard/
├── index.html, vite.config.js, eslint.config.js, package.json, env.example (VITE_API_URL), public/
└── src/
    ├── main.jsx                  # <Provider store>
    ├── App.jsx                   # routes: /login, /register, / (DeliveryPanel), /order/:orderId/:itemId; mount pe verifyToken;
     ToastContainer
    ├── Redux/
    │   ├── store.js
    │   ├── authSlice.js          # login/register/verify. Token: localStorage.deliveryToken (+ riderId)
    │   ├── ordersSlice.js        # delivery history (Delivered/Cancelled/Failed) + search/filter
    │   ├── deliveryTaskSlice.js  # nearby tasks, active tasks, socket se aaye tasks; accept/reject/status thunks
    │   └── activeDelivery.js     # khaali file
    ├── services/
    │   ├── deliveryService.js    # axios.create({baseURL: API/api/delivery}). Request interceptor Bearer token lagata hai, response
     interceptor 401 pe token hatata hai.
    │   ├── socket.js             # io(VITE_API_URL, {autoConnect:false, transports:["websocket"]}), singleton
    │   └── index.js              # barrel export, use nahi hota
    ├── hooks/useRiderSocket.js   # connect hone pe emit("registerRider", riderId) + placeholder location; on("newDeliveryRequest") →
     Redux
    ├── components/
    │   ├── Login/, Register/ (name, phone, city, email, password), Navbar/ (logout pe availability false)
    │   ├── DeliveryPanel/        # 3 tabs: Nearby Radar / Active / History; Nearby tab pe 30 second polling
    │   ├── NearbyDeliveries/     # 5 km ke Pending tasks; "Share My Location" (navigator.geolocation) + Accept
    │   ├── DeliveryRequest/      # socket se aayi request ka popup, 10s countdown, Accept/Dismiss
    │   ├── ActiveDelivery/       # active tasks: Google Maps link, tel: call link, next-status button
    │   ├── StatusUpdater/, CancelOrderModal/, OrderDetailCard/
    └── pages/DeliveryPage/Deliverycomplete.jsx   # naam ke ulat ye OrderDetail export karta hai, yaani task detail page
```